

Neural Network Design: Deep Learning



Hagan
Jafari

MARTIN T. HAGAN AND AMIR H. JAFARI

NEURAL
NETWORK
DESIGN:
DEEP LEARNING

HAGAN

Copyright © 2026 Martin T. Hagan and Amir H. Jafari

PUBLISHED BY HAGAN

TUFTE-LATEX.GOOGLECODE.COM

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First printing, January 2026

Contents

	<i>Preface</i>	1
1	<i>Introduction</i>	1
2	<i>Multilayer Networks</i>	1
3	<i>Multilayer Network Training</i>	1
4	<i>Supplemental Training Procedures</i>	1
5	<i>Python</i>	1
6	<i>TensorFlow Intro</i>	1
7	<i>General Modular Network Notation</i>	1
8	<i>Convolution</i>	1

9	<i>Post Training Analysis</i>	1
10	<i>PyTorch Intro</i>	1
11	<i>Linear Sequence Processing</i>	1
12	<i>Nonlinear Sequence Processing</i>	1
13	<i>Sequence-to-Sequence and Attention</i>	1
	<i>Bibliography</i>	1
	<i>Index</i>	9

Preface

The field of deep learning is currently going through exponential growth. Because there are currently tens of thousand of technical papers being published each year on deep learning, it would not be possible for a single textbook to do justice to this entire volume of work. Our objective will be to provide an introduction to the key concepts that underlie deep learning. We will cover fundamental deep network architectures, such as multilayer perceptrons, convolution networks, recurrent neural networks and transformers. We will also cover the deep learning software frameworks, such as TensorFlow and PyTorch, that are used to train and implement deep networks. Our goal is to help you learn to use the latest deep networks and implementation tools, but also to understand (as much as we can) how and why they work.

Notation and the Neural Network Design Textbook

In our experience, a key to being able to discuss such a wide-ranging topic is to have a clear and consistent notation. Researchers have come to deep learning from varied backgrounds, which leads to a variety of notation and terminology. Fortunately, we have a good foundation on which to base our notation – our introductory textbook on neural networks [Hagan et al., 2014]. We wrote the first edition of that book in 1994, and the second edition was published in 2014. We will see that many of the fundamental concepts behind deep networks were known in the 1990s and were covered in that book. Here we will use the same notation and terminology that we developed for that text, with appropriate extensions as needed. We will often refer to sections of that book when they

Preface

are relevant to our deep learning presentation. (That text was not limited to shallow networks.) For convenience, we will use the identifier [NND2](#) to refer to that textbook. It is available online at <https://hagan.okstate.edu/nnd.html>.

Chapter Organization and Github Site

There are two types of chapters in this text. The first type covers theoretical topics. Each of those chapters is divided into the following sections: Objectives, Theory and Examples, Summary of Results, Solved Problems, Epilogue, Further Reading and Exercises. The Theory and Examples section comprises the main body of each chapter. It includes the development of fundamental ideas as well as worked examples (indicated by the icon shown here in the margin). The Summary of Results section provides a convenient listing of important equations and concepts and facilitates the use of the book as an industrial reference. About a third of each chapter is devoted to the Solved Problems section, which provides detailed examples for all key concepts.

$$\begin{array}{r} 2 \\ +2 \\ \hline 4 \end{array}$$

The second type of chapter covers deep learning software. Deep learning is not something that can be effectively studied from a solely theoretical viewpoint. The fact is that there are many aspects of deep learning that are not well understood at a theoretical level – we do not understand why they work as well as they do. For this reason, it is important to combine the theory with hands-on experimentation. We have chapters on Python, TensorFlow and PyTorch. Currently, Python is the language in which most deep learning work is being done, and TensorFlow and PyTorch are, by far, the most popular current frameworks for deep learning training and deployment. For each software chapter there is a matching jupyter notebook available on the textbook github site. In this way, you can follow along with the chapter and experiment with all of the coding examples. In addition, there are sets of laboratory jupyter notebooks associated with each of the coding chapters, in which you can dig deeper into the topics.

All of the chapters of this text, as well as all of the laboratories are available on the github site <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning>.

The second way in which we use Python is through the Neural Network Design: Deep Learning Demonstrations, which can be accessed from the website <https://pypi.org/project/nndesigndemos>. These interactive demonstrations illustrate important concepts in each chapter. The icon shown in the margin identifies references to these demonstrations in the text.



Book Organization

Deep neural networks can have several different types of inputs, depending on the application area.

- For *tabular* inputs, each item presented to the network consists of a list (vector) of numerical or categorical variables. There is no assumed relationship between the variables, and they can be arranged in any consistent order within an item. The order in which the items are presented to the network does not change the network response to an item.
- For *image* inputs, each item consists of pixels that are arranged in two or three dimensional arrays. The arrangement of the pixels within the arrays is important, and pixels cannot be rearranged. As with tabular inputs, the order of item presentation does not change the network response.
- For *sequential* inputs, the items can be either tabular or image. However, unlike the previous two cases, the order in which the items are presented to the network is important. This can be time series data, text, DNA sequences, audio signals, video, etc. It is assumed that there is a relationship among the items being input to the network, and that the network will use this relationship.

In this text, we will consider all three types of inputs and the networks that are most relevant to each type. We begin with tabular inputs and the multilayer network, which are the focus of Chapters 2-4. (Images can be converted to tabular form, but the result is not the most efficient.) In Chapter 8 we introduce the convolution network, which was especially designed for image processing. In Chapters 11-12 we present dynamic networks, which are designed to operate on sequential inputs.

Preface

The chapters that focus on network architectures and training methods are interspersed with chapters that cover the software that we use to implement and train the networks. The most popular deep learning software frameworks are TensorFlow and PyTorch. These are covered in Chapters 6 and 10. They are both accessed using the Python programming language, which we introduce in Chapter 5.

Chapter 7 presents some additional notation that we need to move from multilayer networks to the more sophisticated convolution and dynamic networks. Chapter 9 discusses how we can analyze a trained network to understand how it works. This analysis can be applied to any type of network.

1 Introduction

<i>Objective</i>	1
<i>What is Deep Learning?</i>	2
<i>History</i>	2
<i>Applications</i>	5
<i>AlphaFold</i>	5
<i>Large Language Models</i>	6
<i>GANs and Diffusion for Image Generation</i>	7
<i>Speech Generation and Recognition</i>	7
<i>Further Reading</i>	8

Objective

There has been a Cambrian explosion in neural network architectures in recent years. Although many of the fundamental concepts underpinning these advances were developed many years ago, the size, complexity and capabilities of these new networks have advanced rapidly since approximately 2010, resulting in a field now referred to as deep learning. In this chapter we will define what we mean by deep learning, discuss some of the reasons for the rapid expansion, describe the history behind it and consider some of the applications.

Introduction

What is Deep Learning?

Deep learning is a branch of machine learning involving algorithms that have many nonlinear processing stages. In most cases, deep learning refers to training neural networks that have many layers. Before approximately 2006, most neural network applications used one or two hidden layers. Since that time the size of the typical neural network has increased dramatically, creating the field of deep learning.

There is no consensus as to how many layers a neural network must have before it is considered deep. A multilayer network with one hidden layer is often referred to as a *shallow network*. A network without a hidden layer can only create linear functions, but it has been shown that shallow networks with one hidden layer can approximate any practical function or create any practical decision boundary, which is why many researchers did not originally concentrate on training deeper networks, even though the algorithms to do so have been available since the mid 1980s.

When deep networks first became popular, the numbers of layers rapidly increased, and many networks have hundreds of layers. However, as the field has matured, most recent efforts have been spent on designing more sophisticated and efficient architectures, rather than just increasing the numbers of layers.

History

Many of the fundamental concepts behind deep learning were well understood in the 1990s. Why has the field exploded in popularity more recently? People did attempt to use deeper networks in the 1990s, but they were difficult to train (as we will discuss in Chapter 3), and the results were rarely better than those achieved with shallower networks. However, in 2006 three different groups published papers ([[Hinton et al., 2006](#)], [[Ranzato et al., 2006](#)], [[Bengio et al., 2006](#)]) that introduced techniques to initialize the weights of deep networks so that the training would converge more efficiently, making deep networks finally practical. Although these initialization procedures were later found to be unnecessary, their initial introduction accelerated deep network research.

The history of neural networks through the 1990s was covered in the introduction of [NND2](#). Here we focus on the advent of deep networks starting around 2006.

Also in 2006, Nvidia introduced the cuda language for multi-purpose programming of graphics processing units (GPUs). Prior to that time GPUs were used almost exclusively for high level 3D graphics. The introduction of cuda allowed the power of GPUs to be applied to general purpose programming. Almost immediately, several groups began to experiment with the use of cuda for neural network training. One of the first applications ([Raina et al., 2009]) demonstrated the use of cuda in training deep belief networks. Their GPU implementation was up to 72 times faster than an optimized cpu implementation.

After researchers began training deep networks with GPUs, they began to win important competitions and to demonstrate that deep networks represented the state of the art in a number of application areas and could, in some cases, produce better-than-human performance. For example, in 2010 [Cireşan et al., 2010] trained a deep multilayer network to achieve the best performance (at that time) on the MNIST data set. This was followed in 2012 by [Krizhevsky et al., 2012], which won the famous Imagenet competition by a large margin.

Another factor that allowed the incredibly fast development of deep learning was the availability of larger and larger data sets. Large data sets, like Imagenet, were beginning to become available in the mid 2000s. In addition, the internet itself has become an enormous storage tank of data. Because deep networks have many more trainable parameters than shallow networks, more data is needed to train them adequately and to prevent overfitting.

In addition to the data sets, a number of deep learning software frameworks have been developed since 2010. Researchers no longer have to write their own cuda code to implement deep network training procedures. Deep learning frameworks, like TensorFlow (developed by Google) and PyTorch (developed by Meta) are free and open-source. They have made it possible for researchers with even a modest programming background to quickly start training and deploying deep networks.

A final factor that has fueled the growth of deep learning is the sheer number of researchers working in the field. When ChatGPT was launched in November of 2022, the platform received approximately 152 million visitors in its first month. There has never before been this level of interest in neural networks. Companies and na-

Before cuda, GPU programming was significantly more complex and limited. Programmers had to essentially "trick" the graphics pipeline into doing general-purpose computations by expressing their computations as graphics operations.

We should note that the deep networks that won many competitions in the early 2010s were essentially just larger versions of networks that had been introduced in the 1980s – multi-layer perceptrons and convolution networks.

Introduction

tional laboratories around the world are scrambling to find the next application area or the next breakthrough network architecture or training algorithm. The explosion in interest is shown in the following graph of the number of monthly google searches for the term *AI* from 2004 through 2024.

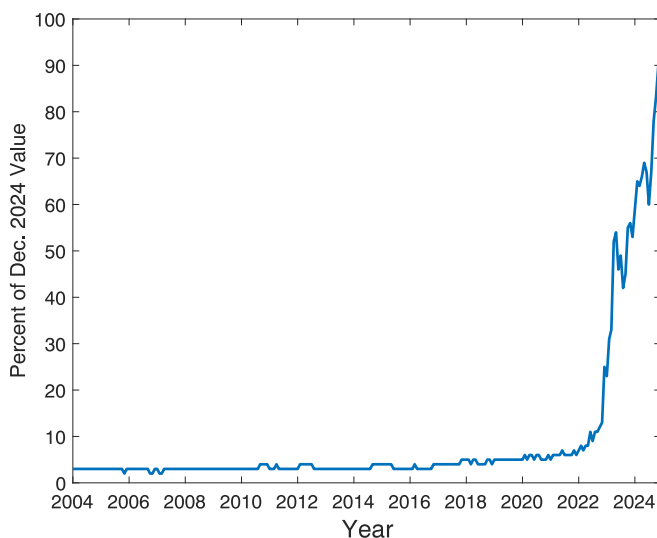


Figure 1.1: Google searches for *AI* from 2004 through 2024

So, the increased computing power of GPUs, the extremely large data sets that have become available, the powerful deep learning frameworks, and the hundreds of thousands of practitioners who have come into the field, have supercharged deep learning progress. It is currently in a virtuous cycle – more success generates more interest, development of faster hardware and efficient software, which leads to more progress.

The following figure (for natural language processing problems) illustrates the idea that has driven deep learning – the scaling law. As the computing power, the size of the data set and the size of the model increase, the performance will increase also. (If any of these stops increasing, progress may slow.)

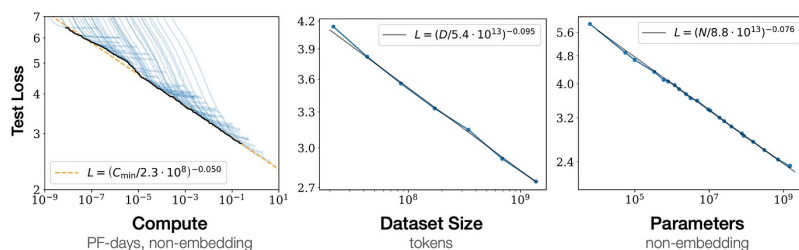


Figure 1.2: Scaling Laws (from [Kaplan et al., 2020])

Applications

Many neural network applications were described in the introduction of [NND2](#). During the 1990s and 2000s neural networks produced very successful solutions to a number of important real-world problems. Since that time, the number of applications has increased, but the principal change is that deep network performance for the most complex problems has improved tremendously. Especially in the areas of computer vision and natural language processing, deep networks are exponentially better than the neural networks (and other machine learning algorithms) that were commonly used before 2006. In many areas now, deep neural networks equal (or surpass) human level performance.

It is impossible to list all of the applications where deep neural networks are currently being used. Instead, let's just briefly review four areas in which deep learning has provided revolutionary breakthroughs. These are developments that even experts in machine learning would not have predicted in 2005.

AlphaFold

AlphaFold is a system developed by DeepMind, a subsidiary of Alphabet (Google's parent company), designed to use deep learning to predict the three-dimensional structures of proteins based on their amino acid sequences. It is one of the most significant advancements in the field of computational biology, and its designers won the Nobel prize in chemistry in 2024.

Introduction

In 2018, AlphaFold debuted at the CASP13 (Critical Assessment of Techniques for Protein Structure Prediction) competition. It used deep learning techniques to predict protein structures with an accuracy that surpassed all previous state-of-the-art methods. In 2020, AlphaFold 2 [Jumper et al., 2021] was able to predict the 3D structure of a protein with a precision that rivaled that of experimental methods like X-ray crystallography and cryo-electron microscopy, but in a fraction of the time. AlphaFold has effectively solved the protein folding problem, which has been a goal of biochemistry for decades.

With the structure of proteins now more accessible, researchers are able to study disease mechanisms at a molecular level. AlphaFold has been particularly impactful in areas like:

- **Understanding diseases:** By predicting the structures of proteins linked to diseases like Alzheimer's, cancer, and COVID-19, AlphaFold has provided a valuable resource for understanding how these proteins might behave in the body.
- **Drug development:** The ability to accurately model protein structures could accelerate the development of drugs that target specific proteins, offering new avenues for treatment.

Large Language Models

Large Language Models (LLMs), enabled by the transformer [Vaswani et al., 2017], have revolutionized natural language processing and enabled human-level performance on many language tasks. This has led to tools like ChatGPT that have transformed how people interact with computers. LLMs have impacted fields from coding to education to customer service. Computer coding has been taught differently since ChatGPT, because LLMs can write code from a natural language description of the desired operation. Human interaction in the workplace has been fundamentally changed – job applicants are using LLMs to write their resumes and cover letters, and companies are using LLMs to screen the applications. LLMs have passed the Uniform Bar Exam and the U.S. Medical Licensing Examination.

GANs and Diffusion for Image Generation

Generative Adversarial Networks (GANs) were first introduced in 2014 [Goodfellow et al., 2014]. Diffusion models were introduced in 2015 [Sohl-Dickstein et al., 2015] and were made practical for standard use in 2022 [Rombach et al., 2022]. Together, they enabled neural network generation of artificial images – what we sometimes refer to as *deepfakes*. Photo-realistic images can be created from a simple text description. This has most obviously disrupted the graphic artist profession, but there are many other applications. It can be used for molecular structure generation and materials design, style transfer, super-resolution, combined text, image, and audio generation. Short films have been generated using this technology. Actors and screenwriters have both recently gone on strike, in part, to prevent the use of this technology.

Speech Generation and Recognition

The evolution of speech technology since 2015 represents a dramatic shift from traditional methods to deep learning, and this has been revolutionary. The first neural network to generate realistic human-like speech as raw audio was WaveNet, which was developed by Google DeepMind in 2016 [Van Den Oord et al., 2016]. There have been significant yearly advances on that technology, with each step producing faster and higher fidelity results. Recently, Microsoft introduced VALL-E 2, which can synthesize high-quality personalized speech using only a three-second recording of an unseen speaker as a prompt. It is so accurate that Microsoft has not released it to the public (as this is being written), citing potential risks of misuse, such as voice spoofing and impersonation.

In 2016 Deep Speech [Amodei et al., 2016] represented a major breakthrough in speech recognition, as it was an end-to-end deep learning model that could recognize either English or Mandarin Chinese, and it was competitive with the transcription of human workers. This was further improved in 2020 with Wav2Vec 2.0 [Baevski et al., 2020], whose innovations included self-supervised learning, contrastive pretraining, and transformer-based architectures. It was a paradigm shift in speech recognition, reducing reliance on labeled data while achieving state-of-the-art performance. It broke the record for accuracy on a key speech recognition benchmark

Introduction

with 100x less labeled data. The Whisper speech recognition system, introduced by OpenAI in 2023 [Radford et al., 2023], moved the state-of-the-art even further. It works on multiple languages – even on languages on which it has not been trained. It can perform speech-to-text translation, converting spoken words from one language into text in another.

Further Reading

[Jumper et al., 2021] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021

This paper introduced AlphaFold, which can accurately predict the three-dimensional structures of proteins based on their amino acid sequences. This effectively solved the protein folding problem, which had been a goal of biochemistry for decades. This work won the Nobel Prize for chemistry in 2024.

[Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017

This paper introduced the transformer architecture, on which all current large language models (LLMs) are based.

[Goodfellow et al., 2014] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014

This paper introduced the generative adversarial network (GAN). This was the first major breakthrough in training deep networks to generate images. It works through two competing neural networks: a generator creating images and a discriminator trying to spot fakes. It led to major breakthroughs like StyleGAN (2019) which could create highly realistic human faces.

[Rombach et al., 2022] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022

This paper introduced methods that made diffusion stable and widely accessible. It is currently the state of the art for image generation. Diffusion works by adding noise to an image in a series of stages. After enough stages, the image becomes random noise. A deep network then learns to reverse the process, removing noise from a random image in stages.

[[Van Den Oord et al., 2016](#)] Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016

The deep neural network introduced in this paper, WaveNet, was the first to produce raw audio.

[[Radford et al., 2023](#)] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023

This paper describes the Whisper speech recognition system of OpenAI. It works for many languages, even languages in which it was never trained.

2 *Multilayer Networks*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Architecture</i>	2
<i>Operation</i>	6
<i>Deep Networks</i>	9
<i>Epilogue</i>	15
<i>Further Reading</i>	16
<i>Summary of Results</i>	17
<i>Solved Problems</i>	20
<i>Exercises</i>	23

Objective

This chapter discusses multilayer networks and associated notation that will be needed in other parts of the book. It describes the components of the multilayer network and the basic architecture. The chapter also describes the functionality of the multilayer network and how increasing the number of layers, creating deep networks, efficiently increases the power of the network.

Architecture

Let's begin by reviewing some of the basic components of a multi-layer neural network. Much of this was also covered in Chapter 2 of [NND2](#). The architecture of this network, as well as the general training concepts (discussed in Chapter 11 of [NND2](#)), have been known since the 1980's. Until about 2010, most people used neural networks with one or two hidden layers, but the theory for an unlimited number of layers was available 30 years before that. Since 2010 a few small additions have been made to facilitate the training of networks with many hidden layers, but the basic concepts have not changed.

The fundamental building block is the single neuron, shown in Figure 2.1. As in [NND2](#), we use p to represent external inputs into the network. In the neuron, p is first multiplied by the weight w . The result is added to the bias b to form the net input n , which is then passed through the activation (or transfer) function f to produce the neuron output a . In future chapters, it will be convenient to consider the result of the weight operation wp as a separate variable, for which we will use the letter z .

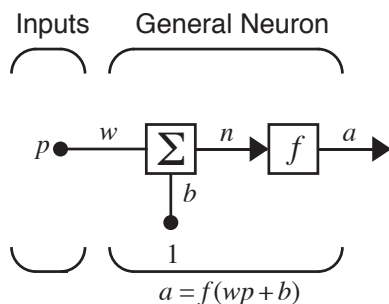


Figure 2.1: Single Neuron

There are four activation functions that we will use for deep networks. The first is the linear activation function, illustrated in Figure 2.2. This is really equivalent to having no activation function, but we include it for consistency. The graph on the right of the figure illustrates the effect of the bias on the neuron response. A positive bias shifts the response to the left, and a negative bias shifts

the response to the right. If the bias is zero, the activation will pass through the origin. Linear activation functions are generally used in the final layer of networks that are used for function approximation, or nonlinear regression.

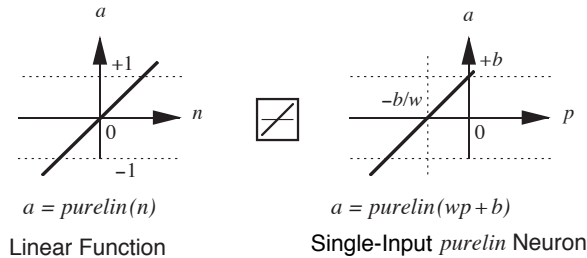


Figure 2.2: Linear Activation Function

The second commonly used activation function is the hyperbolic tangent function. This is a class of function called a sigmoid function (see also the *logsig* activation function in Chapter 2 of [NND2](#)), because of the S-shape. It is illustrated in Figure 2.3. It has been shown that two-layer networks with sigmoid activation functions are universal approximators. This nonlinear function has the advantage that it is differentiable, which is important for network training.

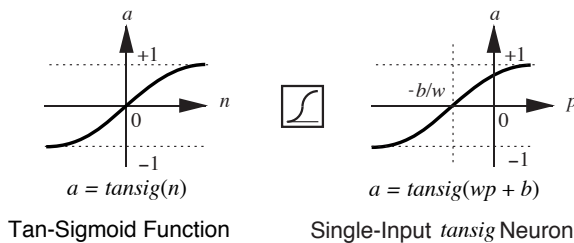


Figure 2.3: Hyperbolic Tangent Activation Function

The *tansig* function was commonly used in the hidden layers of neural networks until larger numbers of hidden layers began to be used (deep networks). When the number of hidden layers becomes larger, sigmoid activation functions lead to a problem in network training called the *vanishing gradient*, which we will discuss later. It occurs because the sigmoid function is limited in magnitude, and therefore its derivative becomes vanishingly small as the net input becomes larger in magnitude. In 2011 a different activation function was suggested for the hidden layers of deep networks (see [[Glorot et al., 2011](#)]). It was called the rectified linear unit, or ReLU. It is another name for the *poslin* (positive linear) activation

Multilayer Networks

function that was described in 1995 in the first edition of [NND2](#).

The function is shown in Figure 2.4. The output is zero when the net input is negative, and it is equal to the net input when the net input is positive.

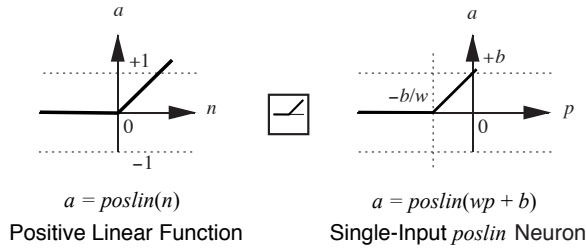


Figure 2.4: Positive Linear (or ReLU) Activation Function

The fourth activation function that we frequently use for pattern classification problems is the *softmax* function, which was described in Chapters 22 and 24 of [NND2](#). This function is used in the final layer of classification networks, and is defined by Eq. 2.1.

$$a_i = f_i(\mathbf{n}) = \frac{e^{n_i}}{\sum_{j=1}^S e^{n_j}} \quad (2.1)$$

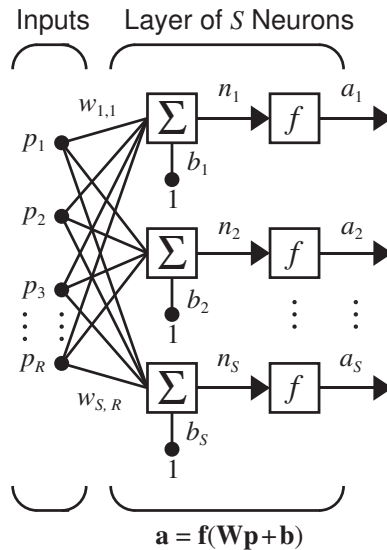


Figure 2.5: Layer of Neurons

This activation function is not defined for single neurons, but only when neurons are stacked together to create a layer, as shown in Figure 2.5. The sum of the layer outputs will be equal to 1 when the softmax activation function is used, and the output of neuron i can be interpreted as the probability that the input belongs to class i .

As in [NND2](#), we prefer to use matrix notation to represent layers of neurons, as shown in Figure 2.6. This provides simpler diagrams in which all operations are clearly visible and emphasizes that matrix operations are taking place.

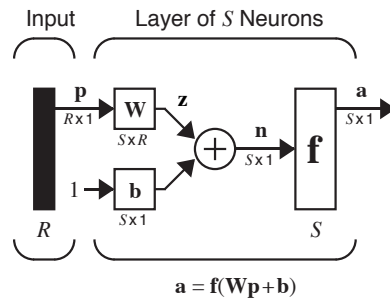


Figure 2.6: Layer of Neurons in Matrix Notation

Overall, there are three components to a layer: 1) a weight operation ($\mathbf{z} = \mathbf{W}\mathbf{p}$ in this case), 2) a net input operation ($\mathbf{n} = \mathbf{z} + \mathbf{b}$ in this case), and 3) an activation function operation ($\mathbf{f}(\mathbf{n})$).

Layers of neurons can be cascaded to obtain multilayer networks, as shown in Figure 2.7. By extending the network to more than one layer, the power of the network is increased dramatically. As described in Chapter 4 of [NND2](#), a single layer network can only implement a linear decision boundary or a linear network response, while, as described in Chapter 11 of [NND2](#), a network with two layers is a universal approximator.

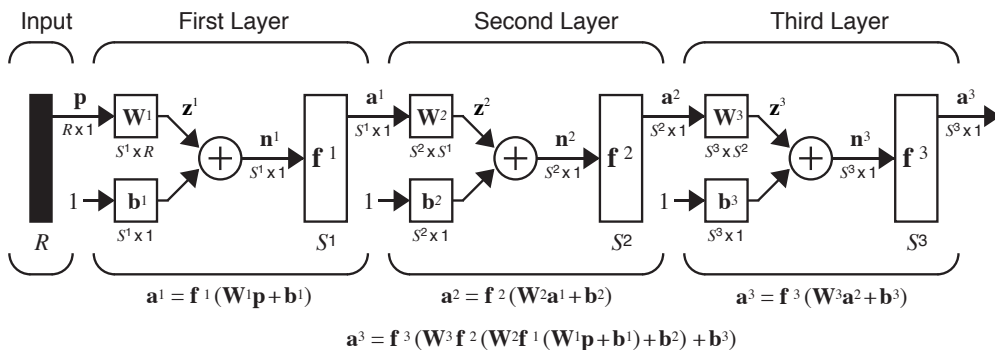


Figure 2.7: Three Layer Network

Multilayer Networks

Operation

To illustrate the operation of a single layer network, consider the network in Figure 2.8. The activation function is the *hardlims*, which outputs -1 for net inputs less than zero, and $+1$ otherwise. This produces a simple decision making network.

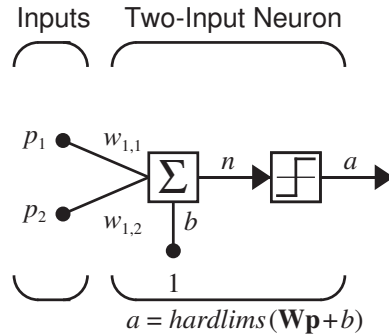


Figure 2.8: Example One Layer Network

If the weight matrix is chosen to be $\mathbf{W} = \begin{bmatrix} -1 & 1 \end{bmatrix}$, and the bias is chosen to be $b = [-1]$, then the network response would be

$$a = f(n) = f(\mathbf{W}\mathbf{p} + b) = \text{hardlims}(-1p_1 + 1p_2 - 1) \quad (2.2)$$

The network output will be 1 whenever the net input is greater than or equal to 0. This defines the decision boundary, which is shown in Figure 2.9. All single layer networks have linear decision boundaries. (See Chapter 4 of [NND2](#).) The function created by the net input is linear, and the locations where that function is equal to a fixed value defines a line (or hyperplane in higher dimensions). If the classes we are trying to recognize cannot be separated by a linear decision boundary, then a multilayer network must be used.

To understand how multilayer networks allow us to create very general nonlinear functions, consider the network in Figure 2.10.

This is like the example in Chapter 11 of [NND2](#), except that the *poslin* activation function is used in the hidden layer. Since the *poslin* (*ReLU*) function was one of the important innovations for deep network training, it is useful to see how it changes the form of the function that is created by the network. Suppose that the weights and biases of the network are set to the following values:

$$\mathbf{W}^1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \mathbf{W}^2 = \begin{bmatrix} -1 & 1 \end{bmatrix}, \mathbf{b}^2 = [0] \quad (2.3)$$

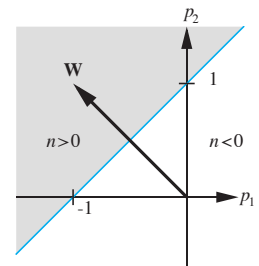


Figure 2.9: Linear Decision Boundary

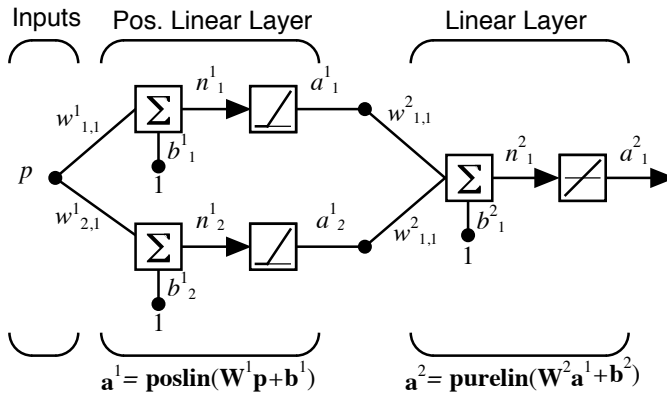


Figure 2.10: Two Layer *poslin* Network

The function created by this network is shown in Figure 2.11. We can see that with the *poslin* activation function in the hidden layer the network creates piece-wise linear functions.

By changing the weights in the network, we can change the slopes of the linear segments of the network function. By changing the biases in the hidden layer, we can move the break points between the line segments, and by changing the bias in the final, linear layer, we can shift the function up or down. This is illustrated in Figure 2.12.

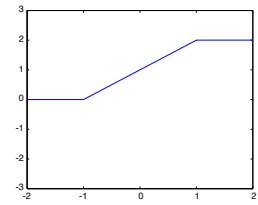


Figure 2.11: Function Created by the *poslin* Network

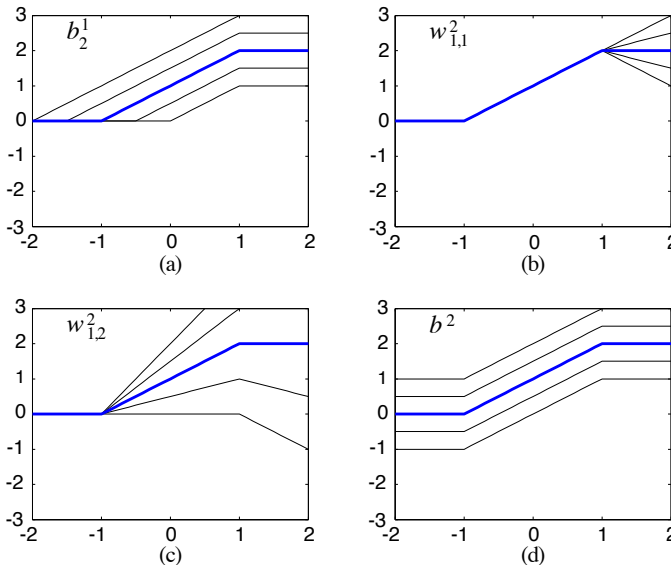


Figure 2.12: Variations in Network Function

Multilayer Networks

The set of functions that can be created by a two-layer network, with *poslin* activation functions in the hidden layer, and a linear output layer, is the set of all piece-wise linear functions. This makes the two-layer *poslin* network a universal approximator.

To experiment with this poslin network, use the Deep Learning Demonstration Poslin Network Function (**dl2pnf**).



In addition to creating functions, we can also use the two layer network to perform pattern classification. Consider the network in Figure 2.13. Let the weights be set as follows:

$$\mathbf{W}^1 = \begin{bmatrix} 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \end{bmatrix}^T, \mathbf{b}^1 = \begin{bmatrix} -1 & 3 & 1 & 1 \end{bmatrix}^T \quad (2.4)$$

$$\mathbf{W}^2 = \begin{bmatrix} -1 & -1 & -1 & -1 \end{bmatrix}, \mathbf{b}^2 = [5] \quad (2.5)$$

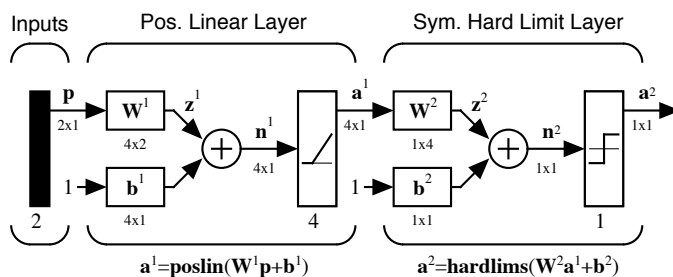


Figure 2.13: Pattern Classification Network

Since this network has two inputs, each row of the first weight matrix points in the direction of a sloping plane, whose starting location is determined by the corresponding bias. Each row of the weight creates a separate plane, and they are all added together at the output. The resulting composite function that is created at the net input of the second layer (n^2) is shown in Figure 2.14(a). When this is passed through the *hardlims* activation function, we obtain the decision region shown in Figure 2.14(b). The solid region shows the area where n^2 is greater than or equal to zero. By including more neurons in the second layer, it is possible to create essentially arbitrary decision regions.

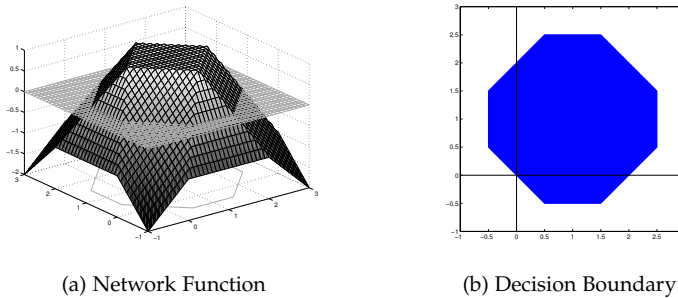


Figure 2.14: Network Response (a) and Decision Boundary (b)

To experiment with these two dimensional decision regions, use the Deep Learning Demonstration Poslin Decision Regions (**dl2pdr**).



Deep Networks

With two-layer networks it is possible to create almost arbitrary functions and decision regions. Then why are deep networks needed? At the current time, this question has not been fully answered to everyone's satisfaction. In practice, it has been shown that networks with many layers have been able to solve problems that have not, until now, been solved with two layer networks, but the theoretical explanation of this is not complete. Some initial theoretical work has been done, and we will describe a couple of these explanations here. The main point of both explanations is that deeper networks can produce more efficient realizations of some complex functions than two layer networks can.

One approach to understanding the benefits of deep networks was taken by Telgarsky. [Telgarsky, 2016] He compared the effect of increasing the number of neurons in a single hidden layer with increasing the number of layers. This involved describing the operation of creating composite functions by cascading functions together. A full description of his theory is beyond the scope of this book, but a simple example can illustrate the concept.

Suppose that we use the network of Figure 2.10, but we change the weights to the following:

$$\mathbf{W}^1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 0 \\ -0.5 \end{bmatrix}, \mathbf{W}^2 = \begin{bmatrix} 2 & -4 \end{bmatrix}, \mathbf{b}^2 = [0] \quad (2.6)$$

$$\begin{array}{|c|} \hline 2 \\ +2 \\ \hline 4 \\ \hline \end{array}$$

Multilayer Networks

This creates the triangle function shown in Figure 2.15. Now, create a composite function by cascading this function with itself: $f \circ f(p) = f(f(p))$. This process is illustrated in Figure 2.16. The blue line represents the original triangle function $f(p)$. The green line at 45° represents p . The red line represents the composite function $f \circ f(p)$. In Figure 2.16(a) we are evaluating the composite function at $p = 0.2$. We first find $f(0.2)$, which is 0.4. This is then input to $f()$ again, which produces 0.8, which means that $f \circ f(0.2) = 0.8$, which falls on the red line. In Figure 2.16(b) the process is repeated for $p = 0.8$.

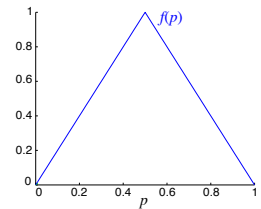


Figure 2.15: Linear Decision Boundary

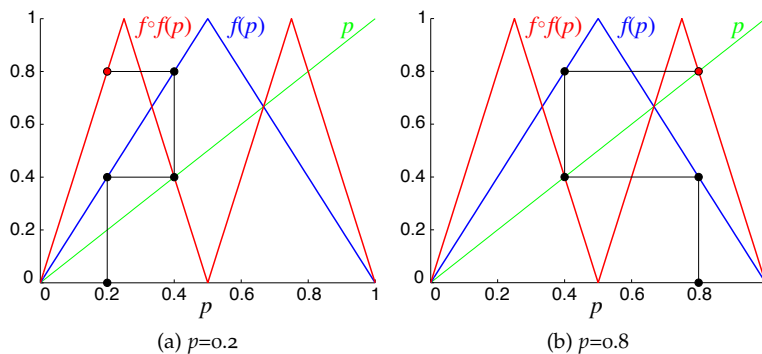


Figure 2.16: Composite function

By cascading the single triangle function, we have created a function with two triangles. It may seem that this requires a four layer network, since we have cascaded two networks, each with two layers. However, since the second layer is linear, it can be merged with the *poslin* layer that follows it, so the resulting network only requires three layers. (The weights of the corresponding three layer network are shown in Eq. 2.7.) Therefore, by adding a single layer to the network we have doubled the complexity of the function we have created.

$$\mathbf{W}^1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 0 \\ -0.5 \end{bmatrix}, \mathbf{W}^2 = \begin{bmatrix} 2 & -4 \\ 2 & -4 \end{bmatrix}, \mathbf{b}^2 = \begin{bmatrix} 0 \\ -0.5 \end{bmatrix}, \mathbf{W}^3 = \begin{bmatrix} 2 & -4 \end{bmatrix}, \mathbf{b}^3 = [0] \quad (2.7)$$

The function shown in Figure 2.16 could be created with a two-layer network, but it would require doubling the number of neurons in the hidden layer (since each *poslin* neuron creates one linear segment). This difference in complexity becomes more pronounced

as we increase the number of layers in the network. Each time we add a layer (which corresponds with a linear increase in the number of neurons), we double the number of neurons that would be required for a two layer network to implement the same function.

*To experiment with increasing the number of layers in a network, use the Deep Learning Demonstration Cascaded Function (**dl2cf**).*



The concept demonstrated in the above example is also used by [Montufar et al., 2014], who show that in deep networks with multiple inputs the number of distinct linear regions in the input space is geometrically larger than for two-layer networks with a similar number of neurons. The take-away is that two-layer networks can create any function that can be created with a deep network. However, the number of neurons required in the two-layer network increase geometrically, while the number of neurons increase linearly in the equivalent deep network.

Another approach for demonstrating the efficiency of deep networks was used by [Eldan and Shamir, 2016]. They also showed that to achieve the equivalent power of approximation of a deep network requires that the number of neurons in the hidden layer of a two-layer network must grow geometrically. They demonstrate this by considering radial functions – functions whose response depends only on the distance of the input from the origin. Even simple radial functions cannot be efficiently approximated by two-layer networks, but can be easily approximated by three-layer networks.

The basic idea is that a radial function depends only on the norm of the input. Therefore, it can be approximated using a three-layer network by first using the first two layers to approximate the squared norm function, and then using the last layer to approximate the function acting on the norm. (To be more precise, the second layer of the three layer network combines parts of both approximations.)

To demonstrate this concept, consider the radial function

$$g(\mathbf{p}) = \cos(\pi \|\mathbf{p}\| / 2), \quad (2.8)$$

$$\begin{array}{r} \sqrt{2} \\ +2 \\ \hline 4 \end{array}$$

Multilayer Networks

which is shown in Figure 2.17. To approximate the squared norm function $\|\mathbf{p}\|^2 = p_1^2 + p_2^2$, we can start with the simple square function p^2 . We can use a two-layer network with *logsig* neurons in the first layer and a linear neuron in the second layer. To approximate the square function over the range $-2 < p < 2$, the weights for this network could be

$$\mathbf{W}^1 = \begin{bmatrix} -0.2 \\ -0.2 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 2 \\ -2 \end{bmatrix}, \mathbf{W}^2 = \begin{bmatrix} -320 & 320 \end{bmatrix}, \mathbf{b}^2 = [243.7] \quad (2.9)$$

The network response and the square function are shown in Figure 8.4a. Even with only two hidden layer neurons, the approximation is almost exact over the range $-2 < p < 2$. We can now stack two of these networks, with p_1 as the input to the top network and p_2 as the input to the bottom network, and sum the outputs to create the function $p_1^2 + p_2^2$. This would be input to another network that would approximate the function $\cos(\pi\sqrt{p}/2)$, so that the total network response approximates $\cos(\pi\|\mathbf{p}\|/2)$.

For the second network, we need to approximate $\cos(\pi\sqrt{p}/2)$ over the range $-8 < p < 8$, since the input to this network is $p_1^2 + p_2^2$. The weights that could approximate this function are given by Eq. 2.10, and the approximation is shown in Figure 2.19.

$$\mathbf{W}^1 = \begin{bmatrix} -0.2489 \\ -0.3541 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 1.2931 \\ -2.6491 \end{bmatrix}, \mathbf{W}^2 = \begin{bmatrix} -6.6898 & 69.3431 \end{bmatrix}, \mathbf{b}^2 = [1.6693] \quad (2.10)$$

Putting the smaller networks together, and combining the linear output layers of the first (square) networks with the input layers of the $\cos(\pi\sqrt{p}/2)$ network, produces a three-layer network with architecture 2-4-2-1. The weights are given by Eq. 2.11.

$$\begin{aligned} \mathbf{W}^1 &= \begin{bmatrix} -0.2 & 0.0 \\ -0.2 & 0.0 \\ 0.0 & -0.2 \\ 0.0 & -0.2 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 2 \\ -2 \\ 2 \\ -2 \end{bmatrix} \\ \mathbf{W}^2 &= \begin{bmatrix} 79.648 & -79.648 & 79.648 & -79.648 \\ 113.312 & -113.312 & 113.312 & -113.312 \end{bmatrix}, \mathbf{b}^2 = \begin{bmatrix} -120.0208 \\ -175.2374 \end{bmatrix} \\ \mathbf{W}^3 &= \begin{bmatrix} -6.6898 & 69.3431 \end{bmatrix}, \mathbf{b}^3 = [1.6693] \end{aligned} \quad (2.11)$$

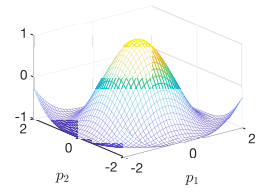


Figure 2.17: Radial Function $\cos(\pi\|\mathbf{p}\|/2)$

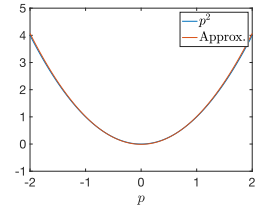


Figure 2.18: Square Function and Network Approximation

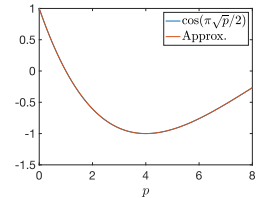


Figure 2.19: $\cos(\pi\sqrt{p}/2)$ and Network Approximation

The resulting three-layer network approximation of the radial function is shown in Figure 2.20. With three layers, even a simple network is able to capture the radial form. However, doing this with a two-layer network is much more difficult, especially if the radial form is more complex.

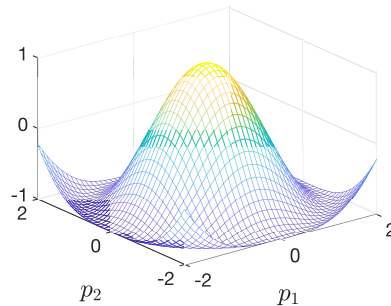


Figure 2.20: Three-Layer Network Approximation of $\cos(\pi \|\mathbf{p}\| / 2)$

The reason that approximating radial functions is more difficult with two-layer networks can be seen if we consider the operation of the simple two-layer network in Figure 2.10. The individual neuron responses from the first layer are combined in the second layer. The total network response is a weighted sum of single neuron responses. With a single input, each neuron response takes form of the activation function. With multiple inputs, the neuron response also takes the form of the activation function, but along the direction of the corresponding row of the weight matrix. In directions orthogonal to the weight direction, the neuron response is constant, forming a ridge.

The response of a single *logsig* neuron with two inputs is illustrated in Figure 2.21. This neuron has a weight $\mathbf{W} = \begin{bmatrix} -5 & 5 \end{bmatrix}$ and a zero bias. The arrow in the figure points in the direction of the weight. The key idea is that the response does not change orthogonal to the weight direction, which creates a ridge function. It is difficult to create radial functions by superimposing ridge functions, since they have fundamentally different characteristics. Radial functions are constant along contours that are equal distance from some origin, and ridge functions are constant along contours that are orthogonal to the weight vector. Approximating a radial function with a combination of ridge functions is a little like approximating a circle with a series of line segments. It can be done, but it is not very efficient.

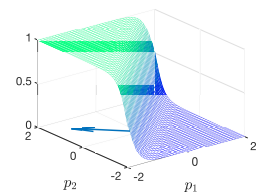


Figure 2.21: Single *logsig* Neuron Response

Multilayer Networks

This is illustrated in the following figures. Figure 2.22 shows the function $\cos(\pi \|\mathbf{p}\| / 2)$, a three-layer approximation of the function, and the corresponding error. The network used here is even simpler than the one described earlier, since there are only three neurons in the first layer. All weights were adjusted together to produce the approximation.

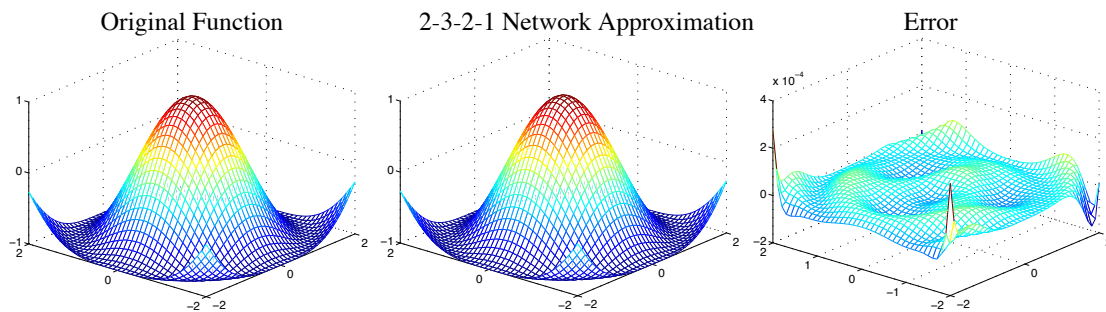


Figure 2.22: Three-layer network approximation of $\cos(\pi \|\mathbf{p}\| / 2)$

Figure 2.23 shows approximations obtained with two-layer networks, and their corresponding errors. The far left subfigure shows the approximation obtained by a 2-5-1 network. This network has 21 parameters (weights and biases), which compares with the 20 parameters of the 2-3-2-1 network demonstrated in Figure 2.22. Even though it has the same number of parameters, the two-layer network approximation is much worse. Even the 2-7-1 network in the right subfigure, which has 29 parameters, does not achieve as accurate an approximation as the three-layer network with 20 parameters.

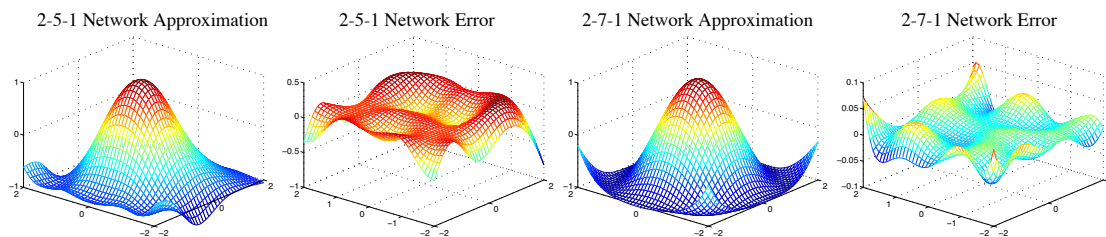


Figure 2.23: Two-layer network approximation of $\cos(\pi \|\mathbf{p}\| / 2)$

To experiment with two- and three-layer network approximations, use the Deep Learning Demonstration Function Approximation (**d12fa**).



Epilogue

This chapter has reviewed the basic structure and operation of the multilayer network. The basic concepts used in deep networks are the same as those for shallow networks. This why much of this chapter has reviewed material from Chapters 2 and 11 of [NND2](#).

One important modification to multilayer networks when used for deep learning is the use of the *poslin*, or ReLU activation function in the hidden layers. This function was described in [NND2](#), but was used principally for the Hamming network. The *poslin* function has become important for deep networks, because of issues with the sigmoid activation function when the number of layers in the multilayer network becomes large. This is because the sigmoid output is limited in magnitude, and therefore its derivative becomes vanishingly small as the net input gets larger. We will have more to say about this in the next chapter.

This chapter also demonstrated how deep networks can be more efficient in approximating certain types of functions than two-layer networks. Essentially, the power of approximation of a multilayer network increases linearly as you increase the number of neurons in a single hidden layer, while the power increases geometrically as you increase the number of layers.

In the next chapter we will discuss the training of the deep multilayer network. We will review some of the concepts from Chapter 11 and 12 of [NND2](#), since the basic training concepts are the same, no matter how many layers are involved. However, we will highlight some recent modifications that are well-suited to deep networks.

Further Reading

[Glorot et al., 2011] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323, 2011

This paper describes the use of the ReLU (*poslin*) activation function in the hidden layers of deep networks. The output of this function is equal to zero for negative inputs and equal to the input for positive inputs. This is useful when training networks with many layers, because it prevents the vanishing gradient problem that occurs when using sigmoid activation functions.

[Telgarsky, 2016] Matus Telgarsky. Benefits of depth in neural networks. In *Proceedings of the 29th Annual Conference on Learning Theory*, pages 1517–1539, 2016

Telgarsky proves that it takes a geometric increase in the number of neurons in the hidden layer of a two-layer network to approximate a function that a deep network can approximate with a linear increase in the number of layers.

[Montufar et al., 2014] Guido F Montufar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in neural information processing systems*, pages 2924–2932, 2014

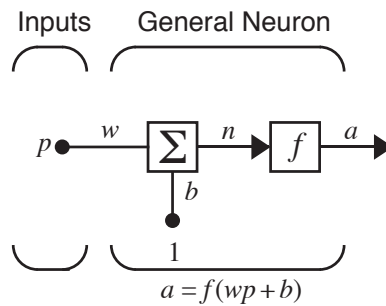
This work is related to Telgarsky's. They show that in deep networks with multiple inputs the number of distinct linear regions in the input space is exponentially larger than for two-layer networks with a similar number of neurons.

[Eldan and Shamir, 2016] Ronen Eldan and Ohad Shamir. The power of depth for feedforward neural networks. In *Conference on Learning Theory*, pages 907–940, 2016

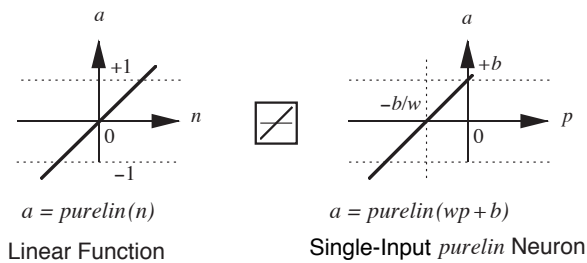
This paper also shows that to achieve the equivalent power of approximation of a deep network requires that the number of neurons in the hidden layer of a two-layer network must grow geometrically. They demonstrate this by considering radial functions – functions whose response depends only on the distance of the input from the origin. Even simple radial functions cannot be efficiently approximated by two-layer networks, but can be easily approximated by three-layer networks.

Summary of Results

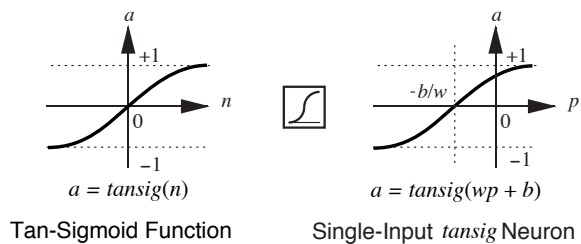
Single Input Neuron



Linear Activation Function

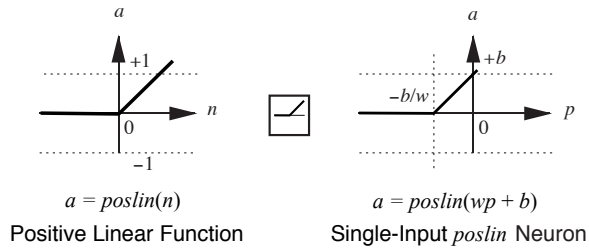


Hyperbolic Tangent Activation Function



Multilayer Networks

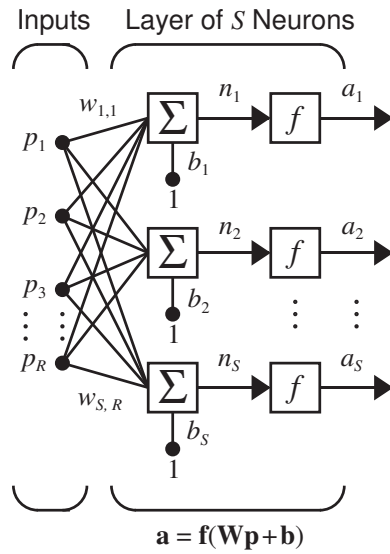
Positive Linear (or ReLU) Activation Function



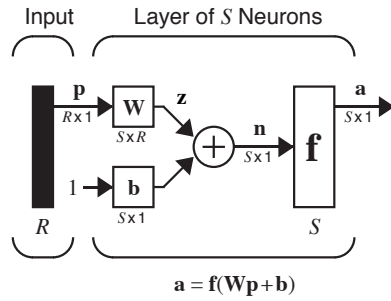
Softmax Activation Function

$$a_i = f_i(\mathbf{n}) = \frac{e^{n_i}}{\sum_{j=1}^S e^{n_j}}$$

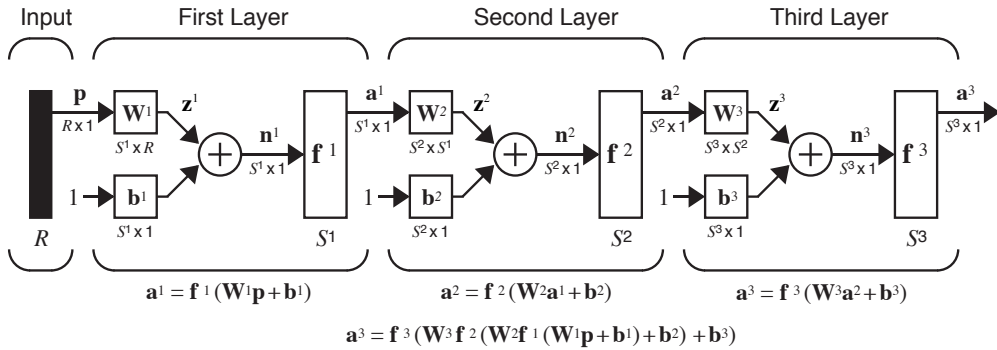
Layer of Neurons



Matrix Notation of Neuron Layer



Three Layer Network

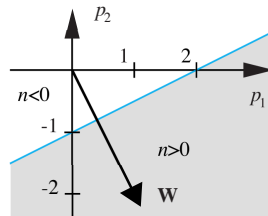
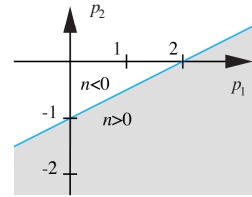


Multilayer Networks

Solved Problems

P2.1 Recall the network shown in Figure 2.8. Find the weights and bias needed for that network to create the decision boundary shown in the adjacent figure.

As described in Chapter 4 of [NND2](#), the weight should be orthogonal to the decision boundary. In this case, the weight $\mathbf{W} = \begin{bmatrix} 1 & -2 \end{bmatrix}$ would be orthogonal to the boundary, as shown in the figure below.

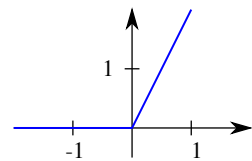


To find the bias, we can solve the equation $n = \mathbf{W}\mathbf{p} + b = 0$ or $b = -\mathbf{W}\mathbf{p}$ at a point on the boundary. If we pick the point $\mathbf{p} = \begin{bmatrix} 0 & -1 \end{bmatrix}^T$, we find that $b = -2$.

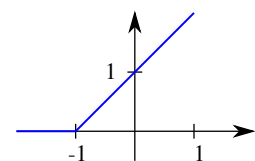
P2.2 Consider again the two-layer *poslin* network of Figure 2.10. Assume the weights and biases are $\mathbf{W}^1 = \begin{bmatrix} 2 & 1 \end{bmatrix}^T$, $\mathbf{b}^1 = \begin{bmatrix} 0 & 1 \end{bmatrix}^T$, $\mathbf{W}^2 = \begin{bmatrix} 1 & -1 \end{bmatrix}$, $\mathbf{b}^2 = [0]$.

- Plot the response of each neuron in the first layer as the input varies in the range $-1 < p < 1$.
- Plot the total network response a^2 as the input varies in the range $-1 < p < 1$.

i. For neuron 1 in the first layer, the neuron net input is $n_1^1 = 2p + 0$. This is a line with slope of 2 that passes through the origin. To get the neuron output, we pass this through the *poslin* activation function, which sets all negative values to zero. The resulting function is zero for $p < 0$, and is equal to $2p$ for $p \geq 0$. For neuron 2 in the first layer, the neuron net input is $n_1^2 = p + 1$. This is a line with slope 1 that is equal to zero when $p = -1$. If we pass this through



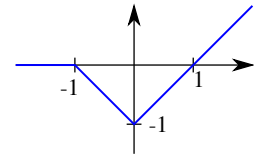
(a) Neuron 1



(b) Neuron 2

the *poslin* function, it will be zero for $p < -1$, and then it will have a slope of $+1$ starting at $p = -1$. (Note that the breakpoint in the *poslin* function is at $p = -b/w$.) The two functions are shown in the margin.

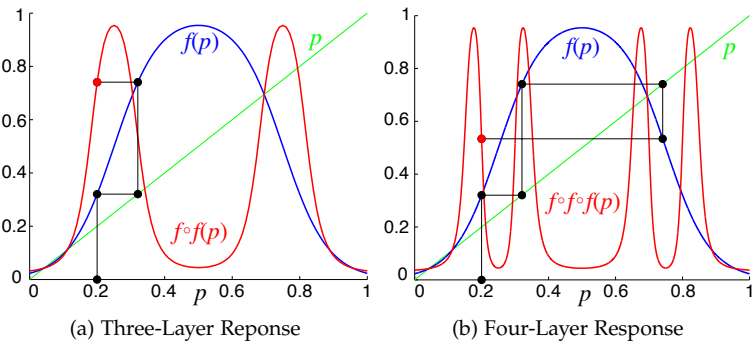
ii. The second layer weights are 1 and -1 , which means that the second neuron output is subtracted from the first neuron output. Since the bias in the second layer is zero, the output is not shifted up or down. Since the first neuron output is zero until zero, the total response will be the negative of the second neuron response until $p = 0$. For $p > 0$, the negative of the neuron two response will have a slope of -1 , and the first neuron response will have a slope of $+2$, so the total response will have a slope of $+1$. The total response is shown in the margin.



P2.3 The example illustrated in Figure 2.16 showed how adding layers of *poslin* neurons created functions of geometrically increasing complexity. How would these functions change if *sigmoid* activation functions replaced the *poslin* functions? What would happen to the total network response as the number of layers became very large? Explain any benefits of using the *poslin* function when the number of layers is large.

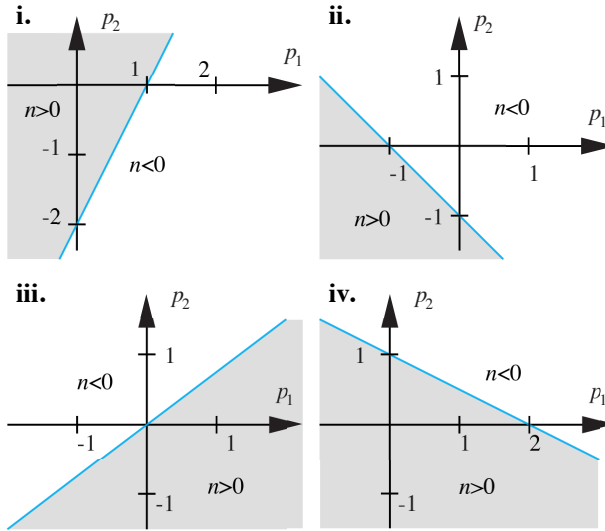
Using the demonstration (**dl2cf**), we can substitute the *logsig* activation function for the *poslin* activation function. The result is shown in the following figure for three-layer (left figure) and four-layer (right figure) networks. The key difference between this response and Figure 2.16 is that when a new *logig* layer is added, not only is the two-layer function repeated, but the pulses become narrower. This is caused by the shape of the *sigmoid* function. If we were to continue to add layers (assuming the same weights were used), we would get more pulses, but the pulses would become narrower. This means that there will be more regions where the slope of the function will be small. This is related to the vanishing gradient problem, which is discussed in the next chapter.

Multilayer Networks

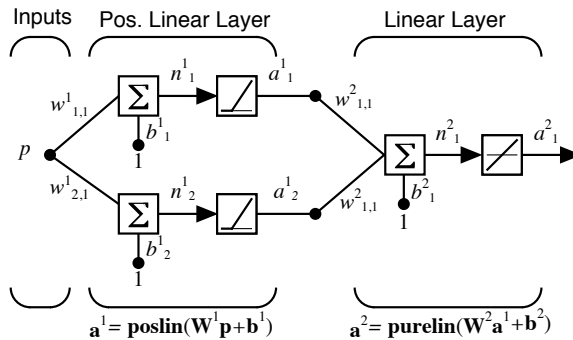


Exercises

E2.1 For the network shown in Figure 2.8, find the weights and bias needed to create the decision boundaries shown in the following figure.

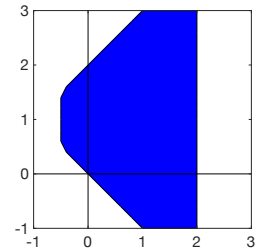
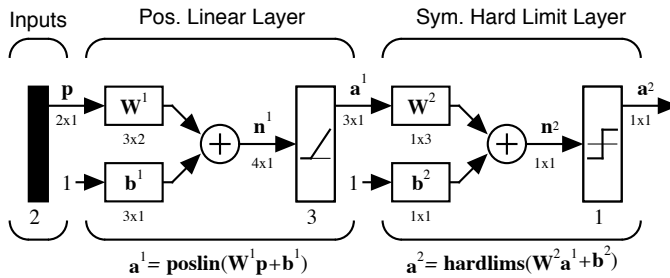


E2.2 Sketch the response of the following network when the weights and biases are $\mathbf{W}^1 = \begin{bmatrix} -1 & 1 \end{bmatrix}^T$, $\mathbf{b}^1 = \begin{bmatrix} 0.5 & 1 \end{bmatrix}^T$, $\mathbf{W}^2 = \begin{bmatrix} 1 & 1 \end{bmatrix}$, $\mathbf{b}^2 = \begin{bmatrix} -1 \end{bmatrix}$.



E2.3 For the following network, find the weights and biases to create the adjacent decision boundaries.

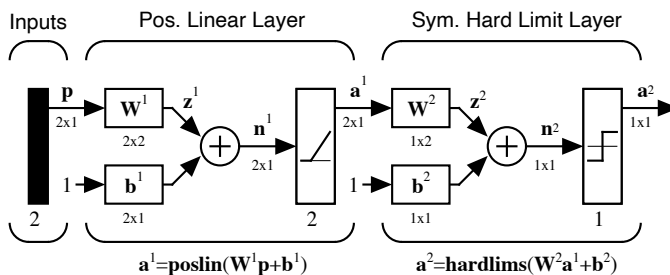
Multilayer Networks



E2.4 Consider the following training data (XOR).

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_1 = 1 \right\}, \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, t_2 = 1 \right\}, \left\{ \mathbf{p}_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, t_3 = -1 \right\}, \left\{ \mathbf{p}_4 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, t_4 = -1 \right\}$$

- You want to use the two-layer poslin network shown below, with two neurons in the hidden layer, to classify these input vectors correctly. Describe what kind of decision regions you can create with this network. Give some examples.
- Draw a decision region that this network can create, and which can correctly classify the training vectors.
- Find the weights and biases that would create that decision region.
- Test your network by applying the four input vectors and demonstrating that the outputs match the targets.



3 Multilayer Network Training

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Performance Index</i>	2
<i>Optimizing Performance - Gradient Descent</i>	4
<i>Variations on Gradient Descent – Adam</i>	8
<i>Computing the Gradient</i>	11
<i>Epilogue</i>	15
<i>Further Reading</i>	16
<i>Summary of Results</i>	17
<i>Solved Problems</i>	19
<i>Exercises</i>	23

Objective

This chapter focuses on the fundamentals of training multilayer networks, and, in particular, deep networks. The training involves choosing a measure of network performance and then determining the network parameters (weights and biases) that optimize that measure of performance. The determination of the optimal parameters is an iterative search process that uses the gradient of performance with respect to the network parameters to decide the direction of search.

Much of the material in this chapter is covered in more detail in Chapters 8-13 of [NND2](#). Here we review the basic concepts and discuss specific topics that are especially important for deep networks.

Performance Index

As discussed in Chapter 4 of [NND2](#), there are three types of network training: supervised, unsupervised and reinforcement. In this chapter we concentrate on supervised training, in which examples of network inputs and correct outputs (targets or labels) are provided. In particular, we consider performance learning, in which network weights and biases are adjusted to optimize some measurable quantity that represents network performance. The measure is usually called the *performance index*, or *loss function*.

All supervised learning begins with a data set of network inputs and target outputs:

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \dots, \{\mathbf{p}_Q, \mathbf{t}_Q\} \quad (3.1)$$

where Q represents the number of examples in the data set. Each input is applied to the network, and the network output is computed as follows:

$$\mathbf{a}_q^0 = \mathbf{p}_q \quad (3.2)$$

$$\mathbf{a}_q^{m+1} = \mathbf{f}^{m+1} \left(\mathbf{W}^{m+1} \mathbf{a}_q^m + \mathbf{b}^{m+1} \right) \text{ for } m = 0, 1, \dots, M-1 \quad (3.3)$$

$$\mathbf{a}_q = \mathbf{a}_q^M \quad (3.4)$$

Next, we define the performance index (or loss function). There are a number of potential performance functions, as discussed in Chapter 22 of [NND2](#). We will consider two popular ones – mean square error and cross-entropy. Mean square error is commonly used for function approximation (regression) applications, and cross-entropy is used for classification problems. Mean square error is defined as

$$F(\mathbf{x}) = \frac{1}{QS^M} \sum_{q=1}^Q \sum_{i=1}^{S^M} (t_{i,q} - a_{i,q})^2 \quad (3.5)$$

where $\mathbf{x}$ is the vector containing all of the weights and biases, S^M is the number of neurons in the output layer (Layer M), $a_{i,q}$ is the i^{th} neuron in the output layer when the q^{th} example input, $\mathbf{p}_q$, is presented, and $t_{i,q}$ is the corresponding element of the target.

Mean square error is a straight-forward description of network performance, and it is minimized when the network outputs exactly match the targets, which produces $F(\mathbf{x}) = 0$. Although mean square error is most-often used for regression problems, it can also be used for classification problems. However, a more commonly used performance function for pattern classification is cross-entropy. This performance function is not quite as intuitive as mean square error, so it is worth taking a few moments to provide some background.

To introduce cross-entropy, we need to view the classification problem from a probabilistic viewpoint. Assume that there exists a conditional probability distribution relating the network input $\mathbf{p}$ to the target output $\mathbf{t}$. In other words, once we know the input, this conditional distribution provides the probabilities associated with any potential target value. The objective of network training is to learn this unknown distribution. If we use the *softmax* activation function at the final layer of a multilayer network, we can interpret the network as an approximation of this conditional distribution – an input $\mathbf{p}_q$ is applied to the network, and the outputs of the last layer represent the probability that the input belongs to each of the target classes:

$$a_{i,q} = P \{ \mathbf{p}_q \text{ belongs to class } i \} \quad (3.6)$$

Now consider the entire training set of inputs and targets:

$$\mathbf{T} = \begin{bmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \cdots & \mathbf{t}_Q \end{bmatrix}, \mathbf{P} = \begin{bmatrix} \mathbf{p}_1 & \mathbf{p}_2 & \cdots & \mathbf{p}_Q \end{bmatrix} \quad (3.7)$$

If the neural network is an accurate estimate of the individual conditional probabilities, then the probability of the training set is:

$$P \{ \mathbf{T} \mid \mathbf{P} \} = \prod_{q=1}^Q \prod_{i=1}^{S^M} a_{i,q}^{t_{i,q}} \quad (3.8)$$

where we have assumed that the targets use *one hot encoding* (or one of K encoding), where one element of the target is 1 (corresponding to the correct class) and the other elements are 0. We also assume that each element of the training set is collected independently, which is why we can multiply the individual probabilities.

Multilayer Network Training

If the network is going to be trained to best fit the data, then $P\{\mathbf{T} \mid \mathbf{P}\}$ can be thought of as the likelihood function (see Chapter 13 of [NND2](#)). We can then adjust the weights and biases to maximize the likelihood. (By maximizing the likelihood, we make the data we collected the most likely data to have occurred.) This is equivalent to minimizing the negative log likelihood, which is the *cross-entropy* performance function:

$$F(\mathbf{x}) = - \sum_{q=1}^Q \sum_{i=1}^{S^M} t_{i,q} \ln a_{i,q} \quad (3.9)$$

Optimizing Performance - Gradient Descent

After the performance function (or loss function) has been selected, the next step is to determine the values of the weights and biases that will optimize performance. Before introducing optimization methods, it is appropriate to first analyze the performance function. (See Chapter 8 of [NND2](#) for a more detailed discussion of this material.)

A tool to analyze the performance function is the *Taylor series* expansion. The first three terms of the Taylor series provide a quadratic approximation to the function:

$$F(\mathbf{x}) \cong F(\mathbf{x}^*) + \nabla F(\mathbf{x})^T|_{\mathbf{x}^*} (\mathbf{x} - \mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T \nabla^2 F(\mathbf{x})|_{\mathbf{x}^*} (\mathbf{x} - \mathbf{x}^*) \quad (3.10)$$

where $\mathbf{x}^*$ is the point where the expansion takes place, $\nabla F(\mathbf{x})$ is the gradient

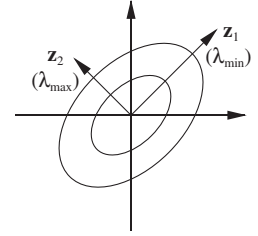
$$\nabla F(\mathbf{x}) = \left[\frac{\partial F(\mathbf{x})}{\partial x_1} \quad \frac{\partial F(\mathbf{x})}{\partial x_2} \quad \dots \quad \frac{\partial F(\mathbf{x})}{\partial x_n} \right]^T \quad (3.11)$$

and $\nabla^2 F(\mathbf{x})$ is the Hessian

$$\nabla^2 F(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 F(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 F(\mathbf{x})}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 F(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 F(\mathbf{x})}{\partial x_2^2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 F(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 F(\mathbf{x})}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_n^2} \end{bmatrix} \quad (3.12)$$

The gradient is a vector that points in the direction in which the performance function is increasing the most rapidly, and the Hessian matrix indicates the curvature of the function. A quadratic function, like the approximation in Eq. 3.10, has elliptical contours

(if the eigenvalues of the Hessian have the same sign), and the major axis of each ellipse is the eigenvector of the Hessian that is associated with the smallest eigenvalue. The eigenvalues of the Hessian represent the curvature of $F(\mathbf{x})$ along the eigenvector directions. This is illustrated in the figure in the margin, where the $\mathbf{z}_i$ are the eigenvectors and the λ_i are the eigenvalues.



The reason that the Taylor series expansion shown in Eq. 3.10 is important is because smooth functions can be closely approximated by these quadratic approximations, at least in small regions. We can use the quadratic approximation, which involves only the gradient and the Hessian, to assist in determining search directions to locate the minimum of $F(\mathbf{x})$.

The basic optimization algorithm takes the form

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \quad (3.13)$$

where $\mathbf{x}_k$ is the current guess for the optimal weights, $\mathbf{p}_k$ is the search direction and α_k is the step size, or learning rate. Optimization algorithms vary in their choice of search direction and learning rate. (See Chapter 9 of [NND2](#) for a more thorough discussion of optimization algorithms.)

The simplest optimization algorithm is *gradient descent* (GD), or steepest descent, in which the search direction is chosen to be the negative of the gradient direction, which is the direction in which $F(\mathbf{x})$ decreases most rapidly:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla F(\mathbf{x}_k) \quad (3.14)$$

To illustrate the performance of GD, consider the network in Figure 3.1.

$$\begin{array}{|c|} \hline 2 \\ \hline +2 \\ \hline 4 \\ \hline \end{array}$$

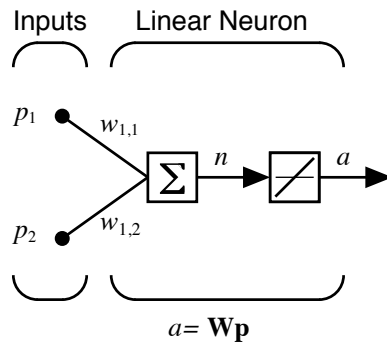


Figure 3.1: Example One Layer Network

Multilayer Network Training

To train this network, we use the following data set:

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \mathbf{t}_1 = [-1] \right\}, \left\{ \mathbf{p}_2 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \mathbf{t}_2 = [-1] \right\} \\ \left\{ \mathbf{p}_3 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \mathbf{t}_3 = [1] \right\}, \left\{ \mathbf{p}_4 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \mathbf{t}_4 = [1] \right\} \quad (3.15)$$

If we then define the following matrices:

$$\mathbf{U} = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \mathbf{p}_3^T \\ \mathbf{p}_4^T \end{bmatrix}, \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} w_{1,1} \\ w_{1,2} \end{bmatrix}, \quad (3.16)$$

we can write the sum square error performance function as

$$F(\mathbf{x}) = \sum_{q=1}^4 (t_q - a_q)^2 = (\mathbf{t} - \mathbf{U}\mathbf{x})^T (\mathbf{t} - \mathbf{U}\mathbf{x}) \quad (3.17)$$

$$= (\mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \mathbf{U}\mathbf{x} + \mathbf{x}^T \mathbf{U}^T \mathbf{U}\mathbf{x}) \quad (3.18)$$

This has the standard form of a quadratic function (see Chapter 8 of [NND2](#)):

$$F(\mathbf{x}) = c + \mathbf{d}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} \quad (3.19)$$

$$(3.20)$$

where

$$c = \mathbf{t}^T \mathbf{t}, \mathbf{d} = -2\mathbf{U}^T \mathbf{t}, \mathbf{A} = 2\mathbf{U}^T \mathbf{U} \quad (3.21)$$

The gradient and Hessian of the quadratic function are given by

$$\nabla F(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{d}, \nabla^2 F(\mathbf{x}) = \mathbf{A} \quad (3.22)$$

Plugging in the numbers for the inputs and targets, we find

$$\mathbf{U} = \begin{bmatrix} -1 & 2 \\ 2 & -1 \\ 0 & -1 \\ -1 & 0 \end{bmatrix}, \mathbf{t} = \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix}, \mathbf{A} = \begin{bmatrix} 12 & -8 \\ -8 & 12 \end{bmatrix}, \mathbf{d} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}, c = 4 \quad (3.23)$$

A plot of the function is given in Figure 3.2.

Now let's set up the GD algorithm with a learning rate $\alpha = 0.1$.

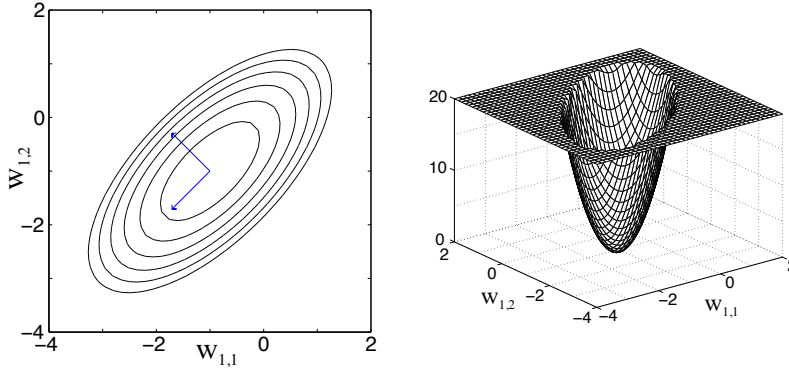


Figure 3.2: Performance Function

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla F(\mathbf{x}_k) = \mathbf{x}_k - 0.01 (\mathbf{A}\mathbf{x}_k + \mathbf{d}) = \begin{bmatrix} 0.88 & 0.08 \\ 0.08 & 0.88 \end{bmatrix} \mathbf{x}_k + \begin{bmatrix} 0.04 \\ 0.04 \end{bmatrix} \quad (3.24)$$

Figure 3.3 shows the trajectory of the GD algorithm with a very small learning rate. The right side of the figure shows a zoomed plot, with arrows indicating the GD directions. The path of GD is always orthogonal to the contour lines.

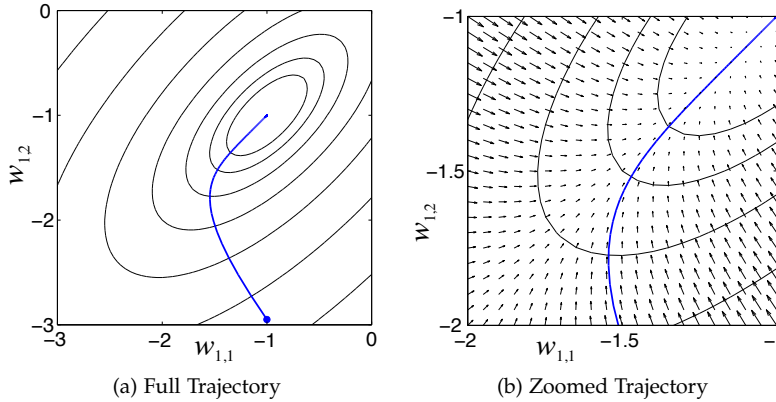


Figure 3.3: Gradient Descent Trajectory

When using GD, there is only one parameter to set: the learning rate α . If this is set too small, it may take a long time to converge. If it is set too large, the algorithm can become unstable (see Chapters 9 and 10 of [NND2](#)). A standard way around this problem is to start with a large value of α , and then reduce it gradually as the number of iterations increases.

Multilayer Network Training

To experiment with the gradient descent algorithm and learning rates, use the Deep Learning Demonstration Gradient Descent (**dl3gd**).



When training deep networks, the data sets are often very large, and it is not practical to compute the gradient of the performance function for the entire data set, which is called *batch* training. Instead, gradients are computed for subsets of the data, which are called *minibatches*. The extreme case of a minibatch occurs when the minibatch contains only a single example. This is called *stochastic gradient descent* (SGD).

For our test problem above, in which a linear network was used, the SGD algorithm would reduce to

$$\hat{F}(\mathbf{x}) = (t_k - a_k)^2 \quad (3.25)$$

$$\nabla \hat{F}(\mathbf{x}) = -2(t_k - a_k) \nabla a_k = -2e_k \mathbf{p}_k \quad (3.26)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + 2\alpha e_k \mathbf{p}_k \quad (3.27)$$

Figure 3.4 shows the path of the stochastic gradient algorithm for the test problem. The trajectory of SGD is a noisy version of the GD trajectory. More details on the convergence of SGD can be found in Chapter 10 of [NND2](#).

The choice of minibatch size depends on the problem, the size of the data set and the memory size of the GPU. It may take some experimentation to select the best minibatch size.

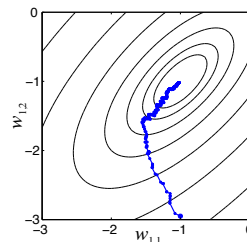


Figure 3.4: Stochastic Gradient Descent Trajectory

Variations on Gradient Descent – Adam

GD is one of the simplest optimization algorithms, and yet it appears to perform reasonably well on deep networks. However, there are variations of the algorithm that can converge faster. A key drawback of GD is that it only uses the slope of $F(\mathbf{x})$ to determine the search direction. It does not use curvature, which can often be quite helpful. If we consider again the example of the network in Figure 3.1 with the data of Eq. 3.15, the eigenvalues of $\nabla^2 F(\mathbf{x})$ (see Eq. 3.23) are 4 and 20. Because the magnitudes of these eigenvalues (which represent curvatures along the eigenvectors – shown as arrows in Figure 3.2) are so different, the contours, as seen in Figure 3.2, are very elliptical. This means that the initial search directions do not point in the direction of the minimum point, although they point in the steepest downhill direction.

There are a number of optimization algorithms that take curvature into account (see Chapters 9 and 12 of [NND2](#)). For example, Newton’s method locates the minimum of the quadratic approximation to $F(\mathbf{x})$, shown in Eq. 3.10. Setting the gradient of that approximation to zero yields the following update:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left[\nabla^2 F(\mathbf{x}_k) \right]^{-1} \nabla F(\mathbf{x}_k) \quad (3.28)$$

This method will converge in one iteration, if $F(\mathbf{x})$ is quadratic like our earlier example (see Chapter 9 of [NND2](#)). However, it requires the calculation of the Hessian matrix and its inverse, which is prohibitive for deep networks. The Levenberg-Marquardt algorithm (see Chapter 12 of [NND2](#)) combines an approximation of Newton’s method with GD, and is very fast for smaller networks. However, like Newton’s method, it requires the inverse of a large matrix and is not practical for deep networks.

Another class of optimization algorithm that takes curvature into account, but which does not require the Hessian calculation, is the conjugate gradient method (see Chapters 9 and 12 of [NND2](#)). Conjugate gradient methods use successive gradient calculations to obtain curvature information. They then find a more efficient search direction by modifying the GD direction to account for the curvature of $F(\mathbf{x})$. Conjugate gradient-type methods are well-suited to deep learning, because they account for curvature without the memory requirements and computational burden of algorithms like Newton’s method and Levenberg-Marquardt.

There are many optimization algorithms that are inspired by the conjugate gradient concepts, and others will certainly be introduced in the future. To illustrate the genre, we will describe one algorithm that has been popular for deep learning. It is called the [Adam](#) method [[Kingma and Ba, 2014](#)] (derived from *adaptive moment estimation*). Like conjugate gradient algorithms, it modifies the search direction from standard GD using curvature information without the Hessian calculations.

As with the GD algorithm, Adam starts with the gradient $\mathbf{g}_k = \nabla F(\mathbf{x}_k)$. In addition, it computes a smoothed gradient and a smoothed squared gradient:

$$\mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1 - \beta_1) \mathbf{g}_k \quad (3.29)$$

$$\mathbf{v}_{k+1} = \beta_2 \mathbf{v}_k + (1 - \beta_2) \mathbf{g}_k \circ \mathbf{g}_k \quad (3.30)$$

Multilayer Network Training

where $\circ$ is the Hadamard (element-by-element) product. These vectors are initialized with zeros: $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{v}_0 = \mathbf{0}$. The smoothed squared gradient $\mathbf{v}_{k+1}$ measures the second moment of the gradient, which is helpful in getting curvature information.

The smoothed values $\mathbf{m}_k$ and $\mathbf{v}_k$ are biased toward zero, because of their initialization. To correct this, a second stage is performed:

$$\hat{\mathbf{m}}_{k+1} = \mathbf{m}_{k+1} / (1 - \beta_1^{k+1}) \quad (3.31)$$

$$\hat{\mathbf{v}}_{k+1} = \mathbf{v}_{k+1} / (1 - \beta_2^{k+1}) \quad (3.32)$$

where $0 < \beta_1, \beta_2 < 1$ are constant exponential decay rates. The adjusted smoothed gradient $\hat{\mathbf{m}}_{k+1}$ is the search direction, and the adjusted smoothed squared gradient $\hat{\mathbf{v}}_{k+1}$ is used to adjust the learning rate:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \hat{\mathbf{m}}_{k+1} \oslash (\sqrt{\hat{\mathbf{v}}_{k+1}} + \epsilon) \quad (3.33)$$

where $\oslash$ is the Hadamard (element-by-element) division and ϵ is a small positive constant to prevent division by zero.

The algorithm can be made more numerically efficient by replacing Eq. 3.31-3.33 with the following:

$$\alpha_{k+1} = \alpha (1 - \beta_1^{k+1}) / \sqrt{1 - \beta_2^{k+1}} \quad (3.34)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_{k+1} \mathbf{m}_{k+1} \oslash (\sqrt{\mathbf{v}_{k+1}} + \epsilon) \quad (3.35)$$

In [Kingma and Ba, 2014], they suggest that appropriate default settings for the algorithm parameters are: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$.

This algorithm has elements of conjugate gradient algorithms as well as momentum algorithms, in which the search direction is a smoothed version of the gradient (see Chapter 12 of NND2). When the gradient is always of the same sign, the smoothed gradient stays large, and the weight adjustment is large. When the gradient is changing sign (e.g., when oscillating across a narrow valley), the weight adjustment gets smaller ($\mathbf{m}_k$ gets smaller because negative and positive values are being averaged, while $\mathbf{v}_k$ stays large because squared values are always positive). This allows us to have a relatively large learning rate, while maintaining stability.

There are other gradient-based algorithms that have been proposed, and there will undoubtedly be more in the future. Our purpose here is to emphasize that the gradient is an important tool in network training, and that variations of GD can offer faster convergence. The next step is to demonstrate how the gradient can be efficiently computed.

To experiment with the Adam algorithm, use the Deep Learning Demonstration Adam Algorithm (**d13aa**).



Computing the Gradient

The key to most optimization algorithms that are used in training deep networks is the gradient of the performance with respect to the weights and biases in the network. It is important to be able to compute the gradient efficiently for any architecture of network. In this section we will focus on the gradient calculation for multilayer networks with an arbitrary number of layers. This was covered in Chapter 11 of [NND2](#), but we will describe it here in a way that can extend to networks with more general connections, as will be discussed in a later chapter.

For one-layer networks, like the one in Figure 3.1, it is easy to compute the derivative of the performance with respect to the weights, because the network output is a direct function of the weights. However, in multilayer networks, like Figure 2.7, the network output is only indirectly a function of the weights in the first layer. Therefore, to efficiently compute the full gradient, we need to use the chain rule of calculus.

Assume that we want to take the derivative of the performance function with respect to a weight in layer m . We would use the chain rule in the following way:

$$\frac{\partial F(\mathbf{x})}{\partial w_{i,j}^m} = \frac{\partial F(\mathbf{x})}{\partial n_i^m} \frac{\partial n_i^m}{\partial w_{i,j}^m} = \frac{\partial F(\mathbf{x})}{\partial n_i^m} \frac{\partial (\sum_{l=1}^{S^{m-1}} w_{i,l}^m a_l^{m-1} + b_i^m)}{\partial w_{i,j}^m} \quad (3.36)$$

$$= \frac{\partial F(\mathbf{x})}{\partial n_i^m} a_j^{m-1} \quad (3.37)$$

The key to using the chain rule is the choice of intermediate variable. In this case, we chose n_i^m . The reason is that n_i^m is a linear function of $w_{i,j}^m$, so the derivative of n_i^m with respect to $w_{i,j}^m$ is simply a_j^{m-1} .

Multilayer Network Training

Eq. 3.37 can be used to compute the elements of the gradient, but we need to find the *sensitivity*

$$\mathbf{s}^m \triangleq \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m} \quad (3.38)$$

With this sensitivity vector, we can compute the elements of the gradient associated with weight matrix $\mathbf{W}^m$ using

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{W}^m} = \mathbf{s}^m \left(\mathbf{a}^{m-1} \right)^T \quad (3.39)$$

The bias gradient elements are simpler, since the bias computation is just an addition:

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{b}^m} = \mathbf{s}^m \quad (3.40)$$

It remains to compute the sensitivities $\mathbf{s}^m$. They can be calculated using a series of applications of the chain rule, starting at the last layer, and moving backward through the layers:

$$\mathbf{s}^M \rightarrow \mathbf{s}^{M-1} \rightarrow \dots \rightarrow \mathbf{s}^1 \quad (3.41)$$

This process is called *backpropagation*. Backpropagation can be performed in stages, as we move across the different components of each layer: weight function, net input function and activation function:

$$\mathbf{z}^{m+1} = \mathbf{W}^{m+1} \mathbf{a}^m \quad (3.42)$$

$$\mathbf{n}^{m+1} = \mathbf{z}^{m+1} + \mathbf{b}^{m+1} \quad (3.43)$$

$$\mathbf{a}^{m+1} = \mathbf{f}^{m+1} \left(\mathbf{n}^{m+1} \right) \quad (3.44)$$

To propagate the sensitivity from layer $m + 1$ to layer m we would use the chain rule as follows:

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m} = \frac{\partial (\mathbf{a}^m)^T}{\partial \mathbf{n}^m} \frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m} \frac{\partial (\mathbf{n}^{m+1})^T}{\partial \mathbf{z}^{m+1}} \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^{m+1}} \quad (3.45)$$

Let's consider each derivative individually. First, for the transfer function we have

$$\frac{\partial (\mathbf{a}^m)^T}{\partial \mathbf{n}^m} = \dot{\mathbf{F}}^m(\mathbf{n}^m) = \begin{bmatrix} \frac{\partial f_1^m(\mathbf{n}^m)}{\partial n_1^m} & \frac{\partial f_2^m(\mathbf{n}^m)}{\partial n_1^m} & \dots & \frac{\partial f_{S^m}^m(\mathbf{n}^m)}{\partial n_1^m} \\ \frac{\partial f_1^m(\mathbf{n}^m)}{\partial n_2^m} & \frac{\partial f_2^m(\mathbf{n}^m)}{\partial n_2^m} & \dots & \frac{\partial f_{S^m}^m(\mathbf{n}^m)}{\partial n_2^m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_1^m(\mathbf{n}^m)}{\partial n_{S^m}^m} & \frac{\partial f_2^m(\mathbf{n}^m)}{\partial n_{S^m}^m} & \dots & \frac{\partial f_{S^m}^m(\mathbf{n}^m)}{\partial n_{S^m}^m} \end{bmatrix}, \quad (3.46)$$

for the weight function we have

$$\frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m} = \frac{\partial (\mathbf{W}^{m+1} \mathbf{a}^m)^T}{\partial \mathbf{a}^m} = (\mathbf{W}^{m+1})^T, \quad (3.47)$$

for the net input function we have

$$\frac{\partial (\mathbf{n}^{m+1})^T}{\partial \mathbf{z}^{m+1}} = \frac{\partial (\mathbf{z}^{m+1} + \mathbf{b}^{m+1})^T}{\partial \mathbf{z}^{m+1}} = \mathbf{I} \quad (3.48)$$

Putting these steps together, we can write Eq. 3.45 as

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m) (\mathbf{W}^{m+1})^T \mathbf{s}^{m+1} \quad (3.49)$$

The process is illustrated in Figure 3.5 for a three layer network. The expressions above the arrows represent the derivatives across the corresponding components of the layer. The backpropagation process involves multiplying by these derivatives. We will see in a later chapter that this can work for feedforward networks with more complex connections between layers. When you propagate the sensitivities backward across a component of the network, you multiply by the derivative of the component.

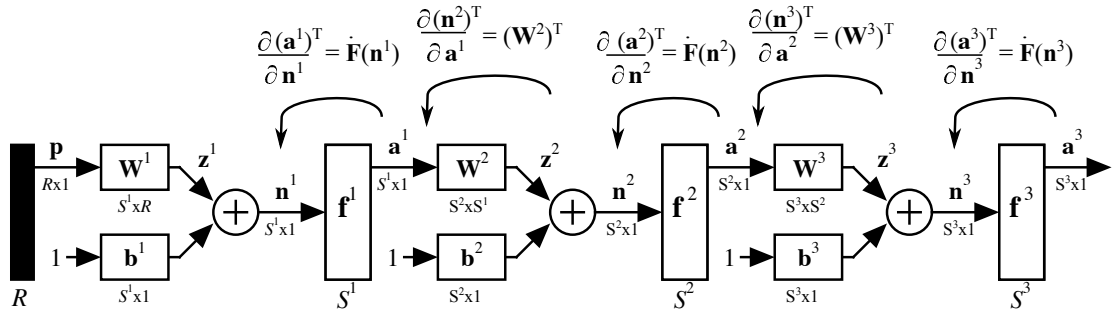


Figure 3.5: Backpropagation Process

We need to begin the backpropagation recursion of Eq. 3.49 at the final layer of the network – Layer M :

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^M} = \frac{\partial (\mathbf{a}^M)^T}{\partial \mathbf{n}^M} \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^M} = \dot{\mathbf{F}}^M(\mathbf{n}^M) \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^M} \quad (3.50)$$

For example, if $F(\mathbf{x})$ is mean square error, we have

$$\frac{\partial F(\mathbf{x})}{\partial a_i^M} = \frac{\partial \frac{1}{S^M} \sum_{j=1}^{S^M} (t_j - a_j^M)^2}{\partial a_i^M} = \frac{-2}{S^M} (t_i - a_i^M) = \frac{-2}{S^M} e_i \quad (3.51)$$

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^M} = \frac{-2}{S^M} \mathbf{e} \quad (3.52)$$

Multilayer Network Training

where we have considered the stochastic gradient, with one example. We can now write Eq. 3.50 as

$$\mathbf{s}^M = \frac{-2}{S^M} \dot{\mathbf{F}}^M(\mathbf{n}^M) \mathbf{e} \quad (3.53)$$

The total backpropagation process begins with Eq. 3.53, followed by Eq. 3.49, which is iterated from $m = M - 1$ to $m = 1$. (Eq. 3.53 applies to mean square error performance, but this process can be adjusted for any performance function by substituting the derivative of the chosen performance function into Eq. 3.50.) Once the sensitivities are computed, the parts of the gradient are then computed from Eqs. 3.39 (for the weights) and 3.40 (for the biases).

From this process, the gradient of performance for a single example is obtained (stochastic gradient). Assuming that the total performance function is produced by summing the performance functions for each example (as is the case for mean square error and cross-entropy), the total gradient is obtained by summing the gradients of each example.

Epilogue

The training of deep networks is an optimization problem. The objective is to determine the weights and biases of the network that optimize network performance. We discussed two popular performance functions: mean square error and cross-entropy. Mean square error is used for function fitting (regression), and cross-entropy is used for pattern recognition.

Once the performance function has been chosen, an iterative search is performed to find the weights and biases that optimize performance. The simplest search method is gradient descent, which moves in the direction where the performance function decreases most rapidly. This method uses only the slope of the performance surface, but convergence can be improved by using curvature information. A popular deep learning algorithm that uses curvature information is the Adam method, which was also described in this chapter.

Training algorithms can be run in batch mode, where performance and gradient are computed on the entire data set. They can also be run in stochastic mode, where performance and gradient are computed on each example in the data set separately and weights are updated after each example. There is also an intermediate mode, in which mini-batches of data are used. The choice of the minibatch size is often determined by the size of available GPU memory.

Almost all deep learning algorithms use the gradient in order to determine search directions. This chapter demonstrated how the gradient can be computed for deep multilayer networks. The gradient is computed in stages, moving backward across each stage of the network. The backward operation for each stage involves multiplying by the derivative of the forward operation of that stage. In a later chapter, we will expand the backpropagation algorithm for networks with more general connections between layers.

When training deep networks on practical problems, the objective is not necessarily to optimize performance on the training set, which was our focus in this chapter. Rather, we want to train networks to perform well on unknown, future data. Networks with this ability are said to generalize well (see Chapter 13 of [NND2](#)). In the next chapter we will discuss auxiliary procedures that are used in combination with basic training algorithms to improve generalization and otherwise improve training.

Multilayer Network Training

Further Reading

[Kingma and Ba, 2014] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014

This is the original description of the Adam algorithm. It also describes AdaMax, which is a variation of Adam that uses the infinity norm.

[Duchi et al., 2011] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011

The Adam algorithm is based, in part, on the AdaGrad algorithm, which is described in this paper. The search direction is modified from the gradient direction using second order information – previous sum of squared gradients.

[Zeiler, 2012] Matthew D Zeiler. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012

The Adadelta algorithm described in this paper is a modification of the AdaGrad algorithm. It limits the sum of squared previous gradients to a window of fixed size, which prevents the effective learning rate from going to zero.

[Liu and Nocedal, 1989] Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1-3):503–528, 1989

This limited memory BFGS algorithm is a type of quasi-Newton method. The standard BFGS algorithm requires storing an approximate inverse Hessian matrix, which can be impractically large for deep networks. The L-BFGS algorithm stores a few vectors that represent the inverse Hessian.

Summary of Results

Training Set

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \dots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

Multilayer Network Forward Operation

$$\begin{aligned} \mathbf{a}_q^0 &= \mathbf{p}_q \\ \mathbf{a}_q^{m+1} &= \mathbf{f}^{m+1} \left(\mathbf{W}^{m+1} \mathbf{a}_q^m + \mathbf{b}^{m+1} \right) \text{ for } m = 0, 1, \dots, M-1 \\ \mathbf{a}_q &= \mathbf{a}_q^M \end{aligned}$$

Mean Square Error Performance Function

$$F(\mathbf{x}) = \frac{1}{QS^M} \sum_{q=1}^Q \sum_{i=1}^{S^M} (t_{i,q} - a_{i,q})^2$$

Cross-Entropy Performance Function

$$F(\mathbf{x}) = - \sum_{q=1}^Q \sum_{i=1}^{S^M} t_{i,q} \ln a_{i,q}$$

Taylor Series Expansion

$$F(\mathbf{x}) \cong F(\mathbf{x}^*) + \nabla F(\mathbf{x})^T|_{\mathbf{x}^*} (\mathbf{x} - \mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T \nabla^2 F(\mathbf{x})|_{\mathbf{x}^*} (\mathbf{x} - \mathbf{x}^*)$$

Gradient

$$\nabla F(\mathbf{x}) = \left[\frac{\partial F(\mathbf{x})}{\partial x_1} \quad \frac{\partial F(\mathbf{x})}{\partial x_2} \quad \dots \quad \frac{\partial F(\mathbf{x})}{\partial x_n} \right]^T$$

Hessian

$$\nabla^2 F(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 F(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 F(\mathbf{x})}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 F(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 F(\mathbf{x})}{\partial x_2^2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 F(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 F(\mathbf{x})}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 F(\mathbf{x})}{\partial x_n^2} \end{bmatrix}$$

Basic Optimization Algorithm

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

Multilayer Network Training

Gradient Descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla F(\mathbf{x}_k)$$

Adam Algorithm

$$\mathbf{g}_k = \nabla F(\mathbf{x}_k)$$

$$\mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1 - \beta_1) \mathbf{g}_k$$

$$\mathbf{v}_{k+1} = \beta_2 \mathbf{v}_k + (1 - \beta_2) \mathbf{g}_k \circ \mathbf{g}_k$$

$$\alpha_{k+1} = \alpha (1 - \beta_1^{k+1}) / \sqrt{1 - \beta_2^{k+1}}$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_{k+1} \mathbf{m}_{k+1} \oslash (\sqrt{\mathbf{v}_{k+1}} + \epsilon)$$

$$\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

Sensitivity

$$\mathbf{s}^m \triangleq \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m}$$

Initialization of Sensitivity

$$\mathbf{s}^M = \dot{\mathbf{F}}^M(\mathbf{n}^M) \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^M}$$

Backpropagation of Sensitivity

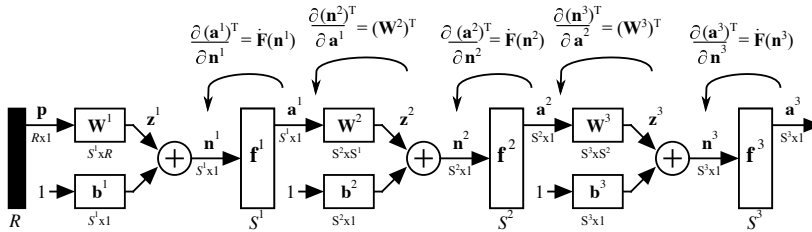
$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m) (\mathbf{W}^{m+1})^T \mathbf{s}^{m+1}, m = M-1, \dots, 1$$

Weight Update

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{W}^m} = \mathbf{s}^m (\mathbf{a}^{m-1})^T$$

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{b}^m} = \mathbf{s}^m$$

Backpropagation Process



Solved Problems

P3.1 The cross-entropy performance function is generally used when the last layer activation function is *softmax*, and the i^{th} output neuron, a_i , is the probability that the input belongs to class i . However, cross-entropy can also be used for a network with a *logsig* activation function, if there is only one neuron in the last layer. Show how this could be done.

Using a network with a single *logsig* output neuron, we can recognize two classes. One class is represented by a target of 1 and the other by a target of 0. The output neuron a represents the probability that the input is in class 1. This means that the probability that the input is in class 0 is $(1 - a)$.

The cross-entropy performance function when there is a *softmax* activation in the last layer is given by Eq. 3.9:

$$F(\mathbf{x}) = - \sum_{q=1}^Q \sum_{i=1}^{S^M} t_{i,q} \ln a_{i,q}$$

If we had two *softmax* neurons in the last layer of the network ($S^M = 2$), then we could use this equation. The first output neuron, a_1 , represents the probability of class 1, and the second output neuron, a_2 , represents the probability of class 0. In a network with one *logsig* neuron, a represents the probability of class 1, and $(1 - a)$ represents the probability of class 0. The cross-entropy equation then becomes

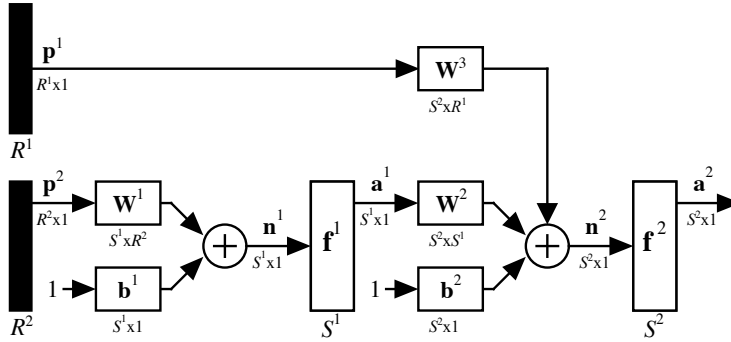
$$F(\mathbf{x}) = - \sum_{q=1}^Q [t_q \ln a_q + (1 - t_q) \ln (1 - a_q)]$$

P3.2 The backpropagation method can be applied to networks that are not strictly multilayer networks, where there is one input and each layer connects only to the following layer, which produces the backpropagation recursion of Eq. 3.49. How would backpropagation change for the network in the figure below, and how would the portion of the gradient for the weight W^3 be computed?

The backpropagation process computes the sensitivities

$$\mathbf{s}^m \triangleq \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m}$$

Multilayer Network Training



For this network, the sensitivities are s^2 and s^1 . They would be computed using Eq. 3.50 and Eq. 3.49, as with a standard multilayer network. The additional input connection does not affect the calculation of the sensitivities. The derivative of n^2 with respect to n^1 depends only on W^2 and f^1 . Other terms that enter the summation junction are just added, and do not change the effect of n^1 on n^2 .

For the elements of the gradient associated with the weight W^3 , we first notice that

$$n^2 = W^3 p^1 + W^2 a^1 + b^2$$

For element i this would produce

$$n_i^2 = \sum_{j=1}^{R^1} w_{i,j}^3 p_j^1 + \sum_{l=1}^{S^1} w_{i,l}^2 a_l^1 + b_i^2$$

The elements of the gradient associated with W^3 would then be computed as

$$\begin{aligned} \frac{\partial F(\mathbf{x})}{\partial w_{i,j}^3} &= \frac{\partial F(\mathbf{x})}{\partial n_i^2} \frac{\partial n_i^2}{\partial w_{i,j}^3} = s_i^2 \frac{\partial (\sum_{j=1}^{R^1} w_{i,j}^3 p_j^1 + \sum_{l=1}^{S^1} w_{i,l}^2 a_l^1 + b_i^2)}{\partial w_{i,j}^3} \\ &= s_i^2 p_j^1 \end{aligned}$$

P3.3 Consider again the network in Figure 3.1, and the data set below.

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -4 \\ 2 \end{bmatrix}, \mathbf{t}_1 = [-1] \right\}, \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{t}_2 = [1] \right\}$$

- Write out an expression for the sum squared error.
- Find the gradient and Hessian of the sum squared error.

- iii. Find the eigenvalues and eigenvectors of the Hessian. The eigenvector with smallest eigenvalue will be the major axis of the elliptical contours of the contour plot.
- iv. Sketch the contour plot, and sketch the path of the GD algorithm if a very small learning rate is used. Start at the initial weight $\mathbf{W} = \begin{bmatrix} 1 & 1 \end{bmatrix}$.
- v. The maximum stable learning rate is 2 divided by the maximum eigenvalue of the Hessian. Find this value. What would the GD path look like for this learning rate?

i. If we define

$$\mathbf{U} = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} = \begin{bmatrix} -4 & 2 \\ 1 & 2 \end{bmatrix}, \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} w_{1,1} \\ w_{1,2} \end{bmatrix},$$

We can write the sum square error as

$$\begin{aligned} F(\mathbf{x}) &= \sum_{q=1}^2 (t_q - a_q)^2 = (\mathbf{t} - \mathbf{U}\mathbf{x})^T (\mathbf{t} - \mathbf{U}\mathbf{x}) \\ &= \left(\mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \mathbf{U}\mathbf{x} + \mathbf{x}^T \mathbf{U}^T \mathbf{U}\mathbf{x} \right) = c + \mathbf{d}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} \end{aligned}$$

Plugging in the numbers, we find

$$c = \mathbf{t}^T \mathbf{t} = 2, \mathbf{d} = -2\mathbf{U}^T \mathbf{t} = \begin{bmatrix} -10 \\ 0 \end{bmatrix}, \mathbf{A} = 2\mathbf{U}^T \mathbf{U} = \begin{bmatrix} 34 & -12 \\ -12 & 16 \end{bmatrix},$$

ii. The Hessian is $\mathbf{A}$ and the gradient is $\mathbf{A}\mathbf{x} + \mathbf{d}$. iii. The eigenvalues

of the Hessian can be found by

$$|\mathbf{A} - \lambda \mathbf{I}| = 0 = \left| \begin{bmatrix} 34 - \lambda & -12 \\ -12 - 12 & 16 - \lambda \end{bmatrix} \right| = \lambda^2 - 50\lambda + 400 = (\lambda - 10)(\lambda - 40)$$

So the eigenvalues are 10 and 40. The corresponding eigenvectors can be found using

$$[\mathbf{A} - \lambda \mathbf{I}] \mathbf{v} = \mathbf{0}$$

For $\lambda_1 = 10$ we have

$$\begin{bmatrix} 24 & -12 \\ -12 & 6 \end{bmatrix} \mathbf{v}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{v}_1 = \begin{bmatrix} -1 \\ -2 \end{bmatrix}$$

Multilayer Network Training

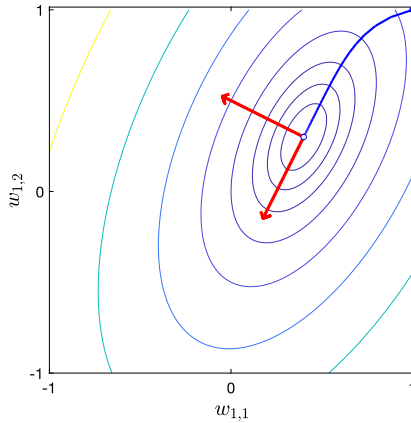
For $\lambda_2 = 40$ we have

$$\begin{bmatrix} -6 & -12 \\ -12 & -24 \end{bmatrix} \mathbf{v}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

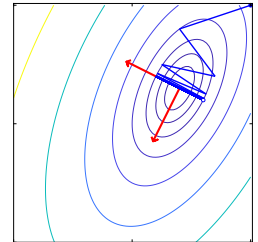
iv. The contours of $F(\mathbf{x})$ will be elliptical, and the major axis of each ellipse will be $\mathbf{v}_1$, since this is the eigenvector with the smallest eigenvalue (curvature). The center of the contours will be at the point where the gradient is equal to zero.

$$\mathbf{A}\mathbf{x} + \mathbf{d} = \mathbf{0} \Rightarrow \mathbf{x} = -\mathbf{A}^{-1}\mathbf{d} = \begin{bmatrix} 34 & -12 \\ -12 & 16 \end{bmatrix}^{-1} \begin{bmatrix} -10 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix}$$

The figure below shows the eigenvectors and the contours. The GD path will move down hill and orthogonal to the contour lines.

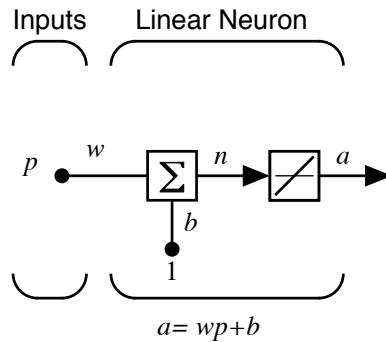


v. Since the largest eigenvalue of the Hessian is 40, the maximum stable learning rate will be $\alpha = 2/40 = 0.05$. With this learning rate, the GD trajectory will oscillate back and forth along the eigenvector that corresponds to the maximum eigenvalue, which would be $\mathbf{v}_2$. The figure in the margin shows what the trajectory would look like.



Exercises

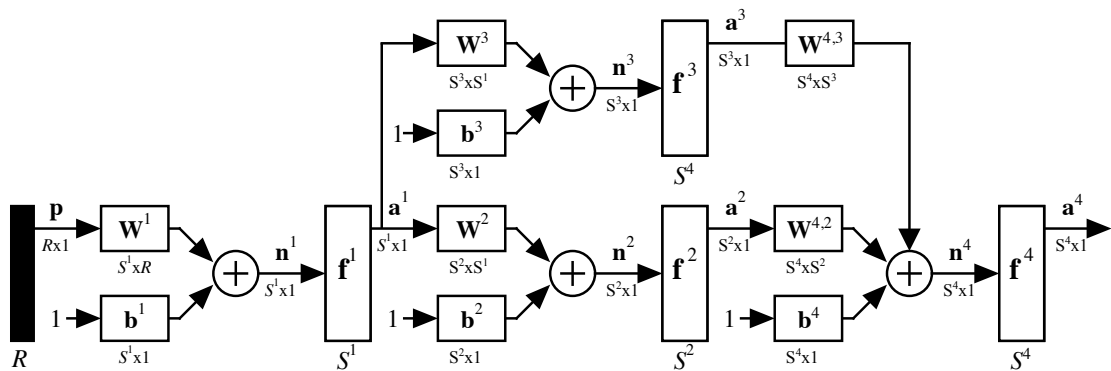
- E3.1** Consider the training data $\{p_1 = [-1], t_1 = [-1.5]\}$, $\{p_2 = [0], t_2 = [0.5]\}$, $\{p_3 = [1], t_3 = [2.5]\}$ and the network below.



- i. Write out an expression for the sum squared error.
 - ii. Starting with initial weight and bias equal to zero, calculate the initial gradient of the sum squared error.
 - iii. Take one step of steepest descent with learning rate of 0.1.
 - iv. Does the error go down?
- E3.2** Use the network and data from Exercise E3.1.
- i. Find the gradient and Hessian of the sum squared error.
 - ii. Starting with initial weight and bias equal to zero, take a step of Newton's method – Eq. 3.28
 - iii. After a step of Newton's method, has the algorithm reached the minimum sum squared error? (Check that the gradient is zero.) Explain how Newton's method has used curvature to make a better first step than steepest descent.
- E3.3** Consider again the network from Exercise E2.3. Use the weights and biases that you found there. Replace the hard limit transfer function with a linear transfer function. For input $\mathbf{p} = \begin{bmatrix} 0 & -1 \end{bmatrix}^T$ and target $\mathbf{t} = [1]$, compute the initial gradient using backpropagation, and perform one iteration of steepest descent with a learning rate of 0.1.

Multilayer Network Training

- E3.4** The network shown in the following figure is not a standard multilayer network, since the output of one layer does not feed into only one succeeding layer. However, it is a purely feedforward network, and the standard backpropagation techniques can be applied to compute the training gradients. (We will say more about computing gradients for arbitrarily connected layers in a future chapter.) Write out the equations needed to compute the sensitivities s^4, s^3, s^2 and s^1 for this network, using the concepts that were used to derive Eq. 3.49.



- E3.5** The initialization of the backpropagation algorithm, Eq. 3.50, depends on the performance function and the last layer activation function. Find the specific initialization step, if the performance function is cross-entropy, and the last layer activation function is *softmax*.
- E3.6** If the cross-entropy performance function is used, show the explicit form for the initialization of the backpropagation process (Eq. 3.50).

4 *Supplemental Training Procedures*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Speeding Up Convergence</i>	2
<i>Generalization</i>	14
<i>Epilogue</i>	17
<i>Further Reading</i>	18
<i>Summary of Results</i>	19
<i>Solved Problems</i>	22
<i>Exercises</i>	26

Objective

The previous chapter focused on deep multilayer network training as an optimization problem, in which the weights and biases of the network are chosen to optimize the performance function. This chapter looks at procedures that can be used to supplement the basic optimization algorithms. These supplementary procedures have two purposes. The first purpose is to select a good starting point for the optimization process, which will speed up the convergence of the optimization algorithm. The second purpose is to ensure that the final trained network will perform as well on new data as it did on the training data.

Theory and Examples

This chapter discusses techniques that can be used to supplement the training algorithms that were discussed in the last chapter. These techniques can: 1) speed up the convergence of the training algorithms, and 2) produce networks that will generalize well to new situations.

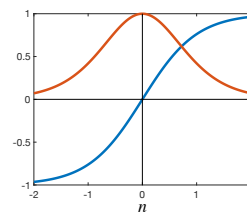
Speeding Up Convergence

As we mentioned in previous chapters, much of the basic theory for deep networks was developed many years ago. For example, the backpropagation algorithm for computing the gradient of network performance with respect to the network weights and biases was first popularized in the mid 1980s. However, the extensive use of neural networks with many layers did not occur until training times became manageable. Two developments, in particular, improved training times: the introduction of the cuda language for general purpose programming of GPUs, and the discovery of methods to overcome the vanishing gradient problem.

The *vanishing gradient* problem occurs when a sigmoid (S-shaped) activation function is used in a network with a large number of layers (or in recurrent networks). The figure in the margin shows the hyperbolic tangent sigmoid function (blue) and its derivative (red). As the net input n becomes large in magnitude, the derivative decreases quickly in size. (The net input will become large if the input to the layer, or the layer weights, become large.) When the sigmoid saturates, the derivative will be very small. Since we multiply by the derivative in the backpropagation process to compute the gradient, the gradient will also become small.

The sigmoid *squashes* the input into the range $[-1, 1]$ and must become flat for large net inputs. When multiple sigmoid layers are cascaded, these multiple squashing operations cause more flat areas in the overall network response. When the gradient is computed by backpropagating through the layers, its magnitude will generally become smaller and smaller, slowing the training convergence.

Some of the material in this chapter is covered in Chapters 13 and 22 of [NND2](#). Here we review the basic concepts and discuss specific topics that are especially important for deep networks.



The flattening operation produced by cascading sigmoid layers is illustrated in Solved Problem P2.3, which demonstrates what happens as the number of sigmoid layers is increased. The process is also shown in Figure 4.1. With a two layer network (with a sigmoid hidden layer), the network response gradually decays to zero. As the number of layers increases, the network response becomes almost flat (derivative almost zero) over a larger portion of the relevant function domain.

To experiment with increasing the number of layers in a network, use the Deep Learning Demonstration Cascaded Function (**dl2cf**).

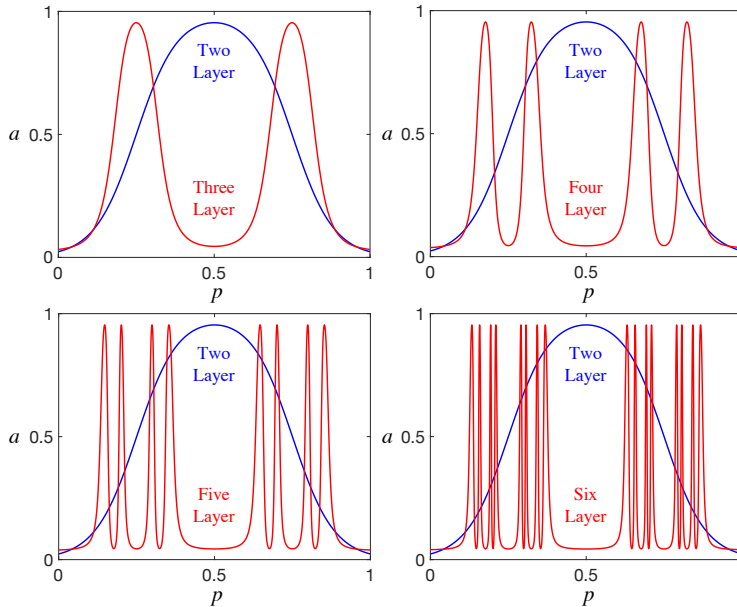


Figure 4.1: Effect of Cascading Sigmoid Layers

Much of the work to make deep network training practical has focused on mitigating the vanishing gradient problem. One very helpful innovation was the use of the *ReLU* (or *poslin*) activation function, which was discussed in Chapter 2. Figure 4.2 demonstrates the operation of the same network used for Figure 4.1, but with the *tansig* layers replaced with *poslin* layers. We no longer have the large areas with diminished slope.

Supplemental Training Procedures

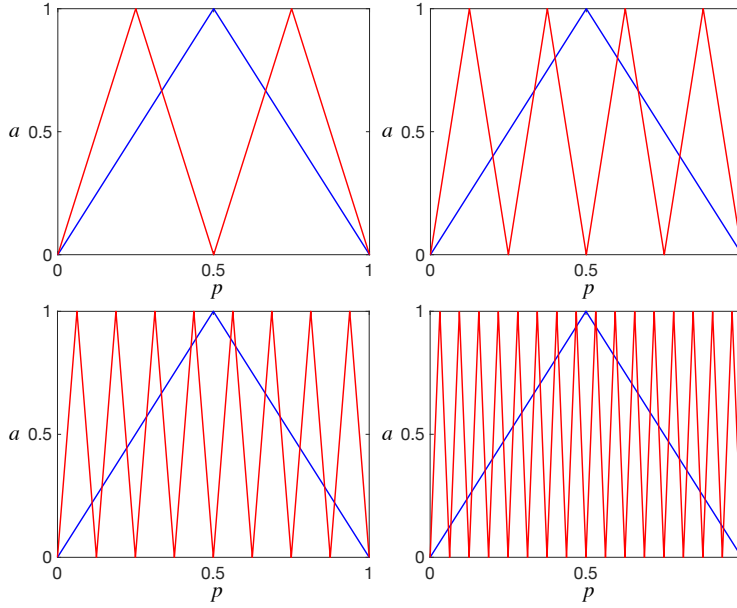


Figure 4.2: Effect of Cascading ReLU (poslin) Layers

In addition to the introduction of the *ReLU* activation function, much additional research has focused on overcoming the vanishing gradient problem. This additional work emphasized two interrelated approaches: normalization and weight initialization.

NORMALIZATION involves pre-processing the input to a layer so that it has a standard distribution. If the inputs to a layer are always of the same size, it is easier to adjust the initial weights so that the net input to the activation function will be in a reasonable range.

Normalization was first introduced for the network inputs. As described in Chapter 22 of [NND2](#), there are two common types of normalization. The first type normalizes the input so that all values fall in some set range – typically $[-1, 1]$. This can be done using

$$\check{\mathbf{p}} = 2 \left(\mathbf{p} - \mathbf{p}^{\min} \right) \oslash \left(\mathbf{p}^{\max} - \mathbf{p}^{\min} \right) - 1 \quad (4.1)$$

where $\mathbf{p}^{\min}$ is the vector containing the minimum values of each element of the input vectors in the dataset, $\mathbf{p}^{\max}$ contains the maximum values, $\oslash$ represents an element-by-element (Hadamard) division of two vectors, and $\check{\mathbf{p}}$ is the resulting normalized input vector.

The other normalization method is to scale the inputs so that they have a specified mean and variance – typically 0 and 1. This can be done with the transformation

$$\check{\mathbf{p}} = (\mathbf{p} - \mathbf{p}^{mean}) \oslash \mathbf{p}^{std} \quad (4.2)$$

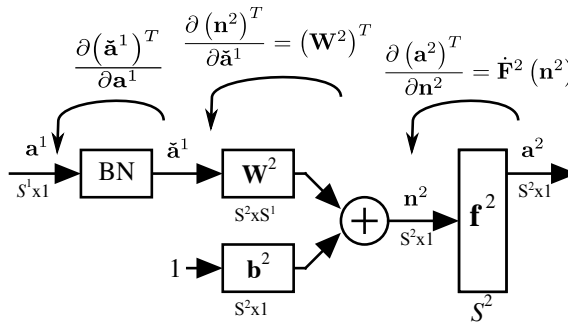
where $\mathbf{p}^{mean}$ is the average of the input vectors in the dataset, and $\mathbf{p}^{std}$ is the vector containing the estimated standard deviations of each element of the input vectors.

These normalizations of the input are helpful for the first layer of the network, and have commonly been used for networks of any size for many years. In 2015 [Ioffe and Szegedy, 2015] proposed that for deep networks the normalizing should be done at each layer in a process they called *batch normalization* (BN). The difficulty is that, unlike the first layer, other layers have inputs (outputs of previous layers) that change during training as the weights and biases are updated. Also, we need to be able to backpropagate across the normalization operations in order to be able to compute the gradient.

For example, consider Figure 4.3. Here we have a layer with a batch normalization block (BN) placed at the beginning of the second layer of a multilayer network. This converts the output of Layer 1, $\mathbf{a}^1$, into its normalized version $\check{\mathbf{a}}^1$.

[Ioffe and Szegedy, 2015] suggested placing the BN block just before the activation function, but others have also placed it before the weight operation, with similar results.

Figure 4.3: Backpropagating Across a Layer With Batch Normalization



To backpropagate across this layer, we need to find the derivative across the BN block:

$$\frac{\partial (\check{\mathbf{a}}^1)^T}{\partial \mathbf{a}^1}. \quad (4.3)$$

Supplemental Training Procedures

However, the backpropagation required here is different than the type discussed in the previous chapter. The backpropagation in the previous chapter was done one example at a time. This was possible because, for example, the derivative of the layer output $\mathbf{a}^2$ with respect to the net input $\mathbf{n}^2$ for the current example does not depend on the previous example. This is not the case when we are doing batch normalization. The normalization of each example depends on the other examples in the batch. We must backpropagate the entire batch at the same time.

The BN operation begins with the following steps:

$$\mathbf{a}^{mean} = \frac{1}{Q} \sum_{q=1}^Q \mathbf{a}_q \quad (4.4)$$

$$\mathbf{a}^{std} = \sqrt{\frac{1}{Q} \sum_{q=1}^Q (\mathbf{a}_q - \mathbf{a}^{mean})^2 + \varepsilon} \quad (4.5)$$

$$\bar{\mathbf{a}}_q = (\mathbf{a}_q - \mathbf{a}^{mean}) \oslash \mathbf{a}^{std} \quad (4.6)$$

This is the same as the input normalization of Eq. 4.2, except that in the calculation of $\mathbf{a}^{std}$ the constant ε is added, to ensure that we do not have a divide by zero. (To obtain an unbiased estimate of standard deviation, divide by $(Q - 1)$, instead of Q .) Here we can see that the normalized value $\bar{\mathbf{a}}_q$ is dependent on all examples in the batch, which are used in the calculation of $\mathbf{a}^{mean}$ and $\mathbf{a}^{std}$.

In addition to normalizing the data so that the average is 0 and the estimated standard deviation is 1 over that batch, using the equations above, the BN process described in [Ioffe and Szegedy, 2015] adds the following additional operation:

$$\check{\mathbf{a}}_q = \mathbf{w}^{bn} \circ \bar{\mathbf{a}}_q + \mathbf{b}^{bn} \quad (4.7)$$

where $\mathbf{w}^{bn}$ is a weight vector, $\circ$ represents the element-by-element (Hadamard) product and $\mathbf{b}^{bn}$ is a bias vector. The weight in Eq. 4.7 is initially set to all 1's and the bias to zeros, so the operation does not initially have any effect, but it allows the BN operation to learn an appropriate overall scaling and offset. Theoretically, $\mathbf{w}^{bn}$ and $\mathbf{b}^{bn}$ could be incorporated into the following weight and bias ($\mathbf{W}^2$ and $\mathbf{b}^2$ in Figure 4.3), if the BN operation is placed before the weight, but having them separate appears to provide some practical advantage.

In the BN operation, each output depends on all inputs within a batch. Therefore, the backpropagation operation must be done using the entire batch. This means that Figure 4.3 would be replaced with Figure 4.4, where $\text{vec}(\mathbf{A})$ transforms the matrix into a vector by stacking the columns of the matrix one underneath the other:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_Q \end{bmatrix} \quad (4.8)$$

$$\text{vec}(\mathbf{A}) = \begin{bmatrix} \mathbf{a}_1^T & \mathbf{a}_2^T & \cdots & \mathbf{a}_Q^T \end{bmatrix}^T \quad (4.9)$$

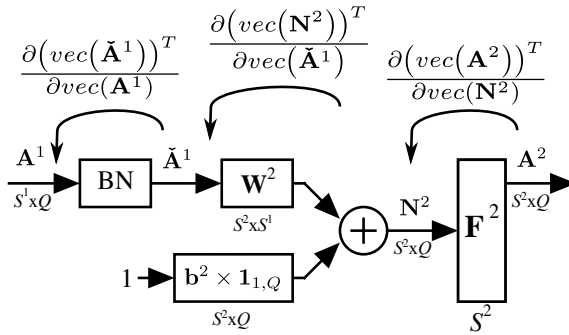


Figure 4.4: Batch Back-propagating Across a Layer With Batch Normalization

The batch backpropagations across the transfer function and the weight are just a combination of the backpropagations of each example in the batch, as described in Eq. 3.49. This is because of the form of the following Jacobians:

$$\frac{\partial (\text{vec}(\mathbf{N}^2))^T}{\partial \text{vec}(\tilde{\mathbf{A}}^1)} = \begin{bmatrix} (\mathbf{W}^2)^T & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & (\mathbf{W}^2)^T & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \cdots & (\mathbf{W}^2)^T \end{bmatrix} \quad (4.10)$$

$$\frac{\partial (\text{vec}(\mathbf{A}^2))^T}{\partial \text{vec}(\mathbf{N}^2)} = \begin{bmatrix} \dot{\mathbf{F}}^2(\mathbf{n}_1^2) & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \dot{\mathbf{F}}^2(\mathbf{n}_2^2) & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \cdots & \dot{\mathbf{F}}^2(\mathbf{n}_Q^2) \end{bmatrix} \quad (4.11)$$

Supplemental Training Procedures

However, the derivative across the BN block will not have this block diagonal form, so the backpropagation must be done with the entire batch. It is possible to compute the derivative across BN in one operation, but it is somewhat complex. It is easier to break the operation into simple steps and use the chain rule (backpropagation) to compute it. Figure 4.5 shows the initial BN operation (Eqs. 4.4 to 4.6) in graphical form involving basic steps.

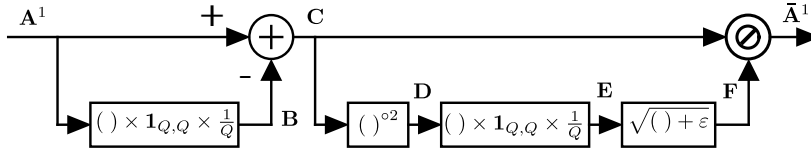


Figure 4.5: First Stage of Batch Normalization Operations

The block between $\mathbf{A}^1$ and $\mathbf{B}$ computes the average of the batch (Eq. 4.4) and broadcasts the result to make Q columns (see Solved Problem P4.2). The $\mathbf{C}$ to $\mathbf{F}$ path computes the estimated standard deviation of the batch (Eq. 4.5) and broadcasts the result to make Q columns.

Using the backpropagation process, the derivative across this part of the BN process can be computed using

$$\begin{aligned} \frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{A}^1)} &= \left[\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{A}^1)} - \frac{\partial (\text{vec}(\mathbf{B}))^T}{\partial \text{vec}(\mathbf{A}^1)} \frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{B})} \right] \\ &\quad \left[\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{C})} + \frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{C})} \frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{F})} \right] \end{aligned} \quad (4.12)$$

The derivatives across the summation point are simple:

$$\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{A}^1)} = \mathbf{I}_{S^1, Q} \quad (4.13)$$

$$\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{B})} = -\mathbf{I}_{S^1, Q} \quad (4.14)$$

where $\mathbf{I}_N$ is an $N \times N$ identity matrix. The averaging operations from $\mathbf{A}^1$ to $\mathbf{B}$ and $\mathbf{D}$ to $\mathbf{E}$ have derivative

$$\frac{\partial (\text{vec}(\mathbf{B}))^T}{\partial \text{vec}(\mathbf{A}^1)} = \frac{\partial (\text{vec}(\mathbf{E}))^T}{\partial \text{vec}(\mathbf{D})} = \mathbf{I}_{S^1}^Q \times \frac{1}{Q} \quad (4.15)$$

where

$$\mathbf{I}_S^Q = \left[\begin{array}{cccc} \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \\ \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \end{array} \right] \left. \vphantom{\begin{array}{c} \mathbf{I}_S^Q \\ \mathbf{I}_S \\ \mathbf{I}_S \\ \vdots \\ \mathbf{I}_S \end{array}} \right\} Q \cdot S \quad (4.16)$$

The derivative across the **C** to **F** path, which computes the estimated standard deviation, can be found by backpropagation:

$$\frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{C})} = \frac{\partial (\text{vec}(\mathbf{D}))^T}{\partial \text{vec}(\mathbf{C})} \frac{\partial (\text{vec}(\mathbf{E}))^T}{\partial \text{vec}(\mathbf{D})} \frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{E})} \quad (4.17)$$

where

$$\frac{\partial (\text{vec}(\mathbf{D}))^T}{\partial \text{vec}(\mathbf{C})} = 2 \cdot \text{diag}(\text{vec}(\mathbf{C})) \quad (4.18)$$

$$\frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{E})} = -\frac{1}{2} \cdot \text{diag}(\text{vec}(\mathbf{E} + \epsilon))^{-\frac{1}{2}} \quad (4.19)$$

where $\text{diag}(\mathbf{x})$ forms a diagonal matrix with the vector $\mathbf{x}$ along the diagonal.

To complete the derivative across the main part of the BN process, using Eq. 4.12, we need only the last two terms (see Solved Problem P4.4):

$$\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{C})} = \text{diag}(\text{vec}(\mathbf{F}))^{-1} \quad (4.20)$$

$$\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{F})} = -\text{diag}(\text{vec}(\mathbf{C}) \oslash \text{vec}(\mathbf{F})^{\circ 2}) \quad (4.21)$$

The final stage of the BN process, Eq. 4.7, involves just a weight multiplication and then the addition of a bias. This is the same as a standard layer in a multilayer network, except that the weight matrix is a vector and the weight operation is a Hadamard product. See Solved Problem P4.1 for a derivation of backpropagation across the Hadamard product.

After training, when the network is being used for inference, it is not reasonable to use the BN block in the same manner that it is used during training. In some cases, only one example (a batch of one) might be applied to the network, and it would not be possible to calculate a standard deviation for a batch of one example. For inference, the BN process should be fixed, which means that fixed values would be used for $\mathbf{a}^{\text{mean}}$ and $\mathbf{a}^{\text{std}}$. (The weight and bias $\mathbf{w}^{\text{bn}}$

Inference refers to the process of using the network on new data after the network has been trained.

Supplemental Training Procedures

and $\mathbf{b}^{bn}$ of Eq. 4.7 would be fixed during inference at their trained values.) One choice would be to use the values obtained from the full training set. However, [Ioffe and Szegedy, 2015] suggests using a smoothed version of the values computed during training, as in

$$\tilde{\mathbf{a}}_k^{mean} = \delta \cdot \tilde{\mathbf{a}}_{k-1}^{mean} + (1 - \delta) \cdot \mathbf{a}_k^{mean} \quad (4.22)$$

$$\tilde{\mathbf{a}}_k^{std} = \delta \cdot \tilde{\mathbf{a}}_{k-1}^{std} + (1 - \delta) \cdot \mathbf{a}_k^{std} \quad (4.23)$$

where $\mathbf{a}_k^{mean}$ and $\mathbf{a}_k^{std}$ are the mean and standard deviation computed at the k^{th} iteration (minibatch) of training, and $0 < \delta < 1$ determines the amount of smoothing (δ close to one averages over more iterations). At the completion of training, the final $\tilde{\mathbf{a}}^{mean}$ and $\tilde{\mathbf{a}}^{std}$ would be used as fixed values during inference.

It should be noted that batch normalization was developed with sigmoid activations in mind – the idea being to keep activations in the non-saturating region of the sigmoid. However, experiments have shown that it also improves performance with *ReLU* (*poslin*) networks. It has been suggested that this is because it tends to improve the generalization abilities of the network by causing random changes in scaling from minibatch to minibatch. In this way, it may have an effect similar to *dropout*, which is discussed in a later section of this chapter. Another reason for the benefits of BN is provided in [Santurkar et al., 2018], which suggests that BN smooths the performance function surface.

Normalization is one way to minimize the vanishing gradient problem, because it helps to keep the inputs to the activation functions in their non-saturating regions. This can also be done by proper weight initialization. This is because it is the product between the weights and the inputs to the layer that determines the input to the activation function. Next we consider several techniques for weight initialization.

WEIGHT INITIALIZATION sets the values of the weights and biases that are used in the first iteration of the training algorithm. If the initial weights are too large, the sigmoid activation functions can be saturated, which means that the algorithm would start in flat regions of the performance surface. Small gradients in this region would lead to very slow convergence. This was illustrated in Chapter 12 of [NND2](#).

It was also shown in Chapter 12 of [NND2](#) that the initial weights should not be set to zero, since this is a saddle point in the performance surface, where the gradient would be zero. One solution is to set the weights to small random values. Of course, small is a relative term. The key is that we do not want to saturate the sigmoid activation function. In addition, as we can see from Eq. 3.49, the sensitivity is multiplied by the weights at each stage of the back-propagation process. This means that if the weights are too small, the gradient will become very small for weights in the early layers of a deep network (*vanishing gradient*).

[[Nguyen and Widrow, 1990](#)] were the first to suggest setting the initial weights to larger values. As described in Chapter 20 of [NND2](#), the idea was to set the magnitude of the weights in the first layer, containing S^1 neurons, so that the linear region of each sigmoid function covers $1/S^1$ of the range of the input. The biases are then randomly set, so that the center of each sigmoid function falls randomly in the input space. This technique was designed for two-layer networks, but could be extended for multiple layers.

[[Glorot and Bengio, 2010](#)] proposed an initialization strategy specifically designed for deep networks with hyperbolic tangent sigmoid activation functions. The idea was to produce weights that would not change the variation in activations as they propagated forward through the network, and at the same time would not change the variation in sensitivities as they propagated backward through the network.

Consider the forward propagation to layer m :

$$\mathbf{a}^m = \mathbf{f}^m (\mathbf{W}^m \mathbf{a}^{m-1} + \mathbf{b}^m) \quad (4.24)$$

if we are operating in the linear region of the hyperbolic tangent sigmoid function, where the slope is 1, we can write

$$\mathbf{a}^m \simeq \mathbf{W}^m \mathbf{a}^{m-1} + \mathbf{b}^m \quad (4.25)$$

Considering one element of the layer output

$$a_i^m \simeq \sum_{j=1}^{S^{m-1}} w_{i,j}^m a_j^{m-1} + b_i^m \quad (4.26)$$

[[Glorot and Bengio, 2010](#)] set the initial biases to zero, so the last term is removed. We also make the assumption that the all activations coming from the previous layer are random, with mean zero and variance $\sigma_a^2(m-1)$ and the initial weights are random values

Supplemental Training Procedures

with mean zero and variance σ_w^2 . To simplify the analysis, we also assume that the activations and weights are independent. This produces the following result.

$$\text{var}(a_i^m) \equiv \sigma_a^2(m) = S^{m-1} \sigma_w^2 \sigma_a^2(m-1) \quad (4.27)$$

Therefore, if we want the activation variance to remain the same from layer to layer (i.e., $\sigma_a^2(m) = \sigma_a^2(m-1)$), we should set

$$\sigma_w^2 = \frac{1}{S^{m-1}} \quad (4.28)$$

Now consider the backpropagation process of Eq. 3.49:

$$\mathbf{s}^{m-1} = \dot{\mathbf{F}}^{m-1}(\mathbf{n}^{m-1})(\mathbf{W}^m)^T \mathbf{s}^m \quad (4.29)$$

If we assume again that we are operating in the linear region of the tangent sigmoid, the derivative will be approximately 1. We can then approximately compute an element of the sensitivity as

$$s_i^{m-1} \simeq \sum_{j=1}^{S^m} w_{j,i}^m s_j^m \quad (4.30)$$

Making the same assumptions concerning independence that were made for the forward propagation, and assuming that the sensitivities at layer m have mean zero and variance $\sigma_s^2(m)$, we can approximate the variance of the sensitivities at layer $m-1$ to be

$$\text{var}(s_i^{m-1}) = \sigma_s^2(m-1) = S^m \sigma_w^2 \sigma_s^2(m) \quad (4.31)$$

If we want the sensitivity variance to remain the same from layer to layer (i.e., $\sigma_s^2(m-1) = \sigma_s^2(m)$), then we should set

$$\sigma_w^2 = \frac{1}{S^m} \quad (4.32)$$

In [Glorot and Bengio, 2010], they suggest combining Eqs. 4.28 and 4.32 so that the variance of the weights should be

$$\sigma_w^2 = \frac{2}{S^{m-1} + S^m} \quad (4.33)$$

So, in *Xavier initialization*, the initial weights in layer m are set as normally distributed random numbers with mean zero and variance set by Eq. 4.33. The biases are initially set to zero.

Xavier initialization was designed for the hyperbolic tangent sigmoid activation function. If the *ReLU* (*poslin*) activation function is used, then a slight adjustment is made to the variance of the weights. In [He et al., 2015], they show that, since the *poslin* function is zero for half of its domain, the variance is cut in half as a signal is passed through that activation function, so the variance of the weights should be doubled to maintain the variance of the activations from layer to layer. The resulting *Kaiming initialization* uses a variance for the initial weights of

$$\sigma_w^2 = \frac{2}{S_{m-1}} \quad (4.34)$$

The combination of normalization and proper weight initialization are helpful in preventing a vanishing gradient.

To experiment with the use of different normalization and initialization effects, use the Deep Learning Demonstration Scaling (dl4sc).



To experiment with different initialization schemes and how they affect the range of layer outputs as the number of layers increase, use the Deep Learning Demonstration Initialization Effect (dl4ie).



GRADIENT CLIPPING is another technique that helps to speed up convergence. First formally proposed in [Pascanu et al., 2013] for training recurrent networks (which were discussed in Chapter 14 of *NND2* and will be discussed in a later chapter of this volume), it involves limiting the magnitude of the gradient. We previously discussed the vanishing gradient problem, in which the gradients can get smaller as the number of layers get large. This is because, as we can see from Eq. 3.49, the sensitivity is multiplied by the weights at each stage of the backpropagation process. This means that if the weights are too small, the gradient will become very small for weights in the early layers of a deep network. For a similar reason, if the weights are too large, the gradients can become very large (*exploding gradient*). This is most likely to occur in recurrent networks, as we saw in Chapter 14 of *NND2*, and as we will see in a later chapter of this volume.

Supplemental Training Procedures

[Pascanu et al., 2013] propose that if the norm of the gradient goes beyond a specified threshold that the gradient be normalized so that the norm is equal to the threshold. In this way, the gradient search algorithm moves in the negative gradient direction, but with a smaller step size. This would be equivalent to reducing the learning rate whenever the gradient becomes too large. Another option would be to limit the individual elements of the gradient to a specific absolute value, although this would change the direction of search. Both of these techniques are referred to as *gradient clipping*.

Generalization

Although the previous chapter focused on network training as optimizing a performance function on a specified training set, the true objective of network training is to produce a network that will perform well on future data that it has not seen before. This ability to generalize was investigated extensively in Chapter 13 of *NND2*, which also surveyed techniques for improving the generalization ability of trained networks.

The two main approaches to improving generalization that were covered in Chapter 13 of *NND2* were *early stopping* and regularization. The idea behind early stopping is that, as the network trains, it is using more and more of its weights, and all of the weights are fully used when the network reaches a minimum of the performance surface. By increasing the number of iterations of training, we are increasing the complexity of the resulting network. The more complex the network is, the more likely it is to overfit the training data and, therefore, to perform poorly on new data that it has not seen. The key question is when to stop the training.

For early stopping, a portion of the training data is set aside as a *validation set*. The performance on the validation set is monitored during training, but it is not used to compute the gradient. An increase in the validation performance indicates that overfitting is occurring, so the weights that produce the minimum validation performance are used as the final trained weights. See Chapter 13 of *NND2* for a more complete description and a software demonstration of early stopping.

At this point, we should clarify that when early stopping is used, the full data set is divided into three components: training, validation and testing. The *test set* is removed from the full data set before training. It is not used in any way during the training process, even for selecting between potential networks. After training is complete, the test set is applied to the network, and the resulting performance is used to measure the generalization ability of the network. One of the most common mistakes in neural network training is misuse of the testing set. It must never be used in any way until the final network has been trained. Otherwise, the performance on the test set will be overly optimistic about future performance.

Regularization, also described in Chapter 13 of [NND2](#), is a method which modifies the performance function by adding a term that penalizes network complexity. By penalizing complexity, the training will produce a network that will be better able to generalize. The most common penalty is the sum squared weights. The addition of this penalty to the mean square error performance function would be

$$F(\mathbf{x}) = \frac{1}{QS^M} \sum_{q=1}^Q \sum_{i=1}^{S^M} (t_{i,q} - a_{i,q})^2 + \rho \sum_{i=1}^N x_i^2 \quad (4.35)$$

where $\mathbf{x}$ is the vector containing the weights of the network. This is often referred to as L2 regularization, because it involves the square of the weights. L1 regularization uses the absolute value of the weights.

The difficulty in using regularization is choosing the regularization coefficient ρ . If ρ is chosen too large, the network will underfit, and if ρ is chosen too small, the network will overfit. One technique for choosing ρ is to minimize the unregularized performance on a validation set. Another technique is to use Bayesian regularization to determine ρ . This method is described in Chapter 13 of [NND2](#).

A method for improving generalization that is very efficient, and therefore well-suited to deep networks, is called *dropout*, which was introduced in [\[Srivastava et al., 2014\]](#). Dropout can be applied to any layer of a network. If it is applied to layer m , then at each iteration of training, each neuron in that layer is activated with a given probability – say ψ_m . If it is not active, then that neuron and its incoming and outgoing connections are removed from the network for both the forward propagation of activations and the backward propagation of sensitivities. (Row i of $\mathbf{W}^m$ and column i

Supplemental Training Procedures

of $\mathbf{W}^{m+1}$ are removed if neuron a_i^m is not active). At each iteration, different neurons are randomly activated. Because a given neuron is only available for ψ_m of the iterations, and a different minibatch is used in each iteration, it is unlikely that the network will overfit by memorizing the training set. This is why dropout improves generalization.

When the network is used for inference (after training is complete), the weights need to be scaled to account for the fact that they were not used during all iterations of training. If a given neuron a_i^m in layer m is activated with probability ψ_m during training, then the outgoing weights for that neuron (column i of $\mathbf{W}^{m+1}$) are multiplied by ψ_m during inference. Assuming that all neurons in layer m are activated with probability ψ_m , then the weight matrix $\mathbf{W}^{m+1}$ is multiplied by ψ_m for inference.

We should note that there is something of a conflict between dropout and the batch normalization process that we described in an earlier section (see [Li et al., 2018]). The standard form of dropout is not normally used in combination with BN. However, [Ioffe and Szegedy, 2015] state that in their experiments dropout is often not needed, because BN improves the generalization ability of networks when used alone. The suggested reason for this is that BN introduces a transformation between layers that changes with each iteration (and minibatch), creating a kind of noise in the training that reduces the chances for overfitting.

To experiment with dropout and how it prevents overfitting, use the Deep Learning Demonstration Dropout (dl4do).



Epilogue

The theoretical principles behind the training of neural networks with an arbitrary number of layers have been widely known since the late 1980s. However, deep networks were not commonly used until techniques were developed for reducing training times. This chapter described several of those techniques.

One way to speed up convergence is to provide the optimization algorithm with a suitable starting point. If sigmoid activation functions are used, then it is important that the initial weights are not so large that the sigmoids are regularly saturated. In this case, the derivatives will be very small, causing gradients to be small and training to be slow. On the other hand, if the weights, which multiply sensitivities during the backpropagation process, are too small, then this will also cause the gradients to be small. The trick is to get into the Goldilocks zone, where the initial neuron activations are neither too large or too small.

The activations are dependent on the inputs to a layer and the layer weights. The inputs to the layer can be brought into the Goldilocks zone through batch normalization, in which the inputs are transformed to have a mean of 0 and a standard deviation of 1. The initial weights can be brought into the Goldilocks zone using Xavier initialization, where they are assigned random values with standard deviation determined by the number of elements of the layer input and layer output. With this choice, activations and sensitivities maintain a similar level throughout the layers of the network.

The main purpose of neural network training is to produce networks that will perform well on new data that were not seen during training – we want the networks to generalize. Dropout is a method to improve network generalization that randomly removes a certain percentage of neurons during each iteration of the training process. Networks are less likely to memorize the training examples, since different neurons may see different minibatches with different frequencies.

Now that we have covered network training procedures, the next step is to implement the algorithms in software and verify their performance on practical data sets. That is the objective of the next couple of chapters.

Supplemental Training Procedures

Further Reading

[Ioffe and Szegedy, 2015] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015

This is the original description of the batch normalization algorithm. It suggests using batch normalization because of covariate shift, in which the distribution of data changes. Others have since questioned that mechanism.

[Santurkar et al., 2018] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization?(no, it is not about internal covariate shift). *arXiv preprint arXiv:1805.11604*, 2018

Batch normalization improves training performance and produces networks that generalize better because it smooths the performance surface.

[Nguyen and Widrow, 1990] Derrick Nguyen and Bernard Widrow. Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights. In *Neural Networks, 1990., 1990 IJCNN International Joint Conference on*, pages 21–26. IEEE, 1990

Describes a method for weight initialization. The initial weights and biases are set so that the sigmoid activations are in their linear range over some specified proportion of the input space.

[Glorot and Bengio, 2010] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256, 2010

Describes a method for initializing the weights of deep networks so that the activations going forward and the gradients propagating backward maintain a reasonable level of variation. The method is usually referred to as Xavier initialization.

[He et al., 2015] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015

Xavier initialization is based on sigmoid activation functions. This paper describes an alternative form of initialization for ReLU networks. It also describes some activation functions that are variations on the ReLU, with adjustable nonzero slopes for negative inputs.

Summary of Results

Normalize to [-1 1]

$$\check{\mathbf{p}} = (\mathbf{p} - \mathbf{p}^{\min}) \oslash (\mathbf{p}^{\max} - \mathbf{p}^{\min}) - 1$$

Normalize to Mean 0 and Variance 1

$$\check{\mathbf{p}} = (\mathbf{p} - \mathbf{p}^{\text{mean}}) \oslash \mathbf{p}^{\text{std}}$$

Batch Normalization

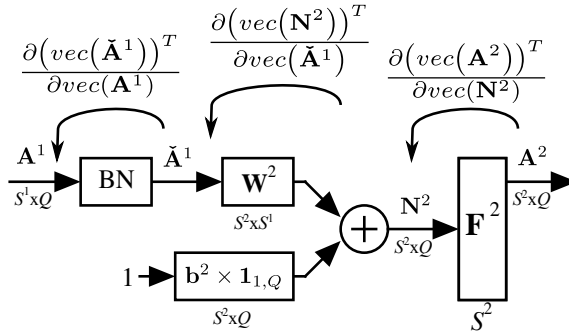
$$\mathbf{a}^{\text{mean}} = \frac{1}{Q} \sum_{q=1}^Q \mathbf{a}_q$$

$$\mathbf{a}^{\text{std}} = \sqrt{\frac{1}{Q} \sum_{q=1}^Q (\mathbf{a}_q - \mathbf{a}^{\text{mean}})^2 + \varepsilon}$$

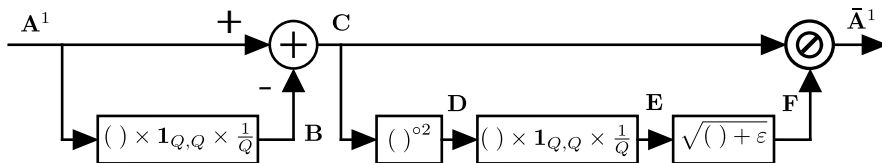
$$\bar{\mathbf{a}}_q = (\mathbf{a}_q - \mathbf{a}^{\text{mean}}) \oslash \mathbf{a}$$

$$\check{\mathbf{a}}_q = \mathbf{w}^{\text{bn}} \circ \bar{\mathbf{a}}_q + \mathbf{b}^{\text{bn}}$$

Batch Backpropagation Across Layer with BN



First Stage of Batch Normalization Operations



Supplemental Training Procedures

Backpropagating Across BN Stage

$$\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{A}^1)} = \left[\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{A}^1)} - \frac{\partial (\text{vec}(\mathbf{B}))^T}{\partial \text{vec}(\mathbf{A}^1)} \frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{B})} \right] \\ \left[\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{C})} + \frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{C})} \frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{F})} \right]$$

Component Derivatives

$$\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{A}^1)} = \mathbf{I}_{S^1 \cdot Q}$$

$$\frac{\partial (\text{vec}(\mathbf{C}))^T}{\partial \text{vec}(\mathbf{B})} = -\mathbf{I}_{S^1 \cdot Q}$$

$$\frac{\partial (\text{vec}(\mathbf{B}))^T}{\partial \text{vec}(\mathbf{A}^1)} = \frac{\partial (\text{vec}(\mathbf{E}))^T}{\partial \text{vec}(\mathbf{D})} = \mathbf{I}_{S^1}^Q \times \frac{1}{Q}$$

$$\mathbf{I}_S^Q = \left\{ \begin{bmatrix} \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \\ \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{I}_S & \mathbf{I}_S & \cdots & \mathbf{I}_S \end{bmatrix} \right\} Q \cdot S$$

$$\frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{C})} = \frac{\partial (\text{vec}(\mathbf{D}))^T}{\partial \text{vec}(\mathbf{C})} \frac{\partial (\text{vec}(\mathbf{E}))^T}{\partial \text{vec}(\mathbf{D})} \frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{E})}$$

$$\frac{\partial (\text{vec}(\mathbf{D}))^T}{\partial \text{vec}(\mathbf{C})} = 2 \cdot \text{diag}(\text{vec}(\mathbf{C}))$$

$$\frac{\partial (\text{vec}(\mathbf{F}))^T}{\partial \text{vec}(\mathbf{E})} = -\frac{1}{2} \cdot \text{diag}(\text{vec}(\mathbf{E} + \varepsilon))^{-\frac{1}{2}}$$

$$\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{C})} = \text{diag}(\text{vec}(\mathbf{F}))^{-1}$$

$$\frac{\partial (\text{vec}(\bar{\mathbf{A}}^1))^T}{\partial \text{vec}(\mathbf{F})} = -\text{diag}(\text{vec}(\mathbf{C}) \oslash \text{vec}(\mathbf{F})^{\circ 2})$$

Smoothed Mean and Standard Deviation for BN Inference

$$\tilde{\mathbf{a}}_k^{\text{mean}} = \delta \cdot \tilde{\mathbf{a}}_{k-1}^{\text{mean}} + (1 - \delta) \cdot \mathbf{a}_k^{\text{mean}}$$

$$\tilde{\mathbf{a}}_k^{\text{std}} = \delta \cdot \tilde{\mathbf{a}}_{k-1}^{\text{std}} + (1 - \delta) \cdot \mathbf{a}_k^{\text{std}}$$

Weight Variance for Xavier Initialization

$$\sigma_w^2 = \frac{2}{S^{m-1} + S^m}$$

Weight Variance for Kaiming Initialization

$$\sigma_w^2 = \frac{2}{S^{m-1}}$$

Supplemental Training Procedures

Solved Problems

- P4.1** The final step of the batch normalization process is Eq. 4.7. This has the appearance of a standard layer, with a linear transfer function. The only difference is that instead of standard matrix multiplication, it uses the Hadamard product. How should you backpropagate across the Hadamard product?

Repeating Eq. 4.7 here, we have

$$\check{\mathbf{a}}_q = \mathbf{w}^{bn} \circ \bar{\mathbf{a}}_q + \mathbf{b}^{bn}$$

This does appear to be a standard linear layer, with Hadamard product as the weight function. From Eq. 3.45, the sensitivity update across a layer would be

$$\mathbf{s}^m = \frac{\partial (\mathbf{a}^m)^T}{\partial \mathbf{n}^m} \frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m} \frac{\partial (\mathbf{n}^{m+1})^T}{\partial \mathbf{z}^{m+1}} \mathbf{s}^{m+1}$$

The term that is different for this layer is the weight function. We just need to compute

$$\frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m}$$

For the Hadamard product, the weight function is

$$\begin{aligned} \mathbf{z}^{m+1} &= \mathbf{w}^{m+1} \circ \mathbf{a}^m \\ z_i^{m+1} &= w_i^{m+1} a_i^m \end{aligned}$$

The elements of the Jacobian would be

$$\left[\frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m} \right]_{i,j} = \frac{\partial z_j^{m+1}}{\partial a_i^m} = \begin{cases} w_i^{m+1} & i = j \\ 0 & i \neq j \end{cases}$$

or, in matrix form

$$\frac{\partial (\mathbf{z}^{m+1})^T}{\partial \mathbf{a}^m} = \text{diag}(\mathbf{w}^{m+1})$$

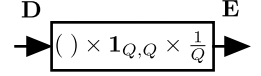
The backpropagation across the layer would then reduce to

$$\mathbf{s}^m = \mathbf{f}^m(\mathbf{n}^m) \text{diag}(\mathbf{w}^{m+1}) \mathbf{s}^{m+1}$$

which is the same as

$$\mathbf{s}^m = \hat{\mathbf{f}}^m(\mathbf{n}^m) \mathbf{w}^{m+1} \circ \mathbf{s}^{m+1}$$

P4.2 The first stage of the batch normalization process is displayed graphically in Figure 4.5. There are two blocks in this process that perform the same averaging operation: A¹ to B and D to E. Demonstrate how the block performs the averaging operation, and show how the derivative is given by Eq. 4.15.



The calculation can be written in matrix form as

$$\begin{aligned} \mathbf{E} &= \mathbf{D} \times \mathbf{1}_{Q,Q} \times \frac{1}{Q} \\ \mathbf{E} &= \begin{bmatrix} d_{1,1} & d_{1,2} & \cdots & d_{1,Q} \\ d_{2,1} & d_{2,2} & \cdots & d_{2,Q} \\ \vdots & \vdots & \ddots & \vdots \\ d_{S^1,1} & d_{S^1,2} & \cdots & d_{S^1,Q} \end{bmatrix} \times \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} \times \frac{1}{Q} \\ &= \begin{bmatrix} \frac{1}{Q} \sum_{q=1}^Q d_{1,q} & \frac{1}{Q} \sum_{q=1}^Q d_{2,q} & \cdots & \frac{1}{Q} \sum_{q=1}^Q d_{S^1,q} \\ \frac{1}{Q} \sum_{q=1}^Q d_{1,q} & \frac{1}{Q} \sum_{q=1}^Q d_{2,q} & \cdots & \frac{1}{Q} \sum_{q=1}^Q d_{S^1,q} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{Q} \sum_{q=1}^Q d_{S^1,q} & \frac{1}{Q} \sum_{q=1}^Q d_{S^1,q} & \cdots & \frac{1}{Q} \sum_{q=1}^Q d_{S^1,q} \end{bmatrix} \\ e_{i,q} &= \frac{1}{Q} \sum_{q'=1}^Q d_{i,q'} \end{aligned}$$

Therefore, each column of $\mathbf{E}$ contains the average of the columns of $\mathbf{D}$. The derivative we want to calculate is

$$\frac{\partial (\text{vec}(\mathbf{E}))^T}{\partial \text{vec}(\mathbf{D})}$$

The elements of this Jacobian have the form

$$\frac{\partial e_{i,q}}{\partial d_{i',q'}} = \begin{cases} \frac{1}{Q} & i' = i \\ 0 & i' \neq i \end{cases}$$

which produces Eq. 4.15.

P4.3 The normalization process changes the mean and variance of incoming data. Suppose that it was known that the input vector $\mathbf{p}$ had a mean μ_p and covariance matrix $\mathbf{Q}_p$. Let the normalization process have the form $\bar{\mathbf{p}} = \mathbf{W}\mathbf{p} + \mathbf{b}$.

Supplemental Training Procedures

- i. Find the mean of $\bar{\mathbf{p}}$, and find the conditions on $\mathbf{W}$ and $\mathbf{b}$ so that the mean of $\bar{\mathbf{p}}$ will be zero.
 - ii. Find the covariance matrix of $\bar{\mathbf{p}}$, and find a $\mathbf{W}$ so that the elements of $\bar{\mathbf{p}}$ have variance of 1 and are uncorrelated with each other.
- i. The mean of $\bar{\mathbf{p}}$ can be written

$$E[\bar{\mathbf{p}}] = \mu_{\bar{\mathbf{p}}} = E[\mathbf{W}\mathbf{p} + \mathbf{b}] = \mathbf{W}\mu_p + \mathbf{b}$$

Therefore, to produce a zero mean result, we should choose

$$\mathbf{b} = -\mathbf{W}\mu_p$$

- ii. The covariance matrix for $\bar{\mathbf{p}}$ can be written

$$\text{cov}\{\bar{\mathbf{p}}\} = \mathbf{Q}_{\bar{\mathbf{p}}} = E\left[\left(\bar{\mathbf{p}} - \mu_{\bar{\mathbf{p}}}\right)\left(\bar{\mathbf{p}} - \mu_{\bar{\mathbf{p}}}\right)^T\right]$$

We can write

$$\bar{\mathbf{p}} - \mu_{\bar{\mathbf{p}}} = \mathbf{W}\mathbf{p} + \mathbf{b} - \mathbf{W}\mu_p - \mathbf{b} = \mathbf{W}\left(\mathbf{p} - \mu_p\right)$$

Substituting into the earlier equation, we have

$$\mathbf{Q}_{\bar{\mathbf{p}}} = E\left[\mathbf{W}\left(\mathbf{p} - \mu_p\right)\left(\mathbf{p} - \mu_p\right)^T \mathbf{W}^T\right] = \mathbf{W}\mathbf{Q}_p \mathbf{W}^T$$

From Chapter 8 of [NND2](#), we know that the eigenvectors of a symmetric matrix $\mathbf{Q}$ are orthogonal, and that $\mathbf{Q}$ can be diagonalized using

$$\Lambda = \mathbf{B}^T \mathbf{Q} \mathbf{B} = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_R \end{bmatrix}$$

where

$$\mathbf{B} = \begin{bmatrix} \mathbf{z}_1 & \mathbf{z}_2 & \cdots & \mathbf{z}_R \end{bmatrix}$$

is the matrix whose columns are the eigenvectors of $\mathbf{Q}$, and the λ_i are the eigenvalues of $\mathbf{Q}$.

From this result, if we choose the transformation matrix $\mathbf{W}$ to be $\mathbf{B}^T$, where the columns of $\mathbf{B}$ are the eigenvectors of $\mathbf{Q}_p$, then we would have

$$\mathbf{Q}_{\bar{\mathbf{p}}} = \mathbf{W}\mathbf{Q}_p\mathbf{W}^T = \mathbf{B}^T\mathbf{Q}_p\mathbf{B} = \Lambda$$

Because the covariance matrix is diagonal, this means that the elements of $\bar{\mathbf{p}}$ are uncorrelated with each other. However, the variances of the elements are equal to the eigenvalues λ_i . If we want the variances to equal 1, we need to multiply the columns of $\mathbf{B}$ by $1/\sqrt{\lambda_i}$, before performing the diagonalization operation.

P4.4 For the first stage of the batch normalization process, show how the derivative across the $\mathbf{F}$ to $\bar{\mathbf{A}}^1$ path is given by Eq. 4.21.

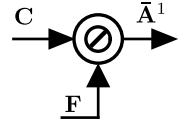
The calculation performed by the Hadamard division is

$$\begin{aligned}\bar{\mathbf{A}}^1 &= \mathbf{C} \oslash \mathbf{F} \\ \bar{a}_{i,j}^1 &= \frac{c_{i,j}}{f_{i,j}}\end{aligned}$$

Taking the derivative across the $\mathbf{F}$ to $\bar{\mathbf{A}}^1$ path, we have

$$\frac{\partial \bar{a}_{i,q}^1}{\partial f_{i',q'}} = \begin{cases} -\frac{c_{i,q}}{(f_{i,q})^2} & i = i', q = q' \\ 0 & else \end{cases}$$

This is the same as Eq. 4.21.



Supplemental Training Procedures

Exercises

- E4.1** Assume that a scalar random variable p has a uniform distribution on the interval $[0, 1]$. We transform p to the variable a using $a = wp + b$.
- Find the values of w and b so that a has mean 0 and variance 1.
 - Sketch the density function of a . Is it possible to adjust w and b so that a has a Gaussian density? Explain.

- E4.2** Suppose that the random variable p has mean 1 and variance 4. Define a second random variable $a = wp + b$. Find the values for w and b so that a has mean 0 and variance 1.

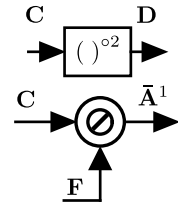
- E4.3** Consider a one-layer network with two neurons and a linear activation function. The input vector $\mathbf{p}$ has mean and covariance matrix given by

$$\mu_p = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$
$$\mathbf{Q}_p = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$$

- Write out an expression for the mean and covariance matrix for the network output $\mathbf{a}$, as a function of the network weight matrix $\mathbf{W}$ and bias vector $\mathbf{b}$.
 - Find a weight matrix $\mathbf{W}$ so that the network outputs will be uncorrelated and with a variance of 1.
 - Find the bias vector $\mathbf{b}$ so that the network outputs will have zero mean.
- E4.4** If w has variance σ_w^2 , b has variance σ_b^2 and p has variance σ_p^2 , what is the variance of $a = wp + b$? Assume that w , b and p are mutually uncorrelated.

E4.5 The first stage of the batch normalization process is displayed graphically in Figure 4.5. The **C** to **D** block performs an element-by-element squaring operation. Show how the derivative across this block is given by Eq. 4.18.

E4.6 For the first stage of the batch normalization process, show how the derivative across the **C** to $\bar{\mathbf{A}}^1$ path is given by Eq. 4.20



5 *Python*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Python</i>	2
<i>NumPy</i>	9
<i>Pandas</i>	12
<i>Epilogue</i>	21
<i>Further Reading</i>	22
<i>Labs</i>	23

Objective

One reason that there has been significant progress in deep learning is the availability of open source software tools (generally referred to as frameworks) for implementing deep networks. In this textbook we will interleave theoretical discussions with discussions of these frameworks. The interface most commonly used to access the frameworks is the Python programming language. In this chapter we provide a brief introduction to Python and a few of its most commonly used packages. This chapter is available as an interactive Jupyter Notebook at the website <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/blob/master/05.PythonChapter/Code/ChapterNotebook/PythonChapter.ipynb>

There are a number of open source software tools for developing deep neural networks. Many of these tools are called *frameworks*. A software framework provides a high-level interface for accessing a library of modular and highly optimized low-level functions. In the case of the deep learning frameworks, the low-level functions implement elementary neural network operations, like matrix multiplication, on CPUs or GPUs.

The two most popular frameworks are TensorFlow and PyTorch, which we will cover in later chapters of this text. Both of these frameworks have a Pythonic interface. In this chapter, to lay the groundwork for the framework chapters, we will first review some of the key Python concepts that are most important to the use of deep learning frameworks. In addition to the Python core, this chapter will also cover two important Python packages, NumPy and Pandas, which are very commonly used in deep learning workflows for loading and preparing data for training.

Of course it would be impossible to provide a complete coverage of Python, NumPy and Pandas in one chapter. Consider it a brief introduction to get you started. Please check out the Further Reading section for more detailed discussions.

Python

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It is currently the most popular language for machine learning, because of its balance of simplicity, flexibility and performance. Almost all of the deep learning frameworks are implemented with Python on the surface and C, C++ or CUDA (for GPU programming) under the hood. Python is open-source, with thousands of third-party modules (extensions, libraries). These include TensorFlow and PyTorch.

In this subsection, we will provide a brief introduction to the key features of Python that we will use for deep learning. To execute cells in the Jupyter Notebook, click into the cell and press Shift+Enter.

VARIABLES in Python are dynamically typed. If you enter an integer value for a variable, it will be typed as integer, but if you include a decimal point, it will be typed as floating point, as in the following example.

```
a = 2
b = 3
c = a + b
print(c)
```

```
5
```

```
print(a/b)
```

```
0
```

```
d = 3.0
print(a/d)
```

```
0.6666666666666666
```

LISTS: An important compound data type in Python is the *list*. A list can be written as a series of items, separated by commas, inside square brackets. They can be *indexed* (accessing individual elements by their position in the list) and *sliced* (selecting a subset of a list), as in the following examples.

```
a = [2, 4, 6, 8]
print(a[0])
```

```
2
```

```
print(a[-1]) # neg. index cnts from right
```

```
8
```

Indexing in Python begins with 0.

You can access the last element of a list with the index `-1`.

You can add comments to code using the `#` symbol.

Python

```
a[1:3] = [] # remove elements 1 and 2
print(a)

-----

[2, 8]
```

TUPLES: A tuple is similar to a list, but the tuple is *immutable*, which means that it cannot be changed after it is created. The list is mutable, which means that it is not write-protected, and therefore not as safe as a tuple. We will sometimes use tuples to pass inputs and targets into a training function.

```
a = 2, 4, 6, 8
print(a)

-----

(2, 4, 6, 8)
```

A tuple is a list of items separated by a comma.

```
print(a[0])

-----

2
```

```
a[0] = 9

-----

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

This error occurs when you try to change a tuple.

DICTIONARIES are another useful data type in Python. A dictionary is a set of key-value pairs, where the key can be any immutable object (object that cannot be changed). A dictionary can be created by putting a comma-separated list of key:value pairs inside a set of braces. Here we create a dictionary that contains inputs and targets that could be used to train a neural network.

```
dict = {'input': [1, 3, 7], 'target': [3, 7, 15]}
print(dict['input'])

-----

[1, 3, 7]
```

You can access a value by using the key.

```
print(dict.items())
```

```
[('input', [1, 3, 7]), ('target', [3, 7, 15])]
```

```
print(dict.keys())
```

```
['input', 'target']
```

IF STATEMENT: In addition to calculator functions, Python has the usual control flow tools that are found in other programming languages, such as the *if* statement. The first *if* can be followed by an *else*, or any number of *elif*'s, or it can be used alone. Unlike some other languages, Python uses indentation to group statements, so there is no end statement. Use the tab or spaces to create the indentation, but be careful to keep the exact spacing, and use a colon (:) at the end of the *if*, *elif* and *else* lines. If in interactive mode, leave a blank line at the end.

The standard Python convention is to use 4 spaces for indenting and not to use tabs.

```
x = -5
if x<0:
    y = -1
elif x>0:
    y = 1
else:
    y = 0
print(y)

-1
```

WHILE LOOP: Another control flow tool is the *while* loop, as shown in the following example.

Python

```
a = [1, 2, 3, 4]
i, x = 0, 0 #Multiple assignment
while i < len(a):
    x = x + a[i]
    i = i + 1

print(x)
```

10

Notice that Python allows multiple variables to be assigned on the same line.

FOR LOOP: Python also has *for* loops that can be used to iterate over the items of any list.

```
x = 0
for num in a:
    x = x + num

print(x)
```

10

The index variable does not have to be an integer.

```
a = ['one', 'two', 'three', 'four']
for q in a:
    print(q)
```

one
two
three
four

For loops can also be used in *list comprehensions* to create lists.

```
a = [1, 2, 3, 4]
b = [x**2 for x in a]
print(b)
```

[1, 4, 9, 16]

LOGICAL OPERATORS: The logical operators in Python are and, or and not.

```
x, y = True, False
print(x and y)
print(x or y)
print(not x)
-----
False
True
False
```

FUNCTIONS: The keyword `def` introduces the function definition. It is followed by the function name and parenthetical parameters. The body must be indented.

```
def f(y,q):
    return [y*y, 1/q]
```

```
zw = f(4.0,2.0)
print(zw)
-----
[16.0, 0.5]
```

The return line indicates which variables will be returned from the function.

ITERATORS AND GENERATORS: Python *containers*, like list, dict and tuple, are *iterables*, which means that they can be iterated over. Technically, the `for` statement calls the `iter()` method with the container as the argument, which returns an *iterator* object. The iterator object has a method `__next__()` that accesses elements in the container one at a time. An iterator keeps an internal state, so it knows how to compute the next element.

The operation of iterating over objects in a container is important for training neural networks, since we generally need to bring data into the training process in an iterative way. The procedures for doing this will be critical, especially when data sets are too large to fit into memory at one time, or when processing is to be distributed across multiple CPUs and GPUs.

Python

A [generator](#) is a type of iterator. It is written like a function, but with return replaced by yield . The yield statement pauses the function so that it can resume where it left off. Here is a generator that returns elements of a list two at a time.

```
def bytwo(x):  
    n = len(x)  
    for i in range(0, n-1, 2):  
        yield x[i:i+2]
```

The range function generates arithmetic progressions from an initial value to a final value by some increment.

```
a=[1, 2, 3, 4]  
zz = bytwo(a)  
for qq in zz:  
    print(qq)
```

```
[1, 2]  
[3, 4]
```

MODULES: After an interactive Python session is closed, all definitions are lost. In order to save definitions, they can be written to a file and then later imported. This type of file is called a [module](#). Definitions in a module can be imported into other modules or the main function. For example, assume the file logic.py contains the following.

```
def a(x,y):  
    print(x and y)  
  
def o(x,y):  
    print(x or y)
```

We can call this module's functions with the following syntax.

```
import logic  
print(logic.a(True,False))  
print(logic.o(True,False))
```

```
False  
True
```

Later we will be importing definitions from TensorFlow and PyTorch.

CLASSES are a way to combine data and functionality. Constructing a new class makes a new object and associated *methods* that can modify the object. For example, the following commands define a new object, which is a simple one-layer network with one input and one neuron. It has a simulation method, which defines how the network operates.

Methods are functions that are associated with a specific class of objects.

```
class simplenet:
    def __init__(self,weight,bias):
        self.w = weight
        self.b = bias

    def sim(self,p):
        return self.w*p + self.b
```

The `__init__` method is executed when an instance is created.

The `sim` function is a method for the `simplenet` class.

You can then create an instance of the class `simplenet` and simulate it.

```
net = simplenet(4.0,2.0)
print(net.sim(3.0))
```

```
14.0
```

NumPy

NumPy is the fundamental module for scientific computing in Python. NumPy functionality works conveniently with the deep learning frameworks we cover in this book, so let's consider some of its key components and concepts.

The key object in NumPy is the multidimensional array (tensor), which can be used to hold inputs and outputs of neural networks. An array can be constructed using the `array` function. The key attributes of an array are: `ndim` – the number of dimensions, `shape` – the sizes of each dimension, and `dtype` – the type of the elements in the array.

Python

```
import numpy as np
x = np.array([[1, 2, 3],[4, 5, 6]])
print(x)
print(x.ndim)
print(x.shape)
print(x.dtype)
```

```
[[1 2 3]
 [4 5 6]]
2
(2, 3)
int64
```

ARRAY OPERATIONS: A number of operations can be performed between arrays: adding, subtracting, multiplying (dot and Hadamard) and dividing. To demonstrate, create some arrays of various sizes.

```
a = np.arange(6)
print(a)
```

```
[0 1 2 3 4 5]
```

The `arange` function returns evenly spaced values in a range. If we use only one argument, the initial value is zero, the increment is one and the argument is the total number of values.

```
b = a.reshape(2,3)
print(b)
c = np.arange(0,12,2).reshape(2,3)
print(c)
d = np.arange(0,24,4).reshape(3,2)
print(d)
```

```
[[0 1 2]
 [3 4 5]]
[[ 0  2  4]
 [ 6  8 10]]
[[ 0  4]
 [ 8 12]
 [16 20]]
```

The `reshape` method rearranges the elements in the array to have the specified numbers of elements in each dimension.

```
e = np.arange(3)
print(e)

[0 1 2]
```

The first dimension can be considered to be the number of rows, so `e` is a column vector, although displayed as a row here.

There are two types of array multiplication. The first is a Hadamard (or element-by-element) multiplication. The second is standard matrix multiplication.

```
print(b*c)

[[ 0  2  8]
 [18 32 50]]
```

The `*` represents Hadamard multiplication. The two arrays should be the same size.

```
print(np.matmul(c,d))

[[ 80 104]
 [224 320]]
```

For standard matrix multiplications (same as dot product for two dimensional arrays), columns of `c` must match rows of `d`.

We can also add arrays. Notice that in the second example the two arrays are not the same dimension.

```
print(b+c)
print(b+e)

[[ 0  3  6]
 [ 9 12 15]]
[[0 2 4]
 [3 5 7]]
```

When the arrays are not the same dimension, the smaller array is *broadcast* across the larger array so that they have compatible shapes.

INDEXING AND SLICING: As with lists, you can access individual elements of an array with *indexing* or subsets of an array with *slicing*.

Python

```
print(b[1, 2])
print(b[0])
print(b[[0, 1],[1, 2]])
```

```
5
[0 1 2]
[1 5]
```

The basic slice syntax is `i:j:k` where `i` is the starting index, `j` is the stopping index, and `k` is the step.

```
print(a[0:5:2])
print(a[:5])
print(a[-4:])
```

```
[0 2 4]
[0 1 2 3 4]
[2 3 4 5]
```

It is also possible to compute the sum or product of values of an array across various dimensions.

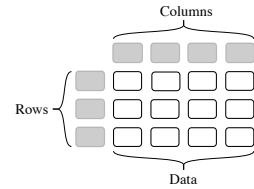
```
print(np.sum(b))
print(np.sum(b,axis=0))
print(np.prod(b,axis=1))
```

```
15
[3 5 7]
[ 0 60]
```

Pandas

Much of the effort (sometimes more than 90%) in any deep learning workflow is devoted to *data wrangling*. This involves loading, formatting and preprocessing the data before it is used to train the deep learning model. A Python module that is commonly used for data wrangling is *Pandas*. Pandas is an open source Python package (built on top of NumPy) that provides data structures as well as data analysis and manipulation tools. In this subsection we summarize some of the most useful features of Pandas.

THE DATAFRAME: The main data structure of Pandas is *DataFrame*. *DataFrame* is a 2-dimensional structure with columns of potentially different types. It is similar to a spreadsheet.



LOADING THE DATA: Pandas data frames are normally created by reading the data in from a file. At the time of this writing, Pandas can read in 14 different types of files:

1. Comma-separated values (CSV)
2. XLSX
3. ZIP
4. Plain Text (txt)
5. JSON
6. XML
7. HTML
8. Images
9. Hierarchical Data Format
10. PDF
11. DOCX
12. MP3
13. MP4
14. SQL

To illustrate how the process works, we will load a sample .csv file. (The file contains information about how patients performed on a particular medical test, as we will see later.) All of the Pandas methods for reading data files begin with the characters `read_`. For .csv files the command is `read_csv`.

```
import pandas as pd
sample_df = pd.read_csv('SampleDF.csv')
```

GETTING BASIC INFO: Now that we have loaded a data frame, let's use some simple commands to investigate its structure and contents. To find the shape of the *DataFrame* we can use `shape`.

This is a small part of a larger file that was used as part of a Kaggle [competition](https://github.com/NNDesignDeepLearning/NNDesignDeepLearning). The sample file is `SamplePulmonary.csv`, and can be downloaded from the textbook companion website <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning>.

Python

```
print(sample_df.shape)
```

```
(100, 7)
```

The DataFrame has 100 rows and 7 columns. Each row corresponds to a patient's test result. We can find the names of the columns using `columns`.

```
print(sample_df.columns)
```

```
Index(['Patient', 'Weeks', 'FVC', 'Percent', 'Age', 'Sex',  
      ↪ 'SmokingStatus'], dtype='object')
```

The first column is a patient id number, followed by the week in which the test was performed, the value of the test, the rating of the test as a percentage of an average patient in the same category, age, sex and smoking status (current, previous or never).

It is also possible to get a description of the DataFrame using the `describe()` method.

```
print(sample_df.describe())
```

	Weeks	FVC	Percent	Age
count	100.00000	100.000000	100.000000	100.000000
mean	35.89000	2759.820000	78.187965	67.110000
std	24.90868	925.766484	21.094723	6.738844
min	0.00000	969.000000	43.352279	49.000000
25%	16.50000	2118.000000	62.821569	64.000000
50%	32.00000	2597.500000	73.989508	68.000000
75%	49.00000	3267.000000	89.005946	72.000000
max	116.00000	5768.000000	153.145378	87.000000

This provides statistics on all of the numerical columns, including minimum and maximum values, mean and standard deviation, and quartiles.

We can also get some summary information using the `info` method.

```
sample_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Patient         100 non-null   object
 1   Weeks           100 non-null   int64
 2   FVC             100 non-null   int64
 3   Percent         100 non-null   float64
 4   Age             100 non-null   int64
 5   Sex             100 non-null   object
 6   SmokingStatus   100 non-null   object
dtypes: float64(1), int64(3), object(3)
memory usage: 5.6+ KB
```

It provides information about each of the columns, including the dtype and the number of missing cells.

In addition to getting statistics on the DataFrame, we can also look at some parts of the DataFrame by using the `head()` and `tail()` methods. The `head()` method shows the first few rows of the DataFrame, and `tail()` shows the last few rows.

```
print(sample_df.head())
```

	Patient	Weeks	FVC	Percent	Age	Sex	\
0	ID00213	32	2972	81.828194	70	Male	
1	ID00129	0	2253	59.622102	71	Male	
2	ID00130	12	1648	68.116062	65	Female	
3	ID00225	23	969	49.075715	77	Female	
4	ID00082	33	2885	98.666211	49	Female	

	SmokingStatus
0	Currently smokes
1	Never smoked
2	Never smoked
3	Never smoked
4	Currently smokes

Python

INDEXING AND SLICING: Data can be extracted from a DataFrame in a number of ways. For example, you can work with a single column. To select the column, provide the column name between square brackets – []. The result is a Pandas [Series](#).

```
fvc = sample_df['FVC']
fvc.describe()
```

count	100.000000
mean	2759.820000
std	925.766484
min	969.000000
25%	2118.000000
50%	2597.500000
75%	3267.000000
max	5768.000000

Name: FVC, dtype: float64

If you want to select multiple columns, you can enter a list of column names within the brackets. The returned object is a DataFrame:

```
twocol = sample_df[['Age', 'FVC']]
print(twocol.head())
```

	Age	FVC
0	70	2972
1	71	2253
2	65	1648
3	77	969
4	49	2885

It is also possible to select rows of a DataFrame. For example, you can consider only patients older than 75.

```
older = sample_df[sample_df['Age']>75]
print(older.shape)
```

(8, 7)

The condition inside the brackets – `sample_df['Age']>75` – checks which rows in the Age column have value greater than 75. The resulting shape shows that there are 8 patients older than 75.

If you want to access a select set of rows and columns, you need to use the `loc/iloc` operators. If you are using column names, row labels or a condition expression, use the `loc` operator. When accessing rows and/or columns based on their position in the table, use the `iloc` operator. For example, if you want to access the Age and FVC value of patients over 75, you could use the following commands.

```
older_fvc = sample_df.loc[sample_df['Age']>75, ['Age', '
↪ FVC']]
print(older_fvc.head())
```

	Age	FVC
3	77	969
7	83	3171
16	87	2220
59	77	1550
82	77	1818

The part before the comma in the selection bracket represents the rows you want, and the part after the comma represents the columns you want.

Here is an example of using the `iloc` operator to access rows 3 through 5 and columns 2 to 3.

```
sample_subset = sample_df.iloc[2:5, 1:3]
print(sample_subset.head())
```

	Weeks	FVC
2	12	1648
3	23	969
4	33	2885

RESHAPING: It is often useful to reshape a DataFrame in order to make the data compatible with a neural network input or target structure. There are several methods for reshaping DataFrames. We will touch on just a few popular ones.

Python

The Pandas `pivot()` method takes three columns of the DataFrame as input arguments and then produces a new data frame that uses the three columns as index, columns and values. For example, we can take the Patient column to be the index, the Weeks column to be the columns and the FVC column to be the values, as in the following.

```
pivoted = sample_df.pivot(index='Patient', values = 'FVC',  
    ↪ columns='Weeks')  
print(pivoted.iloc[:5, :10])
```

Weeks	0	2	4	5	6	7	9	10	12	13
Patient										
ID000116	NaN	NaN	NaN	NaN	NaN	NaN	3541.0	NaN	NaN	3410.0
ID000156	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ID000206	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ID000276	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2472.0	NaN
ID000306	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

There is a column for every week that is indicated in the data set, and there is a row for every patient. If there is no FVC value for a certain patient in a given week in the original DataFrame, then the cell is filled with NaN.

Another reshaping method is `melt()`. It is used to change the DataFrame format from wide to long. One or more of the original columns are used as identifiers, and one or more remaining columns become one value column. For example, we can use Patient as the identifier, and Age as the value variable.

```
melted = sample_df.melt(id_vars='Patient', value_vars='Age')  
print(melted.head())
```

	Patient	variable	value
0	ID00213637202257692916109	Age	70
1	ID00129637202219868188000	Age	71
2	ID00130637202220059448013	Age	65
3	ID00225637202259339837603	Age	77
4	ID00082637202201836229724	Age	49

OPERATIONS ON DATAFRAMES: Before applying data to neural network training, we often need to perform various preprocessing operations on the data. One way to do this would be to use for loops to iterate over the elements of the DataFrame. However, there are Pandas methods that are much more efficient. A very useful Pandas method is `apply()`. It can be used to apply a function along an axis of the DataFrame. There are two arguments to the `apply()` method: the function you want to apply, and the axis you want to apply it to.

For example, consider the DataFrame `old_fvc` that we created earlier. If we want to find the average 'Age' and 'FVC' values, we can apply the NumPy `mean` function to the columns of the DataFrame.

```
print(older_fvc.apply(np.mean, axis=0))
```

```
Age      79.125
FVC     2284.500
dtype: float64
```

In addition to using built-in functions, you can also apply user-defined functions. In the example below we pass a Python `lambda` function to the `apply()` method. The first argument of `apply()` is the function to apply. This can be a built-in function, or a user-defined function. If the function is simple, it can be passed as a `lambda` function, which can be defined inline. A `lambda` expression consists of the *keyword* `lambda`, followed by the *bound variable* name, followed by a colon, and then completed by the *body* of the function. In the example below the bound variable is `x`, and the body is `x.max() - x.min()`. The `apply()` operation will compute the range of values (max minus min) in each column of the old DataFrame.

```
print(older_fvc.apply(lambda x: x.max() - x.min(), axis=0)
      ↪ )
```

```
Age      11
FVC     3081
dtype: int64
```

Python

The `apply()` method can be used to apply any previously defined function. There are also built-in functions that can be used directly. For example, the `min()` method is applied below to find the minimum value in each column of the data frame.

```
print(older_fvc.min(axis=0))
```

```
Age      76  
FVC     969  
dtype: int64
```

There are many more built-in DataFrame methods, including `max()`, `sum()`, `mean()`, `medium()`, `std()`, etc.

Epilogue

Most of the deep learning frameworks, like TensorFlow and PyTorch, can be accessed using Python. This chapter has provided a brief introduction to Python and two of its commonly used packages – NumPy and Pandas. This chapter is meant only as a quick introduction to Python, which will be used extensively in the next chapter on TensorFlow. For more in-depth coverage of this chapters' topics, see the Further Reading section on the next page.

Python

Further Reading

[Python, 2019a] Software Foundation Python. Welcome to python.org. <https://www.python.org/>, 2019a. Accessed: 2019-06-24

This is the main web page for Python. From there you can download the latest releases and access documentation

[Python, 2019b] Software Foundation Python. The python tutorial. <https://docs.python.org/3/tutorial/index.html>, 2019b. Accessed: 2019-06-24

This is the official Python Tutorial. It is an excellent introduction to all of the features of Python.

[NumPy, 2019a] Developers NumPy. Numpy. <https://www.numpy.org/>, 2019a. Accessed: 2019-06-24

The main web page for NumPy. From here you can access downloads and documentation.

[NumPy, 2019b] Developers NumPy. Quickstart tutorial - numpy. <https://www.numpy.org/devdocs/user/quickstart.html>, 2019b. Accessed: 2019-06-24

The official NumPy tutorial. It covers the basic NumPy features and gives numerous examples.

[Pandas, 2021] Pandas. Pandas user guide. https://pandas.pydata.org/pandas-docs/stable/user_guide/index.html, 2021. Accessed: 2021-07-14

The official Pandas User Guide. It has a quick 10 minute introduction, as well as complete documentation of all features.

Labs

The labs that go with this chapter can be found at <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/tree/master/NNDesignDeepLearning/05.PythonChapter/Code/Labs>. Some of the labs will be in Jupyter Notebooks. You can open the labs and run them for free in Google Colab, if you have a gmail account. You can also download and run them on your personal computer.

6 *TensorFlow Introduction*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Loading the Data</i>	2
<i>Constructing the Model</i>	4
<i>Training the Network</i>	5
<i>Advanced Data Loading</i>	7
<i>Epilogue</i>	11
<i>Further Reading</i>	12
<i>Labs</i>	13

Objective

This chapter provides an introduction to the TensorFlow framework for deep learning. There are different interfaces that can be used to access TensorFlow. In this chapter we begin with the simplest one. We will show how you can quickly get started to perform all aspects of the typical deep learning workflow: load and preprocess data, build a neural network, train the network and evaluate the results.

This chapter is available as an interactive Jupyter Notebook at the website <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/blob/master/06.TensorFlowIntroChapter/Code/ChapterNotebook/TensorFlowIntro.ipynb>

Theory and Examples

TensorFlow is one of the most popular deep learning frameworks. It was originally developed by the Google Brain group starting in 2011 as DistBelief. It was further developed into TensorFlow and released as open source software in November of 2015. TensorFlow 2.0 became officially available in Sep 2019. It is written in Python and C++.

There is more than one API (Application Programming Interface) for TensorFlow. We will mainly focus on the *Keras* API in this introduction.

Keras was originally developed independently from TensorFlow. According to its developer, François Chollet, Keras was more of an interface than a framework. This is because Keras was originally designed to act as a frontend for some of the other frameworks. In particular, the TensorFlow, Theano and CNTK frameworks can currently be backends for Keras, and others may be implemented in the future.

Keras was developed during 2014 and 2015, as part of a research project with ONEIROS. It started as an interface to Theano (a predecessor framework to TensorFlow) that enabled the use of recurrent neural networks (RNNs) and convolution neural networks. Its first release was in March of 2015. François Chollet later joined Google, and in 2017 the Google TensorFlow team decided to support Keras in TensorFlow's core library. For TensorFlow 2.0, released in 2019, Keras is the central API for TensorFlow.

Loading the Data

The first step in using any framework is to load, format and preprocess the data set. For Keras, the data can be loaded using NumPy arrays (tensors). For other frameworks the data can be conveniently converted from a NumPy array to a similar tensor format before training. In each case, the important thing to know is what each axis of the tensor represents.

For standard multilayer networks the input is a vector with R elements, which we often refer to as *features*. The training data set will have Q *samples* of the input vector. For Keras this can be stored as a (Q, R) , or (samples, features), tensor.

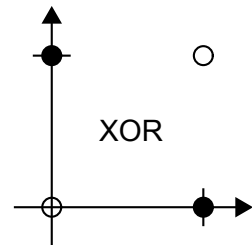
For other types of networks that we will discuss in later chapters the input data can be a *time series*. In this case, the Keras input will be a 3D tensor of the form (samples, timesteps, features). The term *samples* here indicates the number of different time series in the data set, each of which would be of length *timesteps*.

Some networks can also accommodate images, which usually have three dimensions: height, width and color. The 4D input tensors for the network have the form (samples, height, width, color). The color (also called *channel*) dimension is usually three (red, green and blue), but can be one for grayscale images or greater than three for multi-spectral images. (It is possible to have the channel axis come before the height and width axes, but channel-last is the default.)

It is also possible for network inputs to be a time series of images – a video. The 5D input tensors in this case have the form (samples, timesteps, height, width, channels).

In Keras, the network outputs will also be NumPy tensors, which should match the targets. The dimensions for the targets are assigned in the same order as the input dimensions, although the shape of the target is not usually the same as the shape of the input. We will have other examples in the labs and in the case studies.

For simplicity, let's begin by generating data for the XOR problem. This is a binary classification problem with two dimensional inputs. If the two elements of the input vector are equal, the input is from class 1, and if the two elements are different, the input is from class 2. Here we define the inputs and targets.



TensorFlow Introduction

```
from tensorflow.keras.utils import to_categorical
p = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
t = np.array([0, 1, 1, 0])
t = to_categorical(t)
print(t)
```

```
[[1.  0.]
 [0.  1.]
 [0.  1.]
 [1.  0.]]
```

The `to_categorical` function is convenient for creating a one hot encoding of the targets.

Constructing the Model

The core data structure in Keras is the *model*. The model is a way to organize layers of a network. There are three ways to define a model: 1) the sequential class, 2) the functional API, and 3) the model subclass.

To use the *sequential class*, you begin by creating an instance of the class, and then use the method `add` to add layers, as in the following example. Here we create a two-layer network with 10 neurons in the hidden layer and a tanh activation function. The network architecture is 2-10-2.

The sequential class is designed for networks where each layer follows the previous one.

```
from tensorflow.keras import models
from tensorflow.keras import layers
model = models.Sequential()
model.add(layers.Dense(10, activation='tanh', input_shape
    ↪=(2,)))
model.add(layers.Dense(2, activation='softmax'))
```

The Dense layer is standard matrix multiplication.

Notice that for the first layer we need to assign the input size. For the following layers, the input size can be determined from the size of the previous layers. The final layer uses two softmax neurons, which determine two classes.

An API (Application Programming Interface) is a set of methods for communicating between software components. In this case, Keras is communicating with TensorFlow and cuda libraries. There are often several APIs that can be used. For the *Functional API* of Keras, the layers of a network are defined individually with their specific inputs and outputs. For example, the following code implements the same 2-10-2 network using the Functional API.

```
p = layers.Input(shape=(2,))
a1 = layers.Dense(10, activation='tanh')(p)
a2 = layers.Dense(2, activation='softmax')(a1)
model = models.Model(inputs=p, outputs=a2)
```

When you use the *model subclass method*, you create your own fully-customizable models by subclassing the Model class and implementing your own forward pass in the call method. The following code implements the same 2-10-2 network using the model subclass method.

```
class Twolayer(models.Model):
    def __init__(self):
        super(Twolayer, self).__init__()
        self.dense1 = layers.Dense(10, activation='tanh')
        self.dense2 = layers.Dense(2, activation='softmax'
        ↪ )
    def call(self, inputs):
        x = self.dense1(inputs)
        output = self.dense2(x)
        return output

model = Twolayer()
```

The size of the input will not be defined until the model is trained with the fit command.

Training the Network

After the data has been loaded and the network has been created, the next step is to train the network. In the following we will cover the basic training steps. In the case study chapters we will provide more detail.

TensorFlow Introduction

Before training a network, the model needs to be compiled. During compilation you assign the training algorithm and performance (loss) function. In the following example we use the Adam training algorithm and the cross entropy performance function.

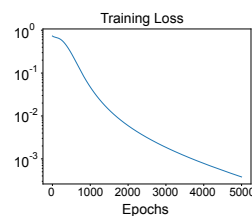
```
model.compile(optimizer='adam',loss='  
↪ categorical_crossentropy')
```

The next step is to train the network using the `fit` method. We supply the inputs and the targets and specify the number of *epochs* for training. An epoch refers to a section of the training process during which the entire training data set is used. A related term is *iteration*, which refers to the updating of the network weights and biases. As we discussed in Chapter 3, the weights can be updated in three ways: 1) one sample at a time (as in the stochastic gradient descent algorithm), 2) on the full training set as a whole (batch training) or 3) on minibatches of the training set. If full batch training is used, then the number of iterations is the same as the number of epochs. However, if minibatches are used, the number of iterations will be larger than the number of epochs. In the example below, we set the number of epochs to 5000 and the batch size to 4. Since the batch size is the same as the size of the full data set, the number of iterations will equal the number of epochs.

```
history = model.fit(p,t,epochs=5000,batch_size=4)
```

The returned history contains information about the progress of the training process. After training is complete, we can plot the progress of the loss function during training.

```
import matplotlib.pyplot as plt  
history_dict = history.history  
loss_values = history_dict['loss']  
epochs = history.epoch  
plt.semilogy(epochs, loss_values)  
plt.title('Training_Loss')  
plt.xlabel('Epochs')  
plt.show()
```



The `history.history` object is a dictionary that contains a history of the loss function, and `history.epoch` contains a list of the epoch numbers.

The predict method for the trained model can be used to calculate the network response to an arbitrary set of inputs. Here we apply the training inputs to the model to check the response.

```
print(model.predict(p))  
-----  
[[9.9955982e-01 4.4020030e-04]  
 [5.6742248e-04 9.9943250e-01]  
 [4.5553621e-04 9.9954444e-01]  
 [9.9940550e-01 5.9446518e-04]]
```

Note

Our example code does not indicate whether the program will execute on a GPU or a CPU. In TensorFlow, the program will automatically run on a GPU, if one is available. If you specifically want some code to run on a CPU, even though a GPU is available, you can manually do that using the following line:

```
with tf.device('/cpu:0')
```

Advanced Data Loading

Loading/formatting/pre-processing data is one of the most important parts of the deep learning workflow. Sometimes this process is referred to as *Extract, Transform and Load* (ETL). First, the data are taken from one or multiple files, which may be distributed across multiple machines. Next, the data is transformed. This process can be as simple as normalizing the data, or may involve augmenting data by rotating or scaling images, adding noise, etc. Finally, the data is loaded into the training process, often in minibatches.

Unfortunately, this ETL process can vary significantly from one application to another, so it is difficult to cover all the options. We will give detailed examples for several specific applications in our case study chapters. However, let's take a quick look at some of the advanced data loading concepts.

TensorFlow Introduction

For more sophisticated data loading, you can make the first argument to the `fit` method a *data generator*. The data generator can be a Python generator, which we described in the Python chapter, or an instance of the `tensorflow.keras.utils.Sequence` class. This class of object is designed for loading data sets. It is expected to have at least two methods: `__getitem__`, which returns the next batch of data, and `__len__`, which returns the number of examples in the data set. It can also have an `on_epoch_end` method, if the data set needs to be modified in some way at each epoch (e.g., by shuffling the data).

There are a number of reasons for using a data generator, instead of passing the entire training set into the `fit` method as a NumPy array. First, the data may be too large to fit into memory, so they may need to be loaded one minibatch at a time. Also, you may want to distribute the computation across multiple processors, which can be done conveniently using the `tensorflow.keras.utils.Sequence` class. In addition, a data loader can modify the data during training (e.g., shuffle data or augment data using transformations or noise).

TENSORFLOW DATASET: TensorFlow has a very useful API for creating input pipelines: `tf.data.Dataset`. These pipelines can also be passed to the `fit` method instead of a data generator. There are many ways to use `tf.data.Dataset`, and we will cover several of these in Chapter 11 and in the Case Study chapters. To give an idea of how `tf.data.Dataset` can be used, let's consider one method from the API: `from_tensor_slices`. This method creates a `Dataset` from a NumPy array, or other types of data structures, like lists and dictionaries. To illustrate the operation, we'll work with the CSV file that we used in the Python chapter and lab.

First, we read the CSV file into a `DataFrame`.

```
import pandas as pd
sample_df = pd.read_csv('SampleDF.csv')
```

Next, we extract two columns that we will use as inputs and targets.

```
P = np.array(sample_df['FVC'])
T = np.array(sample_df['Percent'])
```

Now we load the data into a Dataset, using `from_tensor_slices`.

```
from tensorflow.data import Dataset
dataset = Dataset.from_tensor_slices((P, T))
```

This Dataset can then be passed to the fit method. The Dataset is an iterable, like a data generator, so we can also access the elements with a for loop.

```
for feat, targ in dataset.take(5):
    print ('Features:_{},_Target:_{}'.format(feat, targ))
```

```
Features: 2972, Target: 81.8281938325991
Features: 2253, Target: 59.622102254684
Features: 1648, Target: 68.1160618335125
Features: 969, Target: 49.07571537097999
Features: 2885, Target: 98.66621067031471
```

Once the data are loaded into the Dataset, there are many useful operations that can be efficiently performed. With the batch method, for example, we can group the Dataset into minibatches. In the following example we group the Dataset into minibatches of size 5.

TensorFlow Introduction

```
dataset = dataset.batch(5)
for feat, targ in dataset.take(5):
    print ('Features:_{},_Target:_{}'.format(feat, targ))
```

```
Features: [2972 2253 1648  969 2885], Target: [81.82819383
  ↪ 59.62210225 68.11606183 49.07571537 98.66621067]
Features: [3045 4791 3171 3350 2833], Target: [
  ↪ 76.91724765 153.14537783  92.15880028  83.59952086
  ↪ 77.21029107]
Features: [4029 3410 3346 4251 1383], Target:
  ↪ [100.26378658  88.15925543  86.50465357
  ↪ 118.74301676  60.20634713]
Features: [3255 2220 1845 2756 1389], Target: [84.27402651
  ↪ 96.92630108 67.90577843 82.55451713 56.68924986]
Features: [2416 2917 3303 2327 4574], Target: [
  ↪ 71.57246119  66.70172871 115.39267747  60.56741281
  ↪ 109.13342241]
```

In Chapter 11 and the case study chapters we will investigate other features of the Dataset that allow us to distribute operations across multiple gpus, incorporate augmentation into the data pipeline, prefetch data, etc.

Epilogue

One of the reasons that the field of deep learning has been so productive has been the availability of open source frameworks like TensorFlow. It is possible to implement and train neural networks with relatively few lines of code, while making efficient use of GPUs for fast training with large networks and data sets. This chapter has given a brief introduction to TensorFlow, so that you can quickly begin to experiment with the multilayer network concepts that we have covered in previous chapters.

The next chapter will put the concepts of the current and previous chapters to the test by presenting a practical case study.

Further Reading

[Chollet, 2019] François Chollet. Home - keras documentation. <https://keras.io/>, 2019. Accessed: 2019-06-24

This is the main web page for Keras. It has a quick (30 second) getting started section, as well as links to more extensive documentation.

[Chollet, 2017] François Chollet. *Deep learning with Python*. Manning Publications Company, 2017

This is a book by François Chollet, the developer of Keras, which provides an introduction to deep learning. It focuses on the concepts behind deep learning, and especially their implementation using Keras

[TensorFlow, 2021a] Developers TensorFlow. Tensorflow user guide. <https://www.tensorflow.org/guide>, 2021a. Accessed: 2021-08-05

The official TensorFlow User's Guide. From here you can access information on all aspects of TensorFlow.

[TensorFlow, 2021c] Developers TensorFlow. Keras api user guide. https://www.tensorflow.org/guide/keras/sequential_model, 2021c. Accessed: 2021-08-05

This is the part of the TensorFlow User's Guide that discusses the tf.keras API. It discusses the Sequential class, the Functional API and model subclassing.

[TensorFlow, 2021b] Developers TensorFlow. tf.data api user guide. <https://www.tensorflow.org/guide/data>, 2021b. Accessed: 2021-08-05

This is the part of the TensorFlow User's Guide that discusses the tf.data API. It discusses all aspects of tf.data.Dataset and how it can be used to create an efficient data pipeline.

Labs

The labs that go with this chapter can be found at <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/tree/master/NNDesignDeepLearning/06.TensorFlowIntroChapter/Code/Labs>. Some of the labs will be in Jupyter Notebooks. You can open the labs and run them for free in Google Colab, if you have a gmail account. You can also download and run them on your personal computer.

7 General Modular Network Notation

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Definition of a Layer</i>	3
<i>Multilayer Perceptron</i>	3
<i>Layered Feedforward Network</i>	4
<i>Dynamic Networks</i>	5
<i>Gradient Calculation for GLFNN</i>	8
<i>Epilogue</i>	13
<i>Further Reading</i>	14
<i>Summary of Results</i>	15
<i>Solved Problems</i>	17
<i>Exercises</i>	22

Objective

As we describe more and more complex types of neural networks, it is very important to have clear, consistent and efficient notation and diagrams. The notation is the mathematical language that is used to precisely express the network operation and training. Good diagrams enable us to use visual reasoning, which is complementary to verbal and mathematical reasoning. Deep networks can be very complex, and we need a range of tools to develop a deep understanding. In this chapter we will introduce some general notation and diagrams that we can use to describe the deep networks that will be presented in later chapters. This notation is an extension of the notation from [NND2](#).

Theory and Examples

A variety of types of diagrams and notation have been used to represent neural networks in the years since their first mathematical formulations in the 1940's by McCulloch and Pitts. One difficulty in finding a unified notation for neural networks is that their development has been so interdisciplinary. Contributions have come from many different research and application areas, each with their own notational tendencies. In addition, developments in the neural network field have come in fits and starts over the years, usually with new starts coming from researchers in new disciplines, often with new notation to propose. The result is that notation and diagrams will often change from one paper to another, making it difficult to see the affinity among seemingly different network types.

For this reason, it is important to find a consistent notation to use for all the networks we present in this text. A useful notation should encompass as much of the neural network field as possible, without becoming overly complex and unwieldy. (Of course, beauty is in the eye of the beholder.) In addition, if possible, a diagram of a neural network should provide all the information we need to implement the network.

In the development of the first edition of [NND2](#) in 1993, we made a significant effort to find a notation and associated diagrams that would be suitable for the neural network field at that time, and that could be expanded in the future. Since we were developing an extensive software framework at that time (and extending it through the next 25 years), we sought to make the diagrams/notation consistent with the code. This encouraged the practicality of the notation. Over the years the software has adapted to accommodate new network components and architectures. Some of the notation supporting the new architectures was presented in [NND2](#).

The following sections will describe an update of the notation from [NND2](#). This updated notation maintains the modularity of previous versions and is based on the concept of a layer.

Definition of a Layer

The general notation that we use in this text revolves around layers and the connections between layers. Chapters 2 and 14 of [NND2](#) introduced these ideas, but here we make extensions to allow more general types of weight and net input operations. The first step is to define what we mean by a layer. Over the years, the meaning of the term "layer" in the neural network community has changed multiple times. In order to develop a structure for which we can efficiently perform gradient calculations, we need a careful and consistent definition of *layer*.

For our purposes in this text, a layer consists of the following parts:

- A set of weight matrices, and associated weight functions, that come into that layer (which may connect from other layers or from external inputs),
- Any tapped delay lines that appear at the input of a weight matrix
- A bias vector,
- A net input function (e.g., a summing junction), and
- An activation function.

Multilayer Perceptron

The simplest example of a network of layers is the multilayer network, which we discussed in Chapter 2 and is shown in Figure 7.1 for a three layer network. In this case the weight functions are matrix multiplications between the weight matrices and the inputs, and the net input functions are summations.

The equations of operation for the multilayer network are

$$\mathbf{a}^0 = \mathbf{p} \quad (7.1)$$

$$\mathbf{n}^{m+1} = \mathbf{W}^{m+1} \mathbf{a}^m + \mathbf{b}^{m+1}, m = 0, 1, \dots, M-1 \quad (7.2)$$

$$\mathbf{a}^m = \mathbf{f}^m(\mathbf{n}^m) \quad (7.3)$$

General Modular Network Notation

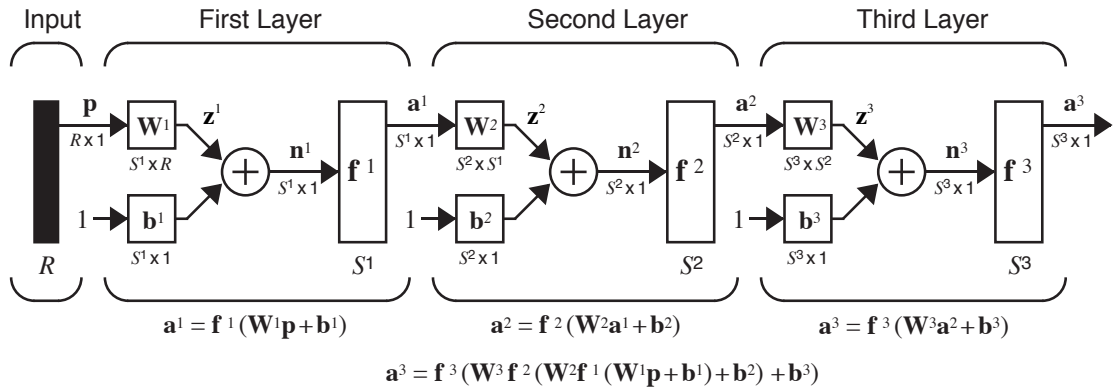


Figure 7.1: Three Layer Network

Layered Feedforward Network

We can generalize the multilayer network by allowing each layer to connect forward to an arbitrary number of other layers. In addition, we can have multiple input vectors, each one of which can be connected to any layer. We call the resulting class of networks *Layered Feedforward Neural Networks* (LFNN). An example is shown in Figure 7.2.

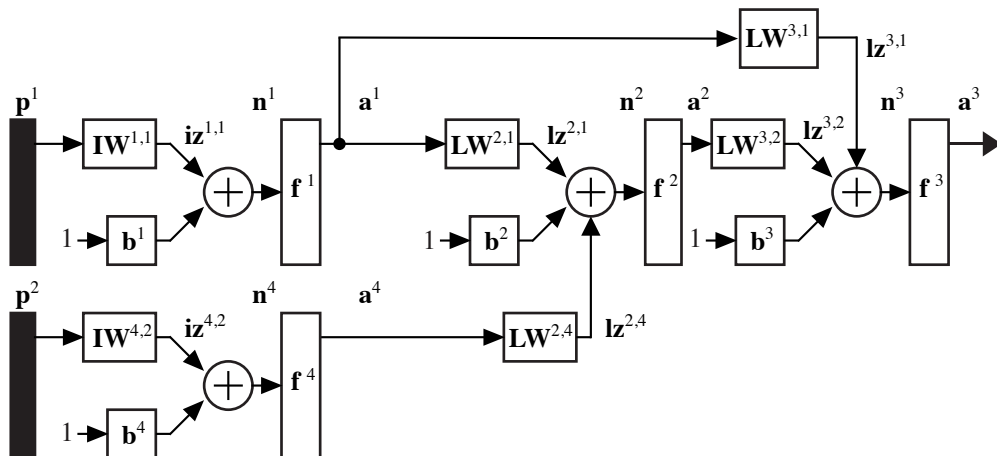


Figure 7.2: Layered Feedforward Network Example

The equations of operation of an LFNN can be written as

$$\mathbf{n}^m = \sum_{l \in I_m} \mathbf{IW}^{m,l} \mathbf{p}^l + \sum_{l \in L_m^f} \mathbf{LW}^{m,l} \mathbf{a}^l + \mathbf{b}^m \quad (7.4)$$

$$\mathbf{a}^m = \mathbf{f}^m(\mathbf{n}^m) \quad (7.5)$$

where $\mathbf{p}^l$ is the l^{th} input vector to the network, $\mathbf{IW}^{m,l}$ is the *input weight* between input l and layer m , $\mathbf{LW}^{m,l}$ is the *layer weight* between layer l and layer m , $\mathbf{b}^m$ is the bias vector for layer m , I_m is the set of indices of input vectors that connect to layer m , and L_m^f is the set of indices of layers that directly connect forward to layer m .

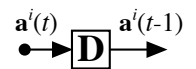
To compute the output of a multilayer network, we start at Layer 1, and proceed in order to Layer M . When we allow general feedforward connections, we need to know what order to compute the layer outputs in Eq. 7.4, so that all of the terms on the right side of the equation are available. We call this order the *Simulation Order*. The simulation order may not be unique; for the network in Figure 7.2, the simulation order could be 1-4-2-3 or 4-1-2-3.

It should be noted that LFNNs, since they are purely feedforward networks, cannot have any feedback loops. If feedback loops existed in a static network, it would not be possible to compute the network output, since a layer output would be needed before it was computed. Feedback can only occur in a dynamic network – a network that contains delays.

Dynamic Networks

The networks that we have discussed to this point in this chapter are static networks. The current outputs of the network can be computed from the current inputs. Such networks have no memory. However, there are many problems, like machine translation and control of dynamic systems, for which it is important for a network to have access to previous inputs, outputs or states. We call such networks *dynamic networks*. (We will devote a later chapter to dynamic networks, but we will describe the notation here.)

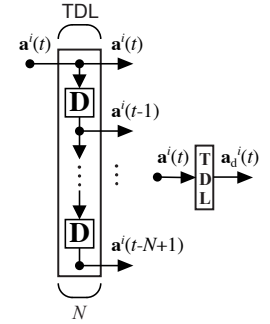
Dynamic networks use delays (for discrete-time, or *digital networks*) or integrators (for continuous-time networks) to actualize memory. In this book we will consider only digital networks. A single delay is shown in the margin. The output of the delay is the input to the delay from the previous time step. When operating a dynamic network, all delays need to be initialized first.



General Modular Network Notation

In the literature on dynamic networks, which are often discussed in the context of recurrent neural networks (RNNs), you will most often see single delays in the network diagrams. However, it is useful on many occasions to stack these delays into a *tapped delay line* (TDL). In this way, a network can have direct access to a history of a given variable. A diagram of a TDL is shown in the margin.

If we generalize the LFNN to include TDLs, which then allows the network to have feedback (recurrent) connections, we have a class of network that we refer to as *Layered Digital Dynamic Networks* (LDDN). (These networks were covered extensively in Chapter 14 of *NND2*.) The equations of operation for these networks are given by



$$\mathbf{n}^m(t) = \sum_{l \in L_m^f} \sum_{d \in DL_{m,l}} \mathbf{LW}^{m,l}(d) \mathbf{a}^l(t-d) + \sum_{l \in I_m} \sum_{d \in DI_{m,l}} \mathbf{IW}^{m,l}(d) \mathbf{p}^l(t-d) + \mathbf{b}^m \quad (7.6)$$

$$\mathbf{a}^m(t) = \mathbf{f}^m(\mathbf{n}^m(t)) \quad (7.7)$$

where most of these expressions have the same meaning as in the LFNN equations, with the addition of $DI_{m,l}$ and $DL_{m,l}$, which represent the sets of the delays to Layer m from Input or Layer l , respectively. Usually, these delays consist of a sequential sequence of numbers from 0 or 1 up to the maximum number of delays. However, the equations allow for any combination of delays.

Unlike the LFNN, an LDDN can have arbitrary connections between layers, but every feedback loop must contain at least one delay. Forward computations are performed forward in time and forward in the simulation order for layers. An example LDDN is shown in Figure 7.3.

The LFNN and the simpler multilayer network are subsets of the more general LDDN, since we can create these networks within the LDDN framework by setting the delays equal to zero. We want to make one more step of generalization by allowing the weight functions and net input functions to be arbitrary operations (as long as they are differentiable), instead of being restricted to be matrix multiplication and addition, respectively. We will refer to this most general formulation as the *Generalized LDDN* (GLDDN).

For the GLDDN, the output of the weight functions (indicated by the letter $\mathbf{z}$) at each layer are computed as follows:

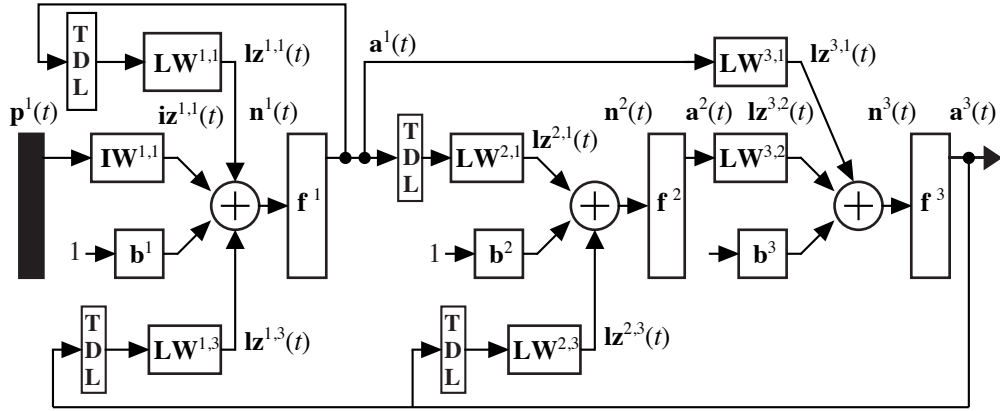


Figure 7.3: Layered Digital Dynamic Network Example

$$\mathbf{iz}^{m,l}(t,d) = \mathbf{ih}^{m,l}(\mathbf{IW}^{m,l}(d), \mathbf{p}^l(t-d)) \quad (7.8)$$

$$\mathbf{lz}^{m,l}(t,d) = \mathbf{lh}^{m,l}(\mathbf{LW}^{m,l}(d), \mathbf{a}^l(t-d)) \quad (7.9)$$

where $\mathbf{ih}^{m,l}()$ and $\mathbf{lh}^{m,l}()$ are input and layer weight functions. They are functions of the corresponding weight matrices $\mathbf{IW}^{m,l}(d)$ and $\mathbf{LW}^{m,l}(d)$, as well as the input $\mathbf{p}^l(t-d)$ or layer output $\mathbf{a}^l(t-d)$. For example, if the weight functions were standard matrix multiplication, then we would have

$$\mathbf{ih}^{m,l}(\mathbf{IW}^{m,l}(d), \mathbf{p}^l(t-d)) = \mathbf{IW}^{m,l}(d) \mathbf{p}^l(t-d) \quad (7.10)$$

The net input for the GLDDN is computed as follows:

$$\mathbf{n}^m(t) = \mathbf{o}^m(\mathbf{iz}^{m,l}(t,d)|_{d \in D_{I_{m,l}}}, \mathbf{lz}^{m,l}(t,d)|_{d \in D_{L_{m,l}}}, \mathbf{b}^m) \quad (7.11)$$

where $\mathbf{o}^m()$ is the net input function at Layer m . For the LDDN the net input is simply computed by summing all of the weight function outputs plus the bias. For the GLDDN, the net input can combine the weight function outputs and the bias in an arbitrary way.

The layer output for the GLDDN is computed in the same way as all of the other formulations:

$$\mathbf{a}^m(t) = \mathbf{f}^m(\mathbf{n}^m(t)) \quad (7.12)$$

The GLDDN layer outputs are computed forward in time and forward in the simulation order for layers, as in the LDDN. An example GLDDN is shown in Figure 7.4.

General Modular Network Notation

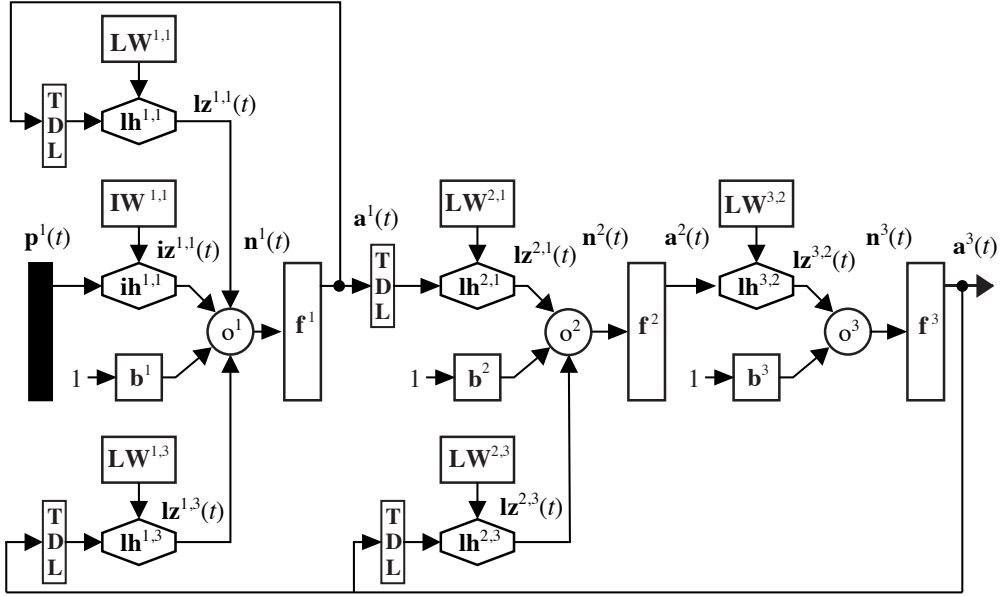


Figure 7.4: General Layered Digital Dynamic Network Example

There are no restrictions on the dimensions of the weight matrices in the GLDDN relative to the dimensions of the input of the weight function and no restriction on the dimension of the bias. Their sizes just need to be consistent with the defined weight and net input functions.

Equations 7.8 through 7.12 for the GLDDN show vector variables. (Our standard notation for a vector is the lowercase bold letter.) However, with a substitution of notation we can interpret these equations as operating on multi-dimensional matrices (tensors). We will take advantage of this in the next chapter on convolution networks.

Gradient Calculation for GLFNN

We are going to investigate GLDDNs in some detail in a later chapter on dynamic networks, including the gradient calculation, which is more complex than for static networks. In this section we want to describe how to compute the gradient for the static version of this network, which would be the Generalized LFNN, or GLFNN. This will be needed in the next chapter on convolution networks.

The gradient for the GLFNN is computed using the standard backpropagation process, except that there may be several backward paths. As we noted in Chapter 3, for multilayer networks the backpropagation process starts with the derivative of the performance (or loss) function $F(\mathbf{x})$ with respect to the net input in the final (output) layer of the network. (See Eq. 3.50.) It starts at the final layer, because the output of that layer is directly compared to the targets in the performance function. In the GLFNN, however, it is possible to have multiple output layers, just as the network can have multiple inputs. We will indicate the set of output layer indices by U . (For the GLFNN networks, a layer is an input layer if it has an input weight. A layer is an output layer if its output will be used in the performance function calculation.)

The elements of the gradient are the derivatives of the performance function with respect to the weights and biases of the network. These derivatives are computed using the chain rule, as in

$$\frac{\partial F(\mathbf{x})}{\partial iw_{i,j}^{m,l}} = \left(\frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m} \right)^T \frac{\partial \mathbf{n}^m}{\partial iw_{i,j}^{m,l}} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial iw_{i,j}^{m,l}} \quad (7.13)$$

$$\mathbf{s}^m \triangleq \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m} \quad (7.14)$$

where we call $\mathbf{s}^m$ the sensitivity, as in Chapter 11 of [NND2](#).

To compute the final term in Eq. 7.13, we need to compute the derivative of the weight function in the following chain rule:

$$\frac{\partial \mathbf{n}^m}{\partial iw_{i,j}^{m,l}} = \frac{\partial \mathbf{n}^m}{\partial (\mathbf{iz}^{m,l})^T} \frac{\partial \mathbf{iz}^{m,l}}{\partial iw_{i,j}^{m,l}} \quad (7.15)$$

$$\frac{\partial \mathbf{n}^m}{\partial lw_{i,j}^{m,l}} = \frac{\partial \mathbf{n}^m}{\partial (\mathbf{lz}^{m,l})^T} \frac{\partial \mathbf{lz}^{m,l}}{\partial lw_{i,j}^{m,l}} \quad (7.16)$$

$$\frac{\partial \mathbf{n}^m}{\partial b_i^m} = \frac{\partial \mathbf{n}^m}{\partial b_i^m} \quad (7.17)$$

The previous equations simplify for the standard multilayer network.

General Modular Network Notation

$$\frac{\partial \mathbf{n}^m}{\partial (\mathbf{iz}^{m,l})^T} = \mathbf{I}, \quad \frac{\partial \mathbf{n}^m}{\partial (\mathbf{lz}^{m,l})^T} = \mathbf{I} \quad (7.18)$$

$$\frac{\partial \mathbf{iz}^{m,l}}{\partial iw_{i,j}^{m,l}} = p_j^l \mathbf{e}_i, \quad \frac{\partial \mathbf{lz}^{m,l}}{\partial lw_{i,j}^{m,l}} = a_j^l \mathbf{e}_i \quad (7.19)$$

$$\frac{\partial \mathbf{n}^m}{\partial b_i^m} = \mathbf{e}_i \quad (7.20)$$

$$(7.21)$$

where $\mathbf{e}_i$ is a vector whose i^{th} element is 1, and the rest of the elements are zero.

The sensitivity $\mathbf{s}^m$ is computed by starting at all $\mathbf{s}^u$, $u \in U$. The performance function $F(\mathbf{x})$ will be an explicit function of these output layers.

$$\mathbf{s}^u = \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^u} = \left(\frac{\partial \mathbf{a}^u}{\partial (\mathbf{n}^u)^T} \right)^T \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^u} = \dot{\mathbf{F}}^u(\mathbf{n}^u)^T \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^u} \quad (7.22)$$

The remaining sensitivities $\mathbf{s}^m$ for $m \notin U$ are computed by following the [backpropagation order](#) (inverse of the simulation order), where L_m^b contains the indices of layers directly connected backward to layer m .

$$\mathbf{s}^m = \left(\frac{\partial \mathbf{a}^m}{\partial (\mathbf{n}^m)^T} \right)^T \sum_{l \in L_m^b} \left(\frac{\partial \mathbf{lz}^{l,m}}{\partial (\mathbf{a}^m)^T} \right)^T \left(\frac{\partial \mathbf{n}^l}{\partial (\mathbf{lz}^{l,m})^T} \right)^T \mathbf{s}^l \quad (7.23)$$

The previous equations simplify for the standard multilayer network:

$$\frac{\partial \mathbf{n}^l}{\partial (\mathbf{lz}^{l,m})^T} = \mathbf{I} \quad (7.24)$$

$$\frac{\partial \mathbf{lz}^{l,m}}{\partial (\mathbf{a}^m)^T} = \mathbf{LW}^{l,m} \quad (7.25)$$

$$\frac{\partial \mathbf{a}^m}{\partial (\mathbf{n}^m)^T} = \dot{\mathbf{F}}^m(\mathbf{n}^m) \quad (7.26)$$

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m)^T \left(\mathbf{LW}^{m+1,m} \right)^T \mathbf{s}^{m+1} \quad (7.27)$$

which is the same as Eq. 3.49.

We can summarize the gradient calculations for the GLFNN as follows. First, we compute the sensitivities at all of the output layers $u \in U$

$$\mathbf{s}^u = \dot{\mathbf{F}}^u(\mathbf{n}^u)^T \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^u} \quad (7.28)$$

Then we compute the remaining sensitivities, $\mathbf{s}^m$, as m follows the backpropagation order (reverse of the simulation order).

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m)^T \sum_{l \in L_m^b} \left(\frac{\partial \mathbf{z}^{l,m}}{\partial (\mathbf{a}^m)^T} \right)^T \left(\frac{\partial \mathbf{n}^l}{\partial (\mathbf{z}^{l,m})^T} \right)^T \mathbf{s}^l \quad (7.29)$$

Finally, we compute the elements of the gradient:

$$\frac{\partial F(\mathbf{x})}{\partial w_{i,j}^{m,l}} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{z}^{m,l})^T} \frac{\partial \mathbf{z}^{m,l}}{\partial w_{i,j}^{m,l}} \quad (7.30)$$

$$\frac{\partial F(\mathbf{x})}{\partial w_{i,j}^{m,l}} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{z}^{m,l})^T} \frac{\partial \mathbf{z}^{m,l}}{\partial w_{i,j}^{m,l}} \quad (7.31)$$

$$\frac{\partial F(\mathbf{x})}{\partial b_i^m} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial b_i^m} \quad (7.32)$$

If we have an LFNN and we use mean square error as the performance function, then the equations simplify to the following:

$$\mathbf{s}^u = -2\dot{\mathbf{F}}^u(\mathbf{n}^u)^T (\mathbf{t}^u - \mathbf{a}^u), u \in U \quad (7.33)$$

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m)^T \sum_{l \in L_m^b} (\mathbf{LW}^{l,m})^T \mathbf{s}^l \quad (7.34)$$

$$\frac{\partial F(\mathbf{x})}{\partial w_{i,j}^{m,l}} = (\mathbf{s}^m)^T p_j^l \mathbf{e}_i = s_i^m p_j^l \quad (7.35)$$

$$\frac{\partial F(\mathbf{x})}{\partial w_{i,j}^{m,l}} = (\mathbf{s}^m)^T a_j^l \mathbf{e}_i = s_i^m a_j^l \quad (7.36)$$

$$\frac{\partial F(\mathbf{x})}{\partial b_i^m} = (\mathbf{s}^m)^T \mathbf{e}_i = s_i^m \quad (7.37)$$

General Modular Network Notation

It is instructive to compare Eq. 7.34 for the LFNN with Eq. 3.49 for the standard multilayer network. Because the LFNN layers can connect forward to several other layers, the backpropagation path needs to sum over all of these connections. In the standard multilayer network each layer only connects forward to one layer, so there is only one term in the backpropagation equation.

Epilogue

The key concept in this chapter is *modularity*. Network outputs are computed one layer at a time in the simulation order. The computation in each layer follows the same order: weight function – net input function – transfer function. The same modularity continues for the computation of the gradient of performance for static networks. It is computed one layer at a time in the backpropagation order. To compute the gradient, for each layer, you need only

- $\dot{\mathbf{F}}^m(\mathbf{n}^m)$ – the derivative of the transfer function
- $\frac{\partial \mathbf{z}^{l,m}}{\partial (\mathbf{a}^m)}$ – the derivative of the weight function with respect to the layer input.
- $\frac{\partial \mathbf{z}^{m,l}}{\partial w_{i,j}^{m,l}}$ – the derivative of the weight function with respect to the weight.
- $\frac{\partial \mathbf{n}^m}{\partial (\mathbf{z}^{m,l})^T}$ – the derivative of the net function with respect to the weight output.
- $\frac{\partial \mathbf{n}^m}{\partial b_i^m}$ – the derivative of the net function with respect to the bias.

In the next chapter we will use the concepts developed in this chapter to introduce the convolution neural network, which takes the form of a GLFNN.

Further Reading

[Hagan et al., 2014] Martin T Hagan, Howard B Demuth, Mark Beale, and Orlando De Jesús. *Neural Network Design (2nd Edition)*. Martin Hagan, 2014

Chapter 14 of this text covers layered digital dynamic networks in detail, including backpropagation through time and real-time recurrent learning algorithms for computing the gradient.

[De Jesús and Hagan, 2007] Orlando De Jesús and Martin T Hagan. Backpropagation algorithms for a broad class of dynamic networks. *IEEE Transactions on Neural Networks*, 18(1):14–27, 2007

This paper covers layered digital dynamic networks. It presents both gradient and Jacobian calculations using both backpropagation through time and real-time recurrent learning. It investigates the complexities of each algorithm as a function of the size of the network and the length of the time sequence.

[Hagan and De Jesús, 2007] Martin T Hagan and Orlando De Jesús. A general framework for dynamic networks. In *Research in Computing Science*, volume 15, pages 3–12, 2007

This paper generalizes the layered digital dynamic network and derives the real-time recurrent learning algorithm for gradient calculation.

[De Jesús and Hagan, 2008] Orlando De Jesús and Martin T Hagan. Backpropagation through time for general dynamic networks. In *IC-AI*, pages 45–51, 2008

This paper generalizes the layered digital dynamic network and derives the backpropagation through time algorithm for gradient calculation.

Summary of Results

Definition of a Layer

- A set of weight matrices, and associated weight functions, that come into that layer (which may connect from other layers or from external inputs),
- Any tapped delay lines that appear at the input of a weight matrix
- A bias vector,
- A net input function (e.g., a summing junction), and
- An activation function.

Multilayer Network Equations

$$\begin{aligned}\mathbf{a}^0 &= \mathbf{p} \\ \mathbf{n}^{m+1} &= \mathbf{W}^{m+1} \mathbf{a}^m + \mathbf{b}^{m+1}, m = 0, 1, \dots, M-1 \\ \mathbf{a}^m &= \mathbf{f}^m(\mathbf{n}^m)\end{aligned}$$

Layered Feedforward Network Equations

$$\begin{aligned}\mathbf{n}^m &= \sum_{i \in I_m} \mathbf{IW}^{m,l} \mathbf{p}^l + \sum_{i \in L_m^f} \mathbf{LW}^{m,l} \mathbf{a}^l + \mathbf{b}^m \\ \mathbf{a}^m &= \mathbf{f}^m(\mathbf{n}^m)\end{aligned}$$

where $\mathbf{p}^l$ is the l^{th} input vector to the network, $\mathbf{IW}^{m,l}$ is the input weight between input l and layer m , $\mathbf{LW}^{m,l}$ is the layer weight between layer l and layer m , $\mathbf{b}^m$ is the bias vector for layer m , I_m is the set of indices of input vectors that connect to layer m , and L_m^f is the set of indices of layers that directly connect forward to layer m .

General Modular Network Notation

Layered Digital Dynamic Network Equations

$$\mathbf{n}^m(t) = \sum_{l \in L_m^f} \sum_{d \in DL_{m,l}} \mathbf{LW}^{m,l}(d) \mathbf{a}^l(t-d) \quad (7.38)$$

$$+ \sum_{l \in I_m} \sum_{d \in DI_{m,l}} \mathbf{IW}^{m,l}(d) \mathbf{p}^l(t-d) + \mathbf{b}^m \quad (7.39)$$

$$\mathbf{a}^m(t) = \mathbf{f}^m(\mathbf{n}^m(t)) \quad (7.40)$$

where $DI_{m,l}$ and $DL_{m,l}$ represent the sets of the delays to Layer m from Input or Layer l , respectively.

Generalized Layered Digital Dynamic Network Equations

$$\mathbf{iz}^{m,l}(t, d) = \mathbf{ih}^{m,l}(\mathbf{IW}^{m,l}(d), \mathbf{p}^l(t-d))$$

$$\mathbf{lz}^{m,l}(t, d) = \mathbf{lh}^{m,l}(\mathbf{LW}^{m,l}(d), \mathbf{a}^l(t-d))$$

$$\mathbf{n}^m(t) = \mathbf{o}^m(\mathbf{iz}^{m,l}(t, d)|_{d \in DI_{m,l}}^{l \in I_m}, \mathbf{lz}^{m,l}(t, d)|_{d \in DL_{m,l}}^{l \in L_m^f}, \mathbf{b}^m)$$

$$\mathbf{a}^m(t) = \mathbf{f}^m(\mathbf{n}^m(t))$$

where $\mathbf{ih}^{m,l}()$ and $\mathbf{lh}^{m,l}()$ are input and layer weight functions, and $\mathbf{o}^m()$ is the net input function at Layer m .

Generalized LFNN Gradient Calculations

$$\mathbf{s}^u = \mathbf{F}^u(\mathbf{n}^u)^T \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^u}$$

$$\mathbf{s}^m = \mathbf{F}^m(\mathbf{n}^m)^T \sum_{l \in L_m^b} \left(\frac{\partial \mathbf{lz}^{l,m}}{\partial (\mathbf{a}^m)^T} \right)^T \left(\frac{\partial \mathbf{n}^l}{\partial (\mathbf{lz}^{l,m})^T} \right)^T \mathbf{s}^l$$

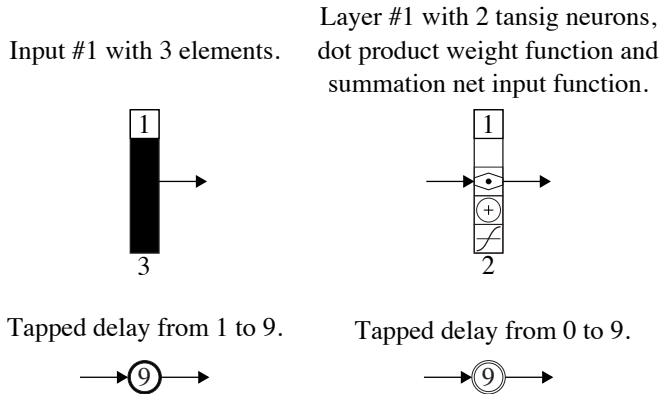
$$\frac{\partial F(\mathbf{x})}{\partial iw_{i,j}^{m,l}} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{iz}^{m,l})^T} \frac{\partial \mathbf{iz}^{m,l}}{\partial iw_{i,j}^{m,l}}$$

$$\frac{\partial F(\mathbf{x})}{\partial lw_{i,j}^{m,l}} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{lz}^{m,l})^T} \frac{\partial \mathbf{lz}^{m,l}}{\partial lw_{i,j}^{m,l}}$$

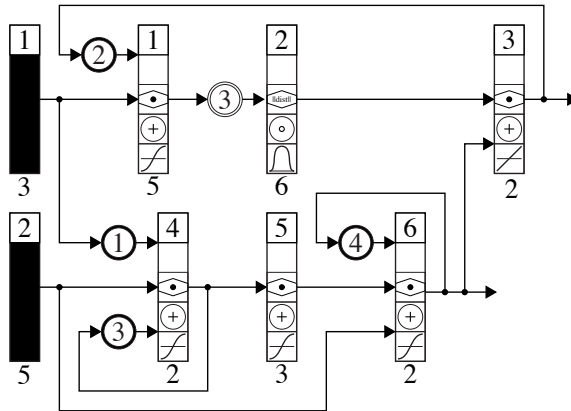
$$\frac{\partial F(\mathbf{x})}{\partial b_i^m} = (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial b_i^m}$$

Solved Problems

P7.1 Before stating the problem, it will be useful to introduce some shorthand notation, based on the notation in Solved Problem P14.1 in [NND2](#).



Using this notation, consider the following network.



- i. Find the sets I_m .
- ii. Find the sets L_m^f .
- iii. Find a Simulation Order.

General Modular Network Notation

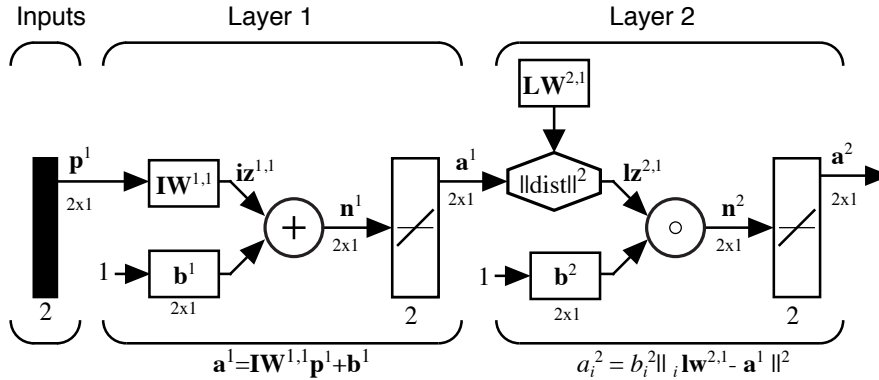
i. Neither of the inputs connect to Layers 2, 3 and 5, so I_m will be the null set for $m = 2, 3, 5$. Layer 1 is connected to the first input only, Layer 4 is connected to both inputs, and Layer 6 is connected to the second input, therefore $I_1 = \{1\}$, $I_4 = \{1, 2\}$ and $I_6 = \{2\}$.

ii. L_m^f is the set of indices of layers that directly connect forward to Layer m . Therefore $L_1^f = \{3\}$, $L_2^f = \{1\}$, $L_3^f = \{2, 6\}$, $L_4^f = \{4\}$, $L_5^f = \{4\}$, $L_6^f = \{5, 6\}$.

iii. The simulation order for this network is not unique, but one possibility is $\{1, 4, 2, 5, 6, 3\}$

P7.2

Consider the following network, and assume that the performance function is sum squared error. The second layer weight function output is computed as $lz_i^{2,1} = \left\| {}_i\mathbf{lw}^{2,1} - \mathbf{a}^1 \right\|^2$. The net input function output is Hadamard product, and is computed as $n_i^2 = lz_i^{2,1} b_i^2$.



i. Find the sensitivities $\mathbf{s}^1$ and $\mathbf{s}^2$.

ii. Find the gradient equations.

i. This network has the general form of a GLFNN, since the weight function in the second layer is not matrix multiplication. The sensitivities can be computed using Equations 7.28 and 7.29. We start with the last layer and Eq. 7.28.

$$\mathbf{s}^2 = \dot{\mathbf{F}}^2(\mathbf{n}^2)^T \frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^2}$$

Since the transfer function in the last layer is linear, and there are two neurons in the last layer, we have:

$$\mathbf{\tilde{F}}^2(\mathbf{n}^2) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The performance function is sum square error:

$$F(\mathbf{x}) = \sum_{j=1}^{S^2} (t_j - a_j^2)^2$$

Therefore

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{a}^2} = -2 (\mathbf{t} - \mathbf{a}^2)$$

Combining the last two equations, we have

$$\mathbf{s}^2 = -2 (\mathbf{t} - \mathbf{a}^2)$$

For $\mathbf{s}^1$ we use Eq. 7.29.

$$\mathbf{s}^1 = \mathbf{\tilde{F}}^1(\mathbf{n}^1)^T \left(\frac{\partial \mathbf{lz}^{2,1}}{\partial (\mathbf{a}^1)^T} \right)^T \left(\frac{\partial \mathbf{n}^2}{\partial (\mathbf{lz}^{2,1})^T} \right)^T \mathbf{s}^2$$

For this network

$$lz_i^{2,1} = \left\| {}_i\mathbf{l}\mathbf{w}^{2,1} - \mathbf{a}^1 \right\|^2 = \sum_{j=1}^{S^1} (lw_{i,j}^{2,1} - a_j^1)^2$$

Therefore

$$\frac{\partial lz_i^{2,1}}{\partial a_j^1} = -2 (lw_{i,j}^{2,1} - a_j^1)$$

In matrix form, this gives us

$$\frac{\partial \mathbf{lz}^{2,1}}{\partial (\mathbf{a}^1)^T} = \begin{bmatrix} -2 \left(lw_{1,1}^{2,1} - a_1^1 \right) & -2 \left(lw_{1,2}^{2,1} - a_2^1 \right) \\ -2 \left(lw_{2,1}^{2,1} - a_1^1 \right) & -2 \left(lw_{2,2}^{2,1} - a_2^1 \right) \end{bmatrix}$$

For the net input $\mathbf{n}^2$, we can write

$$\mathbf{n}^2 = \mathbf{lz}^{2,1} \circ \mathbf{b}^2$$

Therefore,

General Modular Network Notation

$$\frac{\partial \mathbf{n}^2}{\partial (\mathbf{1z}^{2,1})^T} = \begin{bmatrix} b_1^2 & 0 \\ 0 & b_2^2 \end{bmatrix}$$

Finally, because we have a linear activation function in the first layer,

$$\dot{\mathbf{F}}^1(\mathbf{n}^1) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Putting this together, we find

$$\mathbf{s}^1 = \begin{bmatrix} -2 \left(lw_{1,1}^{2,1} - a_1^1 \right) & -2 \left(lw_{1,2}^{2,1} - a_2^1 \right) \\ -2 \left(lw_{2,1}^{2,1} - a_1^1 \right) & -2 \left(lw_{2,2}^{2,1} - a_2^1 \right) \end{bmatrix}^T \times \begin{bmatrix} b_1^2 & 0 \\ 0 & b_2^2 \end{bmatrix}^T \times \mathbf{s}^2$$

Multiplying out the first two terms, we have

$$\mathbf{s}^1 = -2 \begin{bmatrix} b_1^2 \left(lw_{1,1}^{2,1} - a_1^1 \right) & b_2^2 \left(lw_{2,1}^{2,1} - a_1^1 \right) \\ b_1^2 \left(lw_{1,2}^{2,1} - a_2^1 \right) & b_2^2 \left(lw_{2,2}^{2,1} - a_2^1 \right) \end{bmatrix} \begin{bmatrix} s_1^2 \\ s_2^2 \end{bmatrix}$$

If we write this in terms of individual components, we have

$$s_j^1 = -2 \sum_{i=1}^2 b_i^2 \left(lw_{i,j}^{2,1} - a_j^1 \right) s_i^2$$

ii. For the gradient calculations, we use Equations 7.30 through 7.32:

$$\begin{aligned} \frac{\partial F(\mathbf{x})}{\partial iw_{i,j}^{m,l}} &= (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{iz}^{m,l})^T} \frac{\partial \mathbf{iz}^{m,l}}{\partial iw_{i,j}^{m,l}} \\ \frac{\partial F(\mathbf{x})}{\partial lw_{i,j}^{m,l}} &= (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial (\mathbf{1z}^{m,l})^T} \frac{\partial \mathbf{1z}^{m,l}}{\partial lw_{i,j}^{m,l}} \\ \frac{\partial F(\mathbf{x})}{\partial b_i^m} &= (\mathbf{s}^m)^T \frac{\partial \mathbf{n}^m}{\partial b_i^m} \end{aligned}$$

For Layer 1, we have

$$\begin{aligned} \frac{\partial F(\mathbf{x})}{\partial iw_{i,j}^{1,1}} &= (\mathbf{s}^1)^T \frac{\partial \mathbf{n}^1}{\partial (\mathbf{iz}^{1,1})^T} \frac{\partial \mathbf{iz}^{1,1}}{\partial iw_{i,j}^{1,1}} \\ \frac{\partial F(\mathbf{x})}{\partial b_i^1} &= (\mathbf{s}^1)^T \frac{\partial \mathbf{n}^1}{\partial b_i^1} \end{aligned}$$

Since the net input function in the first layer is summation, and $\mathbf{iz}^{1,1}$ is matrix multiplication,

$$\begin{aligned}\frac{\partial F(\mathbf{x})}{\partial iw_{i,j}^{1,1}} &= \left(\mathbf{s}^1\right)^T p_j^1 \mathbf{e}_i = s_i^1 p_j^1 \\ \frac{\partial F(\mathbf{x})}{\partial b_i^1} &= \left(\mathbf{s}^1\right)^T \mathbf{e}_i = s_i^1\end{aligned}$$

For Layer 2, we have

$$\begin{aligned}\frac{\partial F(\mathbf{x})}{\partial lw_{i,j}^{2,1}} &= \left(\mathbf{s}^2\right)^T \frac{\partial \mathbf{n}^2}{\partial (\mathbf{lz}^{2,1})^T} \frac{\partial \mathbf{lz}^{2,1}}{\partial lw_{i,j}^{2,1}} \\ \frac{\partial F(\mathbf{x})}{\partial b_i^2} &= \left(\mathbf{s}^2\right)^T \frac{\partial \mathbf{n}^2}{\partial b_i^2}\end{aligned}$$

The net input function is Hadamard product, and

$$lz_i^{2,1} = \left\| {}_i \mathbf{l} \mathbf{w}^{2,1} - \mathbf{a}^1 \right\|^2 = \sum_{j=1}^{s^1} \left(lw_{i,j}^{2,1} - a_j^1 \right)^2$$

Therefore

$$\frac{\partial lz_i^{2,1}}{\partial lw_{i,j}^{2,1}} = 2 \left(lw_{i,j}^{2,1} - a_j^1 \right)$$

and from earlier we know

$$\frac{\partial \mathbf{n}^2}{\partial (\mathbf{lz}^{2,1})^T} = \begin{bmatrix} b_1^2 & 0 \\ 0 & b_2^2 \end{bmatrix}$$

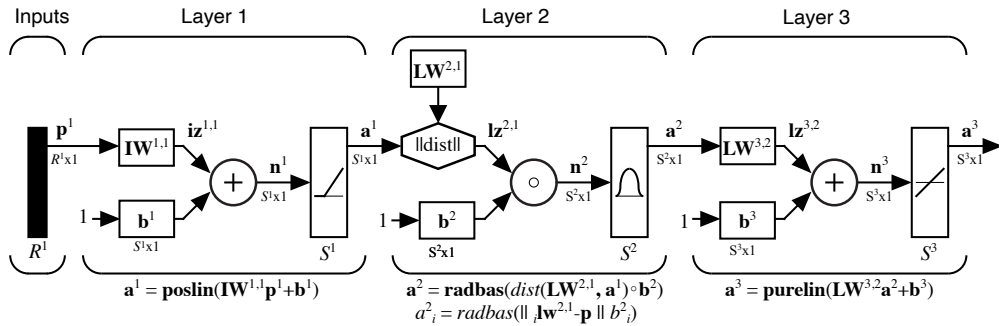
The gradients for the second layer are therefore

$$\begin{aligned}\frac{\partial F(\mathbf{x})}{\partial lw_{i,j}^{2,1}} &= 2s_i^2 b_i^2 \left(lw_{i,j}^{2,1} - a_j^1 \right) \\ \frac{\partial F(\mathbf{x})}{\partial b_i^2} &= 2s_i^2 b_i^2 \sum_{j=1}^2 \left(lw_{i,j}^{2,1} - a_j^1 \right)^2\end{aligned}$$

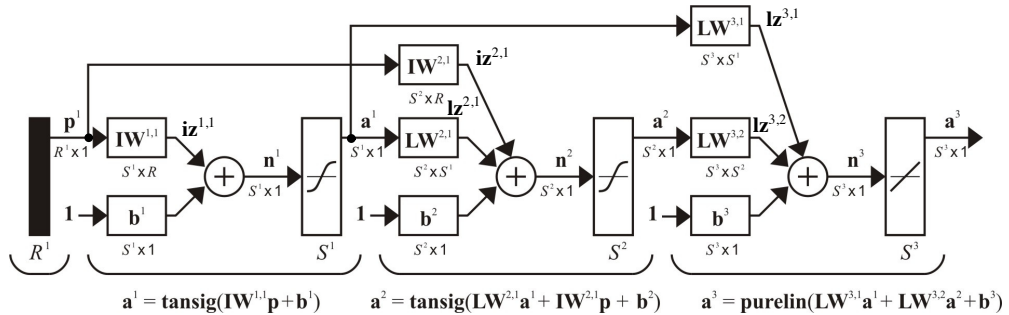
General Modular Network Notation

Exercises

- E7.1** Consider the three layer network shown below. Write out the complete backpropagation equations for computing the gradient for this network. The radbas transfer function is defined as $\text{radbas}(n) = e^{-n^2}$. Assume the performance function is mean square error.

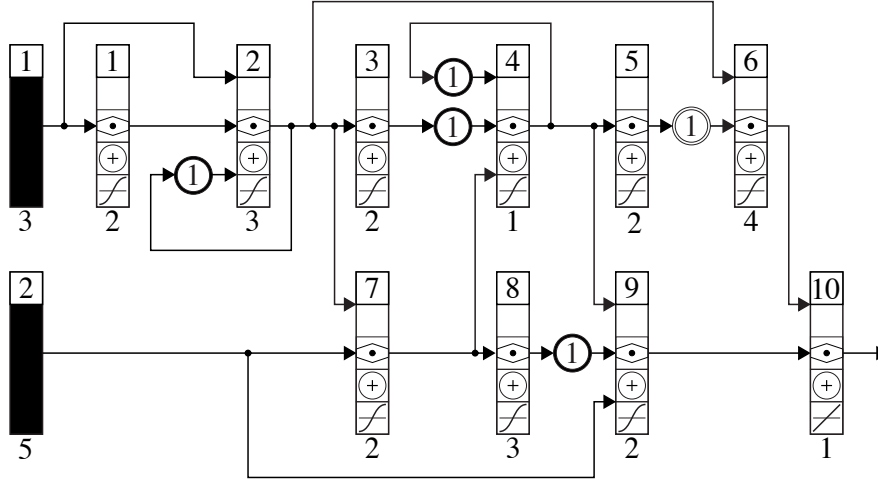


- E7.2** Consider the following network.



- Find the sets I_m , L_m^f and L_m^b .
- Write out the equations for computing the sensitivities $\mathbf{s}^m$, assuming that $U = \{3\}$.
- Write out the equations for computing the gradient.

E7.3 Consider the following GLDDN network.



- i. Find the sets I_m .
- ii. Find the sets L_m^f .
- iii. Find a Simulation Order.

E7.4 Suppose that the net input function for a certain layer is the Hadamard product. This means that all of the weight functions and the bias will be multiplied element by element, maintaining the same dimensions.

$$n_i^m = b_i^m \prod_{l \in L_m^f} l z_i^{m,l} \prod_{l \in I_m} i z_i^{m,l}$$

In order to train a network with this layer, we need to know the derivatives of the net input function.

- i. Find $\frac{\partial \mathbf{n}^m}{\partial (\mathbf{iz}^{m,l})^T}$.
- ii. Find $\frac{\partial \mathbf{n}^m}{\partial (\mathbf{lz}^{m,l})^T}$.
- iii. Find $\frac{\partial \mathbf{n}^m}{\partial b_i^m}$.

8 Convolution

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Why Use Convolution?</i>	2
<i>Details of the Convolution Operation</i>	6
<i>Template Matching–Matched Filter</i>	9
<i>Hierarchical Template Matching</i>	11
<i>Gradient Calculation</i>	16
<i>Epilogue</i>	22
<i>Further Reading</i>	23
<i>Summary of Results</i>	25
<i>Solved Problems</i>	29
<i>Exercises</i>	33

Objective

The convolution neural network is a popular network for image processing applications, including object classification, localization, detection and instance segmentation. It is also used in other applications, such as speech and natural language processing. This chapter describes the basic convolution operation and how it is used in combination with other layers, like pooling and fully connected, to form complete networks. The chapter also explains how to backpropagate through convolution and pooling layers in order to compute the training gradient.

Convolution

Theory and Examples

Why Use Convolution?

One of the drawbacks of using the standard multilayer network, described in Chapter 2, for image processing is that the size of the weight matrix in the first layer can become very large. This can lead to problems with overfitting, in addition to the increased computation. Instead of using standard matrix multiplication, in which every input element is fully connected to every neuron, it would be helpful to constrain the operation in some way.

Of course, if we constrain the operation, we may limit the possible functions that we can create with the network. We need to choose a structure that is suitable for the problem we want to solve. The process of restricting the form of a network is an example of what is called an *inductive bias*. It is a way of embedding *prior* knowledge about how to solve a particular problem into the neural network. If done correctly, this can make the problem easier for the network to solve.

If we consider operations that transform one image into another, it seems reasonable to consider transformations that are *position invariant*, which means that their effect on a point does not depend on the position of the point. (We can also say that a transformation is position invariant if it commutes with translation – the result will be the same whether the image translation occurs before or after the transformation.) Also, we will consider linear transformations, since the layers in a multilayer network consist of a linear transformation followed by a nonlinear activation function.

Recall from Chapter 6 of [NND2](#) that linear transformations can be represented as matrix multiplications. It turns out that any linear, *position invariant* transformation can also be represented by the convolution between the impulse response of the operation (sometimes called the point spread function) and the original image. (See [\[Rosenfeld, 1969\]](#).) This can be shown to be matrix multiplication with a matrix of a particular structure. (See Eq. [8.12](#).)

The convolution operation is ubiquitous throughout math, science and engineering. One of the earliest applications was by d'Alembert in 1754 to derive the Taylor series expansion. (See [\[le Rond d'Alembert et al., 1754\]](#).)

DERIVING THE CONVOLUTION OPERATION

To illustrate that convolution can represent linear, position invariant transformations, we need to define a couple of functions. First, consider an image that consists of a single pixel with a value of 1, with the remaining pixels equal to zero:

$$\delta_{i,j} = \begin{cases} 1 & i = j = 0 \\ 0 & \text{else} \end{cases} \quad (8.1)$$

This δ is called an *impulse* function or a one-point image.

Next we define the *impulse response*, or point spread function. This is the response to a given linear, position invariant transformation when it is applied to the impulse function. For example, if we have a transformation $\mathcal{W}$, the impulse response would be

$$w_{i,j} = \mathcal{W}(\delta_{i,j}) \quad (8.2)$$

(Note that in this equation $\delta_{i,j}$ and $w_{i,j}$ represent the entire images. The transformation $\mathcal{W}$ does not simply use one point in the input image to produce one point in the output image. It uses a set of points in the input image to produce each point in the output image.)

Now we can put these two ideas together. First, write a given image as a sum of impulses (one point images) located at each pixel:

$$p_{m,n} = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} p_{i,j} \delta_{m-i,n-j} \quad (8.3)$$

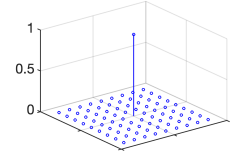
(Note that, although we show infinite sums, the pixel values will be zero outside some range.)

Now we perform the linear, position invariant transformation $\mathcal{W}$:

$$z_{m,n} = \mathcal{W}(\mathbf{P})_{m,n} = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} p_{i,j} \mathcal{W}(\delta_{m-i,n-j}) \quad (8.4)$$

$$= \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} p_{i,j} w_{m-i,n-j} \quad (8.5)$$

where we have used the linearity and position invariant properties of $\mathcal{W}$ and the definition of impulse response in Eq. 8.2. The expression in Eq. 8.5 is the convolution between the image and the impulse response. The convolution operation is commutative, and the order of the functions being convolved is not important, so we can also write



Convolution

$$z_{m,n} = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} w_{i,j} p_{m-i,n-j} \quad (8.6)$$

We can write the result in abbreviated notation as follows:

$$\mathbf{Z} = \mathbf{P} * \mathbf{W} = \mathbf{W} * \mathbf{P} \quad (8.7)$$

where $*$ represents the convolution operator.

CONVOLUTION AND CROSS-CORRELATION

In addition to assumptions about the linearity and position invariance of the transformation, it also seems reasonable to choose transformations such that the value at an output pixel z_{i^*,j^*} should be determined only by pixels in the neighborhood of the corresponding input pixel $p_{i,j}$. (See [Montgomery and Broome, 1962] for an early discussion of this idea, which they called the neighborhood modification process and which we now refer to as a local receptive field.) This would mean that the impulse response of the transformation would be nonzero only over a limited region. In this case the limits of the summation in Eq. 8.6 would be limited to the extent of the impulse response.

$$z_{m,n} = \sum_{i=0}^{r-1} \sum_{j=0}^{c-1} w_{i,j} p_{m-i,n-j} \quad (8.8)$$

where r and c represent the number of rows and columns in the matrix $\mathbf{W}$. We often refer to $\mathbf{W}$ (which represents the impulse response of the desired transformation) as a *kernel* or a *filter*. (We have indicated that the nonzero region of the kernel starts at an index of 0 for both row and column. This is for simplicity of the exposition. We will have more to say about the details of the operation in the next section.)

The convolution operation is very closely related to the *cross-correlation* operation, which is defined as follows:

$$z_{m,n} = \sum_{i=0}^{r-1} \sum_{j=0}^{c-1} w_{i,j} p_{m+i,n+j} \quad (8.9)$$

which we can write in abbreviated notation as

$$\mathbf{Z} = \mathbf{W} \otimes \mathbf{P} \quad (8.10)$$

The only difference between convolution and cross-correlation is in the meaning of the kernel. It is flipped horizontally and vertically between the two operations. (See Solved Problem P8.1 for a discussion of the relationship between convolution and correlation kernels.) Since we will be training the kernels in convolution networks, the orientation of the kernel is not usually of critical importance, since during training the network will learn the appropriate orientation. In most deep learning frameworks, convolution layers are actually implemented as cross-correlations, and most papers use the term convolution to refer to a cross-correlation operation. We will follow this current convention in the remainder of this text. We will use the cross-correlation operation, while referring to it as convolution, unless specifically noted.

WEIGHT SHARING

The cross-correlation (or convolution) operation can be represented as a matrix multiplication. For example, if we have a 3×3 image and a 2×2 kernel, the operation can be represented as

$$\begin{bmatrix} z_{1,1} & z_{1,2} \\ z_{2,1} & z_{2,2} \end{bmatrix} = \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \end{bmatrix} \otimes \begin{bmatrix} p_{1,1} & p_{1,2} & p_{1,3} \\ p_{2,1} & p_{2,2} & p_{2,3} \\ p_{3,1} & p_{3,2} & p_{3,3} \end{bmatrix} \quad (8.11)$$

$$\begin{bmatrix} z_{1,1} \\ z_{2,1} \\ z_{1,2} \\ z_{2,2} \end{bmatrix} = \begin{bmatrix} w_{1,1} & w_{2,1} & 0 & w_{1,2} & w_{2,2} & 0 & 0 & 0 & 0 \\ 0 & w_{1,1} & w_{2,1} & 0 & w_{1,2} & w_{2,2} & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{1,1} & w_{2,1} & 0 & w_{1,2} & w_{2,2} & 0 \\ 0 & 0 & 0 & 0 & w_{1,1} & w_{2,1} & 0 & w_{1,2} & w_{2,2} \end{bmatrix} \begin{bmatrix} p_{1,1} \\ p_{2,1} \\ p_{3,1} \\ p_{1,2} \\ p_{2,2} \\ p_{3,2} \\ p_{1,3} \\ p_{2,3} \\ p_{3,3} \end{bmatrix} \quad (8.12)$$

where Eq. 8.11 shows the operation as a cross-correlation and Eq. 8.12 shows it as standard matrix multiplication. (See Figure 8.2 for a graphical representation of the cross-correlation.)

Note that the same weights appear in each row of the matrix. This is often referred to as *weight sharing*, since each output neuron uses the same weights. It occurs because of the inductive bias we imposed on the network by saying the transformation should be

Convolution

position invariant, and there are many zeros since we specified that the transformation should have a local receptive field. The concept of inductive bias is especially important in problems where the input space is high dimensional. Clever uses of inductive bias will surely continue to expand the use of neural networks to seemingly intractable problems.

Details of the Convolution Operation

Eq. 8.6 describes the general convolution operation in theory, however, many practical details are left out. For example, the input image and the kernel are normally defined over a finite range of indices. In that case, for what values of m and n should $z_{m,n}$ be computed? In this section we want to describe the details of the operation in terms of specific image and kernel sizes.

Consider first the input image $\mathbf{P}$. For network inputs we generally use the letter R to represent the number of elements in the input vector. When dealing with images, we have two input dimensions, which we will represent by R_r and R_c , for the row and column sizes. (We will later add an additional dimension for number of channels [colors], and a depth dimension can be added for volumes in 3D convolution. In this section we want to concentrate on just the basic convolution operation on images.)

The output of the convolution will also be an image. For standard multilayer networks, we indicated the number of neurons by S . Now we will need a row and column size: S_r and S_c .

The dimension of the convolution weight matrix $\mathbf{W}$ (also called the kernel) is not tied to the size of the input, unlike in the multilayer network. We will indicate the number of rows and columns by r and c .

These dimensions are illustrated in Figure 8.1. Note that the bias in this diagram has the same dimension as the output image, since there is one element of the bias for each neuron. This is called an *untied bias*, since the bias is not tied to the kernel, but to the neuron. For a *tied bias*, the bias in this figure would be a scalar, and there would be only one for each kernel. As we will see in a later section, there can be multiple kernels in a layer.

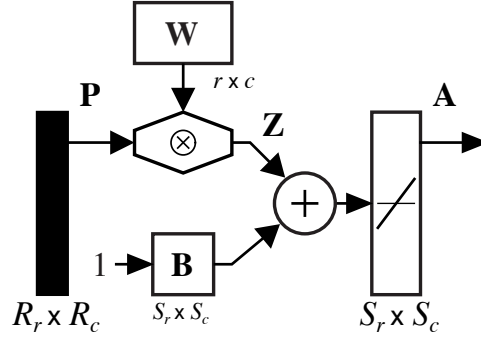


Figure 8.1: Basic Convolution Layer

Next, we should discuss the registration between the pixel locations in the input image and the corresponding pixels in the output image. In Eq. 8.6, the limits on the summation run from $-\infty$ to ∞ , but the kernel or the image will be equal to zero, or undefined, outside some range. When the convolution operation is used in neural networks, the computation of a given output pixel $z_{m,n}$ will not be performed unless the kernel fits completely within the defined region of the input image. This is illustrated in Figure 8.2. Here we have a 3×3 input image P , a 2×2 kernel W , and $Z = W \otimes P$.

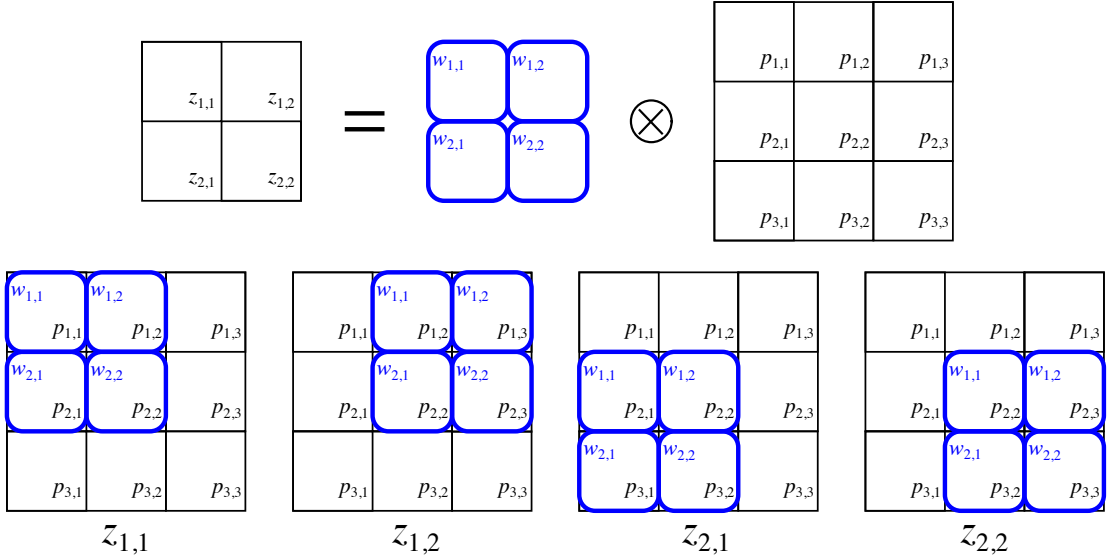


Figure 8.2: Example Convolution Layout

Convolution

Here we use the convention from [NND2](#), where indices begin at 1 for all vectors and matrices. For this reason we modify the indices in Eq. 8.9 to start at 1, instead of 0, and adjust the offsets so that in Eq. 8.13 $w_{1,1}$ multiplies the same pixel of $\mathbf{P}$ as $w_{0,0}$ did in Eq. 8.9. This allows us to start the computation of Eq. 8.13 at $m = 1$ and $n = 1$.

$$z_{m,n} = \sum_{i=1}^r \sum_{j=1}^c w_{i,j} p_{m+i-1, n+j-1} \quad (8.13)$$

The final value of m to be computed in Eq. 8.13 will be when $m + r - 1 = R_r$, or $m = R_r - r + 1$. In a similar way, we can find that the final value for n will be $n = R_c - c + 1$. (For the example in Figure 8.2, the maximum m and n are $m = 3 - 2 + 1 = 2$ and $n = 3 - 2 + 1 = 2$.) Therefore, the number of neurons in the convolution layer (the size of the output image) is determined by the input size and the kernel size:

$$S_r = R_r - r + 1 \quad (8.14)$$

$$S_c = R_c - c + 1 \quad (8.15)$$

It is worth noting that in a convolution layer the input size could be changed, which would change the output size, but the kernel size could remain the same.

PADDING AND STRIDE

The basic convolution operation is often modified in two ways that can change the size of the output image. The first change is called *padding*. From Equations 8.14 and 8.15, we can see that the output image will be smaller than the input image. Padding artificially increases the size of the input image by adding pixels of value 0 around the edge of the image. This can be done in such a way that the output of the convolution will be the same size as the original input image. If we add P^d pixels around the outside of the original image, this will add $2P^d$ to both the row size and the column size. The new sizes of the output image will be

0	0	0	0	0
0				0
0				0
0				0
0	0	0	0	0

$$S_r = R_r + 2P^d - r + 1 \quad (8.16)$$

$$S_c = R_c + 2P^d - c + 1 \quad (8.17)$$

The other size-modifying change to the convolution operation is the adjustment of *stride*. The output image can be made smaller by taking larger strides, or kernel movements. Normally, the kernel is moved one step at a time when performing the convolution, as

shown in Figure 8.2. If the stride is increased to 2, for example, the output image size is reduced by a factor of 2. Figure 8.3 shows an example with a 2×2 kernel, a 4×4 input image and a stride of 2. The output image is 2×2 .

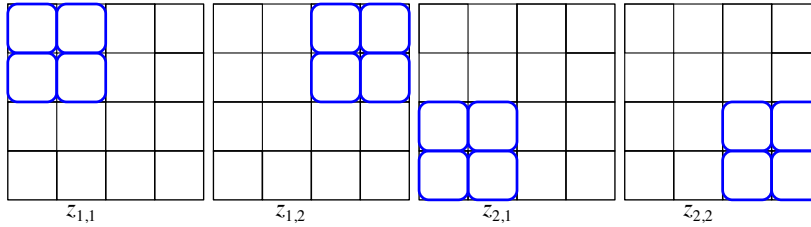


Figure 8.3: Example Convolution with Stride of 2

Equations 8.18 and 8.19 show how the output image size is determined when both padding and non-unity strides are used together. S^t is the stride length.

$$S_r = (R_r + 2P^d - r) / S^t + 1 \quad (8.18)$$

$$S_c = (R_c + 2P^d - c) / S^t + 1 \quad (8.19)$$

Template Matching–Matched Filter

As we showed earlier, the convolution operation can be used to implement any linear, position-invariant transformation on an image. It has been used in image processing since the 1950s for edge detection, noise removal, contrast enhancement, image restoration, etc. (See [Kovácsnay and Joseph, 1955] and [Rosenfeld, 1969].) In this section we want to explore its power for determining when two images match. This process is called *template matching*, or *matched filtering*. (See [Montgomery and Broome, 1962] for an early discussion of matched filters.)

We know from Chapter 5 of [NND2](#) that a good way to determine the similarity between two vectors is to compute their inner product. We can see from Eq. 8.13 and Figure 8.2 that each step of the convolution is an inner product between the kernel and a segment of the input image. Therefore, if we construct the kernel in the form of a template that we would like to match in the input image, the magnitude of the convolution at each output pixel will tell us how close the subimage matches the template.

Convolution

To see how template matching can work, consider a simple problem in which we are trying to distinguish between squares and diamonds. In Figure 8.4, we see two 15×15 black and white images representing examples of our two categories.

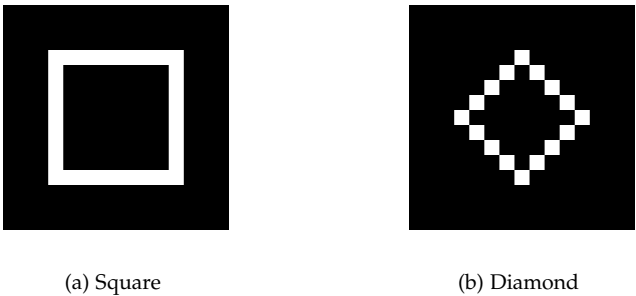


Figure 8.4: Sample Images to be Recognized

If we use 3×3 kernels, what types of templates could we use that would help us distinguish between squares and diamonds? These are very simple objects, so we can select the features manually. Squares consist of horizontal and vertical lines, and diamonds consist of slanted lines. Some possible kernels are shown in Figure 8.5.

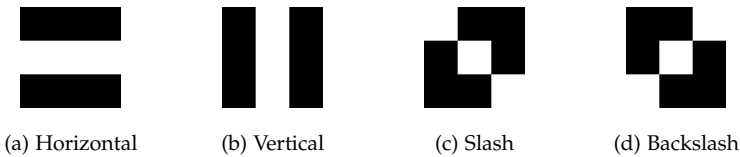


Figure 8.5: Sample Kernels

Figure 8.6 illustrates the results of convolving each of the kernels with the square image. We can see the larger magnitudes in locations where horizontal and vertical lines match segments of the image. The slanted kernels don't match with any segments of the image.

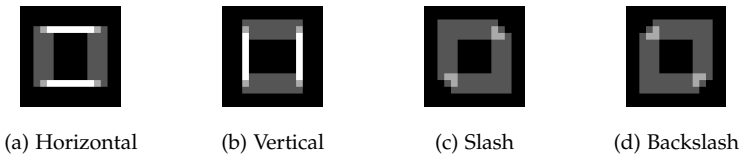


Figure 8.6: Filtered Versions of the Square

Figure 8.7 shows the results of convolving the kernels with the diamond image. In this case there are no matches when convolving with the horizontal and vertical kernels, but there are matches with the slanted kernels in appropriate segments of the diamond image.

To experiment with the convolution operation with different kernels, use the Deep Learning Demonstration Convolution (dl3gd).

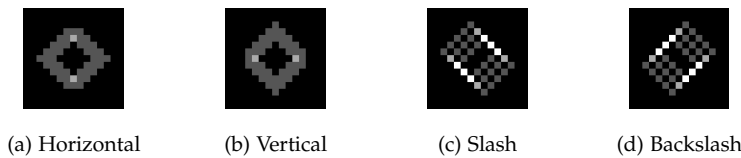


Figure 8.7: Filtered Versions of the Diamond

Hierarchical Template Matching

We now see that the convolution operation can be used to detect elementary features in images. Before deep learning, such matched filters were hand-designed. Now we want to train convolution networks to learn which features to extract. In addition, natural images have complex features that can be formed with combinations of multiple simple elements. We want to build a complete architecture that can be trained to recognize realistic objects, using the convolution operation as a building block.

The idea of stacking convolution layers in a hierarchical architecture was proposed by Kunihiro Fukushima in 1980 (see [Fukushima, 1980]), inspired by the hierarchical model of the cat visual cortex developed by David Hubel and Torsten Wiesel in 1962 (see [Hubel and Wiesel, 1962]). What they found was that neurons in the early visual cortex are organized in a hierarchical fashion, where the first cells connected to the cat's retina are responsible for detecting simple patterns like edges and bars, followed by later layers responding to more complex patterns by combining the earlier neuron outputs. (See Chapter 18 of [NND2](#) for a diagram of the human visual system and discussions of various types of receptive fields.) Their work earned them the Nobel Prize for Physiology or Medicine in 1981.

Convolution

Yann LeCun, in the late 1980s, refined the architecture of Fukushima and demonstrated how all of the network weights could be trained simultaneously using gradient-based algorithms, with backpropagation used to compute the gradient. The components of the convolution network architectures in current use are in large part the same ones first proposed by LeCun.

MULTIPLE KERNELS

The first step in creating the hierarchical architecture is to have multiple kernels in each convolution layer. The idea is to build up complex features by combining many simple features from previous layers. LeCun called the output of one kernel operation a *feature map* (FM). (This is not to be confused with the Self-Organizing Feature Maps of Kohonen, described in Chapter 16 of [NND2](#).) Since each convolution layer will have multiple FMs, this adds an additional dimension to the layer output. This idea can be extended to the input image. For example, a color image consists of red, green and blue planes. Therefore, a convolution layer takes a set of FMs as an input and produces another set of FMs as an output.

We will use the letter H^m to represent the number of FMs in Layer m , with $m = 0$ for the input. If there are H^0 FMs (planes) in the input, and H^1 FMs in the first convolution layer, then there will be $H^0 \times H^1$ kernels in Layer 1. To obtain one FM at the output of Layer 1, H^0 kernels are convolved with the H^0 FMs of the input, and the results are added together. This is done H^1 times to produce all H^1 FMs at the output. The general equation for an input coming from Layer n and connecting to layer m would be the following:

$$z_{i,j}^{m,n,h} = \sum_{l=1}^{H^n} \sum_{u=1}^{r^m} \sum_{v=1}^{c^m} w_{u,v}^{m,n,h,l} a_{i+u-1,j+v-1}^{n,l} \quad (8.20)$$

In abbreviated notation, this would be

$$\mathbf{Z}^{m,n,h} = \sum_{l=1}^{H^n} \mathbf{W}^{m,n,h,l} \otimes \mathbf{A}^{n,l} \quad (8.21)$$

where, in $\mathbf{Z}^{m,n,h}$, m represents the current layer number, n represents the previous layer number and h represents the FM number. In most convolution networks the layers are connected sequentially, so that $m = n + 1$. However, new and more complex architectures are being invented daily.

POOLING

In a standard convolution network, a convolution layer is followed by a *pooling* layer, which changes the resolution of the FMs. It performs a type of subsampling, aggregation, consolidation or coarsening. This allows the different layers of the network to operate at different scales, producing a multi-scale operation. Pooling also reduces the size of the output FM and reduces the number of parameters in the network. It is another example of inductive bias, where we assume that a certain structure in the network is appropriate for the data we are working with.

There are several different types of pooling, but they all generally consolidate several pixels in an input FM to produce one pixel in the output FM. For example, the following equation represents average pooling.

$$z_{i,j} = \left\{ \sum_{k=1}^r \sum_{l=1}^c v_{r(i-1)+k, c(j-1)+l} \right\} w \quad (8.22)$$

In abbreviated notation, we write average pooling like this.

$$\mathbf{Z} = w \boxplus_{r,c}^{ave} \mathbf{V} \quad (8.23)$$

The average pooling operation in Eq. 8.22 looks similar to the convolution operation in Eq. 8.13. The difference is that all of the elements of the kernel are equal to w . However, the stride used in the pooling operation is almost always equal to the size of the kernel. In other words, the stride would equal r in the vertical direction and c in the horizontal direction. Pooling therefore consolidates all of the $r \times c$ pixels into one pixel, reducing the image size by a corresponding amount.

Another type of pooling is max pooling. This also consolidates multiple pixels into one, but it uses the maximum of the input pixels, rather than the sum.

$$z_{i,j} = \max \left\{ v_{r(i-1)+k, c(j-1)+l} \mid k = 1, \dots, r; l = 1, \dots, c \right\} \quad (8.24)$$

Note that there is no trainable parameter in the max pooling function.

The abbreviated notation for max pooling is

$$\mathbf{Z} = \boxplus_{r,c}^{max} \mathbf{V} \quad (8.25)$$

Convolution

Theoretically, any size consolidation window and any stride could be used for pooling. However, in most cases the consolidation window is square, and the stride is equal to the window size, so there is no overlap in the consolidations. The most common choice is $r = 2$, $c = 2$ and $S^t = 2$.

The max pooling operation is illustrated in Figure 8.8, where the window size and stride are equal to 3. The image size goes from 15×15 to 5×5 .

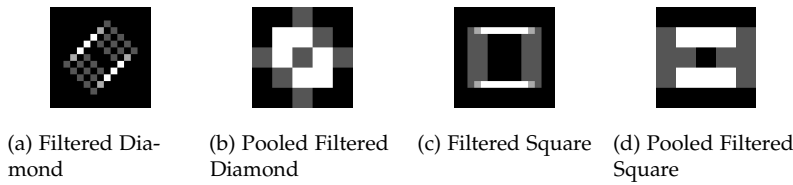


Figure 8.8: Examples of Max Pooling

Figure 8.9 shows a network with a convolution layer followed by a max pooling layer. Note that in the convolution layer we are using a tied bias, so there is one scalar bias for each kernel. The number of kernels is $H^1 \times H^0$, since there are H^0 planes in the input image and H^1 FMs in the first layer. The dimensions of the two terms coming into the summation in the first layer do not have the same dimension, but we will assume that broadcasting will be done to ensure compatibility, as in the Python language (see Chapter 5).

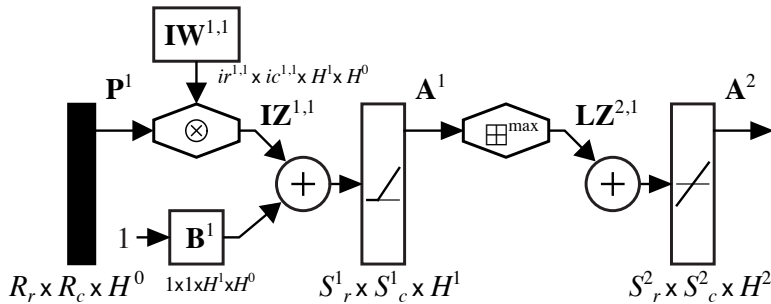


Figure 8.9: Network with Convolution and Pooling Layers

Unlike the convolution layer, in the pooling layer the number of FMs at the input will always be equal to the number of FMs at the output. In Figure 8.9, this means that $H^2 = H^1$. The pooling operation is applied to each FM individually to produce one output

FM. In the convolution layer, separate kernels are applied to each input FM, and the results are summed to produce one output FM. The number of input FMs in that case do not constrain the number of output FMs.

SIMPLIFIED NOTATION

Since convolution networks often have a large number of layers, it is useful to have a shorthand notation. We can build on the notation introduced in Solved Problem P7.1. Figure 8.10 shows the same network as Figure 8.9, but in the shorthand notation.

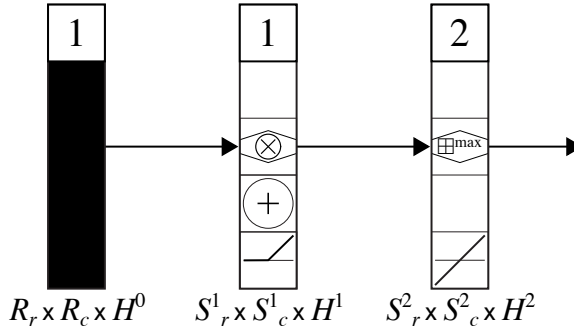
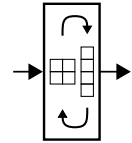


Figure 8.10: Shorthand Notation for Figure 8.9

Convolution and pooling layers take images as input and produce images as output. In order to create a convolution network that performs classification, we need to convert the images to vectors, so that they can be input to a standard dot product (matrix multiplication) layer. In our diagrams, we show this as a conversion block. It converts the output of a layer from a set of FMs to a single vector. It stacks the columns of the FMs on top of each other, starting from the first FM to the last.



$$vec(\mathbf{A}^m) = \left[\left(\mathbf{a}_1^{m,1} \right)^T \left(\mathbf{a}_2^{m,1} \right)^T \cdots \left(\mathbf{a}_{S_c^m}^{m,1} \right)^T \left(\mathbf{a}_1^{m,2} \right)^T \cdots \left(\mathbf{a}_{S_c^m}^{m,H^m} \right)^T \right]^T \quad (8.26)$$

where $\mathbf{a}_i$ indicates the i^{th} column of matrix $\mathbf{A}$.

Figure 8.11 shows a full five layer convolution network classifier. There are two convolution-pooling groups, followed by a conversion to vector and then a fully connected layer (as in the multilayer network) with a softmax activation function to perform the classification.

Convolution

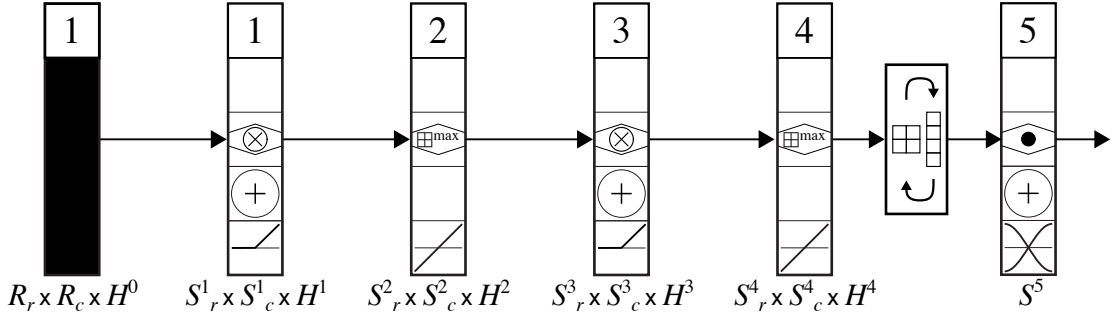


Figure 8.11: Full Convolution Network

Gradient Calculation

Convolution networks are trained using the same gradient-based algorithms we described in Chapter 3, and the gradient is computed using the same backpropagation (chain rule) process we discussed in several chapters (e.g., see Eq. 7.29). In that process, we computed a sensitivity (defined in Eq. 3.38 [and Eq. 11.29 of NND2] and repeated here), which is the derivative of the performance (loss) function with respect to the net input at a given layer.

$$\mathbf{s}^m \triangleq \frac{\partial F(\mathbf{x})}{\partial \mathbf{n}^m} \quad (8.27)$$

In Eq. 7.29 we compute the sensitivities by backpropagating across the layer in stages: across the net input function, across the weight function and across the transfer function. To illustrate this process with convolution networks, it is helpful to perform one step at a time. For this reason, we will introduce a new intermediate variable at each step. We will also change the symbol for the sensitivity to be consistent with the other intermediate variables. The definitions are as follows:

$$\mathbf{dN} \equiv \frac{\partial F}{\partial \mathbf{N}}, \mathbf{dA} \equiv \frac{\partial F}{\partial \mathbf{A}}, \mathbf{dZ} \equiv \frac{\partial F}{\partial \mathbf{Z}}, \mathbf{dW} \equiv \frac{\partial F}{\partial \mathbf{W}}, \mathbf{dB} \equiv \frac{\partial F}{\partial \mathbf{B}} \quad (8.28)$$

where $\mathbf{dN}$ is the original sensitivity, although it is a matrix in the case of the convolution network, since the net input is a matrix.

The process of backpropagating across a layer is shown in Figure 8.12. (For simplicity, this figure shows just one FM in each layer.) In this figure, the process of going backward from $\mathbf{dN}^m$ to $\mathbf{dN}^n$ is analogous to going from $\mathbf{s}^{m+1}$ to $\mathbf{s}^m$ in Eq. 3.49. (See also Figure 3.5.)

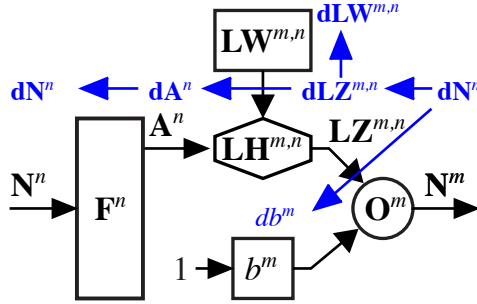


Figure 8.12: Backpropagating Across a Layer

To update the sensitivities, as in Eq. 7.29, we follow these steps:

$$\mathbf{dN}^m \rightarrow \mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n \rightarrow \mathbf{dN}^n \quad (8.29)$$

where we backpropagate across the net input function, the weight function and then the transfer function. Next, to compute the components of the gradient, we perform the following steps:

$$\mathbf{dN}^m \rightarrow db^m \quad (8.30)$$

$$\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n} \quad (8.31)$$

where we again backpropagate across the net input function and the weight function, but in a different direction. This corresponds to the steps given in Equations 7.30 through 7.32.

We will consider each of the above backpropagation steps separately and will show the detailed steps for the summation net input function and the convolution weight function. We will consider other specific net input functions and weight functions in the Solved Problems and Exercises.

ACROSS THE SUMMATION NET INPUT FUNCTION

There are two paths when backpropagating across the net input function: $\mathbf{dN}^m \rightarrow \mathbf{dLZ}^{m,n}$ and $\mathbf{dN}^m \rightarrow db^m$. We begin with $\mathbf{dN}^m \rightarrow \mathbf{dLZ}^{m,n}$. Since

Convolution

$$\mathbf{N}^m = \mathbf{LZ}^{m,n} + b^m \quad (8.32)$$

or in scalar form

$$n_{i,j}^m = lz_{i,j}^{m,n} + b^m \quad (8.33)$$

we can write

$$dlz_{i,j}^{m,n} \equiv \frac{\partial F}{\partial lz_{i,j}^{m,n}} = \frac{\partial n_{i,j}^m}{\partial lz_{i,j}^{m,n}} \times \frac{\partial F}{\partial n_{i,j}^m} = 1 \times \frac{\partial F}{\partial n_{i,j}^m} \equiv dn_{i,j}^m \quad (8.34)$$

or in matrix form

$$\mathbf{dLZ}^{m,n} = \mathbf{dN}^m \quad (8.35)$$

(Note that this equation would be true, even if multiple layers were connecting into Layer m – i.e., there could be more than one Layer n , and the derivative would be the same for all of them.)

Now we consider the $\mathbf{dN}^m \rightarrow db^m$ path.

$$db^m \equiv \frac{\partial F}{\partial b^m} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} \frac{\partial F}{\partial n_{i,j}^m} \times \frac{\partial n_{i,j}^m}{\partial b^m} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} dn_{i,j}^m \quad (8.36)$$

In this case, when we use the chain rule, we need to consider all of the ways in which b^m affects $\mathbf{N}^m$. Since b^m affects each element of $\mathbf{N}^m$, we need to sum over all of these elements in Eq. 8.36. Notice that the resulting operation is the average pooling operation, with the kernel size being equal to the size of the feature map. In abbreviated notation, this can be written

$$db^m = \boxplus_{S_r^m, S_c^m}^{ave} \mathbf{dN}^m \quad (8.37)$$

ACROSS THE CONVOLUTION WEIGHT FUNCTION

There are also two backpropagation paths across the weight function: $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n$ and $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n}$. We begin with $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n$. The convolution operation with multiple kernels is given by Eq. 8.20, repeated here:

$$lz_{i,j}^{m,n,h} = \sum_{l=1}^{H^n} \sum_{u=1}^{r^m} \sum_{v=1}^{c^m} lw_{u,v}^{m,n,h,l} a_{i+u-1,j+v-1}^{n,l} \quad (8.38)$$

Therefore, we can write

$$da_{u',v'}^{n,l} \equiv \frac{\partial F}{\partial a_{u',v'}^{n,l}} = \sum_{h=1}^{H^n} \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} \frac{\partial F}{\partial lz_{i,j}^{m,n,h}} \times \frac{\partial lz_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,l}} \quad (8.39)$$

From Eq. 8.38 we can show that

$$\frac{\partial lz_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,l}} = \begin{cases} lw_{u'-i+1,v'-j+1}^{m,n,h,l} & \text{if } i \leq u' \leq i + r^m - 1 \text{ and } j \leq v' \leq j + c^m - 1 \\ 0 & \text{else} \end{cases} \quad (8.40)$$

Therefore we can simplify Eq. 8.39 to

$$da_{u',v'}^{n,l} = \sum_{h=1}^{H^n} \sum_{i=u'-r^m+1}^{u'} \sum_{j=v'-c^m+1}^{v'} dlz_{i,j}^{m,n,h} lw_{u'-i+1,v'-j+1}^{m,n,h,l} \quad (8.41)$$

If we define new variables

$$i' = i + r^m - u' \quad (8.42)$$

$$j' = j + c^m - v' \quad (8.43)$$

we can rewrite Eq. 8.41 as

$$da_{u',v'}^{n,l} = \sum_{h=1}^{H^n} \sum_{i'=1}^{r^m} \sum_{j'=1}^{c^m} lw_{r^m-i'+1,c^m-j'+1}^{m,n,h,l} dlz_{i'+u'-r^m,j'+v'-c^m}^{m,n,h} \quad (8.44)$$

Compare Eq. 8.44 with Eq. 8.38. We can see that they are both performing convolution operations with the same weights. In Eq. 8.38, $\mathbf{LW}^{m,n,h,l}$ is being convolved with $\mathbf{A}^{n,l}$ to produce $\mathbf{LZ}^{m,n,h}$. In Eq. 8.44, $\mathbf{LW}^{m,n,h,l}$ is being convolved with $\mathbf{dLZ}^{m,n,h}$ to produce $\mathbf{dA}^{n,l}$. However, in Eq. 8.44 the indexing on the weight is different. Here is a comparison:

Forward $\Leftrightarrow$ Backpropagation

Indexes: $u, v \Leftrightarrow$ Indexes: i', j'

$$lw_{u,v}^{m,n,h,l} \Leftrightarrow lw_{r^m-i'+1,c^m-j'+1}^{m,n,h,l}$$

$$a_{i+u-1,j+v-1}^{n,l} \Leftrightarrow dlz_{i'+u'-r^m,j'+v'-c^m}^{m,n,h}$$

The indexes on the weight in the backpropagation step are incrementing down, where the indices in the forward step are incrementing up. Effectively, the weight matrix has been rotated 180 degrees.

We can represent the operation in matrix form as follows:

Convolution

$$\mathbf{dA}^{n,l} = \sum_{l=1}^{H^n} ((rot180)\mathbf{LW}^{m,n,h,l}) \otimes \mathbf{dLZ}^{m,n,h} \quad (8.45)$$

Note that this result is similar to the process for the standard multilayer network: going forward we multiply by the weight matrix (see Eq. 3.42), and going backward we multiply by the transpose of the weight matrix (see Eq. 3.49).

Now let's consider the $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n}$ path, which computes a portion of the gradient.

$$dlw_{u,v}^{m,n,h,l} \equiv \frac{\partial F}{\partial lw_{u,v}^{m,n,h,l}} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} \frac{\partial F}{\partial lz_{i,j}^{m,n,h}} \times \frac{\partial lz_{i,j}^{m,n,h}}{\partial w_{u,v}^{m,n,h,l}} \quad (8.46)$$

From Eq. 8.38, we can see that

$$dlw_{u,v}^{m,n,h,l} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} dlz_{i,j}^{m,n,h} \times a_{i+u-1,j+v-1}^{n,l} \quad (8.47)$$

This is a convolution of $\mathbf{dLZ}^{m,n,h}$ with $\mathbf{A}^{n,l}$. In abbreviated notation, we can write

$$\mathbf{dLW}^{m,n,h,l} = \mathbf{dLZ}^{m,n,h} \otimes \mathbf{A}^{n,l} \quad (8.48)$$

ACROSS ACTIVATION FUNCTION

The final step of backpropagating across a layer is crossing the activation function – $\mathbf{dA}^n \rightarrow \mathbf{dN}^n$.

$$dn_{i,j}^{n,h} \equiv \frac{\partial F}{\partial n_{i,j}^{n,h}} = \frac{\partial a_{i,j}^{n,h}}{\partial n_{i,j}^{n,h}} \times \frac{\partial F}{\partial a_{i,j}^{n,h}} \quad (8.49)$$

By definition,

$$\frac{\partial F}{\partial a_{i,j}^{n,h}} \equiv da_{i,j}^{n,h} \quad (8.50)$$

and

$$\frac{\partial a_{i,j}^{n,h}}{\partial n_{i,j}^{n,h}} = \dot{f}^{n,h}(n_{i,j}^{n,h}) \quad (8.51)$$

Therefore, we can write

$$dn_{i,j}^{n,h} = \dot{f}^{n,h}(n_{i,j}^{n,h}) \times da_{i,j}^{n,h} \quad (8.52)$$

In abbreviated form, this is

$$\mathbf{dN}^{n,h} = \dot{\mathbf{F}}^{n,h} \circ \mathbf{dA}^{n,h} \quad (8.53)$$

where $\circ$ is the Hadamard product, which is an element by element matrix multiplication.

Epilogue

The convolution operation is ubiquitous in science, engineering and mathematics, and it has been in use for centuries. For image processing, it can implement any linear, position invariant transformation. It has been used in this way for template matching (matched filtering) in image processing since the 1950s. The trainable, hierarchical convolution network developed in the 1980s by Yann LeCun has become, with the advent of modern computation resources, one of the most popular neural network architectures for image classification, segmentation and object detection.

These convolution networks are trained using the same gradient-based algorithms that we use for multilayer (fully connected) networks, and the gradient is computed using the same backpropagation algorithm. The difference is only in the specifics of propagating across more general net input functions, weight functions and transfer functions.

One difficulty in using any deep learning model, like a convolution network, is explainability. The model is something of a black box, in that it does not explain why it made a certain decision. To develop confidence in the model, it can be very helpful to be able to explain why a decision is made. In the next chapter we will discuss neural network explainability.

Further Reading

[Rosenfeld, 1969] Azriel Rosenfeld. Picture processing by computer. *ACM Computing Surveys (CSUR)*, 1(3):147–176, 1969

An early survey of digital image processing. Covers convolution, cross-correlation and template matching (matched filtering). Gives a nice review of the classical methods for pattern recognition.

[Duda et al., 1973] Richard O Duda, Peter E Hart, et al. *Pattern classification and scene analysis*, volume 3. Wiley New York, 1973

This was one of the preeminent textbooks on pattern recognition during the 1970s and 1980s. It covers classical methods of pattern recognition, including template matching. An updated version of the book, titled just *Pattern Classification*, replaces the classical sections with neural network material. This demonstrates how the field has changed since the introduction of CNNs.

[Hubel and Wiesel, 1962] David H Hubel and Torsten N Wiesel. Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex. *The Journal of physiology*, 160(1):106–154, 1962

This paper shows that neurons in the early visual cortex are organized in a hierarchical fashion, where the first cells connected to the cat’s retinas are responsible for detecting simple patterns like edges and bars, followed by later layers responding to more complex patterns by combining the earlier neuronal activities. This work inspired Fukushima and, later, LeCun to develop hierarchical convolutional neural networks.

[Fukushima, 1980] Kunihiro Fukushima. A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.*, 36:193–202, 1980

In this paper Fukushima introduced the first artificial convolutional neural network. It was based on the work of Hubel and Weisel describing the receptive fields in the cat’s visual cortex. The network training was somewhat specialized and problem specific. It did not use a backpropagation-based algorithm.

Convolution

[LeCun et al., 1998] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86 (11):2278–2324, 1998

This article gives the best detailed description of LeCun’s original LeNet CNN architectures. The networks are similar in structure to those of Fukushima, but they are trained with standard gradient-based algorithms, with the gradient computed with backpropagation.

Summary of Results

Basic Convolution Operation

$$z_{m,n} = \sum_{i=1}^r \sum_{j=1}^c w_{i,j} p_{m-i+1,n-j+1}$$

Abbreviated Notation

$$Z = W * P$$

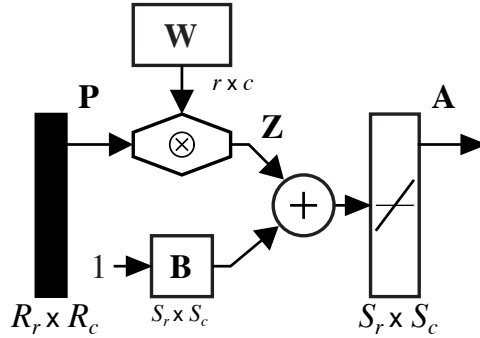
Basic Correlation Operation

$$z_{m,n} = \sum_{i=1}^r \sum_{j=1}^c w_{i,j} p_{m+i-1,n+j-1}$$

Abbreviated Notation

$$Z = W \otimes P$$

Basic Convolution Layer



Affect of Stride and Padding on Output Size

$$S_r = (R_r + 2P^d - r)/S^t + 1$$

$$S_c = (R_c + 2P^d - c)/S^t + 1$$

Convolution with Multiple Feature Maps

$$z_{i,j}^{m,n,h} = \sum_{l=1}^{H^n} \sum_{u=1}^{r^m} \sum_{v=1}^{c^m} w_{u,v}^{m,n,h,l} a_{i+u-1,j+v-1}^{n,l}$$

Convolution

Abbreviated Notation

$$\mathbf{Z}^{m,n,h} = \sum_{l=1}^{H^n} \mathbf{W}^{m,n,h,l} \otimes \mathbf{A}^{n,l}$$

Average Pooling

$$z_{i,j} = \left\{ \sum_{k=1}^r \sum_{l=1}^c v_{r(i-1)+k,c(j-1)+l} \right\} w$$

Abbreviated Notation

$$\mathbf{Z} = w \boxplus_{r,c}^{ave} \mathbf{V}$$

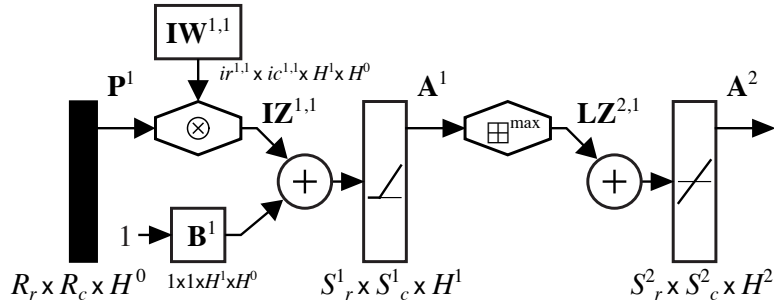
Max Pooling

$$z_{i,j} = \max \left\{ v_{r(i-1)+k,c(j-1)+l} \mid k = 1, \dots, r; l = 1, \dots, c \right\}$$

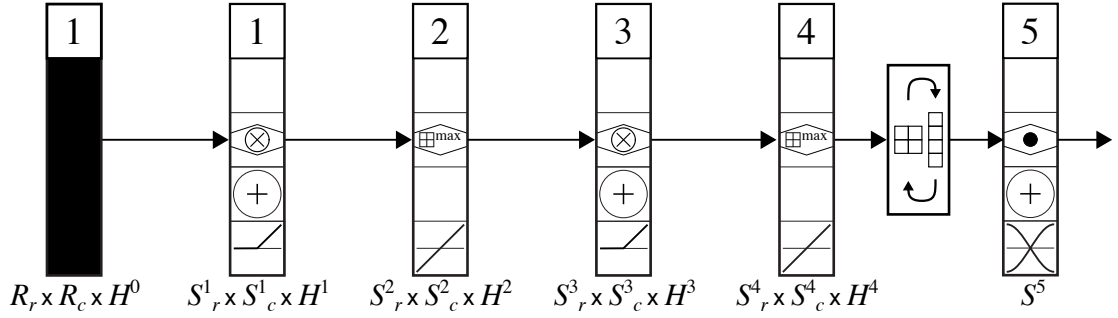
Abbreviated Notation

$$\mathbf{Z} = \boxplus_{r,c}^{max} \mathbf{V}$$

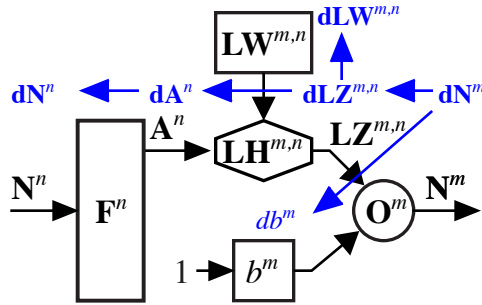
Network with Convolution and Pooling Layers



Full Convolution Network, Simplified Notation



Backpropagating Across a Layer

Backpropagation Path $dN^m \rightarrow dLZ^{m,n}$ (Summation)

$$dlz_{i,j}^{m,n} = dn_{i,j}^m$$

$$dLZ^{m,n} = dN^m$$

Backpropagation Path $dN^m \rightarrow db^m$ (Summation)

$$db^m = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} dn_{i,j}^m$$

$$db^m = \boxplus_{S_r^m, S_c^m}^{ave} dN^m$$

Backpropagation Path $dLZ^{m,n} \rightarrow dA^n$ (Convolution)

$$da_{u',v'}^{n,l} = \sum_{h=1}^{H^n} \sum_{i'=1}^{r^m} \sum_{j'=1}^{c^m} lw_{r^m-i'+1, c^m-j'+1}^{m,n,h,l} dlz_{i'+u'-r^m, j'+v'-c^m}^{m,n,h}$$

$$dA^{n,l} = \sum_{l=1}^{H^n} ((rot180)LW^{m,n,h,l}) \otimes dLZ^{m,n,h}$$

Convolution

Backpropagation Path $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n}$ (Convolution)

$$dlw_{u,v}^{m,n,h,l} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} dz_{i,j}^{m,n,h} \times a_{i+u-1,j+v-1}^{n,l}$$
$$\mathbf{dLW}^{m,n,h,l} = \mathbf{dLZ}^{m,n,h} \otimes \mathbf{A}^{n,l}$$

Backpropagation Path $\mathbf{dA}^n \rightarrow \mathbf{dN}^n$ (Activation Function)

$$dn_{i,j}^{n,h} = \dot{f}^{n,h}(n_{i,j}^{n,h}) \times da_{i,j}^{n,h}$$
$$\mathbf{dN}^{n,h} = \dot{\mathbf{F}}^{n,h} \circ \mathbf{dA}^{n,h}$$

Solved Problems

P8.1 Demonstrate how the kernel in the convolution operation is flipped vertically and horizontally relative to the kernel in the correlation operation.

The convolution operation can be written

$$z_{m,n} = \sum_i \sum_j w_{i,j} p_{m-i,n-j}$$

or, equivalently as

$$z_{m,n} = \sum_i \sum_j p_{i,j} w_{m-i,n-j}$$

where the summations range over all indices where both w and p are nonzero. We will use the second implementation in this demonstration.

Consider the 3x3 kernel $w_{i,j}$ shown in the following figure.

	2	0	0	7	8	9
	1	0	0	4	5	6
j	0	0	0	1	2	3
	-1	0	0	0	0	0
	-2	0	0	0	0	0
		-2	-1	0	1	2
				i		

Let's demonstrate how to go from $w_{i,j}$ to $w_{m-i,n-j}$, which is needed in the equation above. We will follow the steps

$$w_{i,j} \rightarrow w_{-i,j} \rightarrow w_{-i,-j} \rightarrow w_{m-i,n-j}$$

To go from $w_{i,j}$ to $w_{-i,j}$, we need to flip the kernel from left to right, as shown in the following figure.

Convolution

2	9	8	7	0	0
1	6	5	4	0	0
j 0	3	2	1	0	0
-1	0	0	0	0	0
-2	0	0	0	0	0
	-2	-1	0	1	2
	i				

To go from $w_{-i,j}$ to $w_{-i,-j}$, we need to flip the kernel from top to bottom, as shown in the following figure.

2	0	0	0	0	0
1	0	0	0	0	0
j 0	3	2	1	0	0
-1	6	5	4	0	0
-2	9	8	7	0	0
	-2	-1	0	1	2
	i				

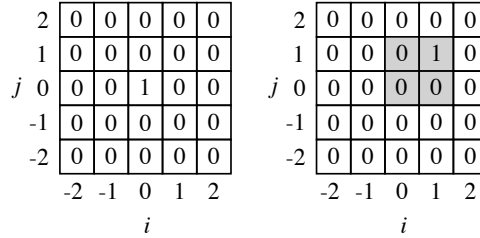
Finally, to go from $w_{-i,-j}$ to $w_{m-i,n-j}$, we need to shift the kernel by m pixels on the x -axis and n pixels on the y -axis, as shown in the following figure.

$n+1$	0	0	0	0	0
n	0	3	2	1	0
j $n-1$	0	6	5	4	0
$n-2$	0	9	8	7	0
$n-3$	0	0	0	0	0
	$m-3$	$m-2$	$m-1$	m	$m+1$
	i				

Therefore, as we calculate $z_{m,n}$ for different values of m and n , the convolution kernel is flipped vertically and horizontally and then moved across the image. At each value of m and n , an inner product is computed between the flipped and shifted kernel and the input image. This operation is equivalent to a correlation operation, if we use the flipped convolution kernel as the correlation kernel.

P8.2 As discussed in this chapter, any linear, position invariant transformation can be represented by a convolution operation, where the convolution kernel is the impulse response of the transformation. Consider the transformation of shifting the input image one pixel to the right and one pixel up. Find the convolution kernel for this transformation by finding the impulse response.

To find the impulse response, we need to apply the transformation to the impulse function (one-point image). In this case, that means that we shift the impulse function one pixel to the left and one pixel up. This process is shown in the following figure.



Based on this result, the convolution kernel will be

$$\mathbf{W} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

P8.3 Find the backpropagation path $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n$ for the average pooling function.

As with the convolution weight function, for the $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n$ path, we use Eq. 8.39. However, each $a_{u',v'}^{n,l}$ affects only one $l_{i,j}^{m,n,h}$, since the stride of the pooling operation is equal to the size of the kernel. Therefore there is no summation required. The equation reduces to

$$da_{u',v'}^{n,h} \equiv \frac{\partial F}{\partial a_{u',v'}^{n,h}} = \frac{\partial F}{\partial l_{i,j}^{m,n,h}} \times \frac{\partial l_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,h}} = dl_{i,j}^{m,n,h} \times \frac{\partial l_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,h}}$$

It remains to find

$$\frac{\partial l_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,l}}$$

Convolution

The average pooling operation is defined in Eq. 8.22, and repeated here:

$$l z_{i,j}^{m,n,h} = \left\{ \sum_{u=1}^{r^m} \sum_{v=1}^{c^m} a_{r^m(i-1)+u, c^m(j-1)+v}^{n,h} \right\} l w^{m,n,h}$$

(Notice that for the average pooling weight function, in contrast to the convolution weight function, each input feature map contributes to one output feature map.) From this equation, we can see that

$$\frac{\partial l z_{i,j}^{m,n,h}}{\partial a_{u',v'}^{n,l}} = l w^{m,n,h} \text{ for } \begin{cases} r^m(i-1)+1 \leq u' \leq r^m(i-1)+r^m \\ c^m(j-1)+1 \leq v' \leq c^m(j-1)+c^m \end{cases}$$

From these equations we can see that each element of $\mathbf{dLZ}^{m,n,h}$ will produce $r^m \times c^m$ elements of $\mathbf{dA}^{n,h}$. We can represent this using an abbreviated notation as

$$\mathbf{dA}^{n,h} = (l w^{m,n,h}) \boxtimes_{r^m, c^m}^{ave} (\mathbf{dLZ}^{m,n,h})$$

where $\boxtimes_{j,k}^{ave} \mathbf{A}$ takes each element of the matrix $\mathbf{A}$ and expands to j rows and k columns (reverse of $\boxplus_{j,k}^{ave} \mathbf{A}$).

P8.4 Find the backpropagation path $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n}$ for the average pooling function.

The $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dLW}^{m,n}$ backpropagation path computes the portion of the gradient for the weight $l w^{m,n,h}$. We can use Eq. 8.46, but it can be simplified, since the weight is a scalar, and each feature map operates separately.

$$d l w^{m,n,h} \equiv \frac{\partial F}{\partial l w^{m,n,h}} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} \frac{\partial F}{\partial l z_{i,j}^{m,n,h}} \times \frac{\partial l z_{i,j}^{m,n,h}}{\partial l w^{m,n,h}}$$

From the average pooling equation we can convert this to

$$d l w^{m,n,h} = \sum_{i=1}^{S_r^m} \sum_{j=1}^{S_c^m} d l z_{i,j}^{m,n,h} \left\{ \sum_{u=1}^{r^m} \sum_{v=1}^{c^m} a_{r^m(i-1)+u, c^m(j-1)+v}^{n,h} \right\}$$

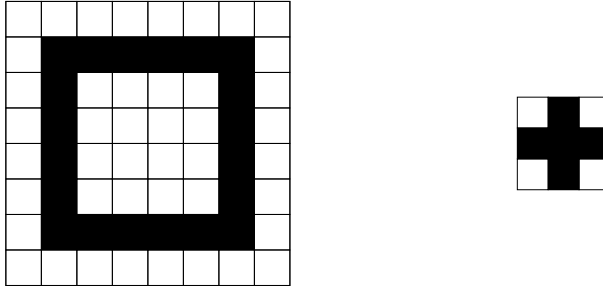
In abbreviated notation, this can be written

$$d l w^{m,n,h} = \boxplus_{S_r^m, S_c^m}^{ave} (\mathbf{dLZ}^{m,n,h} \circ (\boxplus_{r^m, c^m}^{ave} \mathbf{A}^{n,h}))$$

First it performs a reduction operation that produces a matrix of size S_r^m by S_c^m , and then, after a Hadamard matrix multiplication, it performs another reduction to produce the scalar gradient.

Exercises

- E8.1** Consider the 8x8 input image below and the corresponding 3x3 kernel. The black squares correspond to values of 1, and the white squares correspond to values of zero.



- i. Manually perform a convolution of the kernel on the image, with a stride of 1 and a zero padding of 1.
 - ii. What types of objects is this kernel good at detecting?
- E8.2** For your output from Exercise E8.1, manually perform a 2x2 max pooling operation, with a stride of 2.
- E8.3** Design a minimal set of convolution kernels that could be useful in identifying characters from the following font family.

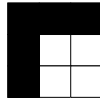
ABCDEFGHIJKLMNOPQRSTUVWXYZ

- E8.4** Demonstrate that the convolution kernel found in Solved Problem P8.2 does perform the required shifting operation by applying it manually to the image below.

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	1	1	0	0
0	0	0	0	0

Convolution

- E8.5** We want to perform a linear, position invariant transformation, which is to average adjacent pixels in an input image to produce the pixels in an output image.
- Using the concepts from Solved Problem P8.2, find the convolution kernel that will perform this transformation.
 - Test your kernel on the input image in Exercise E8.1.
 - What is the corresponding correlation kernel that will perform the same operation? Use the ideas from Solved Problem P8.1 to find it.
- E8.6** The objective of this exercise is to investigate the difference between convolution and correlation. Consider the following 3×3 convolution kernel.



- Manually perform a convolution (not correlation) of this kernel with the image in Exercise E8.1, with a stride of 1 and no padding.
 - Perform a correlation with the same kernel, and compare the results with part i.
 - Find the appropriate correlation kernel that will produce the same output that you obtained in part i. In other words, find the kernel that will produce the same output when used for correlation that the original kernel produces when used for convolution. (See Solved Problem P8.1 for a discussion of the relationship between convolution and correlation kernels.)
 - Perform a correlation with your new kernel, and verify that you obtain the same output as in part i.
- E8.7** Suppose that we have a 15×15 input image. This is input to a convolution layer with one feature map.

- i. If the convolution kernel size is 3×3 , the stride is $S^t = 2$ and the zero padding is $P^d = 1$, what is the size of the output feature map?
- ii. If the convolution kernel size is 3×3 and the stride is $S^t = 1$, what does the padding have to be so that the size of the output feature map is the same as the size of the input image?

- E8.8** Find the backpropagation path $\mathbf{dLZ}^{m,n} \rightarrow \mathbf{dA}^n$ for the max pooling operation.
- E8.9** Consider a layer for which the net input function is the product between a scalar bias and the output of the weight function, instead of the standard summation. Find the two backpropagation paths across this net input function: $\mathbf{dN}^m \rightarrow \mathbf{dLZ}^{m,n}$ and $\mathbf{dN}^m \rightarrow db^m$
- E8.10** Using the kernels in Figure 8.5, design a convolution network to distinguish between squares and diamonds. The network should have a convolution layer, a max pooling layer and a fully connected layer with a softmax activation. The input to the network will be a 15×15 image, like the images in Figure 8.4. Specify all of the weights and biases in the network. Remember that the network is to be designed, not trained.

9 *Post Training Analysis*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>What is Explainability?</i>	2
<i>Some Definitions of Explainability</i>	4
<i>Illustrative Examples</i>	6
<i>Notation and General Explainability Definitions</i>	10
<i>Linearized Model Methods</i>	12
<i>Contribution Propagation Methods</i>	14
<i>Epilogue</i>	20
<i>Further Reading</i>	21
<i>Summary of Results</i>	22
<i>Solved Problems</i>	24
<i>Exercises</i>	37

Objective

After a network has been trained, it is important to analyze the network operation carefully to determine if the training was successful. This is especially important for deep networks, which can be difficult to analyze. One purpose of the analysis is to help explain how the networks are making their decisions. When networks are used for making medical diagnoses or large financial investment decisions, for example, users want to have confidence in the operation of the network. Explaining network decisions is far from an exact science, but in this chapter we will describe several approaches that have been useful.

What is Explainability?

A major purpose of post training analysis is to develop confidence in the trained network. Deep networks are taking on tasks that could formerly only be accomplished by humans. Humans can always be questioned about their decisions, and their explanations can help us gain (or lose) confidence in their decisions. Can we question deep networks, and what form would the resulting explanations take? (If you think human decisions are more transparent than artificial neural network decisions, read the following story.)

Are human decisions transparent?

Much attention has been given to the fact that deep networks are black boxes. The fact is that we have even less understanding about how we humans make our decisions. As we will show in this chapter, at least we can dig around inside deep networks. Our access to the inner workings of the human nervous system is much less advanced. Can you explain how you recognize your mother's face? Do you think your explanation actually describes how your brain is able to pick her out of a crowd in milliseconds?

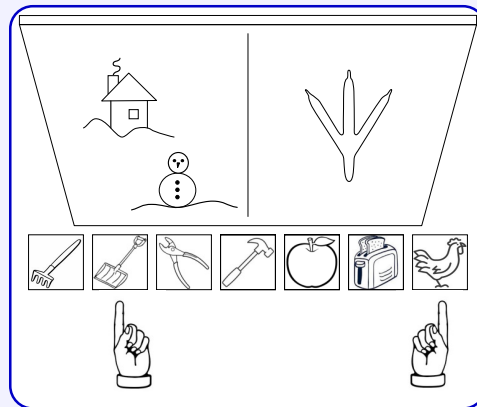
A fascinating set of experiments by Michael Gazzaniga [Gazzaniga and LeDoux, 2013] showed that human decision making is less than transparent.

First, consider some basic brain structure. Your brain has two halves (called hemispheres) that are connected by a bridge of nerve fibers called the corpus callosum. The left side of your brain controls the right side of your body, and the right side of your brain controls the left side of your body. For most right-handed people, the left side of the brain is also responsible for speech.

Gazzaniga worked with special patients who had their corpus callosum surgically cut (this was done to help treat severe epilepsy). This meant the two sides of their brain couldn't communicate with each other as they normally would.

Here is one of his experiments:

- He showed a picture of a chicken claw to the right side of a patient's field of vision (which goes to the left brain)
- At the same time, he showed a snowy scene to the left side of their vision (which goes to the right brain)
- Then he asked the patient to point to related pictures from a collection that included things like a chicken, a snow shovel, an apple, and other objects



The results were surprising. The patient's left hand (controlled by the right brain) pointed to the snow shovel, which makes sense because that side of the brain saw the snow scene. But when asked to explain why they chose the snow shovel, the patient said, "Oh, you need a shovel to clean out the chicken coop!"

Why did this happen? The speaking part of the brain (left side) only saw the chicken claw, not the snow scene. So it made up a logical-sounding explanation to connect the chicken with the shovel, not knowing that the other half of the brain had actually chosen the shovel because of the snow scene!

This experiment shows us something remarkable: our brain creates explanations for our actions after the fact, even when those explanations aren't actually true!

The human brain is exquisitely wired to provide explanations and an inner narrative, so that our world appears comprehensible. The explanations don't need to be true – just somewhat plausible (and plausibility is in the eye of the beholder). Perhaps they should be called rationalizations or justifications, rather than explanations. The same might be said for explanations of neural network decisions.

It seems natural that if deep networks are going to take on complex tasks that were once reserved for humans, the decisions they make may not be more understandable than human decisions. It also seems reasonable that when complex decisions are made the explanations may be just as complex. As Einstein once said, "It would be possible to describe everything scientifically, but it would make no sense." In this chapter we will do our best to describe methods that can be used to provide insights into the operation of trained deep networks. We will not claim that they provide complete explanations. Together, the various post-training analysis tools that we discuss in this chapter help to provide an overall narrative that can provide confidence in decisions that deep networks make.

Some Definitions of Explainability

What does it mean to provide an *explanation* for a deep network decision? To begin, what is the meaning of "explain?" Merriam Webster's dictionary says "to give the reason for or cause of." It also identifies some related words as: clarify, explicate, illustrate, interpret. What would this mean in the context of a deep network? (When considering the "explanations" discussed in this chapter, remember that they are explanations in a narrow, technical, mathematical sense rather than in the everyday meaning of the word.)

Researchers have tried to come up with explanations for the decisions of machine learning models for some time. One of the early efforts was by Irving Good in 1977 [Good, 1977]. He introduced the concept *weight of evidence* in terms of probabilities. If H is an hypothesis and E is evidence for that hypothesis, then Good defined the weight of evidence $W(H : E)$ to be

$$W(H : E) = \ln \frac{P(E|H)}{P(E|\bar{H})} \quad (9.1)$$

where $P(E|H)$ is the conditional probability of E given H , and $\bar{H}$ is the complement of H . So, if E was more likely to occur when H was true, the weight of evidence would be positive.

This idea was eventually applied to Bayesian belief networks by Madigan [Madigan et al., 1997], who also showed how the weight of evidence could be propagated backward through the network using Bayes rule. Poulin [Poulin et al., 2006] extended the idea to what he called "additive" models. An example would be a single layer network with a sigmoid or softmax activation function. To see how this would work, consider a single layer network with two inputs, p_1 and p_2 , and no bias.

$$n_1 = w_{1,1}p_1 + w_{1,2}p_2 \quad (9.2)$$

$$a_1 = \frac{1}{1 + e^{-n_1}} \quad (9.3)$$

In terms of Eq. 9.1, a_1 would correspond to the probability that the hypothesis H is true, given the input $\mathbf{p}$, which would correspond to the evidence E . The probability of $\bar{H}$ given the evidence would be $1 - a_1$. If we then assume that H and $\bar{H}$ have equal prior probability ($P(H) = P(\bar{H})$), we can write

$$\begin{aligned} W(H : E) &= \ln \frac{P(E|H)}{P(E|\bar{H})} = \ln \frac{\frac{P(H|E)P(E)}{P(H)}}{\frac{P(\bar{H}|E)P(E)}{P(\bar{H})}} = \ln \frac{P(H|E)P(\bar{H})}{P(\bar{H}|E)P(H)} \quad (9.4) \\ &= \ln \frac{P(H|E)}{P(\bar{H}|E)} = \ln \frac{\frac{1}{1+e^{-n_1}}}{\frac{e^{-n_1}}{1+e^{-n_1}}} = \ln e^{n_1} = n_1 = w_{1,1}p_1 + w_{1,2}p_2 \quad (9.5) \end{aligned}$$

The weight of evidence is therefore the net input. It has two components: $w_{1,1}p_1$ is the evidence associated with the first input, and $w_{1,2}p_2$ is the evidence associated with the second input. This result is appealing – the evidence associated with each input is the amount that the input contributes to the total net input, which is the weight times the input.

Extending the weight of evidence to deep neural networks is a challenge. It requires distributing the evidence appropriately across layers to each successive layer input. There have been several different approaches to this distribution that we will discuss in this chapter.

Post Training Analysis

Before we get into the details of the specific algorithms, let's look at a couple of simple examples to think about what it would mean to "explain" network operation. (Simple examples can sometimes be misleading when extending the concepts to high-dimensional spaces, but these examples are helpful in introducing some of the key ideas associated with explainability.)

Illustrative Examples

What is the purpose of an explanation? In part, it is to contribute to a narrative that gives confidence to whoever is using the network. To help us think about what an explanation would be for a neural network, consider the simple single layer network we considered in Chapter 2.

The network diagram is repeated in Figure 9.1. The activation function is *logsig*, which is defined in Eq. 9.3. This produces a simple decision making network.

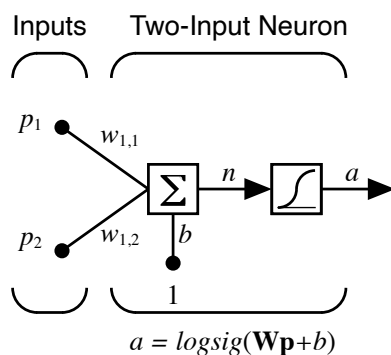


Figure 9.1: Example One Layer Network

Let the weight matrix be $\mathbf{W} = \begin{bmatrix} 1 & 1 \end{bmatrix}$, and the bias $b = [-1]$. The network response is

$$a = f(n) = f(\mathbf{W}\mathbf{p} + b) = \text{logsig}(1p_1 + 1p_2 - 1) \quad (9.6)$$

The network output will be greater than 0.5 (probability of positive detection) whenever the net input is greater than or equal to 0. This defines the decision boundary, which is shown in Figure 9.2. In a way, this figure can be thought of as the *global explanation* for how the network makes its decision. The weight vector is orthogonal to the decision boundary and points in the direction where the

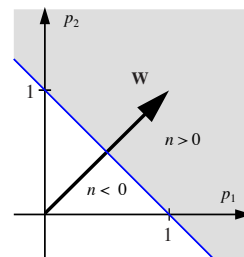


Figure 9.2: Linear Decision Boundary

network output is increasing most rapidly. As the bias changes, the decision boundary moves along the weight vector. (See Chapter 4 of [NND2](#) for a more extensive discussion of one-layer networks and linear decision boundaries.)

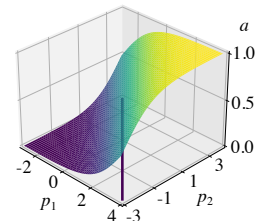
It is also useful to look at the total network response, which we will compare with the response of a multilayer network in another example. The figure in the margin shows the response, and indicates the decision boundary (where $a = 0.5$) in the p_1 - p_2 plane. For a single-layer network the gradient of the network response is always in the direction of the weight, which is orthogonal to the decision boundary. The surface slopes up in the direction of the weight, no matter where we are in the input space.

So, for a single-layer network, the decision boundary or the weight vector provides a global explanation for how the network makes decisions. Because $\mathbf{W} = \begin{bmatrix} 1 & 1 \end{bmatrix}$, the network seems to consider p_1 and p_2 equally in making the decision. However, what if we consider the specific input to the network shown in the margin.

The net input will be $n = 1 \times 2 + 1 \times 0 - 1 = 1$, and the network output will be greater than 0.5, to indicate a positive class. What are the contributions of each input to the result? Although globally the network counts each input equally, in this case the first input contributed 2 to the result and the second input contributed 0. Therefore the first input is more important to this specific decision than the second input. This is a *local explanation* of the result for this particular input vector. Note that this explanation matches with Good's weight of evidence, as shown in Eq. 9.5, where the contributions are equal to the weights times the inputs.

So, from this simple two-input/single-layer network we have some ideas for global and local explanations for network operation. How can we extend these explanations to networks with large numbers of inputs and with more layers? If we have a high-dimensional input vector, it is not possible to view the decision boundary, but we can look at the elements of the weight matrix to see how important each element of the input vector is. What about when the number of layers is increased?

From Chapter 2 (and Chapter 11 of [NND2](#)), we know that the decision boundaries for multilayer networks are not generally linear and can be made arbitrarily complex with increased numbers of neurons and/or layers.



$$\mathbf{p} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\begin{array}{r} 2 \\ +2 \\ \hline 4 \end{array}$$

Post Training Analysis

For example, consider the three-layer network that implemented the radial function in Chapter 2, whose weights are given in Eq. 2.11. If we add a *logsig* activation function in the last layer, we have the network response shown in Figure 9.3. On the contour plot of Figure 9.3b the red curve represents the decision boundary (where the network output is 0.5).

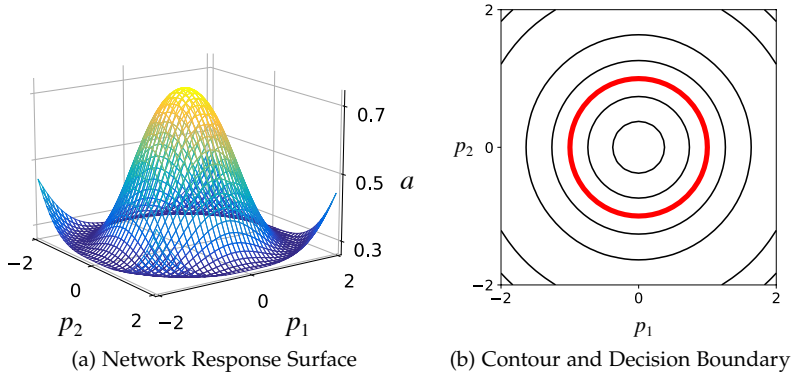


Figure 9.3: Radial Network Response Surface and Contour

In a way, Figure 9.3 is a global explanation for the operation of the network. Any input inside the red circle will be positively classified, any input outside the red circle will be negatively classified, and the closer the input is to the center of the circle the more confident the network decision will be. However, does this help provide a narrative that would give confidence to a user of the network? Especially when the input space is high-dimensional, the decision boundary itself would not provide a practical explanation of network behavior.

In the case of a multilayer network, one way to approach explainability is to linearize the network response at a particular input vector and to use the explainability methods for a one-layer network. This can be done using a Taylor series expansion. (See Chapter 8 of [NND2](#) for a full discussion of Taylor series expansions.) From Eq. 3.10 we can write a linearized approximation of the network response as

$$a(\mathbf{p}) \cong a(\mathbf{p}^*) + \nabla a(\mathbf{p})^T|_{\mathbf{p}^*} (\mathbf{p} - \mathbf{p}^*) \quad (9.7)$$

where $\mathbf{p}^*$ is the input where the expansion takes place, and $\nabla a(\mathbf{p})$ is the gradient

$$\nabla a(\mathbf{p}) = \left[\frac{\partial a(\mathbf{p})}{\partial p_1} \quad \frac{\partial a(\mathbf{p})}{\partial p_2} \quad \dots \quad \frac{\partial a(\mathbf{p})}{\partial p_R} \right]^T \quad (9.8)$$

The gradient is then the weight matrix for the approximate linear network. For global explainability, we could use the components of the gradient as measures of how important each element of the input is. For local explainability, we could use the gradient times the input (as in Good's weight of evidence).

How would this work for the radial network? If we linearize about the input

$$\mathbf{p}^* = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad (9.9)$$

we have

$$a(\mathbf{p}) \cong 0.5 + \begin{bmatrix} 0 & 0.4 \end{bmatrix} \left(\mathbf{p} - \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right) = \begin{bmatrix} 0 & 0.4 \end{bmatrix} \mathbf{p} + 0.9 \quad (9.10)$$

Figure 9.4a shows the linear response surface along with the original network response. The star indicates the p^* point. Figure 9.4b shows the directions where the gradients point at various points in the input space.

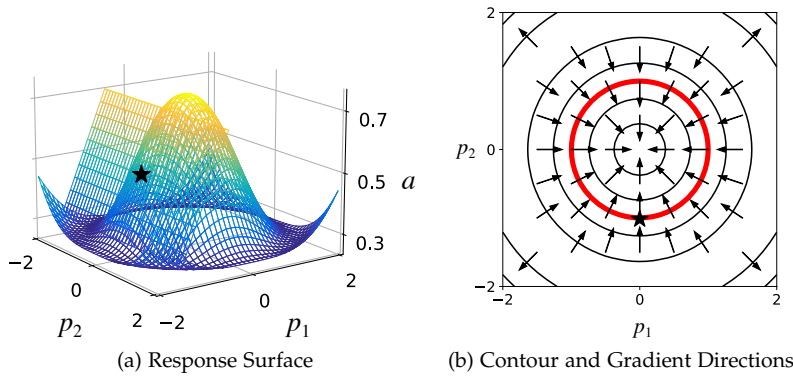


Figure 9.4: Approximate Linear Response Surface

Post Training Analysis

Based on the linearization in Eq. 9.10, the first input p_1 does not contribute to the response. It is only the second input p_2 that is important to the decision. However, if we linearize about a different input location the contributions can change completely. Can you identify an input location where the second input has no contribution to the decision?

Consider the case where the input is $\mathbf{p} = [0 \ 0]^T$. What decision does the network make? Is the network confident in its decision? What contributions do each of the inputs make to the decision? How would you "explain" the network decision?

Keep this example in mind as we go into some detail for specific explainability methods in the remainder of chapter.

To experiment with linearizing this radial network, use the Deep Learning Demonstration Linearized Network Response (dl9lnr).



Using the linearization method, the gradient takes the place of the weight. So the individual elements of the gradient represent global contributions of each input, and the product of the gradient elements with each input represent the local contributions. Of course, since for multilayer networks the gradient will be different in different regions of the input space, the gradient could not be used as a global explanation. For that reason, in the remainder of the chapter we will use gradient times input, instead of gradient alone, as an explanation.

Notation and General Explainability Definitions

To some extent, the explainability approach of Good/Poulin is used in all the methods discussed in this chapter – but extended to multilayer networks. In the Good/Poulin approach the *contribution* of a given input, for a single-layer network, is equal to the product of the input times the corresponding weight.

In this chapter we discuss two methods to extend this approach to multilayer networks. The first method, demonstrated for the radial network in the previous subsection, is to linearize the network about the relevant input vector. The gradient of the network output

with respect to the network input forms the approximate weight of the linearized network. The contribution of each input element is then computed as the product of the input element times the corresponding element of the gradient.

The second method is to backpropagate the contributions through the network, with intermediate contributions being computed at the input to each layer. The intermediate contributions at the last layer can be computed using the standard method for single-layer networks. These must then be backpropagated to each preceding layer until reaching the network input. The backpropagation must be done so as to conserve total contribution at each successive layer.

Before introducing the methods, let's define the notation that we are going to use and describe some properties that we would like explainability methods to satisfy.

First, we will use the word *contribution* to refer to the amount each element of the input contributes to a decision. Taken as a whole, the contributions of all elements of the input make up the *explanation* of the decision. The technical meaning of the word *contribution* will be different for each method that we present, as will the meaning of *explanation*. We will use the letter c to represent the contribution. If the network input is a vector, we will use $\mathbf{c}$. If the network input is a matrix, we will use $\mathbf{C}$.

When discussing local explainability, we will be attempting to explain a particular decision for a specified network input. In other words, why did the network assign that input to that class. This means we will be considering only the network output that corresponds to the relevant class. We will indicate that output as a_c^M , where M is the final layer of the network, and c is the relevant class. (This c can be distinguished from the contribution, because it always appears as a subscript.)

It is difficult to compare the various explainability methods objectively, since there is no *ground truth* to compare the results against. However, there are several properties that we would like an explainability method to have [Sundararajan et al., 2017]. For example,

1. *Continuity* : If $a_c^M(\mathbf{p})$ is continuous, then $\mathbf{c}(\mathbf{p})$ is continuous.

For ease of presentation we will present explainability methods in the context of standard multilayer (fully connected) networks. The methods can be extended in a straightforward way to convolution and other types of networks.

You will find that many authors use names other than contribution, such as attribution, weight of evidence, relevance, importance, explanation, coefficient, etc.

Post Training Analysis

2. *Implementation Invariance* : If two networks are functionally equivalent (same input produces same output), then the computed contributions are the same.
3. *Sensitivity*
 - (a) If an input and a baseline differ in one feature and have different network outputs, a nonzero contribution is assigned to that feature.
 - (b) If the network output does not depend on a feature, the contribution of that feature is zero.
4. *Completeness* : The sum of all contributions is equal to the network output (or the difference between the output and a baseline output).

In the end, explainability methods are usually compared subjectively – do they make sense to human observers, and do they further an understandable narrative? However, it is helpful to know which objective properties each method has. As we cover the different methods, we will note their properties.

Linearized Model Methods

In this subsection we describe two *gradient-based* explainability methods. In the following subsection we describe two *contribution propagation* methods.

SALIENCY: Vanilla *saliency* [Simonyan et al., 2013] is simply the gradient of the key network output with respect to the network inputs. Although some authors will use the gradient directly, we will always multiply by the input to compute the contribution.

$$c^{sal}(p_i) = \frac{\partial n_c^M(\mathbf{p})}{\partial p_i} \times p_i \quad (9.11)$$

The gradient of the network output with respect to the network input is computed using the same technique that we used to compute the gradient of the performance function with respect to the network weights – backpropagation (see page 3-11). Eq. 3.49 again forms the key backpropagation operation, which we repeat here:

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m)(\mathbf{W}^{m+1})^T \mathbf{s}^{m+1} \quad (9.12)$$

where m is decremented from $M - 1$ to 1.

To initialize $\mathbf{s}^M$, since we want the derivative of n_c^M , instead of the derivative of $F(\mathbf{x})$, we use

$$\mathbf{s}^M = \frac{\partial n_c^M}{\partial \mathbf{n}^M} = \boldsymbol{\epsilon}_c \quad (9.13)$$

where $\boldsymbol{\epsilon}_c$ is a vector whose element c is 1 and whose other elements are zero.

Finally, the contribution is computed from the initial sensitivity:

$$\mathbf{c}^{sal}(\mathbf{p}) = \frac{\partial n_c^M}{\partial \mathbf{p}} \circ \mathbf{p} = \left(\frac{\partial (\mathbf{n}^1)^T}{\partial \mathbf{p}} \frac{\partial n_c^M}{\partial \mathbf{n}^1} \right) \circ \mathbf{p} = \left((\mathbf{W}^1)^T \mathbf{s}^1 \right) \circ \mathbf{p} \quad (9.14)$$

INTEGRATED GRADIENT: One of the issues with saliency is that the gradient can be zero at a point where a network decision can be very confident. (For example consider the input $\mathbf{p} = [0 \ 0]^T$ for the radial network example we discussed earlier.) This means that saliency does not satisfy the sensitivity property. (It also fails to satisfy the completeness property, although it does satisfy the implementation invariance property, since the gradient only depends on the function being implemented by the network.) One solution to this problem is to average the gradient over some region. The *integrated gradient* method [Sundararajan et al., 2017] averages the gradient over a line from some baseline input $\bar{\mathbf{p}}$ to the current input $\mathbf{p}$:

$$\mathbf{c}^{ig}(\mathbf{p}) = \left[\frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial n_c^M(\tilde{\mathbf{p}})}{\partial \tilde{\mathbf{p}}} \Big|_{\tilde{\mathbf{p}}=\mathbf{p}^t} \right] \circ (\mathbf{p} - \bar{\mathbf{p}}) \quad (9.15)$$

where

$$\mathbf{p}^t = \bar{\mathbf{p}} + \frac{t}{T-1} (\mathbf{p} - \bar{\mathbf{p}}) \quad (9.16)$$

Eq. 9.15 is an approximation of the line integral of the gradient along a line from $\bar{\mathbf{p}}$ to $\mathbf{p}$. From the fundamental theorem of calculus for line integrals, this means that

$$\sum_{i=1}^R c_i^{ig}(\mathbf{p}) = n_c^M(\mathbf{p}) - n_c^M(\bar{\mathbf{p}}) \quad (9.17)$$

Post Training Analysis

Therefore integrated gradient satisfies the completeness property. It also satisfies the sensitivity properties, since if the input and the baseline differ in one feature and have different outputs, the contribution will equal the difference in the outputs.

There is another algorithm, *SmoothGrad* [Smilkov et al., 2017], that is related to integrated gradients. It averages the gradient at a number of points randomly scattered around the current network input. Unlike integrated gradient, SmoothGrad does not satisfy the completeness property.

A related method that randomly scatters points around the current network input is LIME (Local Interpretable Model-agnostic Explanations) [Ribeiro et al., 2016]. This method does not use the gradient, but it fits a linear model to the network outputs computed at a number of random points in the neighborhood of the current network input. It fits the linear model so that the errors for each random point are weighted inversely to their distance from the current input. After the approximate linear model is found, its weights can be used to compute the contributions. This method does not satisfy the completeness property, because the linear model is only an approximation to the deep network.

Contribution Propagation Methods

In contribution propagation methods the internal structure of the model is used to distribute contributions from higher layers to lower layers until reaching the network input. The output layer (M) can be considered a single-layer network whose inputs are the outputs of the previous layer ($\mathbf{a}^{M-1}$). We can compute the contributions for $\mathbf{a}^{M-1}$ using the concepts for a single-layer network. Then we need to propagate those contributions backward through the network.

Landecker and colleagues [Landecker et al., 2013] first introduced some key concepts for how to backpropagate the contributions. The basic operation for a multilayer (fully connected) network is

$$\mathbf{c}(\mathbf{a}^m) = \mathbf{C}(\mathbf{a}^m | \mathbf{a}^{m+1}) \mathbf{c}(\mathbf{a}^{m+1}), \quad m = M-1, M-2, \dots, 0 \quad (9.18)$$

where

$$\left[\mathbf{C}(\mathbf{a}^m | \mathbf{a}^{m+1}) \right]_{i,j} = c(a_i^m | a_j^{m+1}) \quad (9.19)$$

is the partial contribution of a_i^m to a_j^{m+1} . Eq. 9.18 is initialized with

$$\mathbf{c}(\mathbf{a}^M) = n_c^M \mathbf{e}_c \quad (9.20)$$

In order to satisfy the completeness property, we need to choose the multipliers $c(a_i^m | a_j^{m+1})$ so that the total contributions at each layer remain equal to the key network output:

$$\sum_{i=1}^{S^m} c(a_i^m) = n_c^M, \quad m = 0, 1, \dots, M-1 \quad (9.21)$$

where $c(a_i^0) = c(p_i)$.

This can be guaranteed if we require the columns of $\mathbf{C}(a^m | a^{m+1})$ to sum to 1:

$$\sum_{i=1}^{S^m} c(a_i^m | a_j^{m+1}) = 1 \quad (9.22)$$

To show that this is true, rewrite Eq. 9.18 in scalar form:

$$c(a_i^m) = \sum_{j=1}^{S^{m+1}} c(a_i^m | a_j^{m+1}) c(a_j^{m+1}) \quad (9.23)$$

If we then sum both sides over i and reorder the summations we have

$$\sum_{i=1}^{S^m} c(a_i^m) = \sum_{i=1}^{S^m} \sum_{j=1}^{S^{m+1}} c(a_i^m | a_j^{m+1}) c(a_j^{m+1}) \quad (9.24)$$

$$\sum_{i=1}^{S^m} c(a_i^m) = \sum_{j=1}^{S^{m+1}} c(a_j^{m+1}) \sum_{i=1}^{S^m} c(a_i^m | a_j^{m+1}) \quad (9.25)$$

If Eq. 9.22 is true, then Eq. 9.25 becomes

$$\sum_{i=1}^{S^m} c(a_i^m) = \sum_{j=1}^{S^{m+1}} c(a_j^{m+1}) = \sum_{j=1}^{S^M} c(a_j^M) = n_c^M \quad (9.26)$$

Therefore the total contribution remains constant across layers.

We will describe two different methods for contribution propagation. They differ in how they select the backpropagation multipliers $c(a_i^m | a_j^{m+1})$.

Post Training Analysis

LAYERWISE RELEVANCE PROPAGATION (LRP): [LRP](#) [Bach et al., 2015] is a contribution propagation method that uses Eq. 9.18 to backpropagate the contributions. Within the LRP framework, there are several choices for the multipliers $c(a_i^m | a_j^{m+1})$. The simplest choice is termed LRP-0:

$$c^{lrp-0}(a_i^m | a_j^{m+1}) = \frac{w_{j,i}^{m+1} a_i^m}{\sum_{k=1}^{S^m} w_{j,k}^{m+1} a_k^m} \quad (9.27)$$

We can think of $c(a_i^m | a_j^{m+1})$ as the proportion that a_i^m contributes to a_j^{m+1} . In Eq. 9.27, the numerator is the amount that a_i^m contributes, and the denominator is a normalizing factor. This guarantees that Eq. 9.22 is satisfied, so that LRP has the completeness property. The bias b_j^{m+1} is sometimes also included in the denominator, but then completeness is not exactly satisfied.

There are some variations of the LRP rule. Sometimes the denominator in Eq. 9.22 can be close to zero, which can cause the method to be noisy. One solution for this problem is to add a small number to the denominator. This is called the LRP- ϵ rule:

$$c^{lrp-\epsilon}(a_i^m | a_j^{m+1}) = \frac{w_{j,i}^{m+1} a_i^m}{\epsilon + \sum_{k=1}^{S^m} w_{j,k}^{m+1} a_k^m} \quad (9.28)$$

Another variation is called the LRP- γ rule:

$$c^{lrp-\gamma}(a_i^m | a_j^{m+1}) = \frac{\left(w_{j,i}^{m+1} + \gamma (w_{j,i}^{m+1})^+\right) a_i^m}{\sum_{k=1}^{S^m} \left(w_{j,k}^{m+1} + \gamma (w_{j,k}^{m+1})^+\right) a_k^m} \quad (9.29)$$

where $(.)^+ = \max(0, .)$. The parameter γ can be used to favor positive contributions over negative ones. (The method is appropriate for layers with poslin [ReLU] activation functions, where the layer outputs will always be positive.)

The methods for choosing which LRP version to use and setting the parameters ϵ and γ are heuristic. It is common to use different rules in different layers (e.g., LRP- γ for lower layers, LRP- ϵ for middle layers and LRP-0 for upper layers.) Since there is no ground truth for contribution values, results are assessed qualitatively. Ideally, the significant contributions will be sparse and focused on contiguous sections of an image.

DEEPLIFT: *DeepLIFT* (Deep Learning Important FeaTures) [Shrikumar et al., 2017] is another contribution propagation method. Unlike LRP, DeepLIFT measures contributions relative to a baseline. The LRP contributions sum to n_c^M , and the DeepLIFT contributions sum to Δn_c^M , where

$$\Delta n_c^M = n_c^M(\mathbf{p}) - n_c^M(\bar{\mathbf{p}}) = n_c^M - \bar{n}_c^M \quad (9.30)$$

where, as with the integrated gradient method, $\bar{\mathbf{p}}$ is a baseline input.

The contributions for DeepLIFT are defined in terms of changes and are related to previous contributions as follows:

$$c^{dl}(\Delta a_i^m) = c^{dl}(\Delta a_i^m | \Delta n_c^M) = c(a_i^m) \quad (9.31)$$

$$c^{dl}(\Delta a_i^m | \Delta a_j^{m+1}) = c(a_i^m | a_j^{m+1}) \Delta a_j^{m+1} \quad (9.32)$$

In DeepLIFT there are also modified multipliers, q , that correspond to the standard multipliers, but divided by the change:

$$q(\Delta a_i^m | \Delta a_j^{m+1}) = \frac{c^{dl}(\Delta a_i^m | \Delta a_j^{m+1})}{\Delta a_i^m} = c(a_i^m | a_j^{m+1}) \frac{\Delta a_j^{m+1}}{\Delta a_i^m} \quad (9.33)$$

$$q(\Delta a_i^m) = \frac{c^{dl}(\Delta a_i^m)}{\Delta a_i^m} = c(a_i^m) \frac{1}{\Delta a_i^m} \quad (9.34)$$

By reversing these equations, we can relate the modified multipliers to the original:

$$c(a_i^m | a_j^{m+1}) = q(\Delta a_i^m | \Delta a_j^{m+1}) \frac{\Delta a_i^m}{\Delta a_j^{m+1}} \quad (9.35)$$

$$c(a_i^m) = q(\Delta a_i^m) \Delta a_i^m \quad (9.36)$$

Now substitute these equations into the original contribution backpropagation of Eq. 9.18:

$$q(\Delta a_i^m) \Delta a_i^m = \sum_{j=1}^{S^{m+1}} q(\Delta a_i^m | \Delta a_j^{m+1}) \frac{\Delta a_i^m}{\Delta a_j^{m+1}} q(\Delta a_j^{m+1}) \Delta a_j^{m+1} \quad (9.37)$$

Canceling terms, we get

Post Training Analysis

$$q(\Delta a_i^m) = \sum_{j=1}^{S^{m+1}} q(\Delta a_i^m | \Delta a_j^{m+1}) q(\Delta a_j^{m+1}) \quad (9.38)$$

This is equivalent to Eq. 9.18, but in terms of the modified multipliers.

The DeepLIFT process begins by passing $\mathbf{p}$ and $\bar{\mathbf{p}}$ through the network and computing all the Δa_i^m . Then the backpropagation begins by setting

$$q(\Delta a_c^M) = \frac{c^{dl}(\Delta a_c^M)}{\Delta a_c^M} = \frac{\Delta a_c^M}{\Delta a_c^M} = 1 \quad (9.39)$$

Then we iterate the backpropagation equation, Eq. 9.38, for $m = M - 1$ to $m = 0$ ($\Delta a_i^0 = \Delta p_i$), and compute the contributions:

$$c^{dl}(\Delta p_i) = q(\Delta p_i) \Delta p_i \quad (9.40)$$

or in vector form

$$\mathbf{c}^{dl}(\Delta \mathbf{p}) = \mathbf{q}(\Delta \mathbf{p}) \circ \Delta \mathbf{p} \quad (9.41)$$

Now let's see how the multipliers are computed for DeepLIFT. First, unlike LRP, DeepLIFT considers the activation function when performing the backpropagation. (DeepLIFT only considers activation functions that take a single input, like ReLU.) We can then modify Eq. 9.38 as follows (only for transfer functions with a single input):

$$q(\Delta a_i^m) = \sum_{j=1}^{S^{m+1}} q(\Delta a_i^m | \Delta n_j^{m+1}) q(\Delta n_j^{m+1} | \Delta a_j^{m+1}) q(\Delta a_j^{m+1}) \quad (9.42)$$

We can write this in matrix form

$$\mathbf{q}(\Delta \mathbf{a}^m) = \mathbf{Q}(\Delta \mathbf{a}^m | \Delta \mathbf{a}^{m+1}) \mathbf{q}(\Delta \mathbf{a}^{m+1}) \quad (9.43)$$

where

$$\left[\mathbf{Q}(\Delta \mathbf{a}^m | \Delta \mathbf{a}^{m+1}) \right]_{i,j} = q(\Delta a_i^m | \Delta a_j^{m+1}) \quad (9.44)$$

$$= q(\Delta a_i^m | \Delta n_j^{m+1}) q(\Delta n_j^{m+1} | \Delta a_j^{m+1}) \quad (9.45)$$

When we move across the activation function,

$$c^{dl}(\Delta n_j^{m+1} | \Delta a_j^{m+1}) = \Delta a_j^{m+1} \quad (9.46)$$

Since n_j^{m+1} is the only variable that affects a_j^{m+1} , the change in a_j^{m+1} is totally caused by n_j^{m+1} . We can then compute the corresponding DeepLIFT multiplier

$$q(\Delta n_j^{m+1} | \Delta a_j^{m+1}) = \frac{c^{dl}(\Delta n_j^{m+1} | \Delta a_j^{m+1})}{\Delta n_j^{m+1}} = \frac{\Delta a_j^{m+1}}{\Delta n_j^{m+1}} \quad (9.47)$$

This is called the *rescale rule*. If $\mathbf{p}$ is close to the baseline $\bar{\mathbf{p}}$, then $\Delta n_j^{m+1} \rightarrow 0$, and the rescale rule multiplier approaches the derivative of the activation function.

For the term $q(\Delta a_i^m | \Delta n_j^{m+1})$, the *linear rule* multiplier is

$$q(\Delta a_i^m | \Delta n_j^{m+1}) = w_{j,i}^{m+1} \quad (9.48)$$

By using Eq. 9.35, we can show that this is equivalent to the LRP-0 rule (Eq. 9.27), except that differences are used:

$$c(a_i^m | n_j^{m+1}) = q(\Delta a_i^m | \Delta n_j^{m+1}) \frac{\Delta a_i^m}{\Delta n_j^{m+1}} = \frac{w_{j,i}^{m+1} \cdot \Delta a_i^m}{\sum_{k=1}^m w_{j,k}^{m+1} \cdot \Delta a_k^m} \quad (9.49)$$

We should note that there are variations to the DeepLIFT rules. For example, [Shrikumar et al., 2017] has a variation on the Rescale rule, called the RevealCancel rule, in which positive and negative contributions are addressed separately.

Epilogue

After a network has been trained, it is important to analyze the network to assess its quality. A part of this process is explaining, to the extent possible, how the network makes its decisions. In this chapter we described several different methods for estimating the contribution that each element of the network input makes to the decision.

Further Reading

[Good, 1977] Irving J Good. Explicativity: a mathematical theory of explanation with statistical applications. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 354(1678):303–330, 1977

This is an early paper attempting to find a mathematical formalism to define what an explanation is in terms of probability. He decides to use the term "explicativity" to mean the extent to which one proposition explains why another should be believed. He introduces the concept of "weight of evidence" as part of the overall development. The weight of evidence E for hypothesis H_1 as compared to H_2 is the ratio of the probabilities of the E given H_1 to E given H_2 . This idea lead to the current explainability methods.

[Landecker et al., 2013] Will Landecker, Michael D Thomure, Luís MA Bettencourt, Melanie Mitchell, Garrett T Kenyon, and Steven P Brumby. Interpreting individual classifications of hierarchical networks. In *2013 IEEE symposium on computational intelligence and data mining (CIDM)*, pages 32–38. IEEE, 2013

This paper introduced the idea for backpropagating contributions through the network. LRP and DeepLIFT both used the idea.

[Samek et al., 2021] Wojciech Samek, Grégoire Montavon, Sebastian Lapuschkin, Christopher J Anders, and Klaus-Robert Müller. Explaining deep neural networks and beyond: A review of methods and applications. *Proceedings of the IEEE*, 109(3): 247–278, 2021

This is a good summary of explainability techniques developed before 2021.

Summary of Results

Desired Properties for Explainability

1. *Continuity* : If $a_c^M(\mathbf{p})$ is continuous, then $\mathbf{c}(\mathbf{p})$ is continuous.
2. *Implementation Invariance* : If two networks are functionally equivalent (same input produces same output), then the computed contributions are the same.
3. *Sensitivity*
 - (a) If an input and a baseline differ in one feature and have different network outputs, a nonzero contribution is assigned to that feature.
 - (b) If the network output does not depend on a feature, the contribution of that feature is zero.
4. *Completeness* : The sum of all contributions is equal to the network output (or the difference between the output and a baseline output).

Saliency

$$\mathbf{s}^M = \frac{\partial n_c^M}{\partial \mathbf{n}^M} = \boldsymbol{\epsilon}_c$$

$$\mathbf{s}^m = \mathbf{F}^m(\mathbf{n}^m)(\mathbf{W}^{m+1})^T \mathbf{s}^{m+1}$$

$$\mathbf{c}^{sal}(\mathbf{p}) = \frac{\partial n_c^M}{\partial \mathbf{p}} \circ \mathbf{p} = \left((\mathbf{W}^1)^T \mathbf{s}^1 \right) \circ \mathbf{p}$$

Integrated Gradient

$$\mathbf{c}^{ig}(\mathbf{p}) = \left[\frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial n_c^M(\tilde{\mathbf{p}})}{\partial \tilde{\mathbf{p}}} \Big|_{\tilde{\mathbf{p}}=\mathbf{p}^t} \right] \circ (\mathbf{p} - \bar{\mathbf{p}})$$

$$\mathbf{p}^t = \bar{\mathbf{p}} + \frac{t}{T-1} (\mathbf{p} - \bar{\mathbf{p}})$$

Contribution Propagation

$$\mathbf{c}(\mathbf{a}^M) = n_c^M \epsilon_c$$

$$\mathbf{c}(\mathbf{a}^m) = \mathbf{C}(\mathbf{a}^m | \mathbf{a}^{m+1}) \mathbf{c}(\mathbf{a}^{m+1}), \quad m = M-1, M-2, \dots, 0$$

Requirement for Completeness

$$\sum_{i=1}^{S^m} c(a_i^m | a_j^{m+1}) = 1$$

LRP-0

$$c^{lrp-0}(a_i^m | a_j^{m+1}) = \frac{w_{j,i}^{m+1} a_i^m}{\sum_{k=1}^{S^m} w_{j,k}^{m+1} a_k^m}$$

LRP- ϵ

$$c^{lrp-\epsilon}(a_i^m | a_j^{m+1}) = \frac{w_{j,i}^{m+1} a_i^m}{\epsilon + \sum_{k=1}^{S^m} w_{j,k}^{m+1} a_k^m}$$

LRP- γ

$$c^{lrp-\gamma}(a_i^m | a_j^{m+1}) = \frac{\left(w_{j,i}^{m+1} + \gamma \left(w_{j,i}^{m+1}\right)^+\right) a_i^m}{\sum_{k=1}^{S^m} \left(w_{j,k}^{m+1} + \gamma \left(w_{j,k}^{m+1}\right)^+\right) a_k^m}$$

DeepLIFT

$$q(\Delta a_c^M) = 1$$

$$q(\Delta a_i^m) = \sum_{j=1}^{S^{m+1}} q(\Delta a_i^m | \Delta n_j^{m+1}) q(\Delta n_j^{m+1} | \Delta a_j^{m+1}) q(\Delta a_j^{m+1})$$

Rescale Rule

$$q(\Delta n_j^{m+1} | \Delta a_j^{m+1}) = \frac{\Delta a_j^{m+1}}{\Delta n_j^{m+1}}$$

Linear Rule

$$q(\Delta a_i^m | \Delta n_j^{m+1}) = w_{j,i}^{m+1}$$

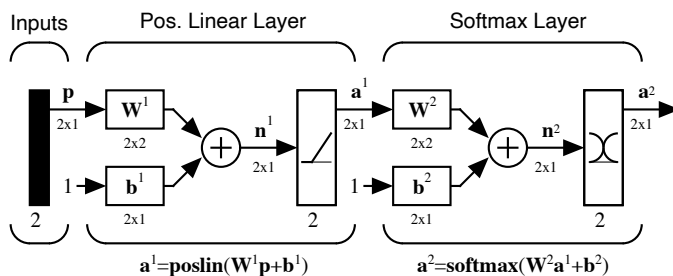
DeepLIFT Contributions

$$c^{dl}(\Delta p_i) = q(\Delta p_i) \Delta p_i$$

Post Training Analysis

Solved Problems

P9.1 Consider the following two-layer network.

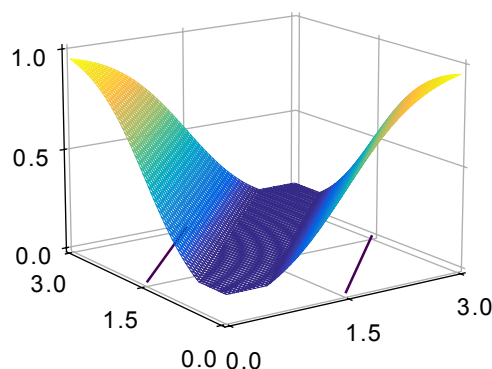


Where the weights and biases are set as follows.

$$\mathbf{W}^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$$

$$\mathbf{W}^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \mathbf{b}^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

The network response surface and decision boundaries are shown below. This is an XOR type of network.



- The input to the network is $\mathbf{p} = \begin{bmatrix} 0 & 3 \end{bmatrix}^T$. Using backpropagation, find the gradient and find the saliency contribution.
- Repeat using the input $\mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T$. Does the contribution "explain" the network decision?

i. We begin by passing the input forward through the network.

$$\mathbf{n}^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 3 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -3 \\ 3 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -3.5 \\ 2.5 \end{bmatrix}$$

$$\mathbf{a}^1 = \text{poslin} \left(\begin{bmatrix} -3.5 \\ 2.5 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}$$

$$\mathbf{n}^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \mathbf{a}^1 + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.5 - 1 \\ -2.5 + 1 \end{bmatrix} = \begin{bmatrix} 1.5 \\ -1.5 \end{bmatrix}$$

$$\mathbf{a}^2 = \text{softmax}(\mathbf{n}^2) = \text{softmax} \left(\begin{bmatrix} 1.5 \\ -1.5 \end{bmatrix} \right) = \begin{bmatrix} 0.9526 \\ 0.0474 \end{bmatrix}$$

which indicates that the input is assigned to the first class.

We then start the backpropagation with Eq. 9.13. Since the input is assigned to the first class, $c = 1$.

$$\mathbf{s}^2 = \epsilon_c = \epsilon_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Backpropagation then continues with Eq. 9.12.

$$\mathbf{s}^1 = \mathbf{F}^1(\mathbf{n}^1)(\mathbf{W}^2)^T \mathbf{s}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

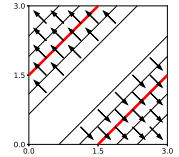
Then the contribution is computed using Eq. 9.14.

$$\mathbf{c}^{sal}(\mathbf{p}) = ((\mathbf{W}^1)^T \mathbf{s}^1) \circ \mathbf{p} = \left(\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) \circ \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \circ \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

The gradient is $\begin{bmatrix} -1 & 1 \end{bmatrix}^T$. The contour plot of the network surface in the margin shows how the gradient varies across the surface (with decision boundary in red).

The contribution is $\begin{bmatrix} 0 & 3 \end{bmatrix}^T$. This says that the second input is most important.

ii. If the input is $\mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T$, the network output is



Post Training Analysis

$$\begin{aligned}\mathbf{n}^1 &= \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} \\ \mathbf{a}^1 &= \text{poslin} \left(\begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ \mathbf{n}^2 &= \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \mathbf{a}^1 + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 - 1 \\ 0 + 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ \mathbf{a}^2 &= \text{softmax}(\mathbf{n}^2) = \text{softmax} \left(\begin{bmatrix} -1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix}\end{aligned}$$

which assigns the input to the second class, so $c = 2$.

$$\mathbf{s}^2 = \epsilon_c = \epsilon_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Backpropagation then continues with Eq. 9.12.

$$\mathbf{s}^1 = \mathbf{F}^1(\mathbf{n}^1)(\mathbf{W}^2)^T \mathbf{s}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

and the contribution is

$$\mathbf{c}^{sal}(\mathbf{p}) = \left((\mathbf{W}^1)^T \mathbf{s}^1 \right) \circ \mathbf{p} = \left(\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) \circ \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \circ \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

There seems to be no contribution, because the gradient is equal to zero. This is clear from the plot of the network surface above. There is a trough in the surface, along which the gradient is zero. Clearly this doesn't mean that neither input contributes to the network decision. This is just a problem with using saliency to compute the contribution.

P9.2 Repeat Solved Problem P.9.1, but use the integrated gradient method to compute the contribution.

i. For the integrated gradient method we average the gradient along a line between the given input and a baseline input. Let's try using baseline $\bar{\mathbf{p}} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$, and for simplicity we will use just three points, $T = 3$, for the average:

$$\begin{aligned}\mathbf{p}^t &= \bar{\mathbf{p}} + \frac{t}{T-1} (\mathbf{p} - \bar{\mathbf{p}}) \\ \mathbf{p}^0 &= \bar{\mathbf{p}} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T \\ \mathbf{p}^1 &= \frac{(\mathbf{p} + \bar{\mathbf{p}})}{2} = \begin{bmatrix} 0 & 1.5 \end{bmatrix}^T \\ \mathbf{p}^2 &= \mathbf{p} = \begin{bmatrix} 0 & 3 \end{bmatrix}^T\end{aligned}$$

We already computed the gradient at $\mathbf{p}$ in the previous problem, and at $\bar{\mathbf{p}}$ the gradient is zero, so we just need to compute the gradient at $\mathbf{p}^1$. Going forward, we have

$$\begin{aligned}\mathbf{n}^1 &= \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1.5 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 1.5 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix} \\ \mathbf{a}^1 &= \text{poslin} \left(\begin{bmatrix} -2 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ \mathbf{n}^2 &= \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \mathbf{a}^1 + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1-1 \\ -1+1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ \mathbf{a}^2 &= \text{softmax}(\mathbf{n}^2) = \text{softmax} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}\end{aligned}$$

We will use $c = 1$, which corresponds to the result for the original $\mathbf{p}$, for all $\mathbf{p}^t$. For the backpropagation we find

$$\begin{aligned}\mathbf{s}^2 &= \epsilon_c = \epsilon_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ \mathbf{s}^1 &= \mathbf{F}'(\mathbf{n}^1)(\mathbf{W}^2)^T \mathbf{s}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}\end{aligned}$$

And the contribution at this point is

$$\mathbf{c}^{sal}(\mathbf{p}) = ((\mathbf{W}^1)^T \mathbf{s}^1) \circ \mathbf{p} = \left(\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) \circ \begin{bmatrix} 0 \\ 1.5 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \circ \begin{bmatrix} 0 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.5 \end{bmatrix}$$

Post Training Analysis

Now we average the three gradients together and multiply by the input deviation to get the contribution, using Eq. 9.15.

$$\begin{aligned} \mathbf{c}^{ig}(\mathbf{p}) &= \left[\frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial n_c^M(\tilde{\mathbf{p}})}{\partial \tilde{\mathbf{p}}} \Big|_{\tilde{\mathbf{p}}=\mathbf{p}^t} \right] \circ (\mathbf{p} - \bar{\mathbf{p}}) \\ &= \left[\frac{1}{3} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right) \right] \circ \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} -\frac{2}{3} \\ \frac{2}{3} \end{bmatrix} \circ \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \end{aligned}$$

This IG result is not very different than the basic saliency result we had earlier. It says that the second input is most important. The advantage of IG can be seen in the next part.

ii. If the input is $\mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T$, we found the saliency contribution to be all zeros. This result showed a limitation of using basic saliency, since some characteristic of the inputs should be affecting the decision. The problem of using basic saliency was that the gradient is zero at the current input. The IG method can help with this problem, if we use an appropriate baseline input. Unfortunately, if we use the origin as a baseline, the gradient will be zero along the entire line between the baseline and the current input, since it follows the valley in the network response. It would be best to choose a baseline that is on the opposite side of a decision boundary from the current input, since that would guarantee a nonzero gradient at some point between the current input and the baseline. In a practical case, we could use a nearby input from the training set that was classified differently from the current input. For demonstration purposes, we can use $\bar{\mathbf{p}} = \begin{bmatrix} 0 & 3 \end{bmatrix}^T$.

If we again use just three points, $T = 3$, for the average:

$$\begin{aligned} \mathbf{p}^t &= \bar{\mathbf{p}} + \frac{t}{T-1} (\mathbf{p} - \bar{\mathbf{p}}) \\ \mathbf{p}^0 &= \bar{\mathbf{p}} = \begin{bmatrix} 0 & 3 \end{bmatrix}^T \\ \mathbf{p}^1 &= \frac{(\mathbf{p} + \bar{\mathbf{p}})}{2} = \begin{bmatrix} 1.5 & 3 \end{bmatrix}^T \\ \mathbf{p}^2 &= \mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T \end{aligned}$$

We previously computed the gradients for $\mathbf{p}^2$, so we can use those numbers. We also computed the gradients for $\mathbf{p}^0$, but that was using $c = 1$ so we need to compute it for $c = 2$. We also need the gradient for $\mathbf{p}^1$ with $c = 2$.

Starting with $\mathbf{p}^0$, we previously computed the forward calculations, so let's do the reverse, starting with $\mathbf{s}^2 = \epsilon_c = \epsilon_2$:

$$\mathbf{s}^2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{s}^1 = \mathbf{F}^1(\mathbf{n}^1)(\mathbf{W}^2)^T \mathbf{s}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

The gradient is then computed as

$$(\mathbf{W}^1)^T \mathbf{s}^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Next we compute the gradient at $\mathbf{p}^1$. Going forward, we find

$$\mathbf{n}^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1.5 \\ 3 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 1.5 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

$$\mathbf{a}^1 = \text{poslin} \left(\begin{bmatrix} -2 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{n}^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \mathbf{a}^1 + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1-1 \\ -1+1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{a}^2 = \text{softmax}(\mathbf{n}^2) = \text{softmax} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

We will use $c = 2$, which corresponds to the result for the original $\mathbf{p}$, for all $\mathbf{p}^t$. For the backpropagation we find

$$\mathbf{s}^2 = \epsilon_c = \epsilon_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{s}^1 = \mathbf{F}^1(\mathbf{n}^1)(\mathbf{W}^2)^T \mathbf{s}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

And the gradient at this point is

$$(\mathbf{W}^1)^T \mathbf{s}^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Post Training Analysis

Now we average the three gradients together and multiply by the input deviation to get the contribution, using Eq. 9.15.

$$\begin{aligned} c^{ig}(\mathbf{p}) &= \left[\frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial n_c^M(\tilde{\mathbf{p}})}{\partial \tilde{\mathbf{p}}} \Big|_{\tilde{\mathbf{p}}=\mathbf{p}^t} \right] \circ (\mathbf{p} - \bar{\mathbf{p}}) \\ &= \left[\frac{1}{3} \left(\begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right) \right] \circ \left(\begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} \frac{2}{3} \\ \frac{2}{3} \\ -\frac{2}{3} \end{bmatrix} \circ \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} \end{aligned}$$

This result says that the first input is most important to this decision.

P9.3 Repeat Solved Problem P.9.1, but use the LRP method to compute the contribution.

i. For the LRP method, the forward calculations are the same, so we have

$$\mathbf{p} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \mathbf{n}^1 = \begin{bmatrix} -3.5 \\ 2.5 \end{bmatrix}, \mathbf{a}^1 = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}, \mathbf{n}^2 = \begin{bmatrix} 1.5 \\ -1.5 \end{bmatrix}, \mathbf{a}^2 = \begin{bmatrix} 0.9526 \\ 0.0474 \end{bmatrix}$$

LRP uses the backpropagation rule of Eq. 9.18:

$$\mathbf{c}(\mathbf{a}^m) = \mathbf{C}(\mathbf{a}^m | \mathbf{a}^{m+1}) \mathbf{c}(\mathbf{a}^{m+1}), \quad m = M-1, M-2, \dots, 0$$

where, from Eq. 9.19, we have

$$\left[\mathbf{C}(\mathbf{a}^m | \mathbf{a}^{m+1}) \right]_{i,j} = c(a_i^m | a_j^{m+1})$$

If we use the LRP-0 method, these terms are computed from Eq. 9.27:

$$c^{lrp-0}(a_i^m | a_j^{m+1}) = \frac{w_{j,i}^{m+1} a_i^m}{\sum_{k=1}^S w_{j,k}^{m+1} a_k^m}$$

We will need $\mathbf{C}(\mathbf{a}^0 | \mathbf{a}^1)$ and $\mathbf{C}(\mathbf{a}^1 | \mathbf{a}^2)$. Let's start with the terms for $\mathbf{C}(\mathbf{a}^1 | \mathbf{a}^2)$.

$$\begin{aligned}
 c(a_1^1 | a_1^2) &= \frac{w_{1,1}^2 a_1^1}{w_{1,1}^2 a_1^1 + w_{1,2}^2 a_2^1} = \frac{1 \cdot 0}{1 \cdot 0 + 1 \cdot 2.5} = 0 \\
 c(a_2^1 | a_1^2) &= \frac{w_{1,2}^2 a_2^1}{w_{1,1}^2 a_1^1 + w_{1,2}^2 a_2^1} = \frac{1 \cdot 2.5}{1 \cdot 0 + 1 \cdot 2.5} = 1 \\
 c(a_1^1 | a_2^2) &= \frac{w_{2,1}^2 a_1^1}{w_{2,1}^2 a_1^1 + w_{2,2}^2 a_2^1} = \frac{-1 \cdot 0}{-1 \cdot 0 - 1 \cdot 2.5} = 0 \\
 c(a_2^1 | a_2^2) &= \frac{w_{2,2}^2 a_2^1}{w_{2,1}^2 a_1^1 + w_{2,2}^2 a_2^1} = \frac{-1 \cdot 2.5}{-1 \cdot 0 - 1 \cdot 2.5} = 1
 \end{aligned}$$

Therefore

$$\mathbf{C}(a^1 | a^2) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

and we can compute the contribution at the first layer, initializing using Eq. 9.20, with $c = 1$:

$$\begin{aligned}
 \mathbf{c}(a^2) &= n_1^2 \epsilon_1 = \begin{bmatrix} 1.5 \\ 0 \end{bmatrix} \\
 \mathbf{c}(a^1) &= \mathbf{C}(a^1 | a^2) \mathbf{c}(a^2) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1.5 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.5 \end{bmatrix}
 \end{aligned}$$

Now we move from the first layer to the input, and will compute $\mathbf{C}(a^0 | a^1) = \mathbf{C}(p | a^1)$.

$$\begin{aligned}
 c(p_1 | a_1^1) &= \frac{w_{1,1}^1 p_1}{w_{1,1}^1 p_1 + w_{1,2}^1 p_2} = \frac{1 \cdot 0}{1 \cdot 0 - 1 \cdot 3} = 0 \\
 c(p_2 | a_1^1) &= \frac{w_{1,2}^1 p_2}{w_{1,1}^1 p_1 + w_{1,2}^1 p_2} = \frac{-1 \cdot 3}{1 \cdot 0 - 1 \cdot 3} = 1 \\
 c(p_1 | a_2^1) &= \frac{w_{2,1}^1 p_1}{w_{2,1}^1 p_1 + w_{2,2}^1 p_2} = \frac{-1 \cdot 0}{-1 \cdot 0 + 1 \cdot 3} = 0 \\
 c(p_2 | a_2^1) &= \frac{w_{2,2}^1 p_2}{w_{2,1}^1 p_1 + w_{2,2}^1 p_2} = \frac{1 \cdot 3}{-1 \cdot 0 + 1 \cdot 3} = 1
 \end{aligned}$$

Therefore

$$\mathbf{C}(p | a^1) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

Post Training Analysis

and we can compute the contribution at the input:

$$\mathbf{c}(\mathbf{p}) = \mathbf{C}(\mathbf{p}|\mathbf{a}^1) \mathbf{c}(\mathbf{a}^1) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.5 \end{bmatrix}$$

This agrees with saliency and IG that the second input is most important to the decision.

ii. When $\mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T$, the forward calculation produces

$$\mathbf{p} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \mathbf{n}^1 = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}, \mathbf{a}^1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{n}^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \mathbf{a}^2 = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix}$$

Because some the calculations of $c^{lrp-0}(a_i^m|a_j^{m+1})$ have zero in the denominator, we will use the LRP- ϵ rule.

$$c(a_1^1|a_1^2) = \frac{w_{1,1}^2 a_1^1}{\epsilon + w_{1,1}^2 a_1^1 + w_{1,2}^2 a_2^1} = \frac{1 \cdot 0}{\epsilon + 1 \cdot 0 + 1 \cdot 0} = 0$$

$$c(a_2^1|a_1^2) = \frac{w_{1,2}^2 a_2^1}{\epsilon + w_{1,1}^2 a_1^1 + w_{1,2}^2 a_2^1} = \frac{1 \cdot 0}{\epsilon + 1 \cdot 0 + 1 \cdot 0} = 0$$

$$c(a_1^1|a_2^2) = \frac{w_{2,1}^2 a_1^1}{\epsilon + w_{2,1}^2 a_1^1 + w_{2,2}^2 a_2^1} = \frac{-1 \cdot 0}{\epsilon - 1 \cdot 0 - 1 \cdot 0} = 0$$

$$c(a_2^1|a_2^2) = \frac{w_{2,2}^2 a_2^1}{\epsilon + w_{2,1}^2 a_1^1 + w_{2,2}^2 a_2^1} = \frac{-1 \cdot 2.5}{\epsilon - 1 \cdot 0 - 1 \cdot 0} = 0$$

Therefore

$$\mathbf{C}(\mathbf{a}^1|\mathbf{a}^2) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

and we can compute the contribution at the first layer, initialing using Eq. 9.20, with $c = 2$:

$$\mathbf{c}(\mathbf{a}^2) = n_1^2 \epsilon_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{c}(\mathbf{a}^1) = \mathbf{C}(\mathbf{a}^1|\mathbf{a}^2) \mathbf{c}(\mathbf{a}^2) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This means that $\mathbf{c}(\mathbf{a}^0) = \mathbf{c}(\mathbf{p})$ will also be zero. This is the same issue that we had with saliency. The result implies that neither input affects the decision, which is not reasonable. Let's see how DeepLIFT addresses this issue in the next problem.

P9.4 Repeat Solved Problem P.9.1, but use the DeepLIFT method to compute the contribution.

i. For the DeepLIFT method, the forward calculations are the same, so we have

$$\mathbf{p} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \mathbf{n}^1 = \begin{bmatrix} -3.5 \\ 2.5 \end{bmatrix}, \mathbf{a}^1 = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}, \mathbf{n}^2 = \begin{bmatrix} 1.5 \\ -1.5 \end{bmatrix}, \mathbf{a}^2 = \begin{bmatrix} 0.9526 \\ 0.0474 \end{bmatrix}$$

If we choose the baseline to be $\bar{\mathbf{p}} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$, the forward calculation gives us

$$\bar{\mathbf{p}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \bar{\mathbf{n}}^1 = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}, \bar{\mathbf{a}}^1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \bar{\mathbf{n}}^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \bar{\mathbf{a}}^2 = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix}$$

The differences are then

$$\Delta \mathbf{p} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \Delta \mathbf{n}^1 = \begin{bmatrix} -3 \\ 3 \end{bmatrix}, \Delta \mathbf{a}^1 = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}, \Delta \mathbf{n}^2 = \begin{bmatrix} 2.5 \\ -2.5 \end{bmatrix}, \Delta \mathbf{a}^2 = \begin{bmatrix} 0.8334 \\ -0.8334 \end{bmatrix}$$

We can initialize the backpropagation with $c = 1$:

$$\mathbf{q}(\Delta \mathbf{a}^2) = \epsilon_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

and then backpropagate with Eq. 9.43:

$$\mathbf{q}(\Delta \mathbf{a}^m) = \mathbf{Q}(\Delta \mathbf{a}^m | \Delta \mathbf{a}^{m+1}) \mathbf{q}(\Delta \mathbf{a}^{m+1})$$

where, from Eq. 9.45, we have

$$q(\Delta a_i^m | \Delta a_j^{m+1}) = q(\Delta a_i^m | \Delta n_j^{m+1}) q(\Delta n_j^{m+1} | \Delta a_j^{m+1})$$

For the last layer, we generally start with n rather than a , so we can skip the $q(\Delta n_j^{m+1} | \Delta a_j^{m+1})$ term. Using the linear rule of Eq. 9.48, we have

$$q(\Delta a_i^m | \Delta n_j^{m+1}) = w_{j,i}^{m+1}$$

so

$$\mathbf{Q}(\Delta \mathbf{a}^1 | \Delta \mathbf{a}^2) = (\mathbf{W}^2)^T = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

Post Training Analysis

and the backpropagation across the last layer produces

$$\mathbf{q}(\Delta \mathbf{a}^1) = \mathbf{Q}(\Delta \mathbf{a}^1 | \Delta \mathbf{a}^2) \mathbf{q}(\Delta \mathbf{a}^2) = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

To backpropagate across the first layer, we need to consider the transfer function in Eq. 9.45. Those two terms are

$$\begin{aligned} q(\Delta n_1^1 | \Delta a_1^1) &= \frac{\Delta a_1^1}{\Delta n_1^1} = \frac{0}{-3} = 0 \\ q(\Delta n_2^1 | \Delta a_2^1) &= \frac{\Delta a_2^1}{\Delta n_2^1} = \frac{2.5}{3} = 0.8333 \end{aligned}$$

So we can compute

$$\mathbf{Q}(\Delta \mathbf{a}^0 | \Delta \mathbf{a}^1) = \begin{bmatrix} w_{1,1}^1 \cdot 0 & w_{2,1}^1 \cdot 0.8333 \\ w_{1,2}^1 \cdot 0 & w_{2,2}^1 \cdot 0.8333 \end{bmatrix} = \begin{bmatrix} 0 & -0.8333 \\ 0 & 0.8333 \end{bmatrix}$$

and the backpropagation across the last layer is

$$\begin{aligned} \mathbf{q}(\Delta \mathbf{a}^0) &= \mathbf{q}(\Delta \mathbf{p}) = \mathbf{Q}(\Delta \mathbf{a}^0 | \Delta \mathbf{a}^1) \mathbf{q}(\Delta \mathbf{a}^1) \\ &= \begin{bmatrix} 0 & -0.8333 \\ 0 & 0.8333 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.8333 \\ 0.8333 \end{bmatrix} \end{aligned}$$

To compute the contribution, we use Eq. 9.41.

$$\mathbf{c}^{dl}(\Delta \mathbf{p}) = \mathbf{q}(\Delta \mathbf{p}) \circ \Delta \mathbf{p} = \begin{bmatrix} -0.8333 \\ 0.8333 \end{bmatrix} \circ \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix} \quad (9.50)$$

This says that the second input is most important to the decision, which agrees with saliency, IG and LRP.

ii. If the input is $\mathbf{p} = \begin{bmatrix} 3 & 3 \end{bmatrix}^T$, we do not want to use the zero baseline, because the network output does not change. In a practical case, it is best to use an input from the training set that is classified differently than $\mathbf{p}$. As with IG, for demonstration purposes, we will use $\bar{\mathbf{p}} = \begin{bmatrix} 0 & 3 \end{bmatrix}^T$. We first need to compute the network responses and the differences.

$$\mathbf{p} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \mathbf{n}^1 = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}, \mathbf{a}^1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{n}^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \mathbf{a}^2 = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix}$$

$$\bar{\mathbf{p}} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \bar{\mathbf{n}}^1 = \begin{bmatrix} -3.5 \\ 2.5 \end{bmatrix}, \bar{\mathbf{a}}^1 = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}, \bar{\mathbf{n}}^2 = \begin{bmatrix} 1.5 \\ -1.5 \end{bmatrix}, \bar{\mathbf{a}}^2 = \begin{bmatrix} 0.9526 \\ 0.0474 \end{bmatrix}$$

The differences are then

$$\Delta \mathbf{p} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}, \Delta \mathbf{n}^1 = \begin{bmatrix} 3 \\ -3 \end{bmatrix}, \Delta \mathbf{a}^1 = \begin{bmatrix} 0 \\ -2.5 \end{bmatrix}, \Delta \mathbf{n}^2 = \begin{bmatrix} -2.5 \\ 2.5 \end{bmatrix}, \Delta \mathbf{a}^2 = \begin{bmatrix} -0.8334 \\ 0.8334 \end{bmatrix}$$

We can initialize the backpropagation with $c = 2$:

$$\mathbf{q}(\Delta \mathbf{a}^2) = \epsilon_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{Q}(\Delta \mathbf{a}^1 | \Delta \mathbf{a}^2) = (\mathbf{W}^2)^T = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

and the backpropagation across the last layer produces

$$\mathbf{q}(\Delta \mathbf{a}^1) = \mathbf{Q}(\Delta \mathbf{a}^1 | \Delta \mathbf{a}^2) \mathbf{q}(\Delta \mathbf{a}^2) = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

To backpropagate across the first layer, we need to consider the transfer function in Eq. 9.45. Those two terms are

$$q(\Delta n_1^1 | \Delta a_1^1) = \frac{\Delta a_1^1}{\Delta n_1^1} = \frac{0}{3} = 0$$

$$q(\Delta n_2^1 | \Delta a_2^1) = \frac{\Delta a_2^1}{\Delta n_2^1} = \frac{-2.5}{-3} = 0.8333$$

So we can compute

$$\mathbf{Q}(\Delta \mathbf{a}^0 | \Delta \mathbf{a}^1) = \begin{bmatrix} w_{1,1}^1 \cdot 0 & w_{2,1}^1 \cdot 0.8333 \\ w_{1,2}^1 \cdot 0 & w_{2,2}^1 \cdot 0.8333 \end{bmatrix} = \begin{bmatrix} 0 & -0.8333 \\ 0 & 0.8333 \end{bmatrix}$$

and the backpropagation across the last layer is

$$\begin{aligned} \mathbf{q}(\Delta \mathbf{a}^0) &= \mathbf{q}(\Delta \mathbf{p}) = \mathbf{Q}(\Delta \mathbf{a}^0 | \Delta \mathbf{a}^1) \mathbf{q}(\Delta \mathbf{a}^1) \\ &= \begin{bmatrix} 0 & -0.8333 \\ 0 & 0.8333 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.8333 \\ -0.8333 \end{bmatrix} \end{aligned}$$

Post Training Analysis

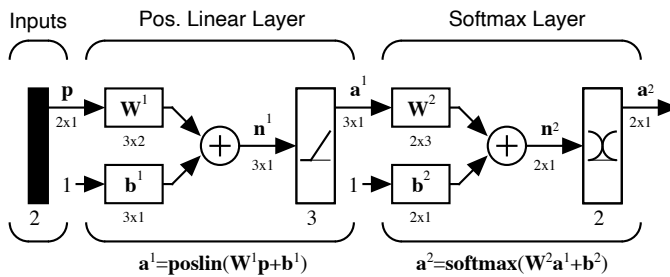
To compute the contribution, we use Eq. 9.41.

$$\mathbf{c}^{dl}(\Delta \mathbf{p}) = \mathbf{q}(\Delta \mathbf{p}) \circ \Delta \mathbf{p} = \begin{bmatrix} 0.8333 \\ -0.8333 \end{bmatrix} \circ \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 0 \end{bmatrix} \quad (9.51)$$

This says that the first input is most important to the decision, which agrees with IG. Since LRP didn't have a baseline to set, it didn't produce a useful result, and saliency also found zero contribution.

Exercises

- Eg.1** In the Solved Problems, which methods produced the most reasonable results? For each of the two input vectors considered in those problems, which element of the input vector do you think is most important to the decision? Explain.
- Eg.2** Consider again the network in Solved Problem P9.1, but with three neurons in the hidden layer.

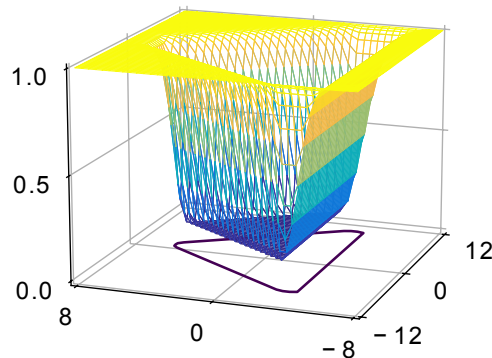


Where the weights and biases are set as follows.

$$W^1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 0 & -1 \end{bmatrix}, b^1 = \begin{bmatrix} -5 \\ -5 \\ -3 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} 1 & 1 & 1 \\ -1 & -1 & -1 \end{bmatrix}, b^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

The network response surface and decision boundaries are shown below.



Post Training Analysis

- i. Using the weights and biases in the first layer, determine approximately where the decision boundaries would be.
- ii. The input to the network is $\mathbf{p} = \begin{bmatrix} 0 & -5 \end{bmatrix}^T$. What class does the network assign to this input? Which element of the input vector do you think is most important to the decision? Explain.
- iii. The input to the network is $\mathbf{p} = \begin{bmatrix} 0 & -5 \end{bmatrix}^T$. From the figure above, what do you think the gradient direction will be? Using backpropagation, find the gradient and find the saliency contribution.
- iv. Repeat parts ii and iii using the input $\mathbf{p} = \begin{bmatrix} 3 & 0 \end{bmatrix}^T$. Does the contribution "explain" the network decision?

- E9.3** Consider again the network and inputs in Exercise E9.2, but use the Integrated Gradient method to compute the contributions. Explain your choices for the baseline input, and demonstrate how different choices can produce significantly different results.
- E9.4** For the network and inputs in Exercise E9.2, use LRP to compute the contributions. Do the contributions seem reasonable? Explain.
- E9.5** For the network and inputs in Exercise E9.2, use DeepLIFT to compute the contributions. Explain how you selected the baseline input. Discuss any differences in the results you obtained with DeepLIFT with results you obtained with other methods.

10 *PyTorch Introduction*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Loading the Data</i>	2
<i>Constructing the Model</i>	3
<i>Training the Network</i>	4
<i>Advanced Data Loading</i>	7
<i>Epilogue</i>	11
<i>Further Reading</i>	12
<i>Labs</i>	13

Objective

This chapter provides an introduction to the PyTorch framework for deep learning. At the moment, TensorFlow and PyTorch are the two most popular frameworks. Over time these two frameworks have become more similar, as they borrow each other's best features. As in Chapter 6, we will show how you can quickly get started to perform all aspects of the typical deep learning workflow: load and preprocess data, build a neural network, train the network and evaluate the results.

This chapter is available as an interactive Jupyter Notebook at the website <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/blob/master/10.PyTorchIntroChapter/Code/ChapterNotebook/PyTorchIntro.ipynb>

The history of *PyTorch* begins around 2002 with Torch, an open-source machine learning library, scientific computing framework, and scripting language based on Lua. It was originally developed at the Idiap Research Institute at EPFL in Lausanne, Switzerland. The main developers were Ronan Collobert, Samy Bengio and Johnny Mariétho. Torch evolved over multiple revisions through version 7 in 2017.

The front-end for Torch was then converted by Soumith Chintala, Adam Paszke, Sam Gross and Gregory Chanan at Facebook (now Meta) from Lua to Python, at which time it was renamed PyTorch. It is written in Python, C++ and CUDA. PyTorch 2.0 was released on 15 March 2023 and, at the time of this writing, is currently governed by the PyTorch Foundation, a subsidiary of the Linux Foundation.

TensorFlow and PyTorch are very similar, and the deep learning workflow (load and process data, build a neural network, train the network and evaluate the results) is the same. Therefore, this chapter will follow the pattern of Chapter 6, but we will emphasize areas where there are differences between the two frameworks.

Loading the Data

PyTorch has the same formats for tensors as TensorFlow. Refer to Chapter 6 to review how *features*, *samples*, *timesteps* and *channels* are indexed in a tensor. PyTorch tensors can be created directly from data, using the `torch.tensor()` command, or from a numpy array, using the `torch.from_numpy()` command.

As in the TensorFlow chapter, we begin by generating data for the XOR problem. For this example we use `torch.tensor()` to create the tensor.

```

import numpy as np
import torch

p = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=
    ↪ torch.float32)
t = torch.tensor([0, 1, 1, 0])
print(p)
print(t)
-----
tensor([[0., 0.],
        [0., 1.],
        [1., 0.],
        [1., 1.]])
tensor([0, 1, 1, 0])

```

We do not need to change the targets using a `to_categorical` operation, because PyTorch will take care of that when we use the cross entropy loss function.

Constructing the Model

In TensorFlow, the core data structure was the model, based on `tensorflow.keras.Model`. The equivalent PyTorch *module* class is `torch.nn.Module`. The module is a way to organize layers of a network. We will describe two techniques, which have equivalent methods in TensorFlow, to define a model: 1) the sequential class and 2) the module subclass. (In the labs we will consider a third method: using a *pretrained model*.)

For the *sequential class*, you use the sequential container `torch.nn.Sequential`. Modules are added in the order they are passed to the constructor, as in the following example. Here we create a two-layer network with 10 neurons in the hidden layer and a tanh activation function. The network architecture is 2-10-2.

The sequential class is designed for networks where each layer follows the previous one.

```

model = torch.nn.Sequential(
    torch.nn.Linear(2, 10),
    torch.nn.Tanh(),
    torch.nn.Linear(10, 2))
print(model)
-----
Sequential(
  (0): Linear(in_features=2, out_features=10, bias=True)
  (1): Tanh()
  (2): Linear(in_features=10, out_features=2, bias=True))

```

The Linear layer is standard matrix multiplication.

Note that we are not using the softmax activation on the last layer, because it is included in the PyTorch `CrossEntropyLoss`.

PyTorch Introduction

When you use the [module subclass method](#), you create your own fully-customizable models by subclassing the Module class and implementing your own forward pass in the forward method. The following code implements the same 2-10-2 network using the model subclass method.

```
class TwoLayer(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.dense1 = torch.nn.Linear(2, 10)
        self.tanh = torch.nn.Tanh()
        self.dense2 = torch.nn.Linear(10, 2)
    def forward(self, x):
        x = self.dense1(x)
        x = self.tanh(x)
        x = self.dense2(x)
        return x
model = TwoLayer()
print(model)

TwoLayer(
  (dense1): Linear(in_features=2, out_features=10, bias=True)
  (tanh): Tanh()
  (dense2): Linear(in_features=10, out_features=2, bias=True))
```

Training the Network

After the data has been loaded and the network has been created, the next step is to train the network. In the following we will cover the basic training steps. In the case study chapters we will provide more detail.

Before training a network, we need to select the optimizer to be used. In the following example we use the Adam optimizer. The first argument of the optimizer contains the variables to be adjusted, which are contained in `model.parameters()`. When setting the optimizer, you can also specify optimizer-specific options such as the learning rate, momentum, etc.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

We also need to define the loss function. Here we will use the cross entropy loss.

```
loss_fn = torch.nn.CrossEntropyLoss()
```

The next step is to train the network. In TensorFlow, we used the `fit` method, but in PyTorch we perform the following steps for each iteration.

PyTorch Training Steps:

1. Zero the gradient
2. Make a forward pass through the network
3. Calculate the loss
4. Compute the gradient
5. Update the weights

The following loop illustrates the process. In this case we pass the entire data set through the network in each iteration. In the labs we will use minibatches. Also, in a difference from TensorFlow, we check to see if a GPU is available, and if so we move the model and the data to the GPU. (If using minibatches, a minibatch would be moved to the GPU at each iteration.) For this simple problem, using the GPU is actually slower than using the cpu.

PyTorch Introduction

```
#Check if a GPU is available, and move model and data
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model.to(device)
p = p.to(device)
t = t.to(device)

epochs = 5000
loss_values = []
for epoch in range(epochs):
    # Zero the gradient
    optimizer.zero_grad()

    # Forward pass through the network
    output = model(p)

    # Calculate the loss
    loss = loss_fn(output, t)

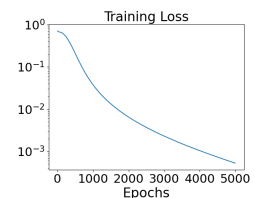
    # Compute the gradient
    loss.backward()

    # Save the loss values for plotting
    loss_values.append(loss.cpu().detach().numpy())

    # Update the weights
    optimizer.step()
```

We saved the loss values at each iteration, so that we could view the process.

```
import matplotlib.pyplot as plt
plt.semilogy(range(epochs), loss_values)
plt.title('Training_Loss')
plt.xlabel('Epochs')
plt.show()
```



To calculate the network response for any input you can invoke the model: `model(p)`. Here we apply the training inputs to the model to check the response. (Is it correct?)

```

n = model(p)
softmax = torch.nn.Softmax(dim=1)
a = softmax(n)
print(a)
-----
tensor([[9.9967e-01, 3.2964e-04],
        [5.4206e-04, 9.9946e-01],
        [2.5702e-04, 9.9974e-01],
        [9.9960e-01, 4.0006e-04]], device='cuda:0',
        ↪ grad_fn=<SoftmaxBackward0>)

```

We manually add the softmax function, because it was not included in the model. The `dim` argument in the `Softmax` command is the dimension along which Softmax will be computed (so every slice along `dim` will sum to 1).

Advanced Data Loading

As described in the TensorFlow chapter, the ETL process of loading/formatting/pre-processing data is one of the most important parts of the deep learning workflow. The procedures for doing this in PyTorch are very similar to those in TensorFlow.

TensorFlow had the `tf.data.Dataset` API for creating input pipelines. PyTorch has `torch.utils.data.Dataset` and `torch.utils.data.DataLoader`. They enable you to use pre-loaded datasets or your own data. `Dataset` defines what the data are and how to access them, while `DataLoader` defines how to load that data efficiently for model training or inference.

PyTorch Dataset:

1. Represents the data and enables access to individual samples
2. Each item of the dataset has an input and a target
3. Defines how to load and preprocess individual data points
4. Implements `__getitem__()` to retrieve a single sample and `__len__()` for the total number of samples
5. Focuses on data representation and access

PyTorch Introduction

PyTorch DataLoader:

1. Wraps a Dataset and provides utilities for batching, shuffling, and parallel data loading
2. Handles the iteration over the dataset, creating batches, and optionally shuffling
3. Allows easy specification of batch size, number of worker processes, etc.
4. Focuses on efficiently feeding data to the model during training/evaluation

To illustrate the operations of Dataset and DataLoader, we'll work with the CSV file that we used in the Python chapter and lab and the TensorFlow chapter. First, we read in the CSV file into a DataFrame.

```
import pandas as pd
sample_df = pd.read_csv('SampleDF.csv')
```

Next, we extract two columns that we will use as inputs and targets.

```
P = np.array(sample_df['FVC'])
T = np.array(sample_df['Percent'])
```

Let's define a Dataset class. It should have `__init__()`, `__getitem__()` and `__len__()` methods. The example below is the simplest possible Dataset. It loads in the features and targets, it determines the length of the dataset, and it retrieves one item, which consists of a feature and a target. There are many data handling/preprocessing tasks that could also be performed in a Dataset: reading data from files, parsing structured data formats (e.g., JSON, XML), normalization, augmentation, etc. We will consider some of these in the labs.

```

class SimpleDataset(torch.utils.data.Dataset):
    def __init__(self, P, T):
        # convert into PyTorch tensors
        self.P = torch.tensor(P, dtype=torch.float32)
        self.T = torch.tensor(T, dtype=torch.float32)

    def __len__(self):
        # this should return the size of the dataset
        return len(self.P)

    def __getitem__(self, idx):
        # return one input and one target sample from the
        # ↪ dataset
        features = self.P[idx]
        target = self.T[idx]
        return features, target

```

Now we load the data into the Dataset.

```
dataset = SimpleDataset(P, T)
```

Now that we have the data loaded into the Dataset, we can wrap the data in a DataLoader. This enables us to control how the data will be fed to the training process (batch size, shuffling).

```

loader = torch.utils.data.DataLoader(dataset, shuffle=True
    ↪ , batch_size=4)

```

The DataLoader is an iterable, like a data generator, so we can access the elements within the training 'for' loop.

```

for feat, targ in loader:
    print ('Features:_{},_Target:_{}'.format(feat, targ))
    break

```

```

Features: tensor([2603., 2762., 1537., 4738.]), Target:
    ↪ tensor([ 71.1591,  72.4174,  63.5281, 113.0464])

```

Here you can see that one minibatch of four inputs and targets has been retrieved, since we set the batch_size to 4.

PyTorch Introduction

In the labs and case studies we will investigate other options of `Dataset` and `DataLoader` that allow us to distribute operations across multiple gpus, incorporate augmentation into the data pipeline, prefetch data, etc.

Epilogue

PyTorch and TensorFlow are currently the most popular deep learning frameworks. They have many similarities, and their performance in terms of computation, training times and memory storage are comparable. They both have strong community support, good documentation and active development. PyTorch has a more intuitive API, and enables faster development and easier debugging. TensorFlow has more deployment options, stronger mobile support and is more established in industry.

PyTorch Introduction

Further Reading

[PyTorch, 2024a] PyTorch. Home - pytorch. <https://pytorch.org/>, 2024a. Accessed: 2024-10-09

This is the main web page for PyTorch. It has access to tutorials, as well as links to more extensive documentation.

[PyTorch, 2024c] PyTorch. Pytorch documentation. <https://pytorch.org/docs/stable/index.html>, 2024c. Accessed: 2024-10-09

The official PyTorch documentation. From here you can access information on all aspects of PyTorch.

[PyTorch, 2024d] PyTorch. Pytorch tutorials. <https://pytorch.org/tutorials/>, 2024d. Accessed: 2024-10-09

This is a link to all PyTorch tutorials.

[PyTorch, 2024b] PyTorch. Pytorch datasets. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html, 2024b. Accessed: 2024-10-09

This page contains links to a wide variety of pre-loaded datasets contained in PyTorch. It also explains how to use the PyTorch DataLoader.

Labs

The labs that go with this chapter can be found at <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/tree/master/NNDesignDeepLearning/10.PyTorchIntroChapter/Code/Labs>. Some of the labs will be in Jupyter Notebooks. You can open the labs and run them for free in Google Colab, if you have a gmail account. You can also download and run them on your personal computer.

11 Linear Sequence Processing

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>What are sequences?</i>	2
<i>Dynamic systems</i>	3
<i>Differences between static and dynamic problems</i>	4
<i>Example: Sequence averaging</i>	6
<i>Impulse response and convolution</i>	9
<i>Infinite impulse response systems</i>	13
<i>State space models</i>	18
<i>Epilogue</i>	22
<i>Further Reading</i>	23
<i>Summary of Results</i>	24
<i>Solved Problems</i>	28
<i>Exercises</i>	35

Objective

Neural network inputs can take several different forms. In previous chapters we considered networks with vector inputs (tabular form) and image inputs. In this chapter we consider networks with sequential inputs. In particular, we will focus on networks that perform linear operations on sequences. By beginning with linear operations, we can focus on some basic concepts that will be important when progressing to more complex networks, which we cover in later chapters.

Theory and Examples

What are sequences?

A *sequence* is an ordered collection of elements, where both their values and their order matter. Sequences can appear in a variety of application areas, as you can see in the following table.

Application	Example Sequence
DNA analysis (nucleotides)	{A, T, C, G, G, T, A, C, C, C, T, T}
Protein folding (amino acids)	{Met, Gly, Trp, Ser, Cys, Ile, Ala, Leu, Phe, Leu}
E-commerce (actions)	{homepage, product search, category browse, product view, add to cart, view cart, start checkout, payment, confirmation}
Stock prediction (prices)	{150.25, 151.50, 153.15, 155.75, 154.30}
Health monitoring (heart rate)	{72, 73, 75, 74, 75, 73, 73, 76, 74}
Translation (words)	{‘A’, ‘sequence’, ‘is’, ‘an’, ‘ordered’, ‘set’, ‘of’, ‘elements’}

The elements in the sequence do not have to be numerical, but before the sequence can be input to a neural network it must be converted to a real number, or a vector of real numbers, in an embedding process. (We will discuss this in later chapters.) For now, we will assume that all network inputs are numerical. We will refer to individual elements of a sequence as either *time points*, or *positions*.

Processing sequences is fundamentally different than processing static examples, in which each example input has no relationship to other examples and can be processed alone without losing any important information. Also, adding temporal (or positional) dimensions creates several cognitive challenges. For example, static problems can be visualized at once, whereas temporal problems require "playing through" sequences, and each time step adds dimensions to the problem space. In addition, sequential processing models can have feedback loops, which require circular (chicken and egg) reasoning.

Dynamic systems

In order to process sequences effectively, we need systems with memory – *dynamic systems*. If a system had no memory it would be a static function mapping inputs to outputs, and the output would be determined solely by the inputs at the the same time point. Dynamic systems are characterized by a state that evolves over time, and the system output depends on both the current input and the state.

Some dynamic systems have finite memory, while others have infinite memory. Infinite memory requires that the system have feed-back (recurrent connections). In this chapter we consider only *linear, discrete-time* systems. We will start with finite memory systems, and then we will extend the conversation to infinite memory systems.

Memory is incorporated into dynamic networks using the *delay*, which we first introduced in Chapter 2 of *NND2*. The output of the delay is computed from its input according to

$$\mathbf{a}(t) = \mathbf{p}(t - 1) \quad (11.1)$$

Note that in Figure 11.1a the initial condition for the delay, $\mathbf{a}(0)$, is indicated by the vertical arrow going into the delay block.

As described in Chapter 10 of *NND2*, it is often useful to stack delays together to obtain a *tapped delay line* (TDL). This produces a history of previous values of the input, as shown in Figure 11.1b.

Chapters 10 and 14 of *NND2* also describe dynamic neural networks.

For *linear* systems, if the input is doubled, then the output is doubled. If two inputs are added together, the total output will be the sum of the two individual outputs.

Discrete-time systems operate on an indexed sequence of numbers, as opposed to continuous-time systems, which operate on continuous input signals. There are continuous-time neural networks, but we don't cover those in this text.

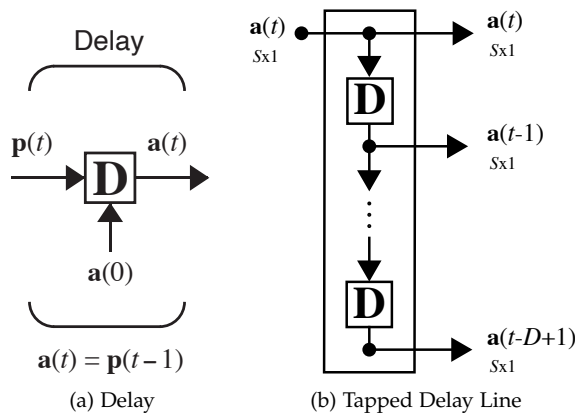


Figure 11.1: Delay and Tapped Delay Line

Linear Sequence Processing

Differences between static and dynamic problems

Since all of the previous chapters have focused on static problems, it is worth taking a moment to reset our thinking to focus on dynamic problems. In static problems the neural network is, in essence, approximating a static function. It could be a static function that maps from characteristics of a neighborhood to the price of a home, or it could be a static function that maps from pixel values to the probability that the image contains a cat. In dynamic problems, however, the neural network is approximating a dynamic system, which maps from input sequences to output sequences.

To illustrate this idea, consider the blue dots in Figure 11.2. They could be targets for a static network whose inputs range from 0 to 10. In that case the network would be approximating the static function $y = f(p) = \sin(2\pi p/10)$, and each blue dot is at a different value of p .

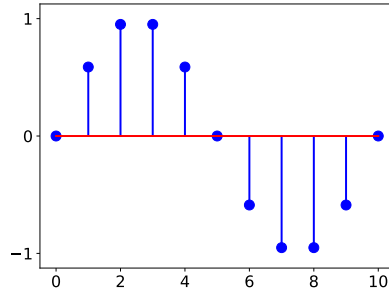


Figure 11.2: Sinusoidal Function

For the static problem, Figure 11.3 would provide a solution, where the weights and biases are set to the values given in Eqs. 11.2 and 11.3.

$$\mathbf{w}^1 = \begin{bmatrix} -0.6246 \\ -0.5410 \\ -0.6246 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 0.3239 \\ 2.7051 \\ 5.9225 \end{bmatrix} \quad (11.2)$$

$$\mathbf{w}^2 = \begin{bmatrix} -5.6565 & 7.4708 & -5.6565 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 1.9211 \end{bmatrix} \quad (11.3)$$

On the other hand, the blue dots could represent a time sequence, produced by passing the input sequence in Figure 11.4 through a dynamic system. In that case, our network would be approximating the dynamic system $y(t) = 1.62y(t-1) - y(t-2) + 0.6p(t-1)$, and each blue dot is at a different value of t .

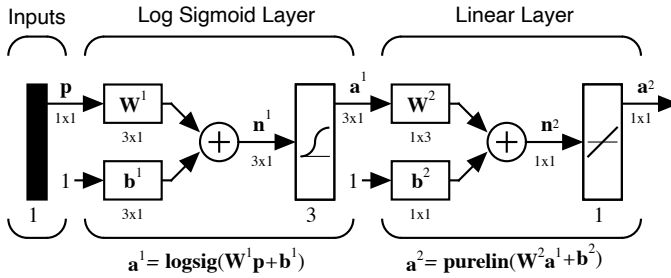


Figure 11.3: Multilayer Network Solution to the Static Problem.

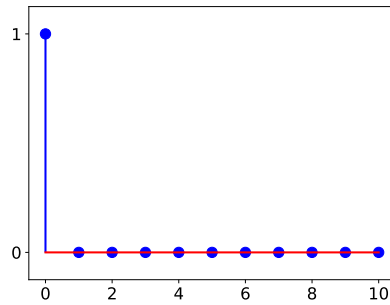


Figure 11.4: Input Sequence

For the dynamic problem, the network in Figure 11.5 would provide a solution, with the weights set to the values given in Eq. 11.4.

$$\mathbf{IW} = [0.6], \mathbf{b} = [0], \mathbf{LW} = [1.62 \quad -1] \quad (11.4)$$

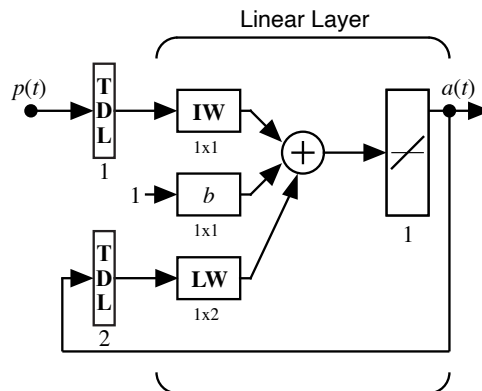


Figure 11.5: Dynamic Network Solution to the Dynamic Problem

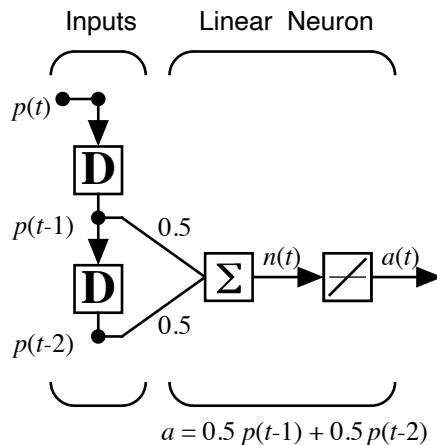
To summarize, here are some differences to keep in mind when approaching static and dynamic problems:

- Static problems:

Linear Sequence Processing

- Example inputs are single scalars, vectors, arrays or tensors.
- The network has no memory.
- Each example input is considered in isolation.
- Static problems use algebraic equations.
- Static networks don't have feedback
- Dynamic sequence processing problems:
 - Example inputs are sequences of scalars, vectors, arrays or tensors.
 - The network has memory.
 - Each time step adds dimensions to the problem space.
 - The network must use relationships between the elements of each sequence.
 - Effects can be separated from causes by many time steps.
 - Dynamic problems require difference equations.
 - Dynamic networks may use feedback.
 - Feedback creates stability issues.

Example: Sequence averaging



$$\frac{2}{+2} \\ 4$$

Figure 11.6: Sequence Averaging Network

Figure 11.6 is an example of a dynamic sequence processing network. A TDL is used at the input of the network to store the previous two time points of the input sequence $p(t)$. The two weights in the network are both equal to 0.5, so the network averages the last two values of the sequence.

Table 11.1 illustrates how the network would respond to a particular input sequence, where each column of the table corresponds to a different time point. Figure 11.7 shows plots of the input and output sequence.

t	0	1	2	3	4	5	6	7	8
$p(t)$	0	1	2	3	2	1	0	0	0
$a(t)$	0	0	$\frac{1}{2}$	$\frac{3}{2}$	$\frac{5}{2}$	$\frac{5}{2}$	$\frac{3}{2}$	$\frac{1}{2}$	0
TDL	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Table 11.1: Sequence Averaging Network Operation

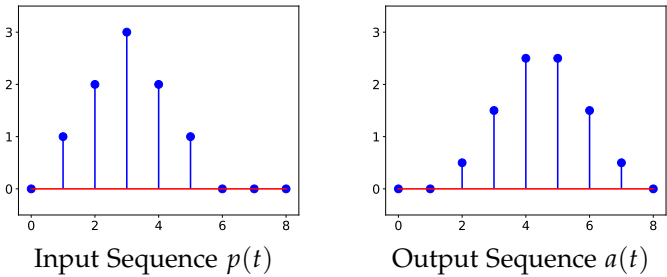


Figure 11.7: Sequence Averaging Network Response

To experiment with this sequence averaging network, use the Deep Learning Demonstration Sequence Averaging Network (**dl11san**).



Linear Sequence Processing

The following code shows how to implement the sequence averaging network in Python.

```
class averaging_network:
    def __init__(self, iw, p_tdl=[]):
        self.iw = np.array(iw)

        if len(p_tdl) > 0:
            self.p_tdl = np.array(p_tdl)
        else:
            if len(self.iw) > 0:
                self.p_tdl = np.zeros(len(self.iw) - 1)
            else:
                self.p_tdl = np.zeros(0)

    def step(self, p):
        a = self.iw[0] * p

        for i in range(len(self.p_tdl)):
            a += self.iw[i + 1] * self.p_tdl[i]

        if len(self.p_tdl) > 0:
            self.p_tdl = np.roll(self.p_tdl, 1)
            self.p_tdl[0] = p

        return a

    def process(self, input_sequence):
        output = np.zeros(len(input_sequence))
        for i in range(len(input_sequence)):
            output[i] = self.step(input_sequence[i])
        return output
```

Initialize the TDL. If no values are provided, initialize with zeros.

Multiply weights time TDL elements to update one time step.

Update the TDL by rolling one element out and shifting in the new time point.

Process the entire input sequence.

The script below invokes the averaging network with the input from the example above and produces the correct network output.

```
# Input weights
iw = [0, 0.5, 0.5]

# Input sequence
p = [0, 1, 2, 3, 2, 1, 0, 0, 0]

# Define the network
net = averaging_network(iw)

# Run the input through the network
a = net.process(p)

print('Input_sequence: _')
print(p)
print('Output_sequence: _')
print(a)
```

```
Input sequence:
[0, 1, 2, 3, 2, 1, 0, 0, 0]
Output sequence:
[0.  0.  0.5 1.5 2.5 2.5 1.5 0.5 0. ]
```

Impulse response and convolution

In this section we introduce three concepts that are important when analyzing linear dynamic systems: the *impulse sequence*, the *impulse response* and the *convolution sum*. We introduced these ideas in the image processing context in Chapter 8, Convolution. There we saw that any linear, position-invariant transformation on an arbitrary image can be represented as the 2-D convolution sum of the impulse response of the transformation with the input image. The impulse in that case was an image with a single nonzero pixel. In this section we show that the response of any linear, *time-invariant* dynamic system to an arbitrary input sequence can be represented as the 1-D convolution sum of the impulse response of the dynamic system and the input sequence.

In the image processing context, the unit impulse was the image with a single nonzero pixel with a value of 1. In the sequence processing context, the unit impulse is a sequence with one nonzero time point point, which occurs at $t = 0$. It is defined as

A *time-invariant system* is one for which the response to an input sequence that has been shifted in time is just a shifted version of the response to the unshifted input sequence.

Linear Sequence Processing

$$\delta(t) = \begin{cases} 1 & t = 0 \\ 0 & t \neq 0 \end{cases} \quad (11.5)$$

Table 11.2 tabulates the impulse sequence $\delta(t)$ and also shows a shifted version $\delta(t - 2)$. A shifted version is nonzero when its argument is zero, in other words when $t - 2 = 0$, or $t = 2$.

t	-4	-3	-2	-1	0	1	2	3	4
$\delta(t)$	0	0	0	0	1	0	0	0	0
$\delta(t - 2)$	0	0	0	0	0	0	1	0	0

Table 11.2: Impulse and shifted impulse

Figure 11.8 shows plots of the impulse sequence and its shifted version.

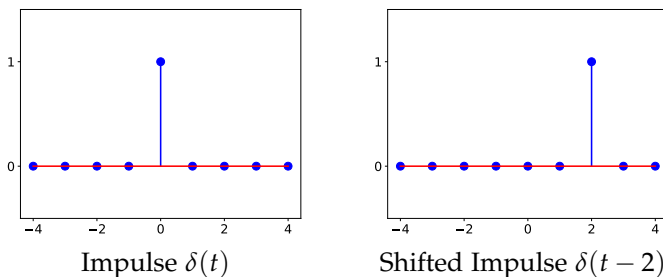


Figure 11.8: Impulse Sequences

The shifted impulse is useful in representing general sequences. For example, we can represent the input sequence in Figure 11.7 with the following formula:

$$p(t) = 1\delta(t - 1) + 2\delta(t - 2) + 3\delta(t - 3) + 2\delta(t - 4) + 1\delta(t - 5) \quad (11.6)$$

To interpret Eq. 11.6, we want to keep in mind that we are combining five different sequences to produce $p(t)$. We can make this more concrete with Table 11.3, which shows the individual sequences and their sum.

We can represent any sequence $p(t)$ in a similar way, as follows:

$$p(t) = \sum_i p(i)\delta(t - i) \quad (11.7)$$

t	0	1	2	3	4	5	6	7	8
$1\delta(t-1)$	0	1	0	0	0	0	0	0	0
$2\delta(t-2)$	0	0	2	0	0	0	0	0	0
$3\delta(t-3)$	0	0	0	3	0	0	0	0	0
$2\delta(t-4)$	0	0	0	0	2	0	0	0	0
$1\delta(t-5)$	0	0	0	0	0	1	0	0	0
$p(t)$	0	1	2	3	4	1	0	0	0

Table 11.3: Summing shifted impulses

Now let's consider what happens when we apply an impulse input to a linear, time-invariant system. First, we generalize the sequence averaging system of Figure 11.6. If we allow the TDL to be of arbitrary size, we would have a system that satisfies Eq. 11.8.

$$a(t) = \sum_{i=1}^{n_\beta} \beta_i p(t-i) \quad (11.8)$$

where the coefficients β_i are fixed numbers (corresponding to weights in a neural network). This can be expanded to the *difference equation* in Eq. 11.9.

$$a(t) = \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta) \quad (11.9)$$

This system can be represented by the neural network in Figure 11.9, where the weight and bias are given in Eq. 11.10.

$$\mathbf{IW} = \begin{bmatrix} \beta_1 & \beta_2 & \dots & \beta_{n_\beta} \end{bmatrix}, b = 0 \quad (11.10)$$

Now apply an impulse sequence input to this system. The response is computed in Eq. 11.11 by substituting $\delta(t)$ for $p(t)$ in Eq. 11.8.

This is a *causal* system, since the output $a(t)$ does not depend on future values of the input sequence.

Linear Sequence Processing

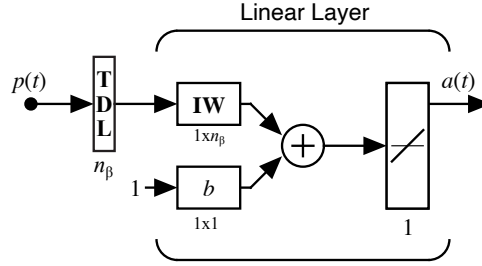


Figure 11.9: FIR Network

$$\begin{aligned}
 h(t) &= \sum_{i=1}^{n_\beta} \beta_i \delta(t-i) \\
 h(1) &= \sum_{i=1}^{n_\beta} \beta_i \delta(1-i) = \beta_1 \\
 h(2) &= \sum_{i=1}^{n_\beta} \beta_i \delta(2-i) = \beta_2 \\
 &\vdots \\
 h(n_\beta) &= \beta_{n_\beta}
 \end{aligned} \tag{11.11}$$

where $h(t)$ represents the system impulse response. We can write the impulse response in terms of the difference equation coefficients, as in Eq. 11.12.

$$h(t) = \sum_{i=1}^{n_\beta} \beta_i \delta(t-i) = \begin{cases} \beta_t & t = 1, \dots, n_\beta \\ 0 & \text{otherwise} \end{cases} \tag{11.12}$$

The impulse response is illustrated in tabular form in Table 11.4.

t	0	1	2	3	4	$\dots$	n_β	$n_\beta + 1$	$t > n_\beta$
$p(t) = \delta(t)$	1	0	0	0	0	0	0	0	0
$a(t) = h(t)$	0	β_1	β_2	β_3	β_4	$\dots$	β_{n_β}	0	0

Table 11.4: Impulse Response

For this system the impulse response is the list of difference equation coefficients. Since $h(t) = 0$ for $t < 1$ and for $t > n_\beta$, the length of the impulse response sequence $h(t)$ is finite. That is why the system in Eq. 11.9 is called a *finite impulse response* (FIR) system.

Since the FIR coefficients in the difference equation are identical to the impulse response values, we can write

$$a(t) = \sum_{i=1}^{n_\beta} h(i)p(t-i) \quad (11.13)$$

Eq. 11.13 is called a convolution sum, and is often represented using the abbreviated notation in Eq. 11.14. The length of the impulse response can be a measure of the memory of the system. In this case, the system looks back at the last n_β values of the input sequence.

$$a(t) = h(t) * p(t) \quad (11.14)$$

We can show that the response of any linear, time-invariant system can be written as a convolution sum of the input sequence and the system impulse response. For example, consider a general input sequence $p(t)$, represented as impulses:

$$p(t) = \sum_i p(i)\delta(t-i) \quad (11.15)$$

and assume that a linear, time-invariant system $\mathcal{W}$ operates on that input:

$$a(t) = \mathcal{W}(p(t)) = \mathcal{W}\left(\sum_i p(i)\delta(t-i)\right) \quad (11.16)$$

Because of linearity and time invariance, the operation can be brought inside the summation, and the operation will be consistent at each time step. The response is a convolution of the impulse response and the input.

$$a(t) = \mathcal{W}(p(t)) = \sum_i p(i)\mathcal{W}(\delta(t-i)) = \sum_i p(i)h(t-i) = h(t) * p(t) \quad (11.17)$$

Infinite impulse response systems

In the FIR system of Eq. 11.9, $a(t)$ depends only on previous values of the input sequence $p(t)$. What if we were to modify the system so that $a(t)$ depends also on previous outputs? A simple system of that form is shown in Figure 11.10.

Linear Sequence Processing

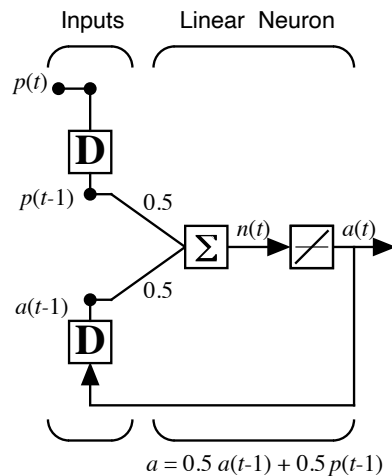


Figure 11.10: Sequence Smoothing Network

If we apply the input from Table 11.1 to this network, we get the response shown in Table 11.5. Here we have two TDLs – one for previous $p(t)$ and one for previous $a(t)$.

t	0	1	2	3	4	5	6	7	8
$p(t)$	0	1	2	3	2	1	0	0	0
$a(t)$	0	0	0.5	1.25	2.125	2.063	1.531	0.766	0.383
p_TDL	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 1 \end{bmatrix}$	$\begin{bmatrix} 2 \end{bmatrix}$	$\begin{bmatrix} 3 \end{bmatrix}$	$\begin{bmatrix} 2 \end{bmatrix}$	$\begin{bmatrix} 1 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$
a_TDL	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0.5 \end{bmatrix}$	$\begin{bmatrix} 1.25 \end{bmatrix}$	$\begin{bmatrix} 2.125 \end{bmatrix}$	$\begin{bmatrix} 2.063 \end{bmatrix}$	$\begin{bmatrix} 1.531 \end{bmatrix}$	$\begin{bmatrix} 0.766 \end{bmatrix}$

Table 11.5: Sequence Smoothing Network Response

A plot of the input and output sequences are shown in Figure 11.11. Note that, although we only have one delay of the input in the TDL, the output sequence extends more than one time step after the input sequence drops to zero. In fact, even if we extend the output sequence indefinitely it will not become zero. This network has infinite memory.

To illustrate that this network has infinite memory, the system impulse response is tabulated in Table 11.6.

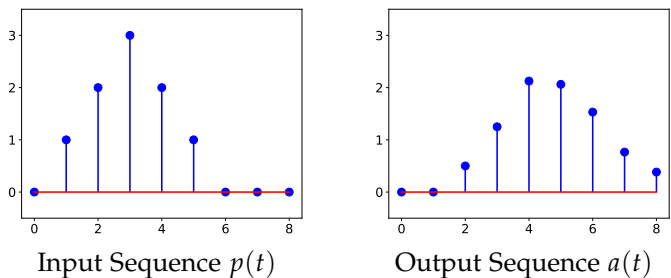


Figure 11.11: Response of Sequence Smoothing Network

t	0	1	2	3	4	5	6	7	8
$p(t) = \delta(t)$	1	0	0	0	0	0	0	0	0
$a(t) = h(t)$	0	$\frac{1}{2}$	$\frac{1}{2^2}$	$\frac{1}{2^3}$	$\frac{1}{2^4}$	$\frac{1}{2^5}$	$\frac{1}{2^6}$	$\frac{1}{2^7}$	$\frac{1}{2^8}$
p_TDL	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 1 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$
a_TDL	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} 0 \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^2} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^3} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^4} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^5} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^6} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{2^7} \end{bmatrix}$

Table 11.6: Impulse Response for Sequence Smoothing Network

The cell for the initial TDL for $a(t)$ is highlighted in red, because the impulse response is defined as the response to the impulse sequence with all initial conditions set to zero.

Figure 11.12 displays the impulse response. The feedback connection produced an *infinite impulse response (IIR)*, therefore the memory of the network is infinite. However, we can see that it decays exponentially to zero. Although it has infinite memory, the effective memory is only six or seven steps, since the impulse response is less than 0.01 by step seven. As the feedback weight comes closer to 1, the effective memory length gets longer.

An IIR system can be approximated by an FIR system, if the TDL length for $p(t)$ is long enough, because their impulse responses can be made approximately the same.

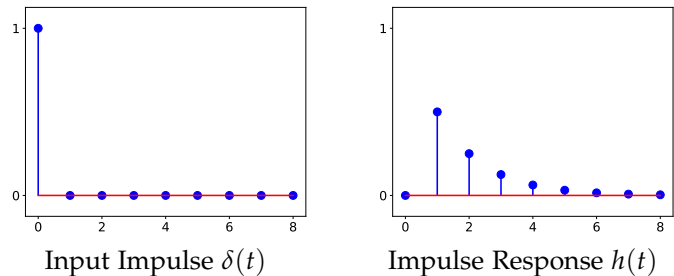


Figure 11.12: Sequence Smoothing Network Impulse Response

To experiment with the sequence smoothing network, use the Deep Learning Demonstration Sequence Smoothing Response (**dl11ssr**).



Linear Sequence Processing

A general IIR system can be written in the following form.

$$a(t) = \sum_{i=1}^{n_\alpha} \alpha_i a(t-i) + \sum_{i=1}^{n_\beta} \beta_i p(t-i) \quad (11.18)$$

If we expand the summation, we have the difference equation.

$$a(t) = \alpha_1 a(t-1) + \alpha_2 a(t-2) + \dots + \alpha_{n_\alpha} a(t-n_\alpha) + \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta) \quad (11.19)$$

This general IIR system can be implemented as a recurrent neural network, as shown in Figure 11.13, where the weights are

$$\mathbf{IW} = \begin{bmatrix} \beta_1 & \beta_2 & \dots & \beta_{n_\beta} \end{bmatrix} \quad (11.20)$$

$$\mathbf{LW} = \begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_{n_\alpha} \end{bmatrix} \quad (11.21)$$

$$b = 0 \quad (11.22)$$

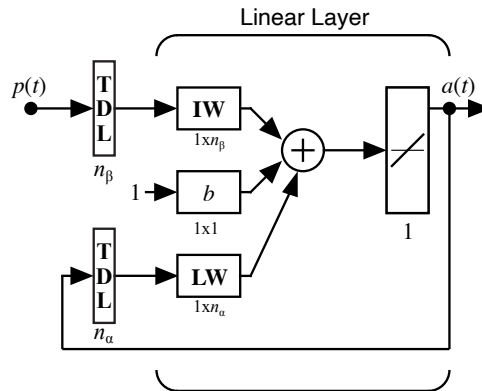


Figure 11.13: Infinite Impulse Response Network

When using IIR systems, which always have recurrent (feedback) connections, we face a problem that does not occur for FIR systems – *stability*. For example, consider the sequence smoothing network of Figure 11.10. If we were to change the feedback weight from 0.5 to 2, the impulse response would double at each time step, rather than decreasing by half.

To investigate stability for general linear dynamic systems, we need to introduce some additional notation. First, we define the delay operator D :

One definition for stability of dynamic systems is that a bounded input will yield a bounded output.

$$Da(t) = a(t-1) \quad (11.23)$$

We can then define the following:

$$\alpha(D) = 1 - \alpha_1 D - \alpha_2 D^2 - \dots - \alpha_{n_\alpha} D^{n_\alpha} \quad (11.24)$$

$$\beta(D) = \beta_1 D + \beta_2 D^2 + \dots + \beta_{n_\beta} D^{n_\beta} \quad (11.25)$$

This allows us to rewrite Eq. 11.19 as

$$a(t) = \frac{\beta(D)}{\alpha(D)} p(t) \quad (11.26)$$

It can be shown that the impulse response of the difference equation of Eq. 11.19 is determined from the roots of the denominator polynomial $\alpha(D)$. Setting $\alpha(D)$ to zero, we obtain the *characteristic equation* [Elaydi, 2005]:

$$\alpha(D) = \prod_{i=1}^{n_\alpha} (1 - \lambda_i D) = 0 \quad (11.27)$$

The impulse response for Eq. 11.19 will have the form

$$h(t) = A_1 (\lambda_1)^t + A_2 (\lambda_2)^t \dots + A_n (\lambda_n)^t \quad (11.28)$$

if the characteristic roots λ_i (also called the system *poles*) are not repeated. In addition, we can say that the system will be stable if the poles are inside the unit circle (have magnitude less than one).

For example, consider the following difference equation.

$$a(t) = a(t-1) - 0.24a(t-2) + p(t-1) \quad (11.29)$$

Using the D operator, we can write:

$$a(t) = \frac{D}{1 - D + 0.24D^2} p(t) \quad (11.30)$$

The characteristic polynomial can be factored.

$$\alpha(D) = 1 - D + 0.24D^2 = (1 - 0.6D)(1 - 0.4D) \quad (11.31)$$

Because the poles $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$ are inside the unit circle, the system is stable.

The impulse response of Eq. 11.29 is given in Eq. 11.32.

$$h(t) = 5(0.6)^t - 5(0.4)^t \quad (11.32)$$

It is plotted in Figure 11.14.



Linear Sequence Processing

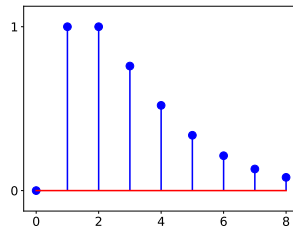


Figure 11.14: Impulse Response for Eq.11.29

To experiment with difference equations and impulse responses, use the Deep Learning Demonstration Impulse Response (**dl11ir**).



State space models

The difference equation models for dynamic systems that we have been considering are often called *input/output models* or *transfer function models*. There is another type of dynamic model - the *state space model*.

In the state space model, instead of a difference equation of order n , we have n first order difference equations:

$$q_i(t+1) = \phi_{i,1}q_1(t) + \phi_{i,2}q_2(t) + \cdots + \phi_{i,n}q_n(t) + \gamma_{i,1}p_1(t) + \gamma_{i,2}p_2(t) + \cdots + \gamma_{i,m}p_m(t) \quad (11.33)$$

$$a_j(t) = \psi_{j,1}q_1(t) + \psi_{j,2}q_2(t) + \cdots + \psi_{j,n}q_n(t) \quad (11.34)$$

$$(11.35)$$

which can be written in matrix form as

$$\mathbf{q}(t+1) = \mathbf{\Phi}\mathbf{q}(t) + \mathbf{\Gamma}\mathbf{p}(t) \quad (11.36)$$

$$\mathbf{a}(t) = \mathbf{\Psi}\mathbf{q}(t) \quad (11.37)$$

where the q_i are the *states* of the system. Knowing $\mathbf{q}(0)$ allows us to solve for all future states and outputs. The eigenvalues of $\mathbf{\Phi}$ are the system poles λ_i .

The state space model can be represented as a recurrent neural network, as shown in Figure 11.15, with the weights given in Eq. 11.38.

$$\mathbf{LW}^{1,1} = \mathbf{\Phi}, \quad \mathbf{IW}^{1,1} = \mathbf{\Gamma}, \quad \mathbf{LW}^{2,1} = \mathbf{\Psi}, \quad \mathbf{a}^1 = \mathbf{q}, \quad \mathbf{a}^2 = \mathbf{a} \quad (11.38)$$

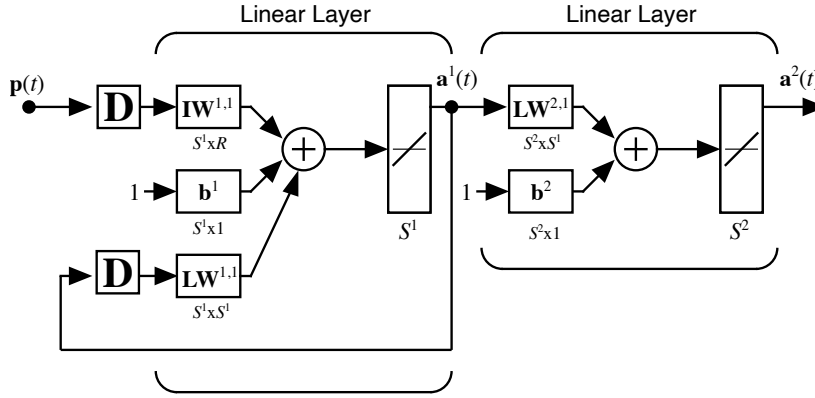


Figure 11.15: State Space Neural Network

For linear systems, it is always possible to convert from an input/output model to a state space model, although the conversion is not unique. In fact, for a given input/output model, there are an infinite number of equivalent state space models. It is worth demonstrating one method for doing the conversion, because it provides some insight into the meaning of the state.

Let's start with the general input/output difference equation of Eq. 11.19, which we repeat here for easy reference.

$$a(t) = \alpha_1 a(t-1) + \alpha_2 a(t-2) + \dots + \alpha_{n_\alpha} a(t-n_\alpha) + \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta) \quad (11.39)$$

This equation can be represented graphically by the simulation diagram in Figure 11.16. The output of the network is $a(t)$. Looking at the summation junction before the right-most delay, we see that two of the inputs are $\alpha_1 a(t)$ and $\beta_1 p(t)$. Since these two terms will be delayed once to become $a(t)$, they produce two terms on the right side of Eq. 11.39: $\alpha_1 a(t-1) + \beta_1 p(t-1)$. The additional terms appear at earlier delays, so that when they reach $a(t)$ they will be delayed the appropriate number of times.

We can use the simulation diagram to find a state space model. The state must contain the minimum number of variables so that if we know those variables at $t = 0$ and we know the input sequence we can solve for the output for all future time. From Figure 11.16,

Linear Sequence Processing

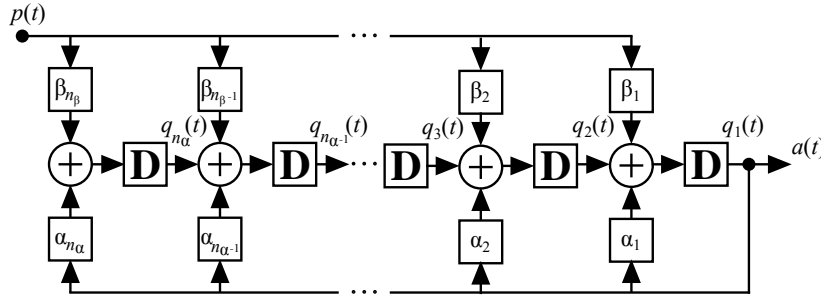


Figure 11.16: Simulation Diagram for IIR System

we can see that if we know the initial conditions on the delay blocks we can solve for $a(t)$ at all future times. This means that we can choose the outputs of the delays to be the states. Using the states as shown in Figure 11.16, the state equations become

$$\mathbf{q}(t+1) = \begin{bmatrix} \alpha_1 & 1 & 0 & \cdots & 0 \\ \alpha_2 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{n_\alpha-1} & 0 & 0 & \cdots & 1 \\ \alpha_{n_\alpha} & 0 & 0 & \cdots & 0 \end{bmatrix} \mathbf{q}(t) + \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_{n_\beta-1} \\ \beta_{n_\beta} \end{bmatrix} p(t) \quad (11.40)$$

$$a(t) = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \end{bmatrix} \mathbf{q}(t) \quad (11.41)$$

This is called the *observer canonical form* for the state equation. Other canonical forms can be found with different arrangements of the simulation diagram.

where each state equation comes from a summation junction in Figure 11.16.

To illustrate the process, consider again the second order system of Eq. 11.29, repeated below.

$$a(t) = a(t-1) - 0.24a(t-2) + p(t-1) \quad (11.42)$$

The simulation diagram for that system is shown in Figure 11.17.

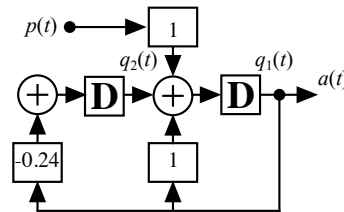


Figure 11.17: Simulation Diagram for Eq. 11.42

If we choose the outputs of the delays to be the state variables, we can write the state equations as follows:

$$q_1(t+1) = 1q_1(t) + 1q_2(t) + 1p(t) \quad (11.43)$$

$$q_2(t+1) = -0.24q_1(t) + 0q_2(t) + 0p(t) \quad (11.44)$$

$$a(t) = 1q_1(t) + 0q_2(t) \quad (11.45)$$

and they can be put in matrix form.

$$\mathbf{q}(t+1) = \begin{bmatrix} 1 & 1 \\ -0.24 & 0 \end{bmatrix} \mathbf{q}(t) + \begin{bmatrix} 1 \\ 0 \end{bmatrix} p(t) \quad (11.46)$$

$$a(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{q}(t) \quad (11.47)$$

Epilogue

Sequence processing is one of the most important applications of deep learning. Language translation, sentiment analysis, chatbots, financial prediction, weather forecasting, protein structure prediction, drug discovery, music generation, speech synthesis, and video captioning are just a few of the ways that deep learning has been used for sequence processing.

This chapter is just the beginning of our investigation into sequence processing. The linear methods introduced in this chapter have introduced many of the key concepts that we will build on over the next few chapters. In those chapters we will again see tapped delay lines, finite memory networks, infinite memory networks, input/output models and state space models.

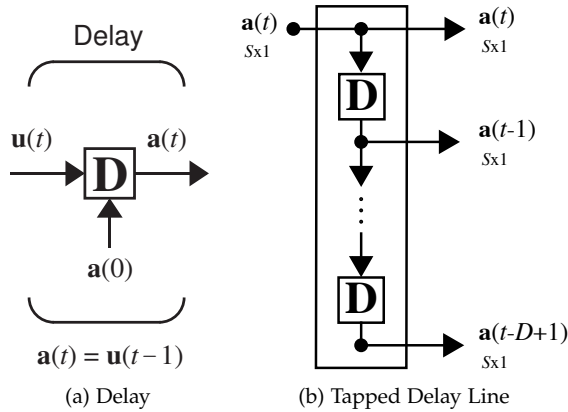
Further Reading

[Elaydi, 2005] Saber N Elaydi. *An Introduction to Difference Equations (3rd Edition)*. New York, NY: Springer New York: Imprint: Springer,, 2005

Chapter 2 of this text, Linear Difference Equations, provides a comprehensive treatment of the theory connecting characteristic polynomials to difference equation solutions, including proofs of the fundamental theorems and extensive examples. Chapter 3, Systems of Linear Difference Equations, covers state space models and how to convert from n th order difference equations to state space models.

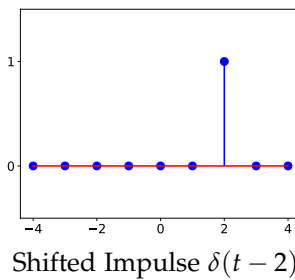
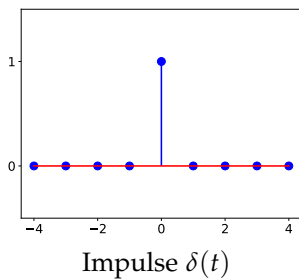
Summary of Results

Delay and Tapped Delay Line (TDL)



Impulse Function

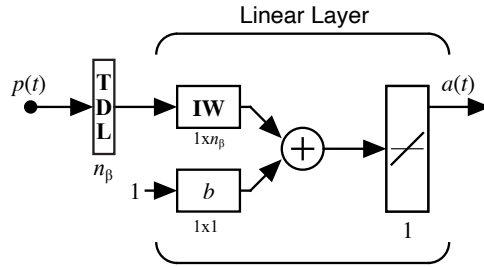
$$\delta(t) = \begin{cases} 1 & t = 0 \\ 0 & t \neq 0 \end{cases}$$



Finite Impulse Response (FIR) System

$$a(t) = \sum_{i=1}^{n_\beta} \beta_i p(t-i)$$

$$a(t) = \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta)$$



$$\mathbf{IW} = \begin{bmatrix} \beta_1 & \beta_2 & \cdots & \beta_{n_\beta} \end{bmatrix}, b = 0$$

$$h(t) = \sum_{i=1}^{n_\beta} \beta_i \delta(t-i) = \begin{cases} \beta_t & t = 1, \dots, n_\beta \\ 0 & \text{otherwise} \end{cases}$$

Convolution Sum

$$a(t) = \mathcal{W}(p(t)) = \sum_i p(i) \mathcal{W}(\delta(t-i)) = \sum_i p(i) h(t-i) = h(t) * p(t)$$

Infinite Impulse Response (FIR) System

$$a(t) = \sum_{i=1}^{n_\alpha} \alpha_i a(t-i) + \sum_{i=1}^{n_\beta} \beta_i p(t-i)$$

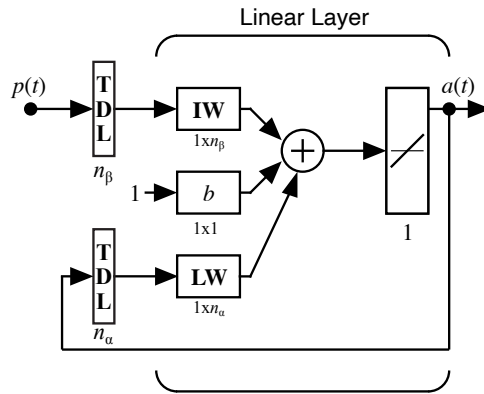
$$a(t) = \alpha_1 a(t-1) + \alpha_2 a(t-2) + \dots + \alpha_{n_\alpha} a(t-n_\alpha) \\ + \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta)$$

$$\mathbf{IW} = \begin{bmatrix} \beta_1 & \beta_2 & \cdots & \beta_{n_\beta} \end{bmatrix}$$

$$\mathbf{LW} = \begin{bmatrix} \alpha_1 & \alpha_2 & \cdots & \alpha_{n_\alpha} \end{bmatrix}$$

$$b = 0$$

Linear Sequence Processing



Stability

$$Da(t) = a(t-1)$$

$$\alpha(D) = 1 - \alpha_1 D - \alpha_2 D^2 - \dots - \alpha_{n_\alpha} D^{n_\alpha}$$

$$\beta(D) = \beta_1 D + \beta_2 D^2 + \dots + \beta_{n_\beta} D^{n_\beta}$$

$$a(t) = \frac{\beta(D)}{\alpha(D)} p(t)$$

$$\alpha(D) = \prod_{i=1}^{n_\alpha} (1 - \lambda_i D) = 0$$

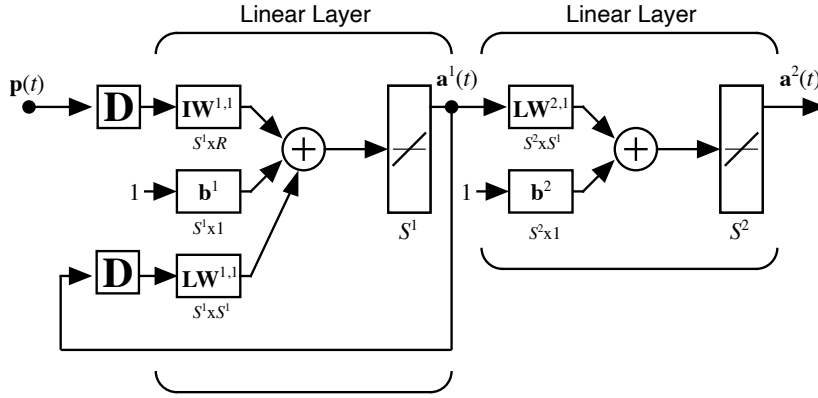
$$h(t) = A_1 (\lambda_1)^t + A_2 (\lambda_2)^t \dots + A_n (\lambda_n)^t$$

The system is stable if $|\lambda_i| < 1$ for all i .

State Space Models

$$\mathbf{q}(t+1) = \mathbf{\Phi} \mathbf{q}(t) + \mathbf{\Gamma} \mathbf{p}(t)$$

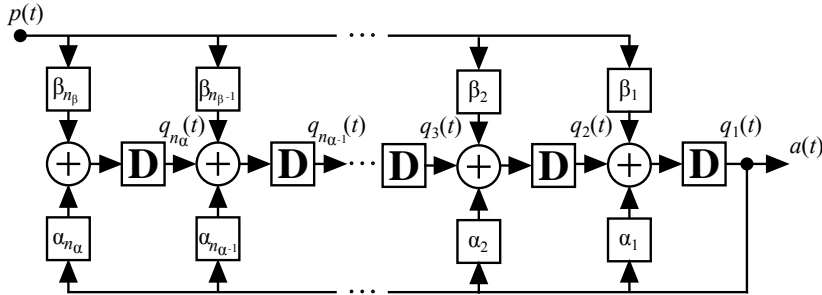
$$\mathbf{a}(t) = \mathbf{\Psi} \mathbf{q}(t)$$



$$LW^{1,1} = \Phi, \quad IW^{1,1} = \Gamma, \quad LW^{2,1} = \Psi, \quad a^1 = \mathbf{q}, \quad a^2 = \mathbf{a}$$

Converting from Input/Output to State Space

$$a(t) = \alpha_1 a(t-1) + \alpha_2 a(t-2) + \dots + \alpha_{n_\alpha} a(t-n_\alpha) \\ + \beta_1 p(t-1) + \beta_2 p(t-2) + \dots + \beta_{n_\beta} p(t-n_\beta)$$



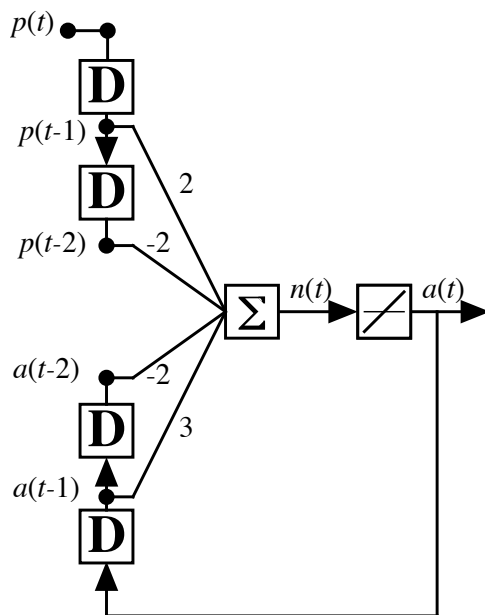
$$\mathbf{q}(t+1) = \begin{bmatrix} \alpha_1 & 1 & 0 & \cdots & 0 \\ \alpha_2 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{n_\alpha-1} & 0 & 0 & \cdots & 1 \\ \alpha_{n_\alpha} & 0 & 0 & \cdots & 0 \end{bmatrix} \mathbf{q}(t) + \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_{n_\beta-1} \\ \beta_{n_\beta} \end{bmatrix} p(t)$$

$$a(t) = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \end{bmatrix} \mathbf{q}(t)$$

P11.1 Consider the following difference equation.

$$a(t) = 3a(t-1) - 2a(t-2) + 2p(t-1) - 2p(t-2)$$

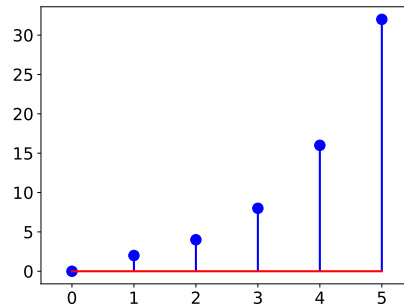
- i. Represent the system as a neural network, using our notation.
 - ii. Find the impulse response by iterating the difference equation by hand up to $t = 5$, and create a table like Table 11.6.
 - iii. Plot the impulse response
 - iv. Find the system poles, and determine if the system is stable.
 - v. Knowing that the impulse response will have the form of Eq. 11.28, if the poles are not repeated, find a closed form representation of the impulse response.
- i. Below is the network diagram.



ii. The following table shows the steps of computing the impulse response by iterating the difference equation.

t	0	1	2	3	4	5
$p(t)$	1	0	0	0	0	0
$a(t)$	0	2	4	8	16	32
p_TDL	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
a_TDL	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 4 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 8 \\ 4 \end{bmatrix}$	$\begin{bmatrix} 16 \\ 8 \end{bmatrix}$

iii. Here is the plot of the impulse response.



iv. We can represent the difference equation as

$$a(t) = \frac{\beta(D)}{\alpha(D)}$$

where

$$\alpha(D) = 1 - 3D + 2D^2$$

$$\beta(D) = 2D - 2D^2$$

We factor the denominator polynomial $\alpha(D)$ to find the system poles.

$$\alpha(D) = 1 - 3D + 2D^2 = (1 - \lambda_1 D)(1 - \lambda_2 D)$$

Linear Sequence Processing

The λ_i are the inverse of the roots of the $\alpha(D)$ polynomial, so we can find the roots of that polynomial first. From the quadratic formula we have

$$\frac{3 \pm \sqrt{9 - 4 \times 2}}{2 \times 2} = \frac{3 \pm 1}{4} = 1, \frac{1}{2}$$

So the poles are $\lambda_1 = 1$ and $\lambda_2 = 2$. Since the poles are not inside the unit circle, the system is not stable.

v. We know from Eq. 11.28 that, since the poles are not repeated, the impulse response should have the form

$$h(t) = A_1 (1)^t + A_2 (2)^t$$

To find the coefficients A_1 and A_2 , we just need two values for $h(t)$. We found $a(1)$ and $a(2)$ in part ii., so we can use those two values.

$$h(1) = 2 = A_1 (1)^1 + A_2 (2)^1 = 1A_1 + 2A_2$$

$$h(2) = 4 = A_1 (1)^2 + A_2 (2)^2 = 1A_1 + 4A_2$$

We can write this in matrix form.

$$\begin{bmatrix} 1 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$
$$\begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 4 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -\frac{1}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Therefore, $A_1 = 0$, $A_2 = 1$ and the impulse response is

$$h(t) = (2)^t, t > 0$$

P11.2 Consider again the difference equation in Solved Problem P.11.1. Assume that the following input is applied to the system.

$$p(t) = 1\delta(t-1) + 1\delta(t-2)$$

- i. Find the response of the system for the first five time steps by iterating the difference equation.

ii. Find the response of the system for the first five time steps by performing a convolution sum between the input and the impulse response.

i. This table shows the steps for iterating the difference equation.

t	0	1	2	3	4	5
$p(t)$	1	1	0	0	0	0
$a(t)$	0	2	6	12	24	48
p_{TDL}	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
a_{TDL}	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 6 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 12 \\ 6 \end{bmatrix}$	$\begin{bmatrix} 24 \\ 12 \end{bmatrix}$

ii. To compute the output using the convolution summation, we use Eq. 11.17.

$$a(t) = \sum_i p(i)h(t-i)$$

Since $p(i) = 0$ for $i < 0$, the lower limit on the sum will be 0. Since $h(j) = 0$ for $j < 1$, the upper limit on the sum will be $i = t - 1$. From the previous problem we know that $h(t) = 2^t$ over the range where it is not zero. Therefore the convolution sum reduces to

$$a(t) = \sum_{i=0}^{t-1} p(i)2^{(t-i)}$$

Also, for this case, $p(t) = 0$ for $t > 1$, so the upper limit of the sum is 1, if $t - 1 > 1$:

$$a(t) = \sum_{i=0}^{\min(t-1,1)} p(i)2^{(t-i)}$$

Linear Sequence Processing

Plugging in values for $p(t)$, we obtain

$$a(1) = p(0)2^1 = (1)(2) = 2$$

$$a(2) = p(0)2^2 + p(1)2^1 = (1)(4) + (1)(2) = 6$$

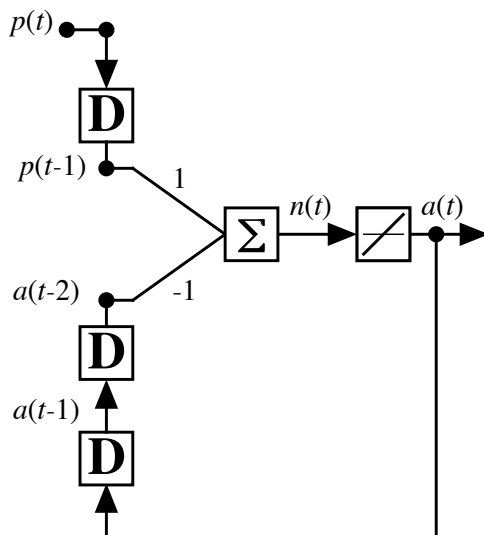
$$a(3) = p(0)2^3 + p(1)2^2 = (1)(8) + (1)(4) = 12$$

$$a(4) = p(0)2^4 + p(1)2^3 = (1)(16) + (1)(8) = 24$$

$$a(5) = p(0)2^5 + p(1)2^4 = (1)(32) + (1)(16) = 48$$

which matches the results from part i.

P11.3 Consider the network shown below.



- i. Find the corresponding difference equation.
- ii. Find the system poles.
- iii. Find the system system impulse response in closed form.

i. The difference equation is

$$a(t) = -a(t-2) + p(t-1)$$

i. To find the poles, we set the $\alpha(D)$ polynomial to zero, and then find the inverse roots

$$\alpha(D) = 1 + D^2 = 0$$

Since we want the inverse roots, we could introduce $E = D^{-1}$

$$1 + D^2 = 1 + E^{-2} = 0$$

$$E^2 + 1 = 0$$

Where we multiplied both sides by E^2 to get the last equation. We can then find the roots of the polynomial in E to find the poles.

From the quadratic equation we have

$$E = \frac{0 \pm \sqrt{0-4}}{2} = \frac{\pm 2j}{2} = \pm j$$

The poles are therefore $\lambda_1 = j$ and $\lambda_2 = -j$, and the impulse response has the form

$$h(t) = A_1 (j)^t + A_2 (-j)^t$$

We need two values for $h(t)$ to solve for A_1 and A_2 . Iterating the difference equation twice, we get

$$a(1) = -a(-1) + p(0) = -0 + 1 = 1$$

$$a(2) = -a(0) + p(1) = -0 + 0 = 0$$

Plugging these values in for $h(1)$ and $h(2)$ in the earlier equation, we have

$$h(1) = 1 = A_1(j) + A_2(-j)$$

$$h(2) = 0 = A_1(j)^2 + A_2(-j)^2 = -A_1 + -A_2$$

We can put this in matrix form:

$$\begin{bmatrix} j & -j \\ -1 & -1 \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} j & -j \\ -1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2j} & -\frac{1}{2} \\ -\frac{1}{2j} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2j} \\ -\frac{1}{2j} \end{bmatrix}$$

Therefore $A_1 = \frac{1}{2j}$ and $A_2 = -\frac{1}{2j}$. We can then write $h(t)$ as

$$h(t) = \frac{1}{2j} (j)^t - \frac{1}{2j} (-j)^t = \frac{(j)^t - (-j)^t}{2j}$$

There is a trigonometric identity

$$\sin(\theta) = \frac{e^{j\theta} - e^{-j\theta}}{2j}$$

Since $j = e^{j\frac{\pi}{2}}$ and $-j = e^{-j\frac{\pi}{2}}$, we can write $h(t)$ as

$$h(t) = \frac{e^{j\frac{\pi}{2}t} - e^{-j\frac{\pi}{2}t}}{2j} = \sin\left(\frac{\pi}{2}t\right)$$

Note: Compare this result to the impulse response of the network in Figure 11.5. That system also has complex poles. Complex poles cause oscillation, while real poles produce exponential responses. When complex poles are on the unit circle, the oscillation does not decay, as in this problem. If the poles are inside the unit circle, the oscillation will decay. If the pole are outside the unit circle, the oscillation will explode.

P11.4 Is there a convolution sum formulation for systems in state space form?

The state space form is

$$\begin{aligned}\mathbf{q}(t+1) &= \mathbf{\Phi}\mathbf{q}(t) + \mathbf{\Gamma}\mathbf{p}(t) \\ \mathbf{a}(t) &= \mathbf{\Psi}\mathbf{q}(t)\end{aligned}$$

If we assume that the system starts with initial state $\mathbf{q}(0)$, we can iterate the state equation as follows.

$$\begin{aligned}\mathbf{q}(1) &= \mathbf{\Phi}\mathbf{q}(0) + \mathbf{\Gamma}\mathbf{p}(0) \\ \mathbf{q}(2) &= \mathbf{\Phi}\mathbf{q}(1) + \mathbf{\Gamma}\mathbf{p}(1) = \mathbf{\Phi}^2\mathbf{q}(0) + \mathbf{\Phi}\mathbf{\Gamma}\mathbf{p}(0) + \mathbf{\Gamma}\mathbf{p}(1) \\ \mathbf{q}(3) &= \mathbf{\Phi}\mathbf{q}(2) + \mathbf{\Gamma}\mathbf{p}(2) = \mathbf{\Phi}^3\mathbf{q}(0) + \mathbf{\Phi}^2\mathbf{\Gamma}\mathbf{p}(0) + \mathbf{\Phi}\mathbf{\Gamma}\mathbf{p}(1) + \mathbf{\Gamma}\mathbf{p}(2)\end{aligned}$$

We can see that a pattern is emerging. We can write a general expression at time point t . The second term is a convolution sum.

$$\mathbf{q}(t) = \mathbf{\Phi}^t\mathbf{q}(0) + \sum_{i=1}^t \mathbf{\Phi}^{t-i}\mathbf{\Gamma}\mathbf{p}(i-1)$$

Exercises

- E11.1** For the following difference equations, a) represent the system as a neural network (using our notation) and b) find and plot (for $0 < t < 8$) the response of the system to the following input sequence.

$$p(t) = \delta(t-2) + \delta(t-3) + \delta(t-4)$$

- i. $a(t) = \frac{1}{3}p(t-1) + \frac{1}{3}p(t-2) + \frac{1}{3}p(t-3)$
- ii. $a(t) = p(t-1) - p(t-2)$
- iii. $a(t) = p(t-1) - 0.5p(t-2) - 0.25p(t-3) - 0.125p(t-4)$

- E11.2** For the following difference equations, a) represent the system as a neural network, using our notation, b) find the impulse response by iterating the difference equation by hand up to $t = 5$, c) plot the impulse response, d) find the system poles and check stability, and e) find a closed form representation of the impulse response.

- i. $a(t) = -5a(t-1) - 6a(t-2) + p(t-1)$
- ii. $a(t) = 1.62a(t-1) - a(t-2) + 0.6p(t-1)$
- iii. $a(t) = -a(t-1) - 0.21a(t-2) + p(t-1)$
- iv. $a(t) = 0.707a(t-1) - 0.25a(t-2) + 2p(t-1)$
- v. $a(t) = -2a(t-1) - 3a(t-2) - 2a(t-3) + p(t-1) + 4p(t-2)$

- E11.3** For the systems in Exercise E.11.2, find the responses (for $0 < t < 4$) to the input $p(t)$ in Exercise E.11.1 using the convolution sums with the impulse responses.

- E11.4** For the system in Exercise E.11.2 iii., find an FIR system that would have approximately the same response to any input sequence. Would this be possible for the system in Exercise E.11.2 i.?

- E11.5** If $p(t-1)$ and $p(t-2)$ are changed to $p(t)$ and $p(t-1)$ in Exercise E.11.2, how will the impulse response change? Explain.

Linear Sequence Processing

E11.6 If every $a(t - i)$ and $p(t - i)$ in Exercise E.11.2 was changed to $a(t + i)$ and $p(t + i)$, how would the resulting systems be implemented? How would the response to input sequences change? Explain.

E11.7 If a dynamic system has the impulse response below, find the corresponding difference equation representation of the system.

$$a(t) = (0.5)^t + (0.75)^t, \text{ for } t > 0$$

E11.8 If a system has complex poles $r \exp(\pm j\theta)$ (in polar form), then, based on Eq. 11.28, the following terms will appear in the impulse response:

$$A_1 [r \exp(j\theta)]^t + A_2 [r \exp(-j\theta)]^t = A_1 r^t \exp(j\theta t) + A_2 r^t \exp(-j\theta t)$$

- i. Since this expression must be real, show that A_2 must be a complex conjugate of A_1 .
- ii. Show that the expression above can then be rewritten as follows:

$$2 |A_1| r^t \cos(\theta t + \angle A_1)$$

- iii. Using this concept, find the impulse response for the network in Figure 11.5. (The impulse response is shown in Figure 11.2.)

E11.9 Consider again the difference equations in Exercise E.11.2.

- i. Find the simulation diagrams.
- ii. Find the state space representations.
- iii. Verify that the eigenvalues of Φ match the system poles you found in Exercise E.11.2, part d).

E11.10 Perform Lab 1, which can be found at <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/tree/master/NNDesignDeepLearning/11.LinearSequenceProcessingChapter/Code/Labs>

12 *Nonlinear Sequence Processing*

<i>Objective</i>	1
<i>Theory and Examples</i>	2
<i>Extending Linear Models to Nonlinear Models</i>	2
<i>Dynamic Backpropagation</i>	7
<i>Learning Long Term Dependencies</i>	12
<i>The Long Short Term Memory (LSTM) Network</i>	15
<i>Epilogue</i>	20
<i>Further Reading</i>	21
<i>Summary of Results</i>	22
<i>Solved Problems</i>	27
<i>Exercises</i>	32

Objective

In this chapter we extend the concepts from the Linear Sequence Processing chapter. The networks covered here are also dynamic. As in the previous chapter, they incorporate memory by using tapped delay lines. Some of the networks are related to linear FIR networks – they are strictly feedforward. Other networks have feedback and are related to linear IIR networks. Some of the networks in this chapter are of the input/output type and use tapped delay lines. Other networks are state space and use a single vector delay.

Theory and Examples

The architectures of nonlinear sequence processing networks are based on the linear sequence processing networks of the previous chapter. We will begin by showing how the FIR networks can be extended to nonlinear networks that have a finite memory. Then we will move on to recurrent networks, which can have either an input/output structure or a state space structure.

This chapter will also discuss how nonlinear dynamic networks are trained. This topic was covered in some depth in Chapter 14 of [NND2](#). In this chapter we concentrate on how the backpropagation-through-time algorithm can be implemented by *unfolding* the network, which is the technique used in TensorFlow and PyTorch.

Extending Linear Models to Nonlinear Models

FOCUSED TIME DELAY NEURAL NETWORK

The simplest nonlinear dynamic network can be obtained by extending the linear FIR network of Figure 11.9. For the linear case, the TDL leads into a single linear layer. For the nonlinear case, we can simply replace the single linear layer with a multilayer network. This is sometimes referred to as a *focused time delay neural network* (FTDNN). The term focused refers to the fact that the TDL is focused at the input to the network. A two-layer version is shown in Figure 12.1.

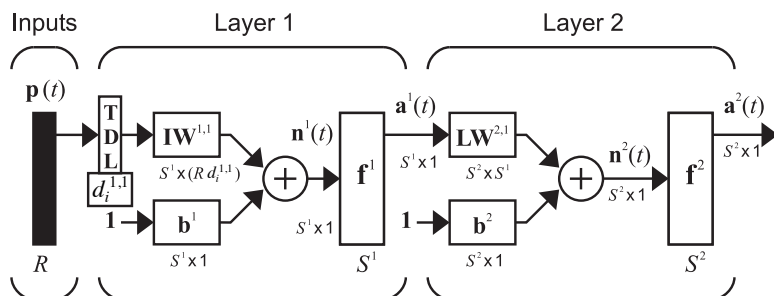


Figure 12.1: Focused Time Delay Neural Network

An advantage of this network is that its gradients can be computed with standard static backpropagation. The inputs are all available before training and can be stacked together to create the input to the weight $IW^{1,1}$. Also, because this network has no feedback, computations can be done in parallel.

DISTRIBUTED TIME DELAY NEURAL NETWORK

A variation on the FTDNN is the *distributed time delay neural network* (DTDNN). (See [Waibel et al., 2013].) This is also a feedforward network, but the TDLs are distributed throughout the network. There is a TDL between each layer, as shown in Figure 12.2 for a two-layer version of the DTDNN.

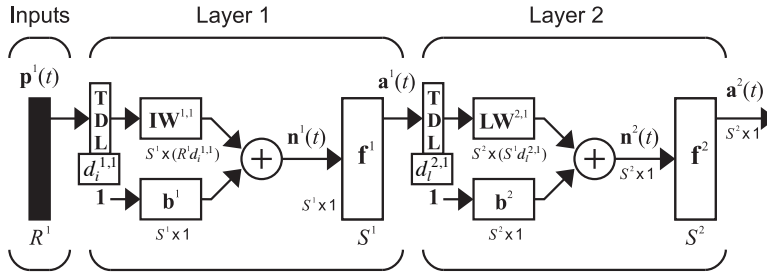


Figure 12.2: Distributed Time Delay Neural Network

This network was initially used for phoneme detection, but has also been used for other applications. It has sometimes been referred to as a 1D convolution network. As we saw in the previous chapter, the single layer FIR network of Figure 11.9 is effectively performing a convolution between the elements of the weight and the input, because the elements in the weight make up the system impulse response, as shown in Eq. 11.12.

The gradients for the DTDNN cannot be computed with the standard backpropagation algorithm. You can use the dynamic backpropagation algorithms described in Chapter 14 of NND2.

THE GAMMA MEMORY NETWORK

The FTDNN and the DTDNN have finite memory. For the FTDNN the length of memory is the maximum delay of the TDL. For the DTDNN, it is the sum of the maximum delays of the TDLs in each layer. To have infinite memory, we need feedback. One network that introduced feedback in a simple way was the *focused gamma memory network* (FGMN). (See [Hu and Hwang, 2018].) To build the FGNN, we need to introduce the gamma element, as shown in Figure 12.3. We can arrange the gamma elements into a tapped gamma memory line (TGML), similar to the TDL. This is shown in Figure 12.4. For the FGMN we can put the TGML at the input of a multilayer network, as shown in Figure 12.5.

Nonlinear Sequence Processing

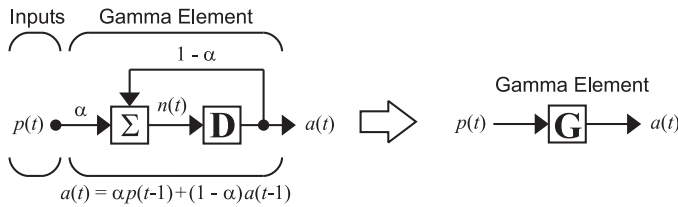


Figure 12.3: Gamma Element

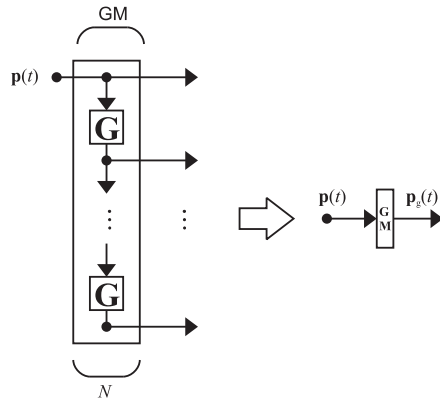


Figure 12.4: Tapped Gamma Memory Line

The FGMN uses the simplest type of recurrent connections, which allows the network to have infinite memory. The effective memory is adjusted during training by the parameter α in the gamma element:

$$a(t) = \alpha p(t-1) + (1-\alpha)a(t-1) \quad (12.1)$$

The gamma element is a low pass filter, or what is sometimes called a *leaky integrator*. Each single gamma element is an IIR system, with impulse response

$$h(t) = \alpha(1-\alpha)^{t-1}, t > 0 \quad (12.2)$$

When $\alpha = 1$ the gamma acts as a simple delay of one time step. As α decreases, the effective memory length increases. During training, the network adjusts its memory length to fit the problem. This network is typically used for adaptive filtering.

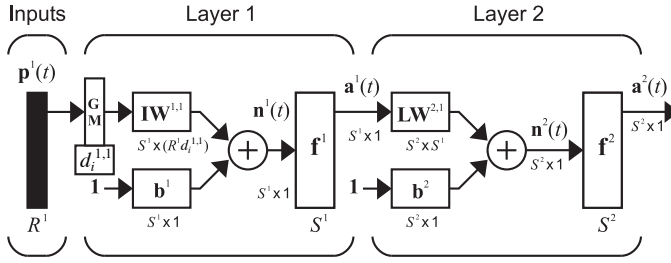


Figure 12.5: Focused Gamma Memory Network

NONLINEAR AUTOREGRESSIVE WITH EXOGENOUS INPUTS (NARX)

There is a nonlinear recurrent network that descends directly from the IIR input/output network of Figure 11.13. It is called the nonlinear autoregressive with exogenous inputs, or **NARX** network, which is shown in Figure 12.6.

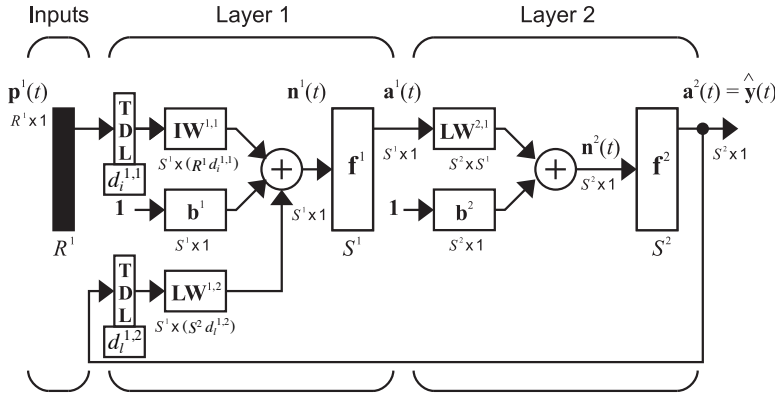


Figure 12.6: NARX Network

The term *autoregressive* comes from the fact that $a(t)$ is regressed on previous values of the same series, as shown in the difference equation in Eq. 12.3. The term *exogenous* refers to the external inputs $p(t)$ in the equation.

$$\begin{aligned} a(t) = & f(a(t-1), a(t-2), \dots, a(t-n_a), \\ & p(t-1), p(t-2), \dots, p(t-n_p)) \end{aligned} \quad (12.3)$$

Although the NARX network is recurrent (has feedback connections), it is possible to compute its gradient with the standard backpropagation algorithm by using a trick, which is illustrated in Figure 12.7. If we are training the network output $a(t)$ to match a target prediction $y(t)$, then we can think of the network output as

Nonlinear Sequence Processing

the estimate of $y(t)$, which we could label $\hat{y}(t)$. After the network has been trained, $\hat{y}(t)$ should be a close match to $y(t)$. This gives us the idea to replace $\hat{y}(t)$ with $y(t)$ in the feedback TDL. This effectively opens the feedback loop, and we can train it in the same way as we can train the FTDDN – as a static network. The NARX network with feedback has been called a *parallel* architecture, and the NARX network with the feedback opened has been called a *series-parallel* architecture, as illustrated in Figure 12.7.

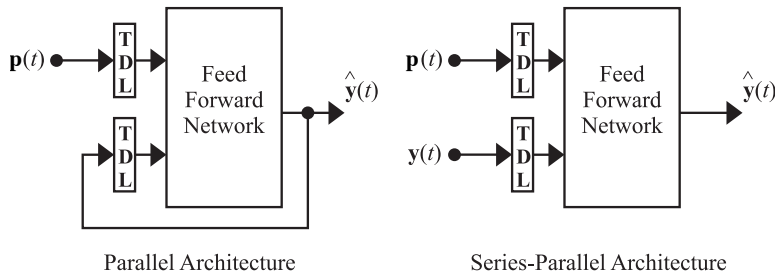


Figure 12.7: Parallel and Series-Parallel Architectures

The two main applications for NARX networks are system control/identification (modeling dynamic systems) [Jafari and Hagan, 2018] and multi-step prediction [Kelley and Hagan, 2024].

RECURRENT NEURAL NETWORK (RNN)

The term *recurrent* generally refers to a feedback operation. Recurrent connections can occur in a variety of ways. For example, the NARX network in Figure 12.6 has a recurrent connection. A more complex example of recurrent connections is shown in Figure 7.4. Historically, however, many have tended to give the name *Recurrent Neural Network* (RNN) to a network with one feedback connection with a single delay, as shown in Figure 12.8. (Sometimes the output layer is not included.) Equations 12.4 and 12.5 define the network operation.

$$\mathbf{a}^1(t) = \text{tansig}\left(\mathbf{IW}^{1,1}\mathbf{p}(t) + \mathbf{LW}^{1,1}\mathbf{a}^1(t-1) + \mathbf{b}^1\right) \quad (12.4)$$

$$\mathbf{a}^2(t) = \mathbf{LW}^{2,1}\mathbf{a}^1(t) + \mathbf{b}^2 \quad (12.5)$$

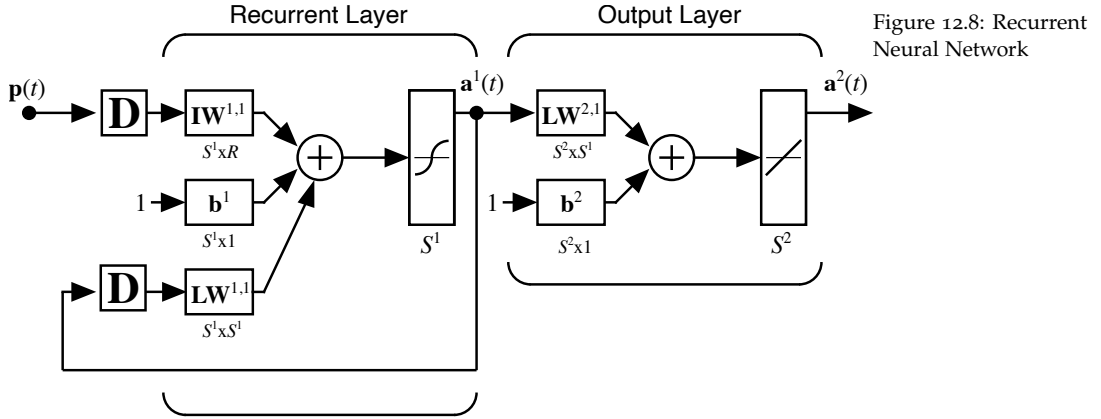


Figure 12.8: Recurrent Neural Network

If we compare this RNN with Figure 11.15, we can see that it is just a nonlinear version of the linear state space model. In fact, most of the networks we have seen in this section are nonlinear versions of the linear networks in Chapter 11. The FTDNN is a nonlinear version of the FIR network of Figure 11.9. The NARX network is a nonlinear version of the IIR network of Figure 11.13.

Dynamic Backpropagation

Chapter 14 of [NND2](#) covers in detail how gradients can be computed for dynamic networks. In this section we focus on the method used in TensorFlow and PyTorch, which is a particular implementation of the *backpropagation-through-time* algorithm. To illustrate the algorithm, we will use the basic RNN, without the output layer, as shown in Figure 12.9.

As discussed in Chapter 14 of [NND2](#), there are two basic approaches to computing the gradients for dynamic networks. One, which is shown in Eqs. 12.6 and 12.7, is called *real time recurrent learning* RTRL.

$$\frac{\partial F}{\partial \mathbf{x}} = \sum_{t=1}^Q \left[\frac{\partial \mathbf{a}(t)}{\partial \mathbf{x}^T} \right]^T \times \frac{\partial^e F}{\partial \mathbf{a}(t)} \quad (12.6)$$

$$\frac{\partial \mathbf{a}(t)}{\partial \mathbf{x}^T} = \frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{x}^T} + \frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{a}(t-1)^T} \frac{\partial \mathbf{a}(t-1)}{\partial \mathbf{x}^T} \quad (12.7)$$

This is RTRL for the network in Figure 12.9. The RTRL algorithm for arbitrary LDDNs is given on page 14-17 of [NND2](#).

Nonlinear Sequence Processing

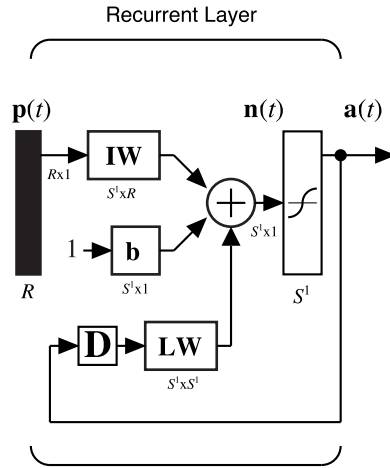


Figure 12.9: Basic Recurrent Neural Network

Both of these equations represent implementations of the chain rule. The term $\mathbf{x}$ represents all of the weights and biases in the network. The superscript e represents an explicit derivative, not accounting for indirect effects through time. The explicit derivative in Eq. 12.6 is the explicit derivative of the loss function with respect to the network output $\mathbf{a}(t)$. If $\mathbf{a}(t)$ does not appear explicitly in the loss function, then this derivative will be zero.

The other derivative in Eq. 12.6 is the complete derivative of the network output at time t with respect to the network weights and biases. This derivative must account for the indirect effect, for example, that $\mathbf{LW}$ would have on $\mathbf{a}(3)$ because $\mathbf{LW}$ directly affects $\mathbf{a}(2)$, which then affects $\mathbf{a}(3)$ through $\mathbf{LW}$ at the next time step. This derivative must be calculated recursively, as shown in Eq. 12.7. For RTRL this recursion moves forward in time.

The other general approach to gradient calculation for dynamic networks is called *backpropagation through time* (BPTT). The equations for this method are given in Eqs. 12.8 and 12.9.

$$\frac{\partial F}{\partial \mathbf{x}} = \sum_{t=1}^Q \left[\frac{\partial^e F}{\partial \mathbf{x}^T} \right]^T \times \frac{\partial F}{\partial \mathbf{a}(t)} \quad (12.8)$$

$$\frac{\partial F}{\partial \mathbf{a}(t)} = \frac{\partial^e F}{\partial \mathbf{a}(t)} + \frac{\partial^e F}{\partial \mathbf{a}(t+1)^T} \times \frac{\partial F}{\partial \mathbf{a}(t+1)} \quad (12.9)$$

This is BPTT for the network in Figure 12.9. The BPTT algorithm for arbitrary LDDNs is given on page 14-26 of [NND2](#).

Note that there are two main differences between BPTT and RTRL. First, in RTRL the recurrence equation moves forward in time, whereas it moves backward in time for BPTT. Second, in RTRL the complete derivative that is updated recursively is $\partial \mathbf{a}(t)/\partial \mathbf{x}^T$, whereas in BPTT $\partial F/\partial \mathbf{a}(t)$ is updated.

In Eq. 12.9, the first term represents the direct effect that $\mathbf{a}(t)$ has on $F(\mathbf{x})$. The second term represents the indirect effect. This is caused because $\mathbf{a}(t)$ contributes to $\mathbf{a}(t+1)$.

It is easiest to follow the steps of BPTT by working through a few steps of the algorithm. Let's set the number of time points to $Q = 2$, and let the loss function be the sum squared error at the last time point:

$$F(\mathbf{x}) = (\mathbf{t} - \mathbf{a}(2))^T (\mathbf{t} - \mathbf{a}(2)) = \mathbf{e}^T \mathbf{e} \quad (12.10)$$

We begin with Eq. 12.9, in which we recursively update $\partial F/\partial \mathbf{a}(t)$. This needs to be initialized at the last time point. Since the loss function doesn't contain any terms beyond $t = 2$, we can set derivatives beyond that point to zero.

$$\frac{\partial F}{\partial \mathbf{a}(t)} = \mathbf{0}, t > 2 \quad (12.11)$$

Now iterate Eq. 12.9 from $t = 3$ to $t = 2$ and then to $t = 1$.

$$\frac{\partial F}{\partial \mathbf{a}(2)} = \frac{\partial^e F}{\partial \mathbf{a}(2)} + \frac{\partial^e \mathbf{a}(3)}{\partial \mathbf{a}(2)^T} \times \frac{\partial F}{\partial \mathbf{a}(3)} = -2\mathbf{e} \quad (12.12)$$

$$\frac{\partial F}{\partial \mathbf{a}(1)} = \frac{\partial^e F}{\partial \mathbf{a}(1)} + \frac{\partial^e \mathbf{a}(2)}{\partial \mathbf{a}(1)^T} \times \frac{\partial F}{\partial \mathbf{a}(2)} = [\mathbf{LW}]^T \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \quad (12.13)$$

The first term in Eq. 12.12 is the explicit derivative of the loss with respect to $\mathbf{a}(2)$. Since $\mathbf{a}(2)$ appears explicitly in $F(\mathbf{x})$, this term is not zero. Also, since $\mathbf{a}(t)$ for $t > 2$ does not appear in the loss, $\mathbf{a}(2)$ cannot have any indirect effects on $F(\mathbf{x})$. Therefore the explicit derivative is the same as the total derivative.

For Eq. 12.13 the explicit derivative of $F(\mathbf{x})$ is zero, since $\mathbf{a}(1)$ does not appear explicitly in $F(\mathbf{x})$. For the explicit derivative of $\mathbf{a}(2)$ with respect to $\mathbf{a}(1)$, this just involves standard static backpropagation. (Compare with Eq. 3.49.)

$$\begin{array}{c} 2 \\ +2 \\ 4 \end{array}$$

We can't begin the BPTT algorithm before computing the network output for all time points.

Nonlinear Sequence Processing

Next we need to implement Eq. 12.8, which computes the derivative of the loss with respect to the weights. Let's consider the particular weight $lw_{i,j}$.

$$\frac{\partial F}{\partial lw_{i,j}} = \sum_{t=1}^2 \left[\frac{\partial^e \mathbf{a}(t)}{\partial lw_{i,j}} \right]^T \times \frac{\partial F}{\partial \mathbf{a}(t)} \quad (12.14)$$

Consider one term in this sum.

$$\frac{\partial^e \mathbf{a}^T(t)}{\partial lw_{i,j}} \frac{\partial F}{\partial \mathbf{a}(t)} = \frac{\partial^e \mathbf{n}^T(t)}{\partial lw_{i,j}} \frac{\partial^e \mathbf{a}^T(t)}{\partial \mathbf{n}(t)} \frac{\partial F}{\partial \mathbf{a}(t)} \quad (12.15)$$

The first term on the right hand side of Eq. 12.15 can be written

$$\frac{\partial^e \mathbf{n}(t)}{\partial lw_{i,j}} = \epsilon_i a_j(t-1) \quad (12.16)$$

since $lw_{i,j}$ connects $a_j(t-1)$ to $n_i(t)$.

The second term on the right hand side of Eq. 12.15 is the derivative of the activation function.

$$\frac{\partial^e \mathbf{a}^T(t)}{\partial \mathbf{n}(t)} = \dot{\mathbf{F}}^T(\mathbf{n}(t)) \quad (12.17)$$

Substituting these two terms back into Eq. 12.15 we have

$$\frac{\partial^e \mathbf{a}^T(t)}{\partial lw_{i,j}} \frac{\partial F}{\partial \mathbf{a}(t)} = a_j(t-1) \dot{f}(n_i(t)) \frac{\partial F}{\partial a_i(t)} \quad (12.18)$$

This is the gradient for one element of the weight. We can represent the gradient for the full weight matrix and substitute into Eq. 12.14 as follows.

$$\frac{\partial F}{\partial \mathbf{LW}} = \sum_{t=1}^2 \dot{\mathbf{F}}^T(\mathbf{n}(t)) \frac{\partial F}{\partial \mathbf{a}(t)} \mathbf{a}^T(t-1) \quad (12.19)$$

$$= \dot{\mathbf{F}}^T(\mathbf{n}(1)) \frac{\partial F}{\partial \mathbf{a}(1)} \mathbf{a}^T(0) + \dot{\mathbf{F}}^T(\mathbf{n}(2)) \frac{\partial F}{\partial \mathbf{a}(2)} \mathbf{a}^T(1) \quad (12.20)$$

Now we can substitute the previous terms that we calculated in Eqs. 12.12 and 12.13.

$$\begin{aligned} \frac{\partial F}{\partial \mathbf{LW}} &= \dot{\mathbf{F}}^T(\mathbf{n}(1)) \mathbf{LW}^T \dot{\mathbf{F}}^T(\mathbf{n}(2)) [-2\mathbf{e}] \mathbf{a}^T(0) \\ &\quad + \dot{\mathbf{F}}^T(\mathbf{n}(2)) [-2\mathbf{e}] \mathbf{a}^T(1) \end{aligned} \quad (12.21)$$

For certain types of architectures, it is possible to implement the BPTT algorithm in a different way, using standard static backpropagation. This can be done by *unrolling* the network in time. Consider again Figure 12.9. Notice that the output of the network at time step t becomes the input to the network at time step $t + 1$. Figure 12.10 illustrates this idea.

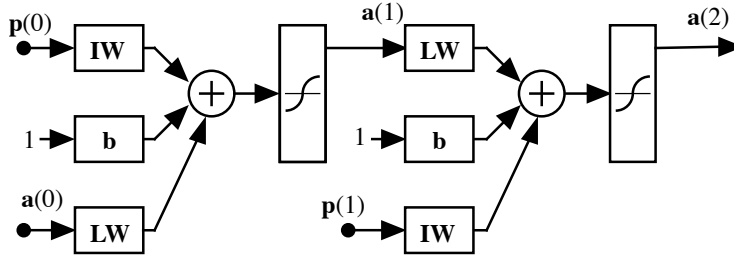


Figure 12.10: Unrolled Network

This is a static network, but the final output for this network will be the same as for the original recurrent network. We can therefore use static backpropagation to compute the gradient. It is true that the same weights and biases appear multiple times in the unrolled network, but we can just add together the gradients from each appearance.

First, recall the static backpropagation equations from Eqs. 3.39, 3.49 and 3.53.

$$\mathbf{s}^M = \dot{\mathbf{F}}(\mathbf{n}^M)^T [-2\mathbf{e}] \quad (12.22)$$

$$\mathbf{s}^m = \dot{\mathbf{F}}(\mathbf{n}^m)^T (\mathbf{W}^{m+1})^T \mathbf{s}^{m+1} \quad (12.23)$$

$$\frac{\partial \hat{F}(\mathbf{x})}{\partial w_{i,j}^m} = s_i^m a_j^{m-1} \quad (12.24)$$

$$\frac{\partial F(\mathbf{x})}{\partial \mathbf{W}^m} = \mathbf{s}^m (\mathbf{a}^{m-1})^T \quad (12.25)$$

Begin by initializing the sensitivity at the last time step.

$$\mathbf{s}(2) = \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \quad (12.26)$$

Now backpropagate one step.

$$\mathbf{s}(1) = \dot{\mathbf{F}}(\mathbf{n}(1))^T \mathbf{LW}^T \mathbf{s}(2) \quad (12.27)$$

$$= \dot{\mathbf{F}}(\mathbf{n}(1))^T \mathbf{LW}^T \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \quad (12.28)$$

By unrolling, we can see that any recurrent network, no matter how many layers, can always be a *deep* network.

Nonlinear Sequence Processing

Now we compute the gradient of the weight for each time step, according to Eq. 12.25.

$$\left. \frac{\partial F(\mathbf{x})}{\partial \mathbf{LW}} \right|_{t=2} = \mathbf{s}(2)\mathbf{a}(1)^T = \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \mathbf{a}(1)^T \quad (12.29)$$

$$\begin{aligned} \left. \frac{\partial F(\mathbf{x})}{\partial \mathbf{LW}} \right|_{t=1} &= \mathbf{s}(1)\mathbf{a}(0)^T \\ &= \dot{\mathbf{F}}(\mathbf{n}(1))^T \mathbf{LW}^T \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \mathbf{a}(0)^T \end{aligned} \quad (12.30)$$

Since the weights at the two time steps are actually the same, we add the gradients from each to get the total.

$$\begin{aligned} \left. \frac{\partial F(\mathbf{x})}{\partial \mathbf{LW}} \right|_{total} &= \dot{\mathbf{F}}(\mathbf{n}(1))^T \mathbf{LW}^T \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \mathbf{a}(0)^T \\ &\quad + \dot{\mathbf{F}}(\mathbf{n}(2))^T [-2\mathbf{e}] \mathbf{a}(1)^T \end{aligned} \quad (12.31)$$

Comparing Eq. 12.31 with Eq. 12.21 we can see that we get the same answer by unrolling the network as we did from the earlier BPTT algorithm. This unrolling technique is currently used in TensorFlow and PyTorch, and it is especially suited to recurrent networks that have a state space form. It is not as well suited to input/output type networks which include TDLs rather than single delays. This is addressed in the Solved Problems and Exercises.

Learning Long Term Dependencies

There are some sequence processing problems where we would like to predict responses that are significantly delayed from the corresponding stimulus. For example, a chess game can be lost 12 moves after the critical mistake. When performing a translation, the words in a previous paragraph might provide important context.

Our network structure must enable the possibility of connecting items that are far removed in time. This means that the network must have long term memory. We have seen in the previous chapter that linear recurrent networks will have long term memory only if the system poles are close to the unit circle. This means that long memory systems can flirt with instability.

As an example, consider the linear recurrent network in Figure 12.11, and let $lw_{1,1} = 0.5$ and $iw_{1,1} = 1$. The impulse response for this network is

$$\begin{array}{r} 2 \\ +2 \\ \hline 4 \end{array}$$

$$h(t) = 0.5^{(t-1)}, t > 0 \quad (12.32)$$

as shown in Figure 12.12.

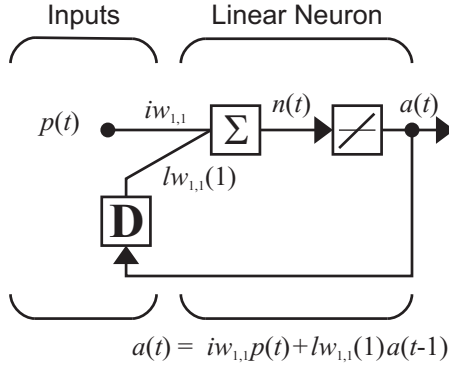


Figure 12.11: Linear Recurrent Network

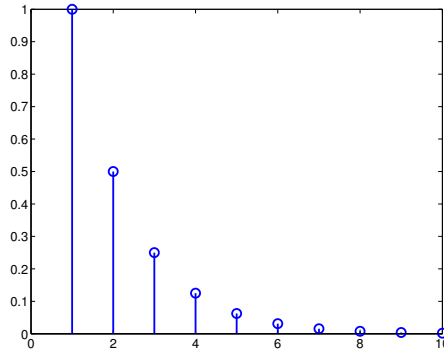


Figure 12.12: Impulse Reponse

As we saw in the last chapter, this system has a pole at 0.5. This produced the $0.5^{(t-1)}$ impulse response. When the poles are all inside the unit circle, the network is stable, and the impulse response will decay with time. In addition, the gradients for this network will also decay with time. Using Eq. 12.9, the update in the BPTT algorithm is

$$\frac{\partial F}{\partial a(t)} = \frac{\partial^e F}{\partial a(t)} + lw_{1,1} \frac{\partial F}{\partial a(t+1)} \quad (12.33)$$

If the error only occurs at the 10th time point, we have

$$\frac{\partial F}{\partial a(t)} = 0.5^{(10-t)} \frac{\partial^e F}{\partial a(10)} \quad (12.34)$$

Nonlinear Sequence Processing

Therefore, if the network is stable, the derivatives will decay as you go back in time. Because the derivatives are small, it will take a long time to learn long term dependencies.

This effect is enhanced when using a saturating nonlinear activation function. Consider the network in Figure 12.13. The BPTT update would now be

$$\frac{\partial F}{\partial a(t)} = \frac{\partial^e F}{\partial a(t)} + \dot{f}(n(t)) l w_{1,1} \frac{\partial F}{\partial a(t+1)}, |\dot{f}(n(t))| \leq 1 \quad (12.35)$$

where the derivative of the activation function will be less than one and will cause the gradient to decay even more quickly.

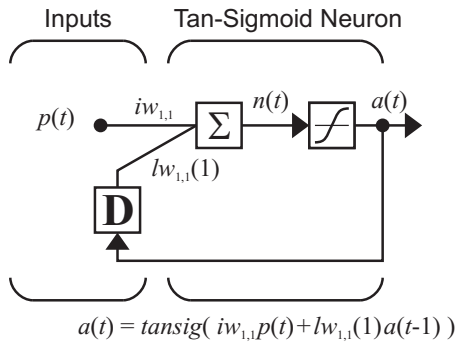


Figure 12.13: Nonlinear Recurrent Network

It is possible to have a recurrent network that can remember forever. For example, consider the parity network in Figure 12.14, with the following weights

To experiment with parity network, use the Deep Learning Demonstration Parity Network (**dl12par**).



$$\begin{aligned} \mathbf{IW}^{1,1} &= \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{LW}^{1,1} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \\ \mathbf{LW}^{2,1} &= \begin{bmatrix} 1 & -1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} -0.5 \\ -1.5 \end{bmatrix}, \mathbf{b}^2 = [-0.5] \end{aligned} \quad (12.36)$$

This network computes the parity of a sequence of bits. It can keep track of the parity, no matter how many bits have been presented.

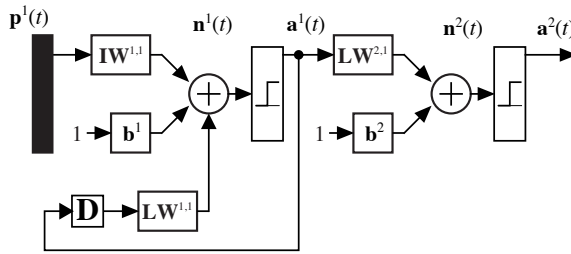


Figure 12.14: Parity Network

The Long Short Term Memory (LSTM) Network

Long term memories are network weights, and short term memories are layer outputs. We need a network that has long short term memories. In recurrent networks, as the weights change during training, the length of the short term memory will change.

If the initial weights produce a network without long short term memory, it will be difficult to increase it. This is because the gradient will become small, as shown in the previous section. On the other hand, if the initial weights produce long short term memory (poles close to the unit circle in the linear case), then instabilities can occur.

The *Long Short Term Memory* (LSTM) was designed to achieve long short term memory while maintaining stability. LSTM is a recurrent network with some special components, which we describe next. [Hochreiter and Schmidhuber, 1997]

CONSTANT ERROR CAROUSEL (CEC)

Consider the recurrent network in Figure 12.15. To maintain a long memory, we would like the feedback matrix $LW^{1,1}$ to have some eigenvalues very close to one, but if they become larger than one, the system will become unstable. These eigenvalues need to stay close to one during training, or the gradients will vanish. Also, we want the derivative of the activation function to remain constant. A solution would be to set $LW^{1,1} = I$ and to use a linear transfer function. The result is called a *constant error carousel* (CEC).

Nonlinear Sequence Processing

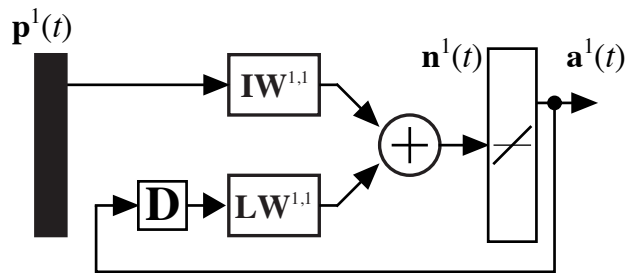
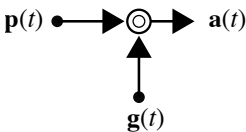


Figure 12.15: Circular Error Carousel

GATING

The problem with using the CEC is that we don't want to indiscriminately remember everything. To remember selectively we introduce several *gates* (switches). In the gate, an input signal $\mathbf{p}(t)$ is multiplied (Hadamard product) by a gating signal $\mathbf{g}(t)$ that has values between zero and one. The gate turns the input signal on or off.



$$\mathbf{a}(t) = \mathbf{g}(t) \circ \mathbf{p}(t) \tag{12.37}$$

Figure 12.16 shows a CEC with gating signals in the input and feedback paths. The gates are turned off when we want to forget and are turned on when we want to remember.

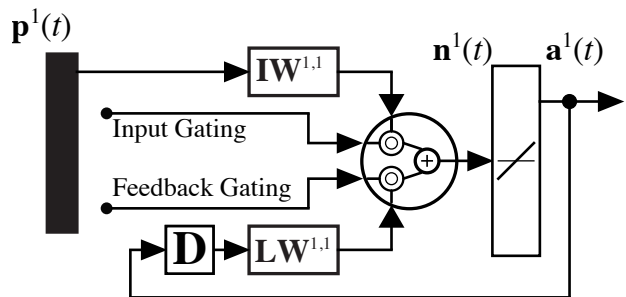


Figure 12.16: Gated CEC

To experiment with gating operations, use the Deep Learning Demonstration CEC Gating (**dl12gat**).



FULL LSTM

In all, we will have three gates: 1) the input gate allows selective inputs into the CEC, 2) the feedback (or forget) gate will clear the CEC, and 3) an output gate will allow selective outputs from the CEC. Each gate will be a layer with inputs from the gated outputs and the network input. The result is called Long Short Term Memory (LSTM) and is shown in Figure 12.17.

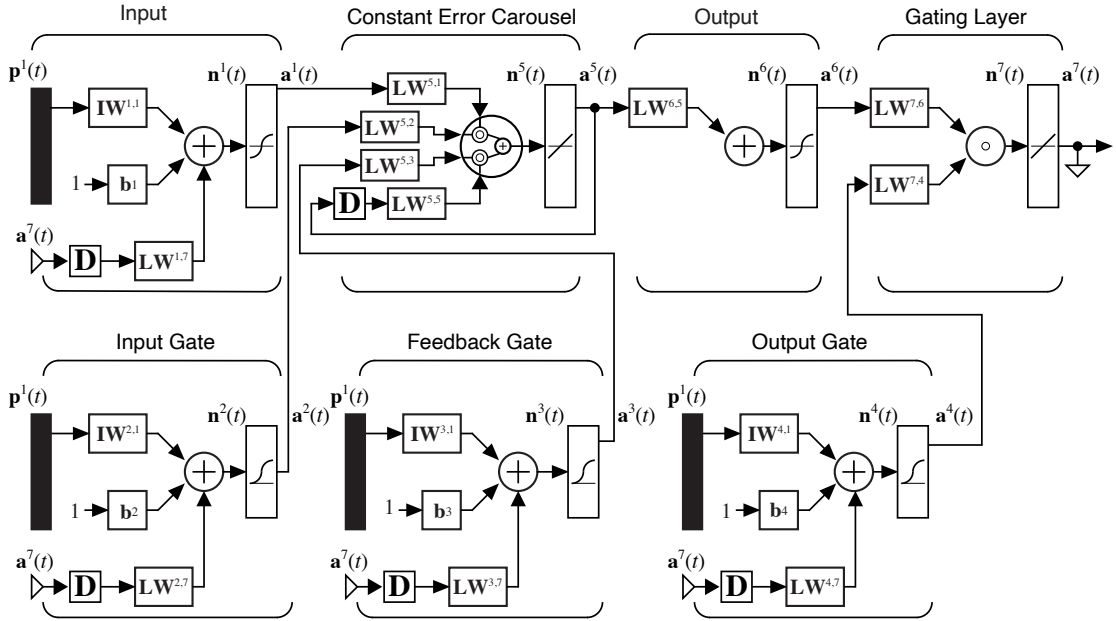


Figure 12.17: Complete LSTM

Although the network looks complex, if we step back, we can see the structure of a basic RNN, as shown in Figure 12.9. The input $p^1(t)$ is multiplied by the weight $IW^{1,1}$, and the output $a^7(t)$ (compare with $a^1(t)$ in Figure 12.9) is multiplied by the weight $LW^{1,7}$ (compare with $LW^{1,1}$ in Figure 12.9). The CEC, with its gating mechanisms is inserted between the first layer and the output of the basic RNN. This enables the network to have long short term memory.

Nonlinear Sequence Processing

The weights in the CEC are all fixed to the identity matrix, and they are not trained. The output and gating layer weights are also fixed to the identity matrix. It has been shown that the best results are obtained by initializing the feedback (forget) gate bias, $\mathbf{b}^3$, to all ones or larger values. This turns the gate on for the beginning of training. Other weights and biases are set to small random values.

The output of the gating layer generally connects to another layer or a multilayer network with a softmax activation function. Also, multiple LSTMs can be cascaded together.

The LSTM is trained with standard gradient-based algorithms, as with other deep networks. The original LSTM paper used RTRL for computing the gradient, although the current frameworks use BPTT. An approximate gradient is sometimes used, in which the derivatives are only propagated in time through the CEC delays. Only static derivatives are computed for the remaining terms.

VARIATIONS ON THE LSTM

There have been a number of variations on the LSTM. The original LSTM did not have the feedback gate, but it is commonly used now. Some variations feedback the CEC output $\mathbf{a}^5(t)$ to the gate layers. These are called *peephole* connections. [Jozefowicz et al., 2015] provides an experimental comparison of many of the variations.

A popular variation is the *Gated Recurrent Unit* (GRU). Figure 12.18 illustrates this network. The differencing layer subtracts the delayed network output $\mathbf{a}^6(t)$ from the processed input $\mathbf{a}^1(t)$ – the weight $\mathbf{LW}^{5,1}$ is set to $\mathbf{I}$, and the weight $\mathbf{LW}^{5,6}$ is set to $-\mathbf{I}$. They are not trained. The gating layer weights are also set to the identity matrix.

The delayed network output is gated by the reset gate, before it is fed into the input layer. This resets the memory. The overall output can be considered a weighted average of a candidate potential output $\mathbf{a}^1(t)$ and the previous output $\mathbf{a}^6(t-1)$:

$$\begin{aligned}\mathbf{a}^6(t) &= \mathbf{a}^4(t) \times [\mathbf{a}^1(t) - \mathbf{a}^6(t-1)] + \mathbf{a}^6(t-1) \\ &= \mathbf{a}^4(t) \times \mathbf{a}^1(t) + [1 - \mathbf{a}^4(t)] \times \mathbf{a}^6(t-1)\end{aligned}$$

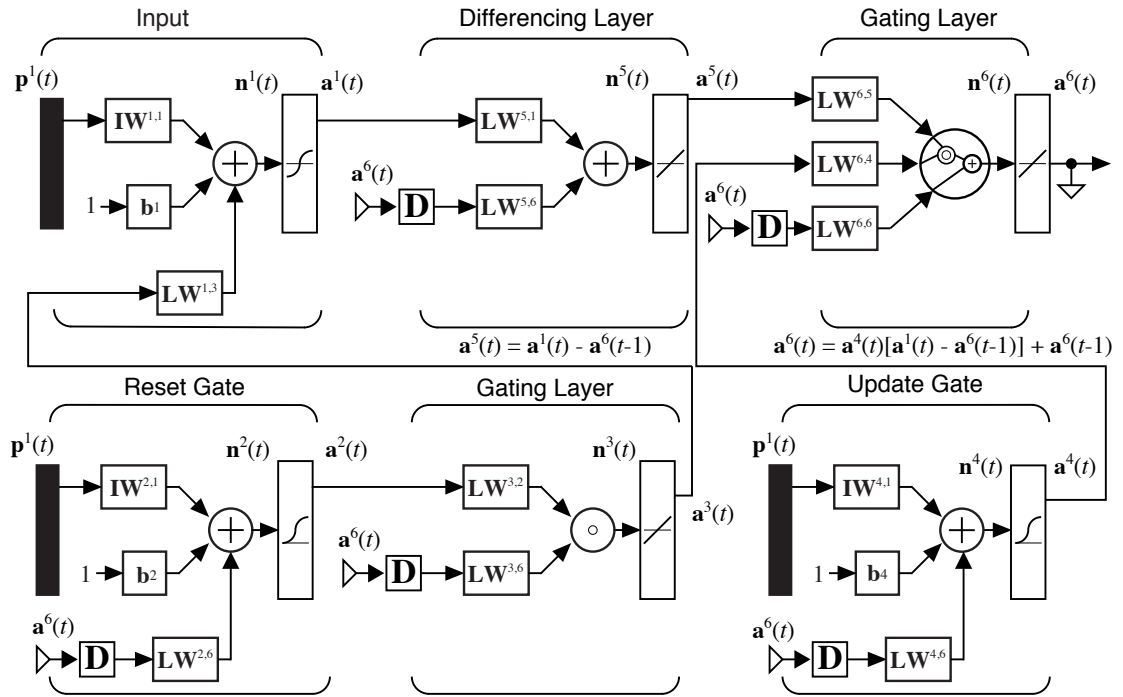


Figure 12.18: GRU Network

Epilogue

This chapter has shown how the linear sequence processing networks of the previous chapter can be extended for nonlinear processing. There are networks that directly descend from linear FIR systems, like the Focused Time Delay neural network. Other networks are related to the linear IIR input-output systems, like the NARX network. Finally, the linear state space models have lead to the RNN, LSTM and GRU.

The next chapter also considers nonlinear sequence processing. It begins with the sequence-to-sequence model, which leads to the concept of attention and then the transformer network, on which most current large language models are based.

Further Reading

[Hochreiter and Schmidhuber, 1997] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997

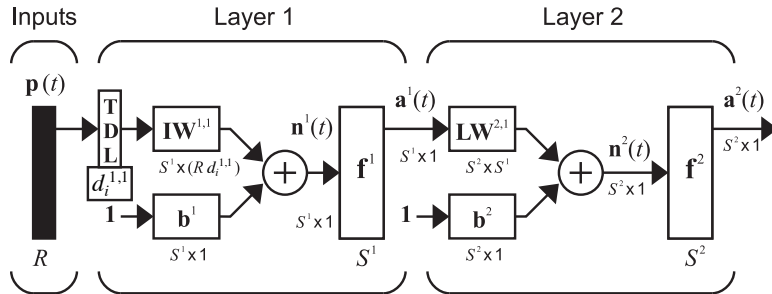
This is the paper that introduced the LSTM network. Although it was written in the 1990s, it found major application after 2015.

[Jozefowicz et al., 2015] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *International conference on machine learning*, pages 2342–2350. PMLR, 2015

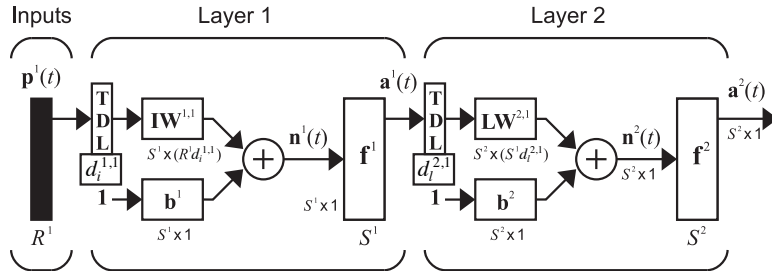
This paper tests a variety of LSTM-like networks across a number of experiments.

Summary of Results

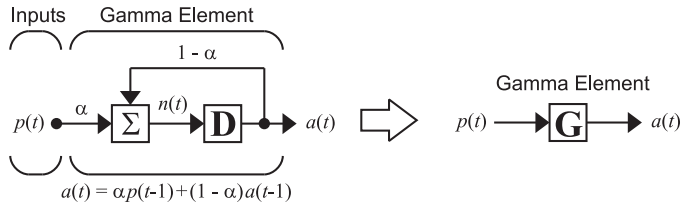
Focused Time Delay Neural Network



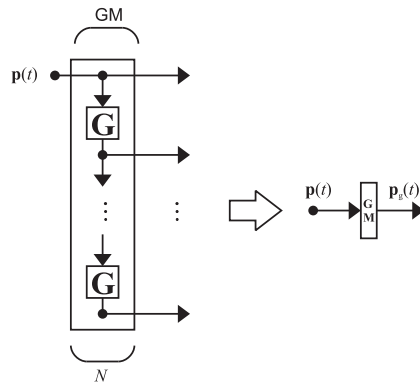
Distributed Time Delay Neural Network



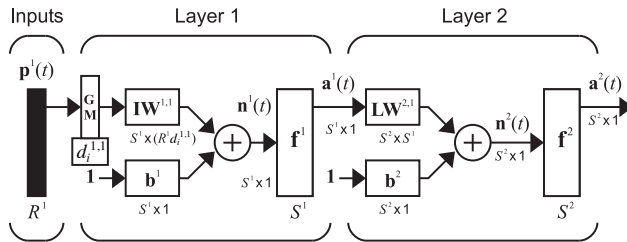
Gamma Element



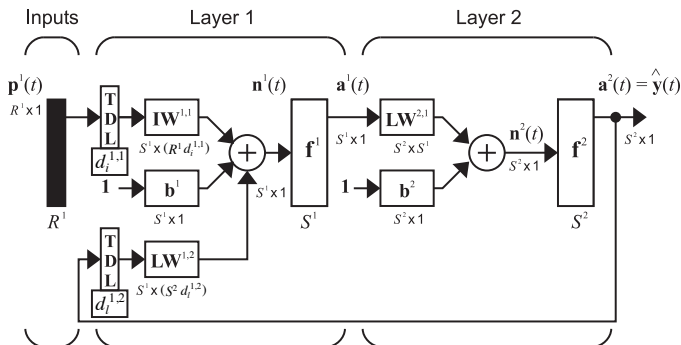
Gamma Memory Line



Focused Gamma Network

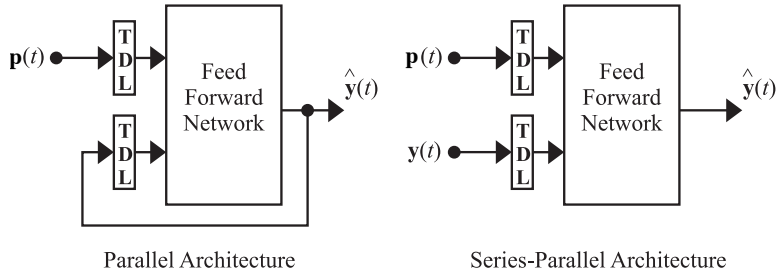


NARX Network

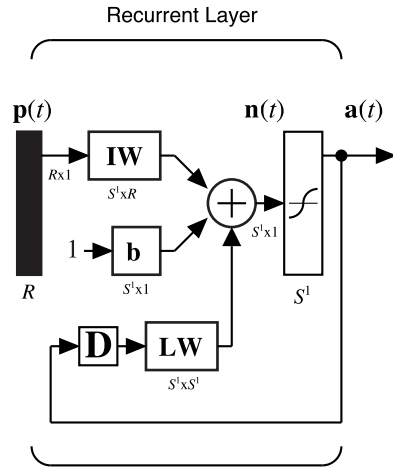


Nonlinear Sequence Processing

Parallel and Series-Parallel Architectures



Recurrent Neural Network



Real Time Recurrent Learning (RTRL)

$$\frac{\partial F}{\partial \mathbf{x}} = \sum_{t=1}^Q \left[\frac{\partial \mathbf{a}(t)}{\partial \mathbf{x}^T} \right]^T \times \frac{\partial^e F}{\partial \mathbf{a}(t)}$$

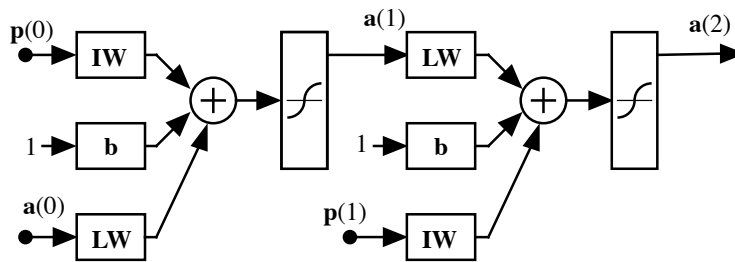
$$\frac{\partial \mathbf{a}(t)}{\partial \mathbf{x}^T} = \frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{x}^T} + \frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{a}(t-1)^T} \frac{\partial \mathbf{a}(t-1)}{\partial \mathbf{x}^T}$$

Backpropagation Through Time (BPTT)

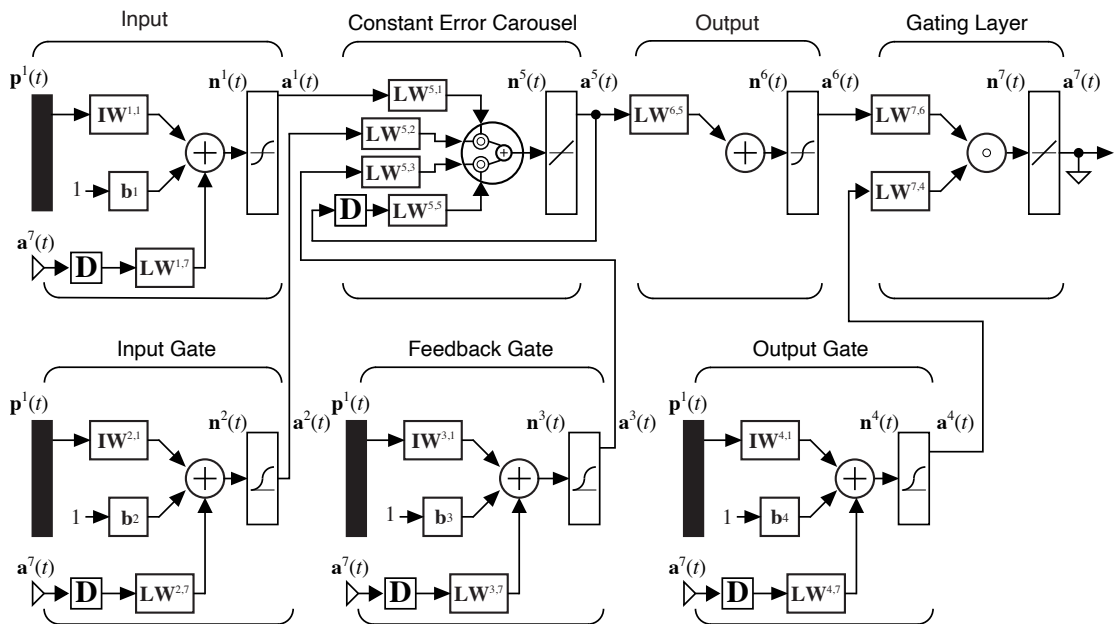
$$\frac{\partial F}{\partial \mathbf{x}} = \sum_{t=1}^Q \left[\frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{x}^T} \right]^T \times \frac{\partial F}{\partial \mathbf{a}(t)}$$

$$\frac{\partial F}{\partial \mathbf{a}(t)} = \frac{\partial^e F}{\partial \mathbf{a}(t)} + \frac{\partial^e \mathbf{a}(t+1)}{\partial \mathbf{a}(t)^T} \times \frac{\partial F}{\partial \mathbf{a}(t+1)}$$

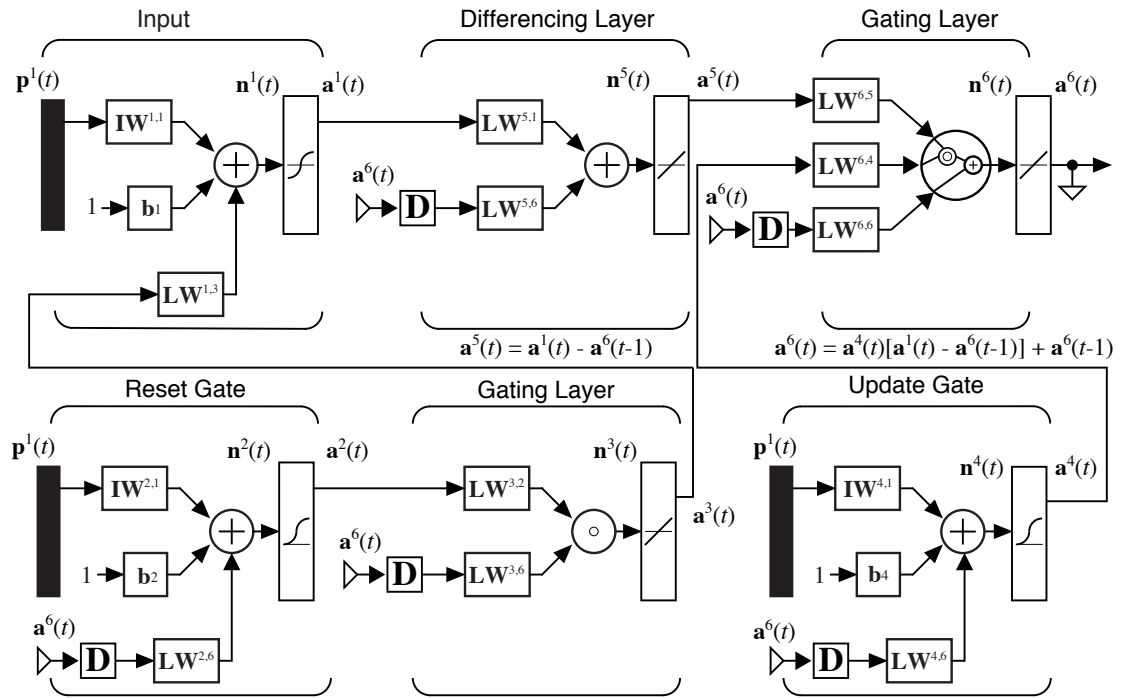
Unrolled RNN



Long Short Term Memory (LSTM) Network



Gated Recurrent Unit (GRU) Network



Solved Problems

P12.1 Consider the FTDNN network of Figure 12.1. This has the form of an LDDN, as discussed in Chapter 7. Write out the equations of operation of this network, using the form of Eqs. 7.6 and 7.7.

The general LDDN equations of operation are

$$\begin{aligned}\mathbf{n}^m(t) &= \sum_{l \in L_m^f} \sum_{d \in DL_{m,l}} \mathbf{LW}^{m,l}(d) \mathbf{a}^l(t-d) \\ &\quad + \sum_{l \in I_m} \sum_{d \in DI_{m,l}} \mathbf{IW}^{m,l}(d) \mathbf{p}^l(t-d) + \mathbf{b}^m \\ \mathbf{a}^m(t) &= \mathbf{f}^m(\mathbf{n}^m(t))\end{aligned}$$

The FTDNN network of Figure 12.1 has two layers. The first layer has a TDL input from $\mathbf{p}^1(t)$. The second layer has an input from $\mathbf{a}^1(t)$ with no delays. The first step is to identify the sets I_1 , L_2^f and $DI_{1,1}$. The TDL in the first layer has length $d_i^{1,1}$. Assuming that the delays go from 1 to $d_i^{1,1}$, we can write:

$$I_1 = \{1\}, L_2^f = \{1\}, DI_{1,1} = \{1, 2, \dots, d_i^{1,1}\}$$

We can then write the equations

$$\begin{aligned}\mathbf{n}^1(t) &= \sum_{d=1}^{d_i^{1,1}} \mathbf{IW}^{1,1}(d) \mathbf{p}^1(t-d) + \mathbf{b}^1 \\ \mathbf{a}^1(t) &= \mathbf{f}^1(\mathbf{n}^1(t)) \\ \mathbf{n}^2(t) &= \mathbf{LW}^{2,1} \mathbf{a}^1(t) + \mathbf{b}^2 \\ \mathbf{a}^2(t) &= \mathbf{f}^2(\mathbf{n}^2(t))\end{aligned}$$

The net input in the first layer could also be written using augmented matrices as follows:

$$\mathbf{n}^1(t) = \begin{bmatrix} \mathbf{IW}^{1,1}(1) & \mathbf{IW}^{1,1}(2) & \dots & \mathbf{IW}^{1,1}(d_i^{1,1}) \end{bmatrix} \begin{bmatrix} \mathbf{p}^1(t-1) \\ \mathbf{p}^1(t-2) \\ \vdots \\ \mathbf{p}^1(t-d_i^{1,1}) \end{bmatrix} + \mathbf{b}^1$$

Nonlinear Sequence Processing

P12.2 We have a NARX network with two delays in the input TDL and two delays in the feedback TDL. The weights, biases and input sequence are provided below. The first layer activation is ReLU, and the second layer activation is linear. Perform two iterations (time steps) of the network.

$$\begin{aligned} \mathbf{IW}^{1,1}(1) &= \begin{bmatrix} 1 & -1 \\ -2 & 2 \end{bmatrix}, \mathbf{IW}^{1,1}(2) = \begin{bmatrix} 3 & 1 \\ 1 & -3 \end{bmatrix} \\ \mathbf{LW}^{1,2}(1) &= \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix}, \mathbf{LW}^{1,2}(2) = \begin{bmatrix} 1 & 3 \\ -3 & 1 \end{bmatrix}, \mathbf{b}^1 = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \\ \mathbf{LW}^{2,1}(0) &= \begin{bmatrix} 2 & -1 \\ 1 & -2 \end{bmatrix}, \mathbf{b}^2 = \begin{bmatrix} -2 \\ 1 \end{bmatrix} \\ \mathbf{p}(t) &= \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix} \right\} \end{aligned}$$

For dynamic networks, the first step is to fill the TDLs. If the network was being trained, and if we had target outputs, we could fill the feedback TDL with target values – we could replace previous values of $\mathbf{a}^2$ with previous targets. Since we don't have targets, we need to initialize the feedback TDL with zeros. We could also fill the input TDL with zeros, if we assume that the inputs were zero before the provided inputs. To demonstrate how the filling would work, we will fill the input TDL with the first two inputs.

If we use the LDDN formulation of Eq. 7.6, we can compute the net input for the first layer for the first time step.

$$\begin{aligned} \mathbf{n}^1(2) &= \mathbf{LW}^{1,2}(1)\mathbf{a}^2(1) + \mathbf{LW}^{1,2}(2)\mathbf{a}^2(0) \\ &\quad + \mathbf{IW}^{1,1}(1)\mathbf{p}^1(1) + \mathbf{IW}^{1,1}(2)\mathbf{p}^1(0) + \mathbf{b}^1 \\ \mathbf{n}^1(2) &= \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ &\quad + \begin{bmatrix} 1 & -1 \\ -2 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 3 & 1 \\ 1 & -3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 5 \\ -4 \end{bmatrix} \end{aligned}$$

Then, using Eq. 7.7, we can compute the first layer output.

$$\begin{aligned}\mathbf{a}^1(2) &= \text{relu}(\mathbf{n}^1(2)) \\ \mathbf{a}^1(2) &= \text{relu}\left(\begin{bmatrix} 5 \\ -4 \end{bmatrix}\right) = \begin{bmatrix} 5 \\ 0 \end{bmatrix}\end{aligned}$$

For the second layer there is no TDL, so the computation is simpler.

$$\begin{aligned}\mathbf{n}^2(2) &= \mathbf{LW}^{2,1}(0)\mathbf{a}^1(2) + \mathbf{b}^2 \\ \mathbf{n}^2(2) &= \begin{bmatrix} 2 & -1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} 5 \\ 0 \end{bmatrix} + \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}\end{aligned}$$

Since the activation for the last layer is linear, the output is the same as the net input.

$$\mathbf{a}^2(2) = \mathbf{n}^2(2) = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$

For the next time step, we repeat the process, beginning with the net input for the first layer. The input and feedback TDLs are shifted from the previous time step.

$$\begin{aligned}\mathbf{n}^1(3) &= \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 8 \\ 6 \end{bmatrix} + \begin{bmatrix} 1 & 3 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ &+ \begin{bmatrix} 1 & -1 \\ -2 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 1 \\ 1 & -3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 9 \\ 21 \end{bmatrix} \\ \mathbf{a}^1(3) &= \text{relu}\left(\begin{bmatrix} 9 \\ 21 \end{bmatrix}\right) = \begin{bmatrix} 9 \\ 21 \end{bmatrix}\end{aligned}$$

For the second layer at the next time step, we have

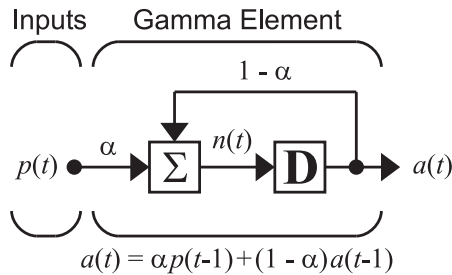
$$\begin{aligned}\mathbf{n}^2(3) &= \begin{bmatrix} 2 & -1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} 9 \\ 21 \end{bmatrix} + \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} -5 \\ 32 \end{bmatrix} \\ \mathbf{a}^2(3) &= \mathbf{n}^2(3) = \begin{bmatrix} -5 \\ 32 \end{bmatrix}\end{aligned}$$

Nonlinear Sequence Processing

Rather than using Eqs. 7.6, we can also use augmented matrices, as shown in the previous problem. For example, for the net input calculation for the first layer in the first time step, we could have written the equation as follows (ignoring the feedback term, which is zero).

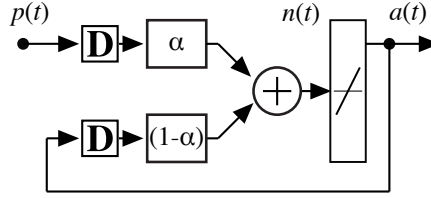
$$\begin{aligned} \mathbf{n}^1(2) &= \begin{bmatrix} \mathbf{IW}^{1,1}(1) & \mathbf{IW}^{1,1}(2) \end{bmatrix} \begin{bmatrix} \mathbf{p}^1(1) \\ \mathbf{p}^1(0) \end{bmatrix} + \mathbf{b}^1 \\ &= \begin{bmatrix} 1 & -1 & 3 & 1 \\ -2 & 2 & 1 & -3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 5 \\ -4 \end{bmatrix} \end{aligned}$$

P12.3 Consider a single gamma element, as shown below.



- i. Assume that this is the complete network, and convert this diagram into the standard LDDN form.
- ii. Write out the BPTT equations to compute the gradient of the loss with respect to the gamma parameter α . Assume that the loss is sum squared error.

i. For the LDDN, each layer could have input and layer TDLs, a summing junction and an activation function. The gamma element can be adjusted to this form if we move the delay to the other side of the summing junction. The input weight would be α , and the layer weight would be $(1 - \alpha)$. The activation function would be linear. The converted network is shown below.



ii. Let's repeat the BPTT equations (Eqs. 12.8 and 12.9) here:

$$\frac{\partial F}{\partial \mathbf{x}} = \sum_{t=1}^Q \left[\frac{\partial^e \mathbf{a}(t)}{\partial \mathbf{x}^T} \right]^T \times \frac{\partial F}{\partial \mathbf{a}(t)}$$

$$\frac{\partial F}{\partial \mathbf{a}(t)} = \frac{\partial^e F}{\partial \mathbf{a}(t)} + \frac{\partial^e \mathbf{a}(t+1)}{\partial \mathbf{a}(t)^T} \times \frac{\partial F}{\partial \mathbf{a}(t+1)}$$

(Note that these equation represent BPTT only for networks with a single feedback delay, which fits the gamma element.)

Let's begin by finding the explicit derivatives first. The network operation equation is

$$a(t) = \alpha p(t-1) + (1-\alpha)a(t-1)$$

From this and the loss function, we can write the three explicit derivatives.

$$\frac{\partial^e a(t+1)}{\partial a(t)} = \frac{\partial^e a(t)}{\partial a(t-1)} = (1-\alpha)$$

$$\frac{\partial^e a(t)}{\partial x} = \frac{\partial^e a(t)}{\partial \alpha} = p(t-1) - a(t-1)$$

$$\frac{\partial^e F}{\partial a(t)} = -2e(t)$$

Now we can write out the BPTT equations.

$$\frac{\partial F}{\partial a(t)} = -2e(t) + (1-\alpha) \frac{\partial F}{\partial a(t+1)}, \text{ initialized with } \frac{\partial F}{\partial a(Q+1)} = 0$$

$$\frac{\partial F}{\partial \alpha} = \sum_{t=1}^Q [p(t-1) - a(t-1)] \times \frac{\partial F}{\partial a(t)}$$

Nonlinear Sequence Processing

Exercises

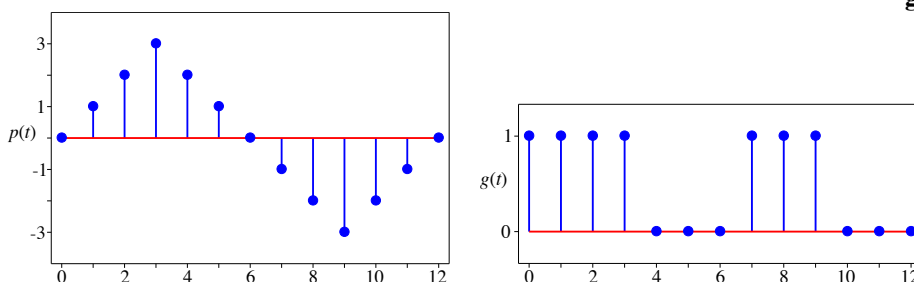
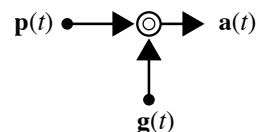
- E12.1** Consider the DTDNN network of Figure 12.2. This has the form of an LDDN, as discussed in Chapter 7. Write out the equations of operation of this network, using the form of Eqs. 7.6 and 7.7.
- E12.2** Consider the NARX network of Figure 12.6. This has the form of an LDDN. Write out the equations of operation of this network, using the form of Eqs. 7.6 and 7.7.
- E12.3** Suppose that we have the NARX network of Figure 12.6, with delays of 1 and 2 in the input TDL and delays of 1, 2 and 3 in the feedback TDL. The training data is given below. Using the series-parallel arrangement of Figure 12.7, arrange the data and the network so that you can compute the gradient for training the network using standard static backpropagation. (Some of the ideas in Solved Problem P.12.1 could be useful here.)

t	0	1	2	3	4	5	6	7	8	9
Input	9	18	13	1	4	6	17	5	3	15
Target	0	9	24	27	14	7	5	20	17	12

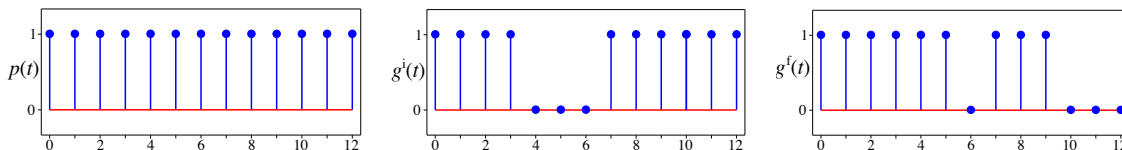
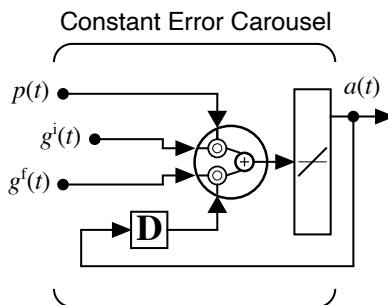
- E12.4** Suppose we have a two-layer FGMN, with two taps on the gamma memory. The first layer of the network has two neurons and a ReLU activation function. The second layer has one neuron and a linear activation function. Draw the network diagram in the form of an LDDN. (Hint. Use ideas from Solved Problem P.12.3, and make a layer for each gamma element.)
- E12.5** Consider the RNN in Figure 12.8, which contains an output layer.
- Unroll the network for two time steps.
 - Write out the backpropagation equations for updating the weight, assuming that there is a target output at Layer 2 for both time steps.

- iii. Perform Lab 1, in which you will implement your backpropagation equations for this network. You can find it at <https://github.com/NNDesignDeepLearning/NNDesignDeepLearning/tree/master/NNDesignDeepLearning/12.NonlinearSequenceProcessingChapter/Code/Labs>

E12.6 Consider the gate shown in the margin and the input signal and gating signals in the figures below. Plot the output of the gate, $a(t)$.



E12.7 Consider the scalar CEC diagram and the input signal $p(t)$, input gating signal $g^i(t)$ and feedback gating signal $g^f(t)$ in the figures below. Plot the network output $a(t)$. Explain how each gate affects network operation.



Nonlinear Sequence Processing

- E12.8** We have a NARX network with two delays in the input TDL and two delays in the feedback TDL. Unfold the network for two time steps. Discuss the difficulty in unfolding a NARX network (or other networks with TDLs) in comparison with unfolding RNNs.

Bibliography

Dario Amodei, Sundaram Ananthanarayanan, Rishita Anubhai, Jingliang Bai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Qiang Cheng, Guoliang Chen, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. In *International conference on machine learning*, pages 173–182. PMLR, 2016.

Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one*, 10(7):e0130140, 2015.

Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33:12449–12460, 2020.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Yoshua Bengio, Pascal Lamblin, Dan Popovici, and Hugo Larochelle. Greedy layer-wise training of deep networks. *Advances in neural information processing systems*, 19, 2006.

François Chollet. *Deep learning with Python*. Manning Publications Company, 2017.

François Chollet. Home - keras documentation. <https://keras.io/>, 2019. Accessed: 2019-06-24.

Dan Claudiu Cireşan, Ueli Meier, Luca Maria Gambardella, and Jürgen Schmidhuber. Deep, big, simple neural nets for handwritten digit recognition. *Neural computation*, 22(12):3207–3220, 2010.

A Bibliography

Orlando De Jesús and Martin T Hagan. Backpropagation algorithms for a broad class of dynamic networks. *IEEE Transactions on Neural Networks*, 18(1):14–27, 2007.

Orlando De Jesús and Martin T Hagan. Backpropagation through time for general dynamic networks. In *IC-AI*, pages 45–51, 2008.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

Richard O Duda, Peter E Hart, et al. *Pattern classification and scene analysis*, volume 3. Wiley New York, 1973.

Saber N Elaydi. *An Introduction to Difference Equations (3rd Edition)*. New York, NY: Springer New York: Imprint: Springer,, 2005.

Ronen Eldan and Ohad Shamir. The power of depth for feedforward neural networks. In *Conference on Learning Theory*, pages 907–940, 2016.

Kunihiko Fukushima. A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.*, 36:193–202, 1980.

Michael S Gazzaniga and Joseph E LeDoux. *The integrated mind*. Springer Science & Business Media, 2013.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256, 2010.

Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323, 2011.

Irving J Good. Explicativity: a mathematical theory of explanation with statistical applications. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 354(1678):303–330, 1977.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

- Martin T Hagan and Orlando De Jesús. A general framework for dynamic networks. In *Research in Computing Science*, volume 15, pages 3–12, 2007.
- Martin T Hagan, Howard B Demuth, Mark Beale, and Orlando De Jesús. *Neural Network Design (2nd Edition)*. Martin Hagan, 2014.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Yu Hen Hu and Jenq-Neng Hwang. *Handbook of neural network signal processing*. CRC press, 2018.
- David H Hubel and Torsten N Wiesel. Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex. *The Journal of physiology*, 160(1):106–154, 1962.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- Amir H Jafari and Martin T Hagan. Application of new training methods for neural model reference control. *Engineering Applications of Artificial Intelligence*, 74:312–321, 2018.
- Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *International conference on machine learning*, pages 2342–2350. PMLR, 2015.
- John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.

A Bibliography

- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Joseph Kelley and Martin T Hagan. Comparison of neural network narx and narmax models for multi-step prediction using simulated and experimental data. *Expert Systems with Applications*, 237:121437, 2024.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Leslie SG Kovásznyai and Horace M Joseph. Image processing. *Proceedings of the IRE*, 43(5):560–570, 1955.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- Will Landecker, Michael D Thomure, Luís MA Bettencourt, Melanie Mitchell, Garrett T Kenyon, and Steven P Brumby. Interpreting individual classifications of hierarchical networks. In *2013 IEEE symposium on computational intelligence and data mining (CIDM)*, pages 32–38. IEEE, 2013.
- Jean le Rond d’Alembert, Michel David, Antoine-Chrétien Boudet, and Jacques Chardon. *Recherches sur differents points importants du système du monde*. Chez David l’aîné, 1754.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Xiang Li, Shuo Chen, Xiaolin Hu, and Jian Yang. Understanding the disharmony between dropout and batch normalization by variance shift. *arXiv preprint arXiv:1801.05134*, 2018.
- Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1-3):503–528, 1989.
- David Madigan, Krzysztof Mosurski, and Russell G Almond. Graphical explanation in belief networks. *Journal of Computational and Graphical Statistics*, 6(2):160–181, 1997.

WD Montgomery and PW Broome. Spatial filtering. *JOSA*, 52(11): 1259–1275, 1962.

Guido F Montufar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in neural information processing systems*, pages 2924–2932, 2014.

Derrick Nguyen and Bernard Widrow. Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights. In *Neural Networks, 1990., 1990 IJCNN International Joint Conference on*, pages 21–26. IEEE, 1990.

Developers NumPy. Numpy. <https://www.numpy.org/>, 2019a. Accessed: 2019-06-24.

Developers NumPy. Quickstart tutorial - numpy. <https://www.numpy.org/devdocs/user/quickstart.html>, 2019b. Accessed: 2019-06-24.

Pandas. Pandas user guide. https://pandas.pydata.org/pandas-docs/stable/user_guide/index.html, 2021. Accessed: 2021-07-14.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318, 2013.

Brett Poulin, Roman Eisner, Duane Szafron, Paul Lu, Russell Greiner, David S Wishart, Alona Fyshe, Brandon Pearcy, Cam MacDonell, and John Anvik. Visual explanation of evidence with additive classifiers. In *Proceedings of the National Conference on Artificial Intelligence*, volume 21, page 1822. Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, 2006.

Software Foundation Python. Welcome to python.org. <https://www.python.org/>, 2019a. Accessed: 2019-06-24.

Software Foundation Python. The python tutorial. <https://docs.python.org/3/tutorial/index.html>, 2019b. Accessed: 2019-06-24.

PyTorch. Home - pytorch. <https://pytorch.org/>, 2024a. Accessed: 2024-10-09.

A Bibliography

PyTorch. Pytorch datasets. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html, 2024b. Accessed: 2024-10-09.

PyTorch. Pytorch documentation. <https://pytorch.org/docs/stable/index.html>, 2024c. Accessed: 2024-10-09.

PyTorch. Pytorch tutorials. <https://pytorch.org/tutorials/>, 2024d. Accessed: 2024-10-09.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023.

Rajat Raina, Anand Madhavan, and Andrew Y Ng. Large-scale deep unsupervised learning using graphics processors. In *Proceedings of the 26th annual international conference on machine learning*, pages 873–880, 2009.

Marc’Aurelio Ranzato, Christopher Poultney, Sumit Chopra, and Yann Cun. Efficient learning of sparse representations with an energy-based model. *Advances in neural information processing systems*, 19, 2006.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. " why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1135–1144, 2016.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

Azriel Rosenfeld. Picture processing by computer. *ACM Computing Surveys (CSUR)*, 1(3):147–176, 1969.

Wojciech Samek, Grégoire Montavon, Sebastian Lapuschkin, Christopher J Anders, and Klaus-Robert Müller. Explaining deep neural networks and beyond: A review of methods and applications. *Proceedings of the IEEE*, 109(3):247–278, 2021.

Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization?(no, it is not about internal covariate shift). *arXiv preprint arXiv:1805.11604*, 2018.

Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *International conference on machine learning*, pages 3145–3153. PMLR, 2017.

Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.

Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*, 2017.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. PMLR, 2015.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.

Matus Telgarsky. Benefits of depth in neural networks. In *Proceedings of the 29th Annual Conference on Learning Theory*, pages 1517–1539, 2016.

Developers TensorFlow. Tensorflow user guide. <https://www.tensorflow.org/guide>, 2021a. Accessed: 2021-08-05.

Developers TensorFlow. tf.data api user guide. <https://www.tensorflow.org/guide/data>, 2021b. Accessed: 2021-08-05.

A Bibliography

Developers TensorFlow. Keras api user guide. https://www.tensorflow.org/guide/keras/sequential_model, 2021c. Accessed: 2021-08-05.

Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Alexander Waibel, Toshiyuki Hanazawa, Geoffrey Hinton, Kiyohiro Shikano, and Kevin J Lang. Phoneme recognition using time-delay neural networks. In *Backpropagation*, pages 35–61. Psychology Press, 2013.

Matthew D Zeiler. Adadelata: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.

Index

- activation function, 2
 - hyperbolic tangent, 3
 - linear, 2
 - poslin, 3
 - relu, 3
 - softmax, 4
 - tangent sigmoid, 3
- Adam, 9
- Attention, 5
 - attention weighting matrix, 7
 - concatenation attention, 8
 - dot product attention, 6
 - general dot product attention, 7
 - key, 6
 - query, 6
 - value, 6
- backpropagation, 12
- backpropagation through time (BPTT), 8
- backpropagation-through-time, 7
- batch, 8
- Batch normalization, 5
- bias, 2
 - tied, 6, 14
 - untied, 6
- broadcast, 11
- causal, 11
- channel, 3
- characteristic equation, 17
- class, 9
- Completeness, 12, 22
- Context, 5
- Continuity, 11, 22
- contribution, 10, 11
- convolution
 - derivation, 2
 - filter, 4
 - kernel, 4
- convolution sum, 9
- cross-correlation, 4
- cross-entropy, 2, 4
- data generator, 8
- data wrangling, 12
- decision boundary, 6
- Decoder, 4
- Deepfake, 7
- DeepLIFT, 17
 - linear rule, 19
 - rescale rule, 19
- delay, 3
- dictionary, 4
- difference equation, 11
- digital networks, 5
- distributed time delay neural network, 3
- Dropout, 10, 15
- dynamic networks, 5
- Dynamic Networks), 1
- dynamic systems, 3
- Early stopping, 14
- encoder, 4
- epoch, 6
- ETL, 7
- explainability, 2
 - contribution propagation, 12
 - gradient-based, 12
- explanation, 4, 11
 - global, 6
 - local, 7
- Exploding gradient, 13
- feature, 3
- feature map, 12
- feedback, 3
- finite impulse response, 12
- focused gamma memory network, 3
- focused time delay neural network, 2
- for loop, 6
- frameworks, 2
- Generalized LDDN, 6
- Gradient clipping, 14
- gradient descent, 5
- if, 5
- Implementation Invariance, 12, 22
- impulse, 3
- impulse response, 3, 9
- impulse sequence, 9
- index, 3
- indexing, 11
- inductive bias, 2, 5
- Inference, 9

A Bibliography

- infinite impulse response (IIR),
 - 15
- input, 2
- Input weight, 5
- input/output models, 18
- integrated gradient, 13
- iteration, 6
- Kaggle, 13
- Kaiming initialization, 13
- Keras, 2
 - compile, 6
 - data format, 2
 - functional API, 4
 - model, 4
 - model subclass, 4
 - sequential, 4
- layer, 5
 - components, 5
- Layer weight, 5
- Layer), 3
- Layered Digital Dynamic Networks, 6
- Layered Feedforward Neural Networks, 4
- license, 2
- list, 3
- local receptive field, 6
- Long , 12
- loss function, 2
- LRP, 16
- matched filter, 9
- mean square error, 2
- memory, 3
- method, 9
- minibatch, 8
- module, 8
- Multilayer Networks, 1
- Multilayer Networks), 1
- NARX, 5
- net input, 2
- Normalization, 4
- NumPy, 9
- observer canonical form, 20
- one hot encoding, 3, 4
- one of K encoding, 3
- output, 2
- padding, 8
- Pandas, 12
 - apply(), 19
 - DataFrame, 13
 - describe(), 14
 - head(), 15
 - iloc, 17
 - info(), 14
 - loc, 17
 - melt(), 18
 - pivot(), 18
 - Series, 16
 - tail(), 15
- performance index, 2
- poles, 17
- pooling, 13
 - average, 13
 - max, 13
- position invariant, 2
- Pretrained model, 3
- Python
 - container, 7
 - generator, 8
 - iterable, 7
 - iterator, 7
 - lambda, 19
 - list comprehension, 6
 - yield, 8
- PyTorch, 2
 - model, 3
 - module subclass, 3
 - optimizer, 4
 - sequential, 3
- real time recurrent learning (RTRL), 7
- recurrent connections, 3
- Recurrent Neural Network (RNN), 6
- Regularization, 15
 - L1, 15
 - L2, 15
- saliency, 12
- samples, 3
- Sensitivity, 12, 22
- sensitivity, 12
- sequence, 2
- Sequence-to-sequence network,
 - 4
- sequences, 2
- Shallow network, 2
- Simulation Order, 5
- slice, 3
- slicing, 11
- SmoothGrad, 14
- stability, 16
- state space model, 18
- states, 18
- static networks, 5
- steepest descent, 5
- stochastic gradient descent, 8
- stride, 8
- Tapped Delay Line, 2
- tapped delay line, 3, 6
- Taylor series, 4
- template matching, 9
- tensor, 2, 9
- TensorFlow, 2
- Test set, 15
- time series, 3
- time-invariant, 9
- transfer function models, 18
- tuple, 4
- unrolling, 10
- Validation set, 14
- Vanishing gradient, 2, 11
- vanishing gradient, 3
- weight, 2
- weight of evidence, 4
- weight sharing, 5
- while, 5
- Xavier initialization, 12